Eine Computersimulation zur Analyse der Auswirkungen von exogenen Schocks auf Versicherungsmärkte – am Beispiel der deutschen Wiedervereinigung –

Dem Fachbereich Wirtschaftswissenschaften der Universität Hannover zur Erlangung des akademischen Grades eines

Doktors der Staatswissenschaften

– Doctor rerum politicarum –

vorgelegte Dissertation

von

Diplom-Ökonom Martin Köhler

geboren am 29.01.1966 in Hannover

Referent: Prof. Dr. J.-M. Graf von der Schulenburg
Koreferent: Prof. Dr. P. Stahlecker

Tag der Promotion: 7. Februar 1996

Geleitwort

Als das Institut für Versicherungsbetriebslehre der Universität Hannover 1988 gegründet wurde, dachte sicher noch niemand an die Wiedervereinigung der seit fast 40 Jahre getrennten beiden deutschen Staaten.

Mit dem Fall der Berliner Mauer begann für die deutsche Versicherungswirtschaft die Zeit des Zusammenwachsens zweier Märkte, wie sie unterschiedlicher nicht hätten sein können.

Für Versicherungsökonomen bekam das Verständnis der Wirkungen von plötzlichen Änderungen der Angebots- und Nachfragesituation nach vielen Jahren einer gleichförmigen Marktentwicklung plötzlich eine ganz neue Bedeutung.

Aufgrund seiner Aktualität war es naheliegend, den Schock der deutschen Wiedervereinigung auf die Versicherungswirtschaft in den Mittelpunkt einer versicherungsökonomischen Promotionsarbeit zu stellen.

Zu den modernen Werkzeugen der Ökonomen gehört in immer stärkerem Maße die Computersimulation. Sie kann in gewissem Sinn als das Forschungslabor der Wirtschaftswissenschaftler bezeichnet werden. Martin Köhler untersucht die Auswirkungen des exogenen Schocks mit Hilfe einer selbstentwickelten Simulation und beschreibt die Entwicklung dieses Programms von den modelltheoretischen Grundlagen bis zu den statistischen Auswertungen der Simulationsergebnisse.

Die Arbeit von Martin Köhler stellt aufgrund der gelungenen Verknüpfung eines prozessorientierten Marktmodells mit einer verhaltensorientierten Simulation einen wichtigen Beitrag zur modernen, empirischen Marktprozessforschung dar.

Prof. Dr. J.-Matthias Graf von der Schulenburg

Vorwort

Die rechnergestützte Analyse ökonomischer Systeme faszinierte mich bereits 1982, als die ersten Heimcomputer in der Lage waren, einfache Modelle zu simulieren. Die Aktualität des Themas und die hervorragenden technischen Voraussetzungen am Institut für Versicherungsbetriebslehre waren entscheidende Voraussetzungen für dieses Forschungsvorhaben.

Die vorliegende Veröffentlichung wurde im Februar 1996 vom Fachbereich Wirtschaftswissenschaften der Universität Hannover als Dissertation angenommen. Mein besonderer Dank gilt meinem Doktorvater Herrn Professor Dr. J.-Matthias Graf von der Schulenburg für seine Unterstützung und seine Diskussionsbereitschaft. Herrn Professor P. Stahlecker vom Institut für Statistik und Ökonometrie der Universität Hamburg danke für die Übernahme des Koreferats und seine Anregungen, die weit über den üblichen Einfluß eines Koreferenten hinausreichten. Danken möchte ich auch Prof. Dr. F. Haslinger, für seine Mitwirkung in der Prüfungskommission.

Zu danken habe ich weiterhin Frau Dr. Susanne Wähling und Frau Dipl.-Ök. Ute Schattschneider, die als Mitarbeiterinnen des Instituts für Versicherungsbetriebslehre ihre knappe Zeit für Korrekturlesen opferten.

Für die finanzielle Unterstützung der Arbeit danke ich der Graduiertenförderung der Universität Hannover, die diese Arbeit mit einem zweijährigen Stipendium unterstützt hat.

Dem Verein zur Förderung der Versicherungswissenschaft an der Universität Hannover e.V. danke ich für die finanzielle Unterstützung zur Veröffentlichung der Ergebnisse.

Mein größter Dank aber gilt meiner Familie, meiner Mutter, meiner Schwester, meinem Schwager und meinen Freunden, deren Liebe, Zuneigung und Freundschaft die eigentliche Voraussetzung für das Gelingen dieser Arbeit waren.

Hannover, im März 1996 Martin Köhler

Inhaltsverzeichnis

Abkürzungsverzeichnis **VIII**

Abbildungsverzeichnis **IX**

Tabellenverzeichnis **X**

1 Einleitung **1**

1.1 Aktueller Bezug der Arbeit 1

1.2 Thematik der Arbeit 2

1.3 Aufbau der Arbeit 4

2 Die Modellbasis **6**

2.1 Ziele des Kapitels 6

2.2 Gleichgewichtstheorien 6

2.3 Marktprozesse 10

2.4 Besonderheiten von Versicherungsmärkten 12

2.5 Ansätze zur Schockanalyse 14

2.6 Elemente der Computersimulation 18

2.6.1 Der Simulationsbegriff 18

2.6.2 Der Ablauf von Simulationstudien 20

2.6.3 Simulationsarten 23

2.6.4 Simulationssprachen 25

2.7 Simulation in der Versicherungswissenschaft 28

3 Das Modell **31**

3.1 Ziele des Kapitels 31

3.2 Der Modellrahmen 31
3.2.1 Das Marktmodell 31
3.2.2 Das Angebotsmodell 35
3.2.3 Das Nachfragemodell 37
3.2.4 Schocks 41
3.3 Die Modellbausteine 43
3.3.1 Die Verhaltensregeln der Anbieter 43
3.3.1.1 Informationsfreie Verhaltensregeln 43
3.3.1.2 Verhaltensregeln mit Eigeninformation 44
3.3.1.3 Verhaltensregeln mit Fremdinformation 48
3.3.2 Die Verhaltensregeln der Nachfrager 51
3.3.2.1 Informationsfreie Verhaltensregeln 51
3.3.2.2 Verhaltensregeln mit Eigeninformation 53
3.3.2.3 Verhaltensregeln mit Fremdinformation 55

4 Das Programm 58
4.1 Ziele des Kapitels 58
4.2 Programmbedienung 59
4.2.1 Das Hauptmenü 59
4.2.2 Die Modellkonfiguration 60
4.2.3 Die Modellauswertung 63
4.3 Programmaufbau 65
4.3.1 Die Programmiersprache 65
4.3.2 Die Datenstrukturen 69
4.3.3 Die Unterprogramme 72

5 Die Programmanwendung 78
5.1 Ziele des Kapitels 78
5.2 Grundlagen zur Bewertung der Simulationsergebnisse 78
5.3 Untersuchungsablauf 80
5.4 Die Simulation 84
5.4.1 Modellkonfiguration 84

5.4.2 Datenauswertung 88
5.4.3 Ableitung der Modellaussagen 95
5.5 Die Variation 97
5.5.1 Modellkonfiguration 97
5.5.2 Datenauswertung 99
5.5.3 Ableitung der Modellaussagen 102

6 Die Ergebnisse **103**
6.1 Die Ergebnisse in Thesen 103
6.2 Ausblick und Schluß 107

Quelltext von imsdata.c **111**

Quelltext von ims.e **119**

Literaturverzeichnis **257**

Abkürzungsverzeichnis

ALGOL	Algorithmic Language
ANSI	American National Standards Institute
ASIR	Advanced Simulation Modell of Insurance and Re-insurance Operations
AWK	Aho, Kernighan, Weinberger
Aufl.	Auflage
BASIC	Beginner's All-Purpose Symbolic Instruction Code
BAV	Bundesaufsichtsamt für das Versicherungswesen
DDR	Deutsche Demokratische Republik
DYNAMO	Dynamic Models
Diss.	Dissertation
ESL	Economic Simulation Language
FORTRAN	Formular Translator
FWU	Forschungsgesellschaft für Wettbewerb und Unternehmensorganisation
GASP	Graph Algorithm and Software Package
GDV	Gesamtverband der Deutschen Versicherungswirtschaft e.V
GIM	Geneva Insurance Modell
GPSS	General Purpose Simulation System
Hrsg.	Herausgeber
IBM	International Business Machines
IMS	Insurance Market Simulation
INSGAME	The Insurance Game
INSULA	Insurance Long Range Analysis
Jg.	Jahrgang
LISP	List Processing
No.	Nummer (englisch: numero)
Nr.	Nummer
S.	Seite
SIMULA	Simulation Language
SK	Subjektklasse
SMS	Stochastic Market Simulation
SOFI	Simulation of Financial Institutions
STELLA	Systems Thinking, Experimental Learning Laboratory
TROLL	Time-Shared Reaktive On-Line Laboratory
VK	Verhaltensregelklasse
VN	Versicherungsnehmer
VR	Verhaltensregel
VU	Versicherungsunternehmen
Vol.	Volume

Abbildungsverzeichnis

1.1 Der Aufbau der Arbeit 4

4.1 Beispiel der Eingabemaske für einen Versicherer 61
4.2 Beispiel der Eingabemaske für einen Versicherungsnehmer 62
4.3 Abhängigkeit der Unterprogramme 77

5.1 Aggregatzuordnung der Versicherer 83
5.2 Autokorrelationsfunktionen der `VU[14]` 88
5.3 Autokorrelationsfunktionen der Subjektklasse `VU` 89
5.4 Werbeausgaben der `VU[14]` 90
5.5 Prämie der `VU[14]` 90
5.6 Marktanteil der `VU[14]` in Sparte 1 91
5.7 Marktanteil der `VU[14]` in Sparte 2 91
5.8 Marktanteil und gleitende Durchschnitte des Marktanteils `VU[14]` 92
5.9 `VU[14]`: Signifikanz der Schockwirkung auf den Marktanteil 93
5.10 Vergleich der Reserven auf unterschiedlichen Aggregatstufen 93
5.11 Vergleich der gezahlten Prämien unterschiedlicher Aggregate 94
5.12 Signifikanzvergleich der Schockwirkung auf die gezahlte Prämie 95
5.13 Durchschnittliche Schadenhöhe in beiden Risiken 98
5.14 Durchschnittsprämie in beiden Sparten 98
5.15 Durchschnittsreserven in beiden Sparten 99
5.16 Reserven der Einzelversicherer in Sparte 1, Zins 3 % 100
5.17 Reserven der Einzelversicherer in Sparte 2, Zins 3 % 100
5.18 Reserven der Einzelversicherer in Sparte 1, Zins 0 % 101
5.19 Reserven der Einzelversicherer in Sparte 2, Zins 0 % 101

Tabellenverzeichnis

3.1 Synopsis der Modellspezifikation 57

4.1 Inhaltsverzeichnis der Unterprogramme im Quelltext von *ims.e* 73

5.1 Zuordnung der Verhaltensregeln zu den Versicherungsnehmern 85

5.2 Zuordnung der Verhaltensregeln zu den Versicherern 86

Kapitel 1

Einleitung

Man brauche gewöhnliche Worte
und sage ungewöhnliche Dinge.
Schopenhauer
Über Schriftstellerei und Stil

1.1 Aktueller Bezug der Arbeit

Als am 9. November 1989 mit dem Fall der Berliner Mauer das Symbol der deutschen Teilung Geschichte wurde, begann eine wirtschaftliche Umbruchphase, die in der Geschichte der deutschen Wirtschaft einmalig ist. Anders als in den Aufbaujahren nach den beiden Weltkriegen ging es nicht um die Rekonstruktion eines ökonomischen Systems, sondern um einen Systemwechsel. Die dabei entstehenden Probleme sind ungleich komplexer und werden sich noch auf Jahre hinaus auf die gesamtdeutsche Wirtschaft auswirken.

Nach der Anfangseuphorie der Wirtschafts- und Währungsunion wurde bald deutlich, daß die ökonomische Aussteuer des Arbeiter- und Bauernstaates eher eine Mogelpackung war. Die Verteilung dieser Mitgift wurde wenig systemkonform einer riesigen Staatsholding übergeben, deren Arbeit am 31.12.1994 mit einem Abschlußdefizit von mehr als 270 Milliarden DM beendet war.

Aber es gab nicht nur Verlustbetriebe abzuwickeln und Entlassungen auszusprechen, sondern auch neue Märkte zu entdecken. Doch dieser Prozeß verlief recht einseitig. Fast immer wurden die ostdeutschen Märkte unter westdeutschen Firmen in einem mehr oder weniger marktwirtschaftlichen Akt verteilt.

Ein besonderes Verteilungsproblem entstand bei Betrieben, die in einem Markt als Monopolist auftraten. Hier standen Treuhandanstalt und Bundeskartellamt vor dem Dilemma entweder ein bestehendes Monopol zu übernehmen oder durch dessen Zerschlagung eine große Anzahl von Beschäftigten in die Arbeitslosigkeit zu entlassen. Der Versicherungs-

markt der ehemaligen DDR war ein Monopolmarkt. Das Angebot des Staatsmonopolisten umfaßte aber nur einen kleinen Teil der Produktpalette westdeutscher Versicherer und nur wenige Produkte der Industrieversicherung. Eine Beteiligung des westdeutschen Allianz-Konzerns an diesem Staatsmonopolisten wurde noch vor der Unterzeichnung des ersten Staatsvertrags zur Wirtschafts- und Währungsunion durch die Regierung Modrow am 3. März 1990 gebilligt[1].

Vielen Vertretern westdeutscher Versicherer erschien dies wie das Ende des Wirkens der Kräfte des freien Marktes für das wiedervereinigte Deutschland. Andere formulierten vorsichtiger, daß "damit der Marktwirtschaft ein schlechter Dienst erwiesen wurde"[2]. Unstrittig ist jedoch, daß die deutsche Versicherungswirtschaft mit der Wiedervereinigung "vor den tiefgreifendsten Änderungen ihrer Rahmenbedingungen seit Ende des Zweiten Weltkrieges"[3] stand. Für die Befürworter der Allianz-Beteiligung dagegen gab es aufgrund der Finanzschwäche und der fehlenden Überlebensfähigkeit der Staatsversicherung kaum eine wettbewerbspolitische Alternative[4].

Eine mehrheitliche Übernahme der Staatsversicherung durch einen westdeutschen Versicherer war aber nicht die einzige Lösung, die damals diskutiert wurde. Alternativ war die Aufteilung der Verträge der Staatsversicherung auf die westdeutschen Versicherer im Gespräch oder die Einbringung der Verträge in einen Versicherungspool[5].

Abstrahiert man von den stattgefundenen Ereignissen und versucht sie in einen theoretischen Rahmen zu bringen, dann kann die Wiedervereinigung als ein exogener Schock auf die Versicherungsmärkte betrachtet werden. Ein Schock wird im Duden als ein Stoß oder plötzlicher Schlag bezeichnet [6]. Ein Schock auf einen Markt bedeutet eine wesentliche Änderung der Rahmenbedingungen für die in diesem Markt Agierenden.

1.2 Thematik der Arbeit

Im Mittelpunkt dieser Arbeit steht die Entwicklung eines Modells, daß Marktprozesse eines Versicherungsmarktes simulieren kann. Schocks werden als exogene Elemente von Marktprozessen aufgefaßt. Die Untersuchung ihrer Auswirkungen auf endogene Variablen muß daher zwingend auf der Analyse von Marktprozessen basieren. Marktprozesse sind immer das Ergebnis von Interaktionen der Marktteilnehmer. Eine Simulation von Marktprozessen muß die Interaktion der Marktteilnehmer beinhalten. Zu diesem Zweck werden die Angebots- und Nachfrageseite des Marktes disaggregiert. Die Interaktion der Marktteilnehmer wird auf einen Minimalprozeß zurückgeführt, dessen Ergebnisse das Verhalten der einzelnen Versicherer und Versicherungsnehmer beeinflussen.

[1] vgl. Bach (1990), S. 38
[2] Klein (1991), S. 203
[3] Wesselkock (1991), S. 1450
[4] vgl. Haasen (1990), S. 435
[5] vgl. Gerlach (1991), S. 893
[6] vgl. Wissenschaftlicher Rat der Dudenredaktion (Hrsg.)(1973), S. 610

Der Minimalprozeß wird ständig wiederholt, so daß die Ergebnisse der Interaktion eines Zeitabschnitts die Verhaltensgrundlage des nächsten Zeitabschnitts sein können. Die Marktteilnehmer besitzen ein Gedächtnis und eine Verhaltensfunktion, die ihre Entscheidungen im Marktprozeß steuern.

In diesem Marktprozeß werden Schocks simuliert und deren Auswirkungen anhand der Änderungen der Verhaltensergebnisse von Marktteilnehmern untersucht. Nach der kompletten Disaggregation können die Ergebnisse aus verschiedenen Perspektiven der Aggregation betrachtet werden. Durch die Gruppenbildung innerhalb der Anbieter und Nachfrager wird eine differenzierte Betrachtung der Schockwirkungen möglich.

Versicherungsmärkte besitzen im Vergleich zu Gütermärkten einige Eigenschaften, die die Untersuchung von Schocks zusätzlich erschweren und damit besonders interessant machen. Das Hauptelement der Versicherung ist das Risiko. Es bestimmt den Preis der Ware und ihren Austausch. Weiterhin ist der Anteil der Kosten der Vorleistungen für die Produktion dieses Gutes relativ klein[7]. Kapazitätsgrenzen und Produktionsengpässe sind nicht mit anderen Märkten vergleichbar. Es gibt keine Lagerkosten und die Anpassungsgeschwindigkeit der Marktteilnehmer an Änderungen der Rahmenbedingungen ist hoch. Es ist daher als wahrscheinlich anzusehen, daß Schocks auf Versicherungsmärkten anders wirken als auf Gütermärkten.

In dieser Arbeit wird ein künstlicher Versicherungsmarkt geschaffen, der es ermöglicht, alternative Szenarien von Marktschocks miteinander zu vergleichen. Der thematische Schwerpunkt liegt in der Modellierung und Programmierung einer Simulation, die in der Lage ist, verschiedene Marktteilnehmer auf einen Versicherungsmarkt in einem Marktprozeß interagieren zu lassen. Weiterhin ist das Programm in der Lage, verschiedene Schocks auf den Versicherungsmarkt einwirken zu lassen und ermöglicht dem Anwender, die Parameter des Marktes und der Schocks zu beeinflussen. Ziel ist es, ein Analyseinstrument für dynamische Marktuntersuchungen von Versicherungsmärkten schaffen und es auf das Szenario der Wiedervereinigung anzuwenden. Ein weiteres Ziel dieser Arbeit ist die Dokumentation des Erstellungsprozeßes dieses Programms. Der Leser soll nicht nur die ökonomischen Prozesse besser verstehen und bewerten können, sondern auch in die Lage versetzt werden, den Vorgang der Simulationsentwicklung nachvollziehen zu können.

Das erstellte Simulationsprogramm ist portabel, d.h. eine Umsetzung auf andere Rechnerstrukturen und Betriebssysteme ist ohne zusätzlichen Aufwand möglich. Es muß flexibel genug sein, um den komplexen Modellanforderungen gerecht werden zu können. Die Teilnehmer des Versicherungsmarktes werden mittels ihrer verhaltensrelevanten Daten und Regeln abgebildet. Der Versicherungsmarkt wird als Ganzes betrachtet. Um die Komplexität zu begrenzen, wird von speziellen Eigenschaften bestimmter Sparten oder Spezialversicherungen abstrahiert. Die Simulation soll nicht eine spezielle Variable des Versicherungsmarktes berechnen, sondern dem Anwender eine Vielzahl von Daten zur Verfügung stellen. Diese Daten werden dann programmintern und -extern weiter ausgewertet.

[7] vgl. Flöthmann (1989), S. 40

Nach der Erstellung des Programms wird das Marktprozeßmodell der deutschen Wiedervereinigung damit simuliert. Das Prozeßmodell besteht aus Parametern und Startwerten, die das real beobachtbare Verhalten von Marktteilnehmern in Modellverhalten übersetzen. Die dabei erzeugten Daten sind die Basis für die grafische und statistische Analyse der Schockwirkungen. Die Verifikation der Ergebnisse richtet sich an Plausibilitätsüberlegungen und an während der Modellierungsphase gebildeten Erwartungen aus.

Die Motivation dieser Arbeit steht auf zwei Säulen: Den Besonderheiten der Analyse von Versicherungsmärkten und den Unzulänglichkeiten der bisherigen Aussagen der Versicherungstheorie zu Marktprozessen. Marktverhalten und Marktprozeßtheorie von Märkten unter Risiko sind bislang kaum untersucht worden und bieten dadurch ein weites Betätigungsfeld[8]. Schocks werden bislang fast ausschließlich als Störungen eines Gleichgewichtszustandes am Markt aufgefaßt. Die Annahme eines permanenten Ungleichgewichts und die Ableitung der Modelle der Marktteilnehmer aus der Verhaltenstheorie anstatt aus der Entscheidungstheorie sind weitere Grundbausteine dieser Arbeit[9].

1.3 Aufbau der Arbeit

Die einzelnen Kapitel sind hierarchisch aufgebaut. Die Kapitel 3 bis 5 bauen jeweils auf den Ergebnissen der Vorkapitel auf oder setzen sie um. Kapitel 6 faßt diese Ergebnisse in Thesen zusammen und gibt einen Ausblick auf Weiterentwicklungen des hier vorgestellten Modellansatzes.

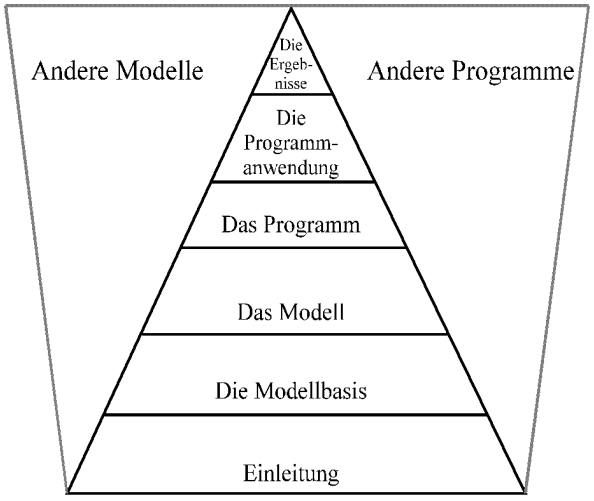

Abbildung 1.1: Der Aufbau der Arbeit

Jedes der vier Kernkapitel beginnt mit der Definition seiner Ziele. Im zweiten Kapitel wird die Modellbasis geschaffen. Analysen der bestehenden Gleichgewichts- und Markt-

[8] vgl. Schulenburg (1993), S. 526
[9] vgl. Schmalensee (1986), S. 384

prozeßtheorien bilden die Grundlage des zu erstellenden Modells. Die Besonderheiten des Versicherungsmarktes werden dabei ebenso aufgegriffen, wie die in der bisherigen ökonomischen Modellwelt verwendeten Ansätze zur Schockanalyse. Der Simulationsbegriff wird definiert und der Ablauf der Simulationsstudie festgelegt. Danach wird das eigene Konzept eines Simulationsmodells und die zu seiner Erstellung verwendete Simulationssprache mit existierenden Modellen und Simulationssprachen verglichen.

Im dritten Kapitel wird das IMS-Modell[10] vorgestellt. Dieses Modell bildet die Grundlage für das Simulationsprogramm, daß im vierten Kapitel ausführlich dargestellt wird. Zunächst werden die grundlegenden Marktprozesse vorgestellt und im Anschluß daran die Teilmodelle der Angebots- und der Nachfrageseite beschrieben. Im Anschluß an die Einführung der grundlegenden Modelleigenschaften werden die unterschiedlichen Verhaltensregeln von Anbietern und Nachfragern vorgestellt, die im Rahmen einer Simulation interagieren können.

Das vierte Kapitel beschreibt die Programmbedienung und den Programmaufbau. Es dient einerseits zum Verständnis der Umsetzung des in Kapitel 3 geschaffenen Modells und andererseits zur Beschreibung der Programmbedienung. Die Benutzung des Programms wird anhand der typischen Anwendungszyklen beschrieben. Neben der Programmbedienung werden die Formate der erzeugten Simulationsdateien erläutert, in denen die Simulationsrohdaten gespeichert werden. Bei der Erstellung der Simulation stehen zwei Aspekte im Vordergrund: Portabilität auf andere Betriebssysteme und Weiterentwicklung durch den Anwender. Die Programmstrukturen werden soweit erläutert, daß ein Anwender mit grundlegenden Programmierkenntnissen einerseits und hinreichend spezifizierten Erweiterungswünschen andererseits das Programm um eigene Verhaltensregeln, Schockdefinitionen und Auswertungen ergänzen kann.

Im fünften Kapitel wird die Simulation auf ein Modell der Wiedervereinigung der deutschen Versicherungsmärkte angewendet. Das Kapitel beginnt mit der Vorstellung grundlegender Überlegungen zur Bewertung der Simulationsergebnisse. Daran schließt sich eine Beschreibung des Untersuchungsablaufs an, der auch als methodische Basis für weitere Anwendungen des Modells dienen kann. Danach erfolgt die eigentliche Simulation und die Auswertung der dabei erzeugten Simulationsdaten. Aus der Analyse dieser Daten werden dann in einem letzten Schritt die Modellaussagen abgeleitet.

Im Anhang dieser Arbeit findet sich ein ausgedruckter Programmquelltext. Die Dateien, die für eigene Programmweiterentwicklungen notwendig sind und die Binärdateien der Simulation können über das INTERNET abgerufen werden unter:

`http://www.econ.uni-hannover.de/~ivbl/fofeims.htm`

[10] IMS ist die Abkürzung für **I**nsurance **M**arket **S**imulation

Kapitel 2

Die Modellbasis

2.1 Ziele des Kapitels

In diesem Kapitel wird die Modellbasis der Arbeit beschrieben. Zunächst wird ein Überblick zu den Teilgebieten Gleichgewichtstheorie und Marktprozeßtheorie gegeben. Dabei werden die bestehenden Theorieansätze gemäß den Anforderungen des zu schaffenden Marktmodells auf ihre Verwendbarkeit geprüft. Daran schließt sich ein Abschnitt an, der die Besonderheiten von Versicherungsmärkten im Vergleich zu anderen Märkten untersucht. Danach werden die Elemente von Simulationen analysiert. Der Simulationsbegriff wird eingeführt, der Ablauf von Simulationsstudien untersucht und Unterschiede zur eigenen Vorgehensweise herausgearbeitet. Die Typisierung der Simulationen und der Simulationssprachen führt zu einem Überblick der in der Versicherungswissenschaft bislang verwendeten Simulationen.

2.2 Gleichgewichtstheorien

Nahezu alle großen Theorieströmungen der Ökonomie haben bislang das Gleichgewicht in den Mittelpunkt ihrer Untersuchung gestellt. Zwar waren ihre Prämissen und axiomatischen Ausgangspositionen zum Teil höchst unterschiedlich, doch die Philosophie war sehr ähnlich. Sie alle postulieren einen Endzustand der Stationarität: Das Gleichgewicht. Im Rahmen dieser Arbeit sind die bestehenden Gleichgewichtstheorien nicht der Mittelpunkt, sondern der Ausgangspunkt für einen eigenen Modellansatz: Dem Modell eines Versicherungsmarktes mit exogenen Schocks.

Grundlage der Neoklassik ist die Allgemeine Gleichgewichtstheorie[1]. Mit ihr hat *Walras* im letzten Jahrhundert den Existenzbeweis für ein Gleichgewicht in einem statischen

[1] vgl. Holleis (1985), S. 23

Marktsystem unter vollständiger Konkurrenz geliefert[2].

Obwohl diese Theorie in sich mathematisch geschlossen war, besaß sie jedoch entscheidende Restriktionen, die sich aus ihren Prämissen ergeben. Diese Prämissen lassen sich in zwei Gruppen unterteilen: **Verhaltensprämissen** und **Umweltprämissen**. Zu den Verhaltensprämissen zählen[3]:

1. Maximierendes Verhalten
2. Rationales Verhalten
3. Unendlich hohe Reaktionsgeschwindigkeit
4. Mengenanpassung

Unter den Umweltprämissen sind folgenden Annahmen zusammengefaßt:

1. Unendlich viele Marktteilnehmer
2. Keine "barriers to entry" für Ressourcen und Faktoren
3. Keine Transaktionskosten
4. Homogene Güter
5. Bruttosubstituierbarkeit
6. Auktionator
7. Erstausstattung ist gegeben
8. Vollständige Teilbarkeit der Güter und Faktoren
9. Neutralität des Geldes

Unter diesen Prämissen läßt sich für eine *Walras*-Ökonomie eine geschlossene algebraische Lösung angeben. Die allgemeine Gleichgewichtstheorie war geistig im Newtonschen Weltbild verankert. Newton hatte durch die Entdeckung der mechanischen Theorie ein deterministisches Weltbild geschaffen, daß außer der Physik auch die Nationalökonomie stark beeinflußte. Mit zunehmender Verbreitung des Ansatzes von *Walras*, begann die Mathematisierung der Ökonomie. Sein statischer Ansatz wurde von *Arrow* und *Debreu*[4] auf intertemporale Gleichgewichte erweitert. Doch auch danach war die neoklassische Gleichgewichtstheorie weitgehend statisch, denn die Integration intertemporaler Gleichgewichte

[2] vgl. Schumann (1987), S.182
[3] vgl. Holleis (1985), S. 28
[4] vgl. Arrow, Debreu (1954), S. 266

erforderte als zusätzliche Prämisse die Existenz von Märkten für alle zukünftigen, risikobehafteten Güter[5].

Seit Arrow und Debreu hat die Ökonomie zwei große Hauptströmungen: die "Erweiterung des Erkenntnisgegenstands" und die "Ausnahmenökonomie". In der ersten Hauptströmung wird versucht, Erkenntnisse der neoklassischen Theorie auf andere Lebensbereiche anzuwenden. Grundannahme dieses Zweiges ist, daß sich eine gleichgewichtige Marktlösung in allen diesen Bereichen durchsetzen wird. Als einer der Hauptvertreter dieser Theorierichtung ist *Garry S. Becker* anzusehen, der versucht, alle verhaltenstheoretischen Ansätze der Ökonomie in nutzentheoretische Ansätze umzuwandeln[6]. Die zweite Hauptrichtung der Ökonomie beschäftigt sich mit Änderungen der allgemeinen Gleichgewichtstheorie bei veränderten Prämissen.

Schon früh erkannte man die geringe Praktikabilität der allgemeinen Gleichgewichtstheorie und untersuchte die Auswirkungen von Prämissenänderungen. Die Prämisse der vollständigen Konkurrenz wurde zunehmend fallengelassen. Die Variation der Anzahl der Marktteilnehmer auf der Angebots- und Nachfrageseite führte zur Entwicklung der Marktformenlehre. Die so gefundenen Marktformen waren nun zwar realitätsnäher, stellten aber auch neue Anforderungen an die Preistheorie. War Preisbildung im Monopol mit der Lösung von *Cournot* analytisch noch einfach zu handhaben[7], so stand man bei der monopolistischen Konkurrenz vor schwierigeren Problemen.

Auf Märkten mit monopolitischer Konkurrenz haben Anbieter jeweils nur einen sehr geringen Marktanteil und Nachfrager nur eine geringe Nachfragemenge. Die einzelnen Marktteilnehmer können Präferenzen haben, die wiederum von anderen Marktteilnehmer beeinflußt werden können[8]. Der Preis ist nicht mehr der einzige Aktions- oder Strategieparameter, sondern wird um absatzpolitische Instrumente erweitert. Da die Marktmacht der Teilnehmer vernachlässigbar klein ist, trifft jeder seine strategischen Entscheidungen autonom. *Chamberlin*[9] fand für diese Marktform als erster eine gleichgewichtige Preis-Mengenkombination, aber unter der sehr restriktiven Symmetrieannahme. Diese Annahme besagte, daß alle Anbieter immer den gleichen Preis setzen. *Gutenberg*[10] begründete in seinem Ansatz die doppelt geknickte Preis-Absatzfunktion. Jeder Anbieter kann seine Preise im einem bestimmten Bereich erhöhen und verliert aufgrund der bestehenden Präferenzbindung nur einen Teil seiner Nachfrager. Erhöht er den Preis zu stark, dann verliert er alle Nachfrager. Senkt er dagegen den Preis zu sehr, um neue Nachfrager an sich zu binden, erhält er keine ausreichende Vergütung für seine Produktionskosten.

Im Gegensatz zur monopolistischen Konkurrenz muß ein Marktteilnehmer im Oligopol sehr wohl mit Einflüssen der Entscheidungen anderer Marktteilnehmer auf seine eigenen Handlungen rechnen. Dadurch wurde diese Marktform zur großen mathematischen

[5] vgl. Debreu (1959), S. 100
[6] vgl. Holleis (1985), S. 61
[7] vgl. Schumann (1987), S. 250
[8] vgl. Schumann (1987), S. 274
[9] vgl. Chamberlin (1962), S. 28
[10] vgl. Gutenberg (1965), S. 294

Herausforderung der Ökonomie. Die ersten untersuchten Formen des Mengendyopols von *Cournot*[11] und von *von Stackelberg*[12] waren analytisch recht einfach gehalten. In späteren Modellen konnte vor allem die Spieltheorie von *von Neumann, Morgenstern*[13] einen erheblichen Beitrag zur Lösung komplexerer Oligopolmodelle leisten.

Zur Vereinfachung wird oftmals das sogenannte Gefangenen-Dilemma[14] als Beispiel für strategische Entscheidungen auf oligopolistischen Märkten herangezogen. Dieses statische, nicht-kooperative Zwei-Personenspiel hat aufgrund seiner einfachen Lösung (Gleichgewicht in reinen Strategien) aber nur Lehrcharakter. Für realistischere Märkte werden spieltheoretische Modelle schnell algebraisch anspruchsvoll. Bei diesen Modellen werden wiederholte, mitunter mehrstufige kooperative oder auch gemischte Modelle verwendet.

Die Oligopoltheorie wird noch komplizierter, wenn man von statischen Modellen zu dynamischen Ansätzen übergeht. In dieser Schnittmenge von Oligopoltheorie, dynamischer Gleichgewichtstheorie und Preistheorie hat sich ein eigener Theoriezweig entwickelt: Die Marktprozeßtheorie[15]. Diese Theorie untersucht einen Markt immer unter der Prämisse des Ungleichgewichts und versucht Modelle zu entwickeln, die den Marktprozeß endogen darstellen. Ein Annährungsprozeß an ein Gleichgewicht wird in diesem Ansatz nur als eine mögliche Entwicklung des Marktprozesses angesehen. In der neoklassischen Theorie wurde dieser Anpassungsprozeß durch entsprechende Prämissen ausgeklammert. *Walras* führte den Tâtonnement-Prozeß ein, bei dem ein Auktionator solange Preise "ausruft" bis alle Märkte im Gleichgewicht sind. *Edgeworth* erweiterte diesen Modellansatz um das "recontracting", was bedeutete, daß alle Kontrakte, die nicht zum Gleichgewichtspreis vorgenommen worden waren, als vorläufig und somit als aufhebbar angesehen werden konnten[16].

Auch wenn die Oligopoltheorie realitätsnäher ist als die Theorie der vollkommenen Konkurrenz, kommt auch sie nicht ohne Prämissen aus, die sie analytisch handhabbar macht. Eine ihrer Hauptprämissen ist das Rationalitätspostulat und eng damit zusammenhängend: Das Gewinnmaximierungspostulat. Das Rationalitätspostulat sorgt dafür, daß jeder Marktteilnehmer rationale Annahmen über das Verhalten der anderen Marktteilnehmer trifft. Eine weitere implizite Annahme kommt noch dazu: Jeder Marktteilnehmer nimmt an, daß sich alle anderen Marktteilnehmer ebenfalls rational verhalten. Somit muß die Theorie keinerlei Aussagen über den Such-, Lern- und Adaptionsprozeß der einzelnen Marktteilnehmer machen.

Die Adaptive Oligopoltheorie[17] ersetzt das Gewinnmaximierungspostulat durch das Gewinnmotivationspostulat und untersucht explizit Lern- und Suchprozeße und bezieht diese in die Gleichgewichtsbildung mit ein.

[11] vgl. Schumann (1987), S. 291
[12] vgl. Schumann (1987), S. 297
[13] vgl. Neumann, Morgenstern (1973), S. 523
[14] vgl. Holler, Illing (1991), S. 4
[15] vgl. Witt (1980), S. 43
[16] vgl. Fischer (1988), S. 8
[17] vgl. Klopstech (1980), S. 22

Die Einbeziehung von Such- und Lernprozeßen in ein Marktmodell macht eine analytische Modellbildung zu einem fast hoffnungslosen Unternehmen[18]. Aus diesem Grund muß oftmals auf eine größtmögliche Allgemeingültigkeit der Analytik zugunsten einer nummerischen Einzelfallbetrachtung verzichtet werden. Die immer stärkere Mathematisierung der Ökonomie hat seit Walras dazu geführt, daß zur analytischen Lösbarkeit komplexerer Konkurrenz- und Wettbewerbsmodelle immer stärkere Modellprämissen eingeführt werden müssen, die die Modelle zwar analytisch lösbar, aber meistens realitätsferner machen.

2.3 Marktprozesse

Dynamische Marktmodelle sind ohne Aussagen über ihre Marktprozesse unvollständig. Die Theorie des allgemeinen Gleichgewichts besitzt, obwohl sie als rein statische Theorie konzipiert wurde, für die Steuerung dieser Marktprozesse einen Koordinationsmechanismus: Das Tâtonnement. Diese Koordinationsvorschrift wirkt auf die Preise und Transaktionen auf dem Markt und sorgt dafür, daß bei Überschußnachfrage die Transaktionen solange ausgesetzt werden, bis diese durch Preisanpassungen abgebaut worden ist und ein Gleichgewicht erreicht wird. Da die Transaktionen als Hauptmerkmal eines Marktprozesses bei ungleichgewichtigen Preisen durch das Tâtonnement ausgesetzt sind, kann eigentlich nicht von einem Marktprozeß gesprochen werden. Weil die allgemeine Gleichgewichtstheorie auf konsistenten Aktionen der Transaktoren des Marktes beruht, sind Aktionen bei ungleichgewichtigen Preisen nicht definiert. Erweitert man das Tâtonnement um eine Rationierungsvorschrift, die die Nettotransaktionspläne begrenzen kann und von den Transaktoren berücksichtigt wird, dann lassen sich die Inkonsistenzen zumindest teilweise auflösen. Das um die Rationierungsvorschrift erweiterte Tâtonnement wird als Negeshi-Prozeß[19] bezeichnet. Im Negeshi-Prozeß sind bei jedem Preissystem, auch bei ungleichgewichtigem, Transaktionen zugelassen. Durch die nur unvollständige Erfüllung der Nettotransaktionspläne wird aber die Weltbeschreibung aus der Sicht der Transaktoren unvollständig. Zusammengefaßt ist der Marktprozeß in der allgemeinen Gleichgewichtstheorie auf einen zentralen Preisausgleichsmechanismus reduziert und dient der Erhaltung eines Koordinationsgleichgewichts[20].

Interaktion und Konkurrenz stehen im Mittelpunkt der Marktprozeßtheorie[21]. In der klassischen Vorstellung von Adam Smith wird der Marktprozeß durch kurzfristig inflexible Angebots- und Nachfragemengen ausgelöst und durch die langfristig wirkende, vollkommen flexible Anpassung des Preises an einen "natürlichen Preis" des jeweiligen Gutes beendet. Der natürliche Preis war in der Vorstellung der klassischen Ökonomen gleichzusetzen mit den langfristigen Opportunitätskosten der Anbieter.

Im Zuge der Entwicklung der neoklassischen Theorie kam es zu einer Neudefinition der

[18] vgl. Klopstech (1980), S. 61
[19] vgl. Otruba (1981), S. 34
[20] vgl. Lehmann-Waffenschmidt (1992), S. 94
[21] vgl. Witt (1980), S. 40

Aufgaben der Marktprozeßtheorie. Da die Neoklassik das Weltbild Newtons auf die Ökonomie übertragen wollte, mußte sich die Erklärung des Marktprozeßes von einer unscharfen Fluktuation hin zu einer idealen Konvergenz zum Gleichgewicht wandeln[22]. Die Neoklassik sah das Gleichgewicht eines Marktes als seinen naturgegebenen Zustand an. Dadurch verkümmerte die Marktprozeßtheorie zu einem Theoriezusatz zur Erklärung des Konvergenzverhaltens des Gleichgewichts.

In der neoklassischen Theorie hat besonders *Alfred Marshall* Aussagen zu Marktprozessen formuliert. Im Gegensatz zu Walras konnten in seinem Ansatz auch bei ungleichgewichtigen Preisen Transaktionen stattfinden. Auch im Ungleichgewicht wird produziert und werden Preiserwartungen gebildet[23]. Der Anbieter maximiert aber seinen Gewinn nicht auf der Basis tatsächlicher Preise, sondern nach Preiserwartungen. Die Differenz zwischen erwartetem und tatsächlichem Preis initialisiert einen Anpassungsprozeß, der letztlich zum Gleichgewicht führt. Im Gegensatz zu Walras, der in der Überschußnachfrage*menge* den Anpassungsmotor sah, stellt Marshall den Exzess-Preis an den Anfang des Konvergenzprozesses[24]. Die Kernannahme auf der Mikroebene, die dieses Verhalten ermöglicht, besagt, daß die Marktnachfrage unabhängig von den vergangenen Transaktionen ist[25]. Nur wenn die Nachfrager keine schlechten Erfahrungen behalten können, also nach jeder Periode ihr Gedächtnis verlieren, kann der Prozeß zum Gleichgewicht führen. Ihre völlige Erinnerungslosigkeit läßt die Nachfrager auch während des Anpassungsprozesses zu überhöhten Preisen kaufen.

In realen Marktprozessen spielen imperfekte Informationen und beschränktes Erinnerungsvermögen eine entscheidende Rolle[26]. Mangelnde Kenntnis von Mengen und Preisen führt in der verhaltensorientierten Mikroökonomie zur Bildung von Suchverhalten. Die ökonomische Untersuchung von Suchprozessen, die durch *Stigler* zu Beginn der 60er Jahre initiiert wurde, hat wichtige Beiträge zum Verständnis von Marktprozessen geleistet[27].

Obwohl die Theorie *Marshalls* Marktprozesse im Ungleichgewicht ausdrücklich vorsieht, sind sie immer nur Anpassungsmarktprozesse. Schocks auf der Makro- oder Mikroebene sind immer Auslöser von Marktprozessen. Aber nicht jeder Marktprozeß muß zum Gleichgewicht führen. Im Marshall-Ansatz ist die vollkommen imperfekte Information der Garant für einen Anpassungsprozeß.

Im Marktprozeß auf dem IMS-Markt[28] hängt das Verhalten eines Anbieters oder Nachfragers von seiner Informationsmenge ab. Ist die Informationsmenge eines Transaktors auf dem IMS-Markt leer, dann trifft er Zufallsentscheidungen. Das Modell sieht Transaktionen bei Ungleichgewicht ausdrücklich vor. Ein Marktprozeß auf dem IMS-Markt ist aber kein Anpassungsprozeß an ein Gleichgewicht, weil es keine zentrale Preiskoor-

[22] vgl. Witt (1980), S. 43
[23] vgl. Witt (1980), S. 78
[24] vgl. Witt (1980), S. 80
[25] vgl. Witt (1980), S. 81
[26] vgl. Schulenburg (1993), S. 518
[27] vgl. Stigler (1961), S. 218
[28] IMS ist die Abkürzung für **I**nsurance **M**arket **S**imulation

dinationsinstitution gibt und die Anbieter und Nachfrager keinen verhaltensimmanenten Gleichgewichtsdrang besitzen.

Die bestehenden Untersuchungen zur Ungleichgewichtstheorie sind in der Makroökonomie angesiedelt. In einer makroökonomischen Partialanalyse lassen sich exogene Schocks auf einzelnen Märkte durch einmaliges Verschieben von Angebots- oder Nachfragekurve darstellen. Wird nur ein Markt betrachtet, dann führen die keynesianischen Marktprozeßhypothesen immer wieder zum Gleichgewicht[29]. Wird dagegen eine Totalanalyse angestrebt, muß die allgemeine Gleichgewichtstheorie Anwendung finden. Für sie stellt jeder Angebots- oder Nachfrageschock einen neuen Beobachtungszustand dar, an dem sie prüft, ob es wieder zu einem Gleichgewicht auf allen Märkten kommen kann. Über die auf möglicherweise allen Märkten stattfindenden Marktprozesse und deren Zusammenspiel macht die allgemeine Gleichgewichtstheorie keine Aussagen[30].

In der post-keynesianischen makroökonomischen Ungleichgewichtstheorie werden permanente Marktungleichgewichte untersucht. Aber selbst auf dem hochaggregierten Niveau der Makroökonomie kann die Ungleichgewichtstheorie keine analytisch-exakten Aussagen zu den Marktprozessen bei Ungleichgewicht machen[31].

Faßt man nun die bestehenden Aussagen der analytischen Wirtschaftstheorie zusammen, dann wird deutlich, daß eine theoretisch-analytische Untersuchung der Auswirkungen von Schocks auf ein mikroökonomisches Partialmarktmodell keine Aussichten auf Erfolg hat. Die Betrachtung der Marktprozesse ist für die Untersuchung von Schockwirkungen unerläßlich, weil Schocks immer die Marktprozesse beeinflussen. Da weder die Gleichgewichts- noch die Ungleichgewichtstheorie analytische Ansätze bieten, wird das IMS-Modell zugunsten einer numerischen Speziallösung auf eine allgemeine analytische Betrachtung verzichten müssen. Im Mittelpunkt der numerischen Lösung steht ein Ansatz des permanenten Ungleichgewichts, in dem die Wirkungen von Schocks durch die Erfolge oder Mißerfolge der handelnden mikroökonomischen Einheiten gemessen wird. Durch die Abkehr von der entscheidungstheoriebasierten Marktprozeßtheorie und die Hinwendung zur verhaltenstheoriebasierten Marktprozeßtheorie können realistischere Annahmen über das Verhalten der Marktteilnehmer getroffen werden, ohne das die analytische Lösbarkeit die Realitätsnähe des Marktmodells nachhaltig beeinträchtigt.

2.4 Besonderheiten von Versicherungsmärkten

Um dynamische Wettbewerbsmodelle von Versicherungsmärkten zu formulieren ist es notwendig, die Besonderheiten von Versicherungsmärkten zu analysieren. Für die spätere Modellformulierung ist die dort herrschende Konkurrenzsituationen unter Berücksichtigung der deutschen Marktverhältnisse zu erfassen.

[29] vgl. Flassbeck, Horn, Zwiener (1992), S. 43
[30] vgl. Flassbeck, Horn, Zwiener (1992), S. 46
[31] vgl. Rothschild (1981), S. 128

Die Besonderheiten von Versicherungsmärkten ergeben sich überwiegend aus den besonderen Eigenschaften des gehandelten Gutes. Versicherungen sind als Dienstleistungen nicht lagerfähig und für sie gilt das uno-actu-Prinzip[32]. Das Risiko ist das bestimmende Element dieses Marktes. Im Gegensatz zu Gütermärkten, auf denen die Anbieterseite nur Produzentenfunktion innehat, ist der Anbieter von Versicherungen auch Vermittler im Risikotausch zwischen den Marktteilnehmern[33]. Durch sein Auftreten sinken die Transaktionskosten beim Risikotausch zwischen den Nachfragern.

Der Produktionsprozeß auf Versicherungsmärkten weist eine weitere Spezialität auf: Der Hauptkostenfaktor einer Versicherung sind die Schadenkosten. Von der hohen Anpassungsgeschwindigkeit der Prämien hängt das langfristige Überleben des Unternehmens ab. Die Untersuchung und Modellierung von Wettbewerbsprozessen hat auch die Dauer der Vertragsverhältnisse von Versicherungen zu berücksichtigen. Der Wettbewerb findet meist nur um das Neugeschäft oder kurzfristig abgeschlossene Verträge statt[34]. Der hohe Anpassungsdruck und die notwendige hohe Anpasungsgeschwindigkeit lassen die Untersuchung von Schocks auf diesem Markt besonders lohnenswert erscheinen.

Um die Elemente Risiko und hohe Anpassungsgeschwindigkeit in ein Wettbewerbsmodell mit der Einbeziehung exogener Schocks integrieren zu können, müssen einige Abstraktionen vorgenommen werden. Das Modell sollte sich auf das Neugeschäft beschränken, weil das Bestandsgeschäft realer Märkte kaum Dynamik aufweist. Weiterhin sollten neben den Schadenkosten nur wenige andere Kostenarten berücksichtigt werden, weil die Prämienanpassungen weitgehend auf die Änderungen der Schadenkosten reagieren.

Eine Analyse dynamischer Wettbewerbsprozesse auf Versicherungsmärkten ist bisher in der Literatur kaum zu finden. Die Versicherungsmarkttheorie basiert weitgehend auf der neoklassischen Gleichgewichtstheorie. Einzelaspekte wie die theoretische Begründung staatlicher Regulierung greifen auf den Tauschprozeß von *Edgeworth* zurück um einen Existenzbeweis für ein Marktgleichgewicht zu führen[35]. *Strassl* definiert ein kompetitives Gleichgewicht auf Versicherungsmärkten als einen Zustand, in dem kein Versicherer mehr mit Neuabschlüssen Gewinne machen kann[36]. In diesem Theorieansatz ist das Marktgleichgewicht theorieimmanent. Durch den Wettbewerbsprozeß bewegt sich der Marktpreis immer zum Gleichgewicht hin.

In einer deskriptiven Betrachtung der deutschen Versicherungsmärkte lassen sich die Prämissen der vollkommenen Konkurrenz, die die Existenzgrundlage für das Gleichgewicht der allgemeine Gleichgewichtstheorie bilden, kaum wiederfinden. Auf den Märkten herrscht "– von den wenigen Fällen des institutionellen Monopols (Bannrechte) abgesehen – monopolitische Konkurrenz, ein unvollkommener Wettbewerb, denn jede Gesellschaft bietet ihren Versicherungsschutz als einzigartig an"[37], ihre Form hält *Mahr* dagegen für

[32] vgl. Schulenburg (1989), S. 321
[33] vgl. Finsinger (1983), S. 59
[34] vgl. Farny (1979), S. 66
[35] vgl. Strassl (1980), S. 25
[36] vgl. Strassl (1980), S. 8
[37] Mahr (1971), S. 370

oligopolistisch[38].

Mahr identifiziert den Beziehungswettbewerb als "das eigentliche Kampffeld der Versicherungsfirmen"[39]. Die Pfege der persönlichen Kontakte zwischen Versicherern und Kunden steht dabei im Mittelpunkt. Dieses strategische Instrument läßt sich zur Beeinflussung der Präferenzen durch die Unternehmen abstrahieren.

Unabhängig, ob es sich um Privatkunden- oder Firmengeschäft handelt, sind als Hauptwettbewerbsarten Qualitäts-, Preis- und Präferenzwettbewerb in der Versicherungstheorie identfiziert worden[40].

Für die Modellierung von Versicherungsmarktmodellen bleibt somit festzuhalten:

1. Mögliche Marktformen der Versicherungsmärkte
   - 1.1. Monopolistischer Konkurrenz
   - 1.2. Oligopol
2. Mögliche Hauptwettbewerbsarten
   - 2.1. Präferenzwettbewerb
   - 2.2. Qualitätswettbewerb
   - 2.3. Preiswettbewerb

In dem hier zu entwickelnden Wettbewerbsmodell eines Versicherungsmarktes kämpfen Versicherer in unterschiedlichen Marktformen um Markterfolge. Ziel des Modells ist es nicht, Existenz- und Eindeutigkeitsaussagen über Gleichgewichte auf dynamischen Märkten mit exogenen Schocks zu machen. Vielmehr sollen ungleichgewichtige oligo- und monopolistische Marktprozesse bei unterschiedlichem Anbieter- und Nachfragerverhalten analysiert werden[41], um aus diesen Ergebnissen Rückschlüsse auf die Wirkungen der Schocks ziehen zu können.

2.5 Ansätze zur Schockanalyse

In der Ökonomie hat es bereits frühzeitig Überlegungen zur Untersuchung von plötzlichen Änderungen von Marktvariablen gegeben. Im folgenden Abschnitt werden drei Ansätze zur Schockanalyse auf ihre Eignung für den eigenen Modellansatz überprüft:

1. Der Ansatz der komparativen Statik

[38] vgl. Mahr (1971), S. 365
[39] Mahr (1971), S. 375
[40] vgl. Meyer-Kahlen (1985), S. 243
[41] vgl. Fischer (1988), S. 45

2. Der Ansatz der Industrieökonomie
3. Der Simulationsansatz

Dabei soll die Eignung dieser Ansätze anhand von drei Kriterien überprüft werden:

1. Möglichkeiten der Marktprozeßmodellierung
2. Prognosefähigkeiten
3. Möglichkeiten der Szenariobildung

Den einfachsten Ansatz zur Schockanalyse findet man in der Partialanalyse eines Marktes mit einem Gut. Ausgehend von einem Marktgleichgewicht wird eine Preisänderung oder die Änderung von Angebot und Nachfrage durch Verschieben der Angebots- und Nachfragekurven dargestellt. Dieser Schock löst sowohl im klassischen als auch im neoklassischen Marktmodell Anpassungsreaktionen aus, die durch vollständig flexible Preise und Mengen und vollkommene Information der Marktteilnehmer begründet werden[42]. Versucht man nun den Ansatz der komparativen Statik auf den Modellrahmen dieser Arbeit anzuwenden, dann stößt man sehr schnell an seine Grenzen. Die Marktprozeßhypothesen der komparativen Statik postulieren aufgrund der Annahme vollkommener Informationen ein völlig homogenes Verhalten aller Marktteilnehmer. Die daraus resultierenden Marktprozesse verlaufen vollkommen gleichförmig. Die Prognosefähigkeit dieses Ansatzes für einen Markt hängt davon ab, inwieweit sich die Modellprämissen wie Preis-, Mengenflexibilität und vollkommene Information aller Markteilnehmer empirisch verifizieren lassen. Die Möglichkeiten, verschiedene Marktszenarien miteinander zu vergleichen, sind begrenzt. Außer einer kurzfristigen Senkung oder Erhöhung von Angebot oder Nachfrage lassen sich kaum andere Szenarien denken. Aufgrund der theorieimmanenten Anpassungsprozesse sind langfristige Ungleichgewichtssituationen nicht möglich. Verglichen werden immer nur zwei gewählte Zeitpunkte: Der Zeitpunkt des Schocks und der Zeitpunkt des neuen Gleichgewichts. Durch das bewußte Ausklammern von Aussagen zu Marktprozessen ist die Analyse von Schocks mittels komparativ-statischer Partialanalysen ein weit verbreiteter Ansatz, dessen Lehrcharakter aber eine vertiefte Anwendung auf reale Märkte verbietet.

In der empirisch ausgerichteten ökomischen Forschung stehen zwei grundlegende Analysekonzepte nebeneinander. Das traditionelle Konzept stellt eine Theorie in den Mittelpunkt und versucht sie an der Wirklichkeit zu verifizieren. In der Erforschung von Marktprozessen, die mit der Wettbewerbsdynamik eng verknüpft sind, hat sich seit den 40er Jahren ein weiterer Ansatz etabliert: Das Industrial Organization-Konzept. Als Pionier in diesem Gebiet ist *J.S. Bain*[43] anzusehen.

[42] vgl. Flassbeck, Horn, Zwiener (1992), S. 41
[43] vgl. Böbel (1984), S. 27

Der Industrial Organization-Ansatz ist ein Konzept zur empirischen Untersuchung von Märkten, der ohne Anspruch auf Verifizierung einer vorher formulierten Theorie abläuft. Industrial Organization[44] wird heute als empirisch ausgerichtete Wettbewerbstheorie verstanden[45]. Ziel der Industrieökonomie ist die kausale Verknüpfung von:

1. Marktstruktur
2. Marktverhalten
3. Marktergebnis

Forschungen im Bereich der Industrieökonomie befassen sich mit Fragen über:

1. Konzentration-Gewinn Beziehungen
2. Markteintritts-Gewinn Beziehungen
3. Strategie-Marktergebnis Beziehungen
4. Marktmacht-Wohlfahrtsverlust Beziehungen
5. Gewinn-Wettbewerbsparameter Beziehungen

Auch in der Versicherungswissenschaft ist dieses Konzept der theoriefreien empirischen Wirtschaftsforschung angewendet worden. Ein Themenschwerpunkt lag dabei in der Untersuchung der Beziehung von Konzentration und Unternehmensgröße auf Versicherungsmärkten[46]. Durch die Deregulierungsdiskussion in Deutschland wurde ein weiterer Schwerpunkt bei der Untersuchung von Skalen- und Verbundvorteilen gesetzt. *Holzheu* analysierte die mit Größenvorteilen verbundenen Effekte zwischen Erst- und Rückversicherungsmarkt[47]. *Houston* und *Simon* überprüften die Existenz von Skalenvorteilen anhand von 237 kalifornischen Lebensversicherern und wiesen sie bei kleineren Versicherern nach[48]. *Dirrheimer* und *Junghanns* wendeten industrieökonomische Methoden zur Wettbewerbsanalyse der deutschen Lebens- und Kraftfahrzeugversicherung an[49]. *Kaluza* untersuchte die Beziehungen zwischen Betriebsgröße und Stückkosten in der privaten Krankenversicherung in Deutschland und kam zu dem Ergebnis, daß aufgrund der starken Kapitalanlageregulierung Kapitalmachtpositionen nicht ausgenutzt werden können[50]. *Szopo* konnte für die österreichische Lebensversicherungsgesellschaften einen L-förmigen Verlauf der Kostenfunktion im Bezug auf die Unternehmensgröße ableiten[51].

[44] Im deutschsprachigen Raum wird sie oft auch als Industrieökonomie bezeichnet
[45] vgl. Böbel (1984), S. 12
[46] vgl. Hax (1972), S. 261
[47] vgl. Holzheu (1992), S. 112
[48] vgl. Houston, Simon (1970), S. 864
[49] vgl. Dirrheimer, Junghanns (1989), S. 84
[50] vgl. Kaluza (1990), S. 269
[51] vgl. Szopo (1987), S. 90

Die Basishypothese der Industrieökonomie besagt, daß das Marktergebnis durch die Marktstruktur und das Marktverhalten erklärt wird. Die einzelnen Untersuchungen betrachten jeweils Teilaspekte dieses Erklärungsmusters. Von zentraler Bedeutung im Industrieökonomie-Ansatz sind die Rückkopplungen zwischen Marktstruktur, Marktverhalten und Marktergebnis. Diese Rückkopplungen steuern die Hypothesenbildung und den Modellanpassungsprozeß in der Industrieökonomie.

Die Grundlage aller industrieökonomischen Analysen ist eine umfassende Datenbasis. In den empirischen Untersuchungen wird eine Marktstruktur als gegeben betrachtet und stellt die Ausgangsbasis für die Beurteilung von Wirkungen des Marktverhaltens auf das Marktergebnis dar[52]. Für eine industrieökonomische Untersuchung der Auswirkungen der deutschen Wiedervereinigung auf die Versicherungsmärkte müßte eine Informationsbasis mit allen relevanten Marktdaten für einen größeren Zeitraum vor und nach der Wiedervereinigung beschafft werden. Diese Daten könnten dann entweder getrennt untersucht oder als Gesamtstichprobe benutzt werden.

Das Hauptinstrument der Industrieökonomie ist die Ökonometrie. Die Qualität ihrer Ergebnisse steht und fällt mit der Qualität der im Rahmen der Analysen durchgeführten Schätzungen. Eine plötzliche Änderung der exogenen Variablen, der durch das Modell nicht erklärt werden kann, wird in der Ökonometrie als Strukturbruch bezeichnet[53]. Die Wiedervereinigung als Strukturbruch beeinträchtigt die Möglichkeiten einer industrieökonomischen Untersuchung erheblich. Zwei mögliche Ansätze zur Schockanalyse der deutschen Wiedervereinigung mittels industrieökonomischer Methoden bieten sich dennoch an:

1. Untersuchung der Strategie-Marktergebnis Beziehungen mittels zweier Schätzungen, vor und nach der Wiedervereinigung

2. Untersuchung der Strategie-Marktergebnis Beziehungen mittels einer Gesamt–schätzung

In beiden Fällen ist die Prognosefähigkeit dieses Ansatzes durch die statistischen Auswirkungen des Strukturbruchs stark beeinträchtigt, weil die Zeitreihen nach der Wiedervereinigung noch zu kurz sind und ein Strukturbruch in den Gesamtreihen nicht auf alle Variablen zum gleichen Zeitpunkt wirken muß. Wählt man den ersten Ansatz, dann lassen sich, ähnlich wie beim Ansatz der komparativen Statik, die Ergebnisse vergleichen. Über die Marktprozesse, die die Wiedervereinigung ausgelöst hat, sind aber kaum Aussagen möglich. Würde der zweite Ansatz gewählt, dann könnte zwar der Schock endogenisiert werden, ein Vergleich der beiden Szenarien ist dann aber kaum möglich.

Der Hauptvorteil einer Simulation ist, daß relevante historische Ergebnisse, die in "glatten" ökonomischen Standardmodellen nicht berücksichtigt werden können, explizit in das Modell integriert werden können. Fehlende Daten oder Parameterwerte können durch

[52] vgl. Böbel (1984), S. 9
[53] vgl. Bonhoeffer, F.O., Goldrian, G. (1975), S. 86

plausible Annahmen angenährt werden. Durch die Wahl einer zeitdiskreten Simulation als Modellbasis können sehr genaue Untersuchungen von Marktprozessen vorgenommen werden.

Der größte Vorteil des Simulationsansatzes bei der Schockanalyse sind die umfangreichen Möglichkeiten der Szenariobildung. Durch Sensitivitätsanalysen kann festgestellt werden, welche Marktparameter besonders durch den Schock beeinflußt werden. Die Simulationsparameter können als Vorinformationen für die Hypothesenbildung herangezogen werden. Durch die Simulationsparameter wird eine Marktstruktur definiert und das Marktverhalten der Anbieter und Nachfrager festgelegt. Daduch wird ein künstlicher Datenkranz geschaffen, der auch mit industrieökonomischen Methoden untersucht werden kann. Die Parametrisierung der Simulation und der simulierten Schocks ist dann nur als eine Vorstufe dieser "simulierten" industrieökonomischen Untersuchung anzusehen.

Stellt man nun die Vor- und Nachteile der drei Ansätze zur Schockanalyse gegenüber, dann sind die besseren Möglichkeiten der Marktprozeßmodellierung und die bessere Szenariobildung des Simulationsansatzes ausschlaggebend für seine Wahl als Untersuchungsinstrument im Rahmen dieser Arbeit.

2.6 Elemente der Computersimulation

2.6.1 Der Simulationsbegriff

Der Simulationbegriff ist in der Literatur nicht einheitlich definiert. *Grams* beschreibt sie als "Durchführung von Experimenten mit Modellen dynamischer Systeme"[54]. *Mresse* versteht unter Simulation "die Untersuchung der Verhaltensweise eines Systems mit einem *Ersatzsystem*, dem Modell"[55]. Er sieht drei Gründe für die Erstellung einer Simulation:

1. Das System, was untersucht werden soll, existiert noch nicht
2. Ein Experiment mit dem realen System ist nicht möglich
3. Ein Experiment mit dem realen System ist zwar möglich, die Aufwendungen dafür aber zu hoch[56]

Ludewig faßt unter Simulation "die Erklärung und Prognose realer Systeme auf der Grundlage dynamischer Modelle"[57]. Damit rückt, neben dem Experiment, die Erklärung und die Prognose in den Mittelpunkt der Begriffsabgrenzung. *Zwicker* differenziert drei Auslegungen des Simulationsbegriffs[58]:

[54] Grams (1992), S. 25
[55] Mresse (1984), S. 25
[56] vgl. Mresse (1984), S. 25
[57] Ludewig (1975), S. 2
[58] vgl. Zwicker (1981), S. 102

1. Die zielgerichtete Variation eines dynamischen Modells
2. Die Bestimmung der Zeitpfade einer oder mehrerer endogener Variablen eines parametrisch-singulären Modells
3. Die Bestimmung einer generellen Inplikation durch die Bestimmung einer endlichen Anzahl von singulären Implikationen

Eine andere Art, den Begriff der Simulation zu definieren, erfolgt über die Festlegung ihrer Ziele. Eine sehr allgemeine Abgrenzung sieht "Aussagen über das Verhalten eines Ausschnitts aus der realen Welt zu erhalten"[59] als das Hauptziel der Simulation an.

Möller definiert Simulation "als die Reproduktion des statischen und/oder dynamischen Verhaltens eines realen Systems, basierend auf einem materiellen oder immateriellen Abbild der Realität, dem Modell, welches diejenigen Aspekte des realen Systems beschreibt, die für den angestrebten Erkenntnisgewinn von Bedeutung sind, um aus den Simulationsergebnissen auf die Eigenschaften des realen Systems rückschließen zu können"[60]. Während andere Autoren den experimentellen Aspekt betonen, sieht *Möller* sie nicht als experimentelle Methode an, die den Bedingungen eines naturwissenschaftlich begründeten Experiments genügen. Er sieht zwei Gründe für ein Experiment:

1. Den zu untersuchenden Gegenstand selbst
2. Ein unabhängiges Meßverfahren

Ein mathematisches Modell der Realität ist aber immer nur eine Annährung an die Realität[61]. Da eine vollkommene Simulation der Realität nicht möglich ist, wären die Meßergebnisse immer auch von der Konstruktion des simulierten Modells abhängig. Den Vergleich der simulierten Ergebnisse mit realen Daten betrifft diese Kritik aber nicht, da dieser Vergleich der Modellverifikation dient.

Nach Meinung von *Pidd* basiert die Computersimulation auf der Idee des Experiments[62]. Darin ist auch die Motivation für ökonomische Simulationen zu sehen. Ökonomen verfügen nur sehr bedingt über laborähnliche Experimentiermöglichkeiten. Um eine neue wirtschaftspolitische Maßnahme oder eine einzelwirtschaftliche Unternehmenstrategie bewerten zu können, muß man sie entweder in einem Teilgebiet (Testmarkt) tatsächlich durchführen oder ist auf theoretische Wirkungsmodelle angewiesen. Beide Methoden haben Nachteile. Die Verifizierung theoretischer Wirkungsmodelle kann nur mit empirischen Daten erfolgen und Testfelder oder -märkte können die Maßnahmenwirkungen verzerren.

Computersimulationen können eine Ergänzung oder ein Ersatz für fehlende Experimentiermöglichkeiten sein. Die hier diskutierten Abgrenzungen des Simulationsbegriffs haben Gemeinsamkeiten, die einer allgemeinen Definition dienen können:

[59] Siegert (1991), S. 9
[60] Möller (1992), S. 119
[61] vgl. Möller (1992), S. 119
[62] vgl. Pidd (1989), S. 2

1. Ein Modell muß vorhanden sein
2. Die Simulation muß dynamisch sein
3. Experimente mit der Simulation müssen möglich sein

Von tragender Bedeutung für die Erstellung einer Simulation ist die Existenz eines Modells. Dieses Modell muß nicht zwingend analytisch formuliert sein. Eine Mischung aus Formeln und verbaler Beschreibung kann bereits ausreichen. Die Anforderung der Dynamik ist von ökonomischen Modellen meist erfüllt. Die Experimentiermöglichkeiten einer Simulation hängen in hohem Maße von der Umsetzung des Modells in ein Computerprogramm ab. Experimente mit Computersimulationen bedeuten immer Variation der Simulationsparameter und Vergleich der Simulationsergebnisse.

2.6.2 Der Ablauf von Simulationstudien

Die Bestimmung des methodischen Ablaufs einer Simulation ist entscheidend für ihren Erfolg. Der Ablauf ist durch drei Hauptschritte festgelegt:

1. Modellierung
2. Programmierung
3. Experiment[63]

Meissner formuliert einen Ablauf in sieben Schritten:

1. Problemformulierung
2. Datensammlung
3. Problemformalisierung
4. Modellparameter bestimmen
5. Programmformulierung
6. Auslegen des Experiments
7. Ergebnisauswertung[64]

[63] vgl. Pidd (1989), S. 2
[64] vgl. Meissner (1970), S. 387

In diesem Ablaufschema werden die Punkte 1 und 3 des allgemeineren Schemas ausdifferenziert. Auffällig ist die Linearität der Schemata. Dem natürlichen Ablauf der Erstellung einer Simulation kommt aber die Methode der schrittweisen Verfeinerung am nächsten. Durch diesen Prozeß wird das Erstellungsschema zu einem Rückkopplungsvorgang:"

1. Abstraktion und Modellierung
2. Untersuchung
3. Interpretation
4. Rückschlüsse und Änderungen"[65]

Die in dieser Arbeit verwendete Vorgehensweise orientiert sich am letzten Ablaufschema. Ausgehend von einem Grundmodell und einer Programmbasis verlaufen die Modellentwicklung und die Programmentwicklung in einem zyklischen Prozeß. Die oben aufgeführten Schemata dienen einer ersten Orientierung bei der Projektierung einer Simulation und stellen keine Gesetzmäßigkeit dar.

Nicht nur der Rückkopplungsprozeß zwischen Modell, Programm und Ergebnissen beeinflußt den Ablauf von Simulationsstudien, sondern auch die Abhängigkeiten zwischen den einzelnen Phasen. Die getrennte Betrachtung der Modellierung einer Simulation als abgeschlossene Phase ist in realen Projekten kaum zu finden. Bei der Erstellung von IMS ist die verwendete Programmiersprache ein beeinflussender Faktor für die Modellformulierung. Wie bei der Formulierung analytischer Modelle dem Modellbildner die Analytik Grenzen setzt, so gibt auch die in einer Simulation verwendete Programmiersprache Grenzen vor. Bei analytischen Modellen müssen Prämissen aufgestellt werden, die die analytische Lösbarkeit der Modelle sicherstellen. Bei Modellen, die Grundlage für Computersimulationen sind, müssen ebenfalls Prämissen zur Komplexitätsbegrenzung integriert werden.

Grundlegend für das IMS-Modell ist die Abstraktion von der Komplexität realer Versicherungsmärkte, durch die in einer ersten Modellierungsphase Modellstrukturen und Formeln geschaffen werden. In der zweiten Phase werden diese Strukturen und Formeln in Datenstrukturen und Algorithmen umgesetzt. Die in der zweiten Phase entwickelten Programmbausteine müssen in einem Entwickungsprozeß an die dritte Modellphase angepaßt werden. Die dritte Phase des IMS-Modells umfaßt die Formulierung und Umsetzung der Hypothesen an das Modell in Eingaben, die von einem Simulationsprogramm verstanden werden können. Die Form und Vielfalt der Hypothesen wird durch die Analyseanforderungen einerseits und die möglichen Formen der Datenausgabe andererseits begrenzt.

Das Grundmodell von IMS sieht einen starren Ablauf von strategischen Entscheidungen der einzelnen Subjektklassen vor. Die Abstraktionsleistung der Modellierungsphase besteht darin, bestehende Theorien und Abläufe auf Versicherungsmärkten an dieses Ablaufschema anzupassen. Es besteht eine starke Abhängigkeit von den exo- und endogenen

[65] Mresse (1984), S. 25

Variablen des Modells und den Datenstrukturen des Programms. Oftmals werden aber im Programm zusätzliche Variablen benötigt, um die aus der Theorie abgeleiteten mikroökonomischen Abläufe umzusetzen. In diesem Fall ist im Abstraktionsvorgang der Modellbildung die bereits vorhandene Datenstruktur des Programm zu berücksichtigen, da sonst der Speicher- und Verwaltungsaufwand zu groß wird.

Zwischen der Programmierphase und der Experimentierphase bestehen ebenfalls Abhängigkeiten. Experimente mit Simulationen bedeuten im weitesten Sinne: Dateneingabe und Datenausgabe. Der Anwender gibt der Simulation eine bestimmte Datenmenge ein und wertet die Programmergebnisse aus. Dadurch muß schon in der Programmierphase die Form der Ausgabe, die Parameter, die variiert werden sollen und die Art der zu verarbeitenden Hypothesen berücksichtigt werden.

Interdependenzen ergeben sich auch zu der Simulationsart[66]. Computersimulationen makroökonomischer Modelle können sich weitgehend an den von der Theorie vorgegebenen Formeln orientieren. Mikroökonomische Simulationen müssen die Strategien und Aktionen der Marktteilnehmer beeinhalten. Diese individualistischen Verhaltensabläufe sind in der mikroökonomischen Theorie nur unzureichend beschrieben, so daß eine einfache Übernahme der Formeln nicht möglich ist. Es besteht ein Zusammenhang zwischen den Aggregationsgrad der simulierten Ökonomie und dem Modellierungs- und Programmierungsaufwand. Je höher aggregiert die simulierten Abläufe sind, um so einfacher zu erstellen sind daraus abgeleitete Simulationen. Ein keynesianisches Makromodell liegt in geschlossener analytischer Form vor. Eine Simulation muß nur die Parameter und Startwerte einlesen, die dynamischen Gleichungen über eine Anzahl von Perioden fortschreiben und die Ergebnisse darstellen[67]. Bei mikroökonomischen Simulationen stehen disaggregierte Größen im Mittelpunkt. Formeln und Modelle an denen man "entlangprogrammieren" kann, gibt es für Versicherungsmärkte nicht.

Die Bildung von Hypothesen ist ein Prozeß, der außerhalb des Zyklus der Simulationsstudien liegt. Hypothesen müssen entweder an das Modell oder an das Programm gestellt werden. Die Simulation stellt keine Hypothesen auf, sondern Hypothesen müssen mit der Simulation überprüft werden[68]. Die Modellierungsphase muß die Arten möglicher Hypothesen berücksichtigen und in der Programmierphase sind die technischen Voraussetzungen für ihre Prüfung zu schaffen.

Durch die Zyklen zwischen den einzelnen Phasen der Simulationsstudie wird ein Abbruchkriterium für den Erstellungsprozeß benötigt. Ein eindimensionales quantitatives Kriterium (z.B. Anzahl der Modellvariationen oder Programmumfang) wird dem Problem kaum gerecht. Geeigneter erscheint ein mehrdimensionales Abbruchkriterium, welches die Modellkomplexität und die definierten Ziele in Beziehung zum Entwicklungsstand setzt. Dadurch wird einerseits die gewünschte Zielerreichung gesichert, aber auch der Zeitaufwand für Wartung, Pflege und Erweiterung der Simulation unter Kontrolle gehalten.

[66] vgl. Abschnitt 2.6.3
[67] vgl. Külp (1989), S. 144
[68] vgl. Meissner (1970), S. 395

2.6.3 Simulationsarten

IMS ist eine dynamische Simulation. Statische Simulationen sind in der Analyse von Marktprozessen kaum sinnvoll. Weiterhin ist diese Simulation zeitdiskret. Im Gegensatz zu kontinuierlichen Simulationen, bei der die Simulationsuhr beliebig kleine Schritte machen kann, ist der Abstand zwischen zwei Zeitpunkten in IMS immer gleich. Simulationen mit dieser Eigenschaft werden als äquidistant bezeichnet[69]. IMS ist somit eine dynamische, zeitdiskrete, äquidistante Simulation.

Eine weitere Klassifizierung unterscheidet zwischen deterministischen und stochastischen Simulationen. Da IMS einen Versicherungsmarkt simuliert, ist es zwingend, daß dabei Pseudo-Zufallszahlen verwendet werden. Damit Schadenverteilungen generiert werden können, müssen stochastische Variablen erzeugt werden. IMS zählt zu den stochastischen Simulationen.

In der Klasse der Wirtschaftssimulationen waren makroökonomische Simulationen die ersten computergestützen Modelle. Ausgehend von ökonometrischen Schätzmodellen wurden erste Prognosen mittels programmierter Makromodelle ermittelt. Makroökonomische Modelle betrachten keine einzelnen Wirtschaftseinheiten, sondern fassen alle gleichartigen Einheiten zu einer Gruppe zusammen[70]. Mikroanalytische Modelle gehen genau den umgekehrten Weg. Sie simulieren das Verhalten einer Anzahl individueller Einheiten über die Zeit[71]. Jede dieser Einheiten besitzt einen Vektor von Merkmalen. Verhaltensänderungen der Einheiten werden durch Änderungen der Merkmalsausprägungen modelliert[72]. Mikroanalytische Modelle beruhen auf drei Grundannahmen[73]:

1. Aggregation führt zu Fehlern, da die menschliche Handlungsweise durch persönliche Bestimmungsfaktoren erklärt wird. Diese Faktoren werden bei der Aggregation nicht berücksichtigt.

2. Auf geschlossene Lösungen wird von Beginn an verzichtet. Die Lösungsmethode dieser Modelle ist immer die Simulation.

3. Komplexe, institutionelle Regeln werden in Algorithmen umgesetzt.

An diesen Grundannahmen wird bereits der fundamentale Unterschied zwischen Makrosimulationen und mikroanalytischen Simulationen deutlich. Erstere setzen auf bestehenden Theorien und analytischen Lösungen auf, während letztere explizit auf die Formulierung geschlossener Lösungen verzichten und sofort bei der Programmierung der Simulation ansetzen. Dieses Konzept hat auch Auswirkungen auf den im letzten Abschnitt beschriebenen Entwicklungszyklus von Simulationsstudien[74].

[69] vgl. Zwicker (1981), S. 28
[70] vgl. Klösgen, Quinke (1985), S. 328
[71] vgl. Vetterle (1986), S. 1
[72] vgl. Vetterle (1986), S. 14
[73] vgl. Krupp, Wagner (1982), S. 6–8
[74] vgl. 2.6.2

Während Makrosimulationsstudien Rückkopplungen zwischen der Phase der Modellierung und der Phase der Programmierung besitzen, sind in mikroanalytischen Simulationsstudien diese Rückkopplungen nicht oder nur stark vermindert vorhanden, da die eigentliche Modellbildung bei der Programmierung erfolgt.

Nach dieser Klassifizierung ist IMS eine mikroanalytische Simulation. Sie simuliert individuelle Markteinheiten: Versicherer und Versicherungsnehmer. Eine analytisch feststehende Theorie wird nicht einfach in ein Programm umgesetzt, sondern auch in IMS wird die Lösung des Modells über eine Simulation erreicht.

In ihrer ursprünglichen Definition hat die mikroanalytische Simulation noch eine weitere Eigenschaft: Die einfließenden Verhaltensinformationen über die simulierten Entscheidungseinheiten werden aus einer Stichprobe realer Individuen abgeleitet[75]. Das ist bei IMS nicht der Fall. Im Modell von IMS werden keine realen Daten verarbeitet und fortgeschrieben. Die simulierten Einheiten werden aus einer abstrakteren Perspektive betrachtet. Die simulierten Merkmale gleichen teilweise den realen Merkmalen von Versicherungsmarktteilnehmern, können aber auch Approximationen oder künstliche Merkmale darstellen. Die Daten haben somit keinen direkten Bezug zu realen Unternehmens-, Konsumenten- oder Marktdaten. Die erzeugten Daten sind künstlich.

Hier wirken sich die unterschiedlichen Ziele von mikroanalytischer Simulation und dem IMS-Modell aus. Das Ziel mikroanalytischer Simulationen ist die genauere Prognose und Politikfolgenabschätzung. Das Ziel von IMS ist die Untersuchung der dynamischen Eigenschaften eines Marktes unter Risiko bei plötzlichen exogenen Änderungen der Marktparameter.

Die Betonung der Dynamik als Untersuchungsziel verdeutlicht die Nähe des IMS-Ansatzes zu System Dynamics-Modellen. System Dynamics-Modelle wurden besonders in den 70er Jahren zu langfristigen Prognosen hochaggregierter dynamischer Modelle verwendet. Die Kritik an der Prognosegenauigkeit dieser Modelle geht aber eigentlich am Kern ihrer Auslegung vorbei. Das System Dynamics-Konzept ist nur eine Interpretationsweise dynamischer Systeme. Sie geht davon aus, das sich dynamische soziale und technische Systeme immer durch Bestandsgrößen (Level) und deren verzögerte Zu- und Abgänge (Flows) modellieren lassen[76]. Besondere Prognosefähigkeiten sind dem Konzept nicht vorgesehen. Seine Stärke liegt in der Untersuchung der Dynamik von komplexen Systemen. System Dynamics-Modelle lassen sich immer als Differenzengleichungen formulieren. Ihre Umsetzung in Programme ist deshalb einfach. Stochastische Modellelemente sind in der Grundkonzeption von System Dynamics nicht vorgesehen.

Betrachtet man nun die Konzeption des IMS-Modells, dann fallen Parallelen zum System Dynamics-Ansatz auf. Die Datenstrukturen und die zentralen Variablen der Simulation sind Bestandsgrößen: Das Vermögen bei den Versicherungsnehmern und die Reserven bei den Versicherern. Diese Bestandsgrößen ändern sich verzögert um Zu- und Abgänge:

[75] vgl. Krupp (1986), S. 32
[76] vgl. Zwicker (1981), S. 399

Prämien und Schäden.

IMS verbindet die Hauptvorteile von System Dynamics und mikroanalytischen Modellen (Dynamikanalyse und Disaggregation) und umgeht das Hauptproblem der Mikrosimulation, die Datenbeschaffung, indem künstliche anstatt realer Daten verwendet werden. Da die Zielsetzung von IMS die Analyse der Dynamik eines Versicherungsmarktes bei exogenen Schocks ist, kann die Verifizierung des Modells an realen Daten in den Hintergrund treten.

2.6.4 Simulationssprachen

Die zweite Phase des Modellierungszyklus ist die Programmierphase. In ihr wird das Simulationsmodell in eine für den Computer verständliche Form gebracht. Der Programmierer muß das Modell in die Syntax der verwendeten Computersprache übersetzen. Diese Computersprache muß dann in einem zweiten Übersetzungsschritt durch den Rechner in ein ausführbares Programm umgesetzt werden.

Im Laufe der letzten 30 Jahre sind eine Reihe von Computersprachen entwickelt worden. Einige wurden speziell für die Simulationsprogammierung ausgelegt. Generell unterscheidet man höhere von niederen Programmiersprachen. Höhere Sprachen wie C, PASCAL, FORTRAN, BASIC können unmittelbar zur Erstellung von Simulationen genutzt werden. Die Vorteile der direkten Programmierung liegen in:

1. der problemgerechten Wahl der mathematischen Algorithmen,
2. der direkten Kontrolle der Rechengenauigkeit,
3. der Kontrolle des Rechenaufwands[77].

Ein Nachteil liegt in dem erhöhten Aufwand zur Programmierung der Programmein- und -ausgabe.

Die erste Simulationssprache war GPSS (General Purpose Simulation System). GPSS wurde in den frühen 60er Jahren von IBM entwickelt und hat eine weite Verbreitung gefunden[78]. GPSS ist eine blockorientierte Hochsprache für diskrete, dynamische Simulationen. Der Anwender definiert die einzelnen simulierten Einheiten in Blöcken und ordnet ihnen Eigenschaften zu. GPSS führt alle ihm bekannten Blöcke mit allen Transaktionen zwischen ihnen nacheinander aus[79]. GPSS ist ein weitgehend geschlossenes System. Programmerstellung und -ausführung sind zu einer Arbeitsumgebung zusammengefaßt. Diese Abgrenzung gegen bestehende Sprachen schränkt die Flexibilität von GPSS ein[80]. Die Akzeptanz einer neuen Simulationssprache steigt erheblich, wenn bestehende Kenntnisse und

[77] vgl. Grams (1992), S. 17
[78] vgl. Neelamkavil (1987), S. 161
[79] vgl. Pidd (1988), S. 159
[80] vgl. Law, Kelton (1982), S. 130

Programme weiter angewendet werden können. SIMSCRIPT und GASP sind als Erweiterungen allgemeiner Hochsprachen konzipiert. Sie ergänzen die Syntax dieser Sprachen um die Befehle, die die Erstellung diskreter Simulationen stark erleichtern, schränken den Programmierer aber in der eigentlichen Erstellung der Simulation kaum ein. GASP ist eine Spracherweiterung zu FORTRAN[81], die in Form einer Programmbibliothek vorliegt, SIMSCRIPT ist ein Präprozessor für FORTRAN-Programme[82]. SIMULA stellt eine Spracherweiterung der Sprache ALGOL 60 dar, einem Vorläufer von PASCAL. Neben einer Erweiterung von ALGOL 60 um Sprachelemente zur Simulation diskreter dynamischer Modelle stehen in SIMULA Koroutinen zur Verfügung, die parallele Simulationsmodelle ermöglichen[83]. SIMULA stellt eine Verwaltung dynamischer Listen und Klassen zur Verfügung[84]. Datenobjekte wie Listen oder Klassen werden dann als dynamisch bezeichnet, wenn die Anzahl ihrer Elemente oder Mitglieder zum Zeitpunkt ihrer Initialisierung nicht bekannt sein muß, sondern erst zur Laufzeit des Programm bekannt wird. Durch dynamische Datenverwaltung und parallele Prozesse ist SIMULA eine sehr flexible Simulationssprache.

Für große mikroanalytische Modelle ist der Zugriff auf große Datenmengen von zentraler Bedeutung. Bei Systemen für diese Simulationen handelt es sich weniger um reine Programmiersprachen, sondern mehr um ökonometrische Endanwendersysteme. Beispiele hierfür sind: TROLL, IAS, MBS[85].

Für den System Dynamics-Ansatz[86] wurde eine spezielle Programmiersprache entwickelt: DYNAMO[87]. Durch die zunehmende Verbreitung von Personal Computern mit grafischer Benutzeroberfläche konnte die Simulationsprogrammierung interaktiver werden. Das Programm STELLA[88] läßt den Anwender ohne Progammiererfahrung mausgesteuert eine Simulation erstellen und ausführen. Die Datenformatierung und -ausgabe übernimmt ebenfalls STELLA. Diese Bequemlichkeit geht aber auf Kosten der Flexibilität und der Rechnerleistung. Anwendungen von STELLA gibt es für die Ökonomie, Biologie, Ingenieurswissenschaften und für Warteschlangenprozesse[89].

Die zur Programmierung von IMS verwendete Sprache heißt ESL[90]. ESL ist eine Erweiterung der Hochsprache C um Sprachelemente, die zur Erstellung diskreter, klassenorientierter Simulationen nötig sind[91]. Sie ist als Präprozessor zur Sprache C implementiert und speziell für die Simulation mikroökonomischer Prozesse und stochastischer Märkte

[81] vgl. Law, Kelton (1982), S. 116
[82] vgl. Pidd (1988), S. 154
[83] vgl. Ferstl (1979), S. 144
[84] vgl. Siegert (1991), S. 117
[85] vgl. Klösgen, Quinke (1985), S. 334
[86] vgl. 2.6.2
[87] vgl. Ferstl (1979), S. 140
[88] vgl. Bossel (1992), S. 184
[89] vgl. Lindfield (1991), S. 102
[90] **E**conomic **S**imulation **L**anguage
[91] vgl. Ohlendorf (1987), S. 5

konzipiert worden[92]. Sie ist ähnlich wie SMS[93] (Stochastic Market Simulation) in der Lage, komplexe stochastisch-dynamische Marktumgebungen zu simulieren, vermeidet aber die Nachteile von FORTRAN und ist als C-Präprozessor weitaus flexibler.

Die Vorteile von ESL lassen sich an folgendem Kriterienkatalog für Simulationssprachen prüfen[94]:

1. *Die Sprache muß für die Rechner der Anwender verfügbar sein:* Der Quelltext von ESL ist über den Autor verfügbar und ist durch seine hohe Portabilität auf viele Rechner und Betriebsystemen übertragbar.

2. *Kosten der Installation und Pflege:* Der Quelltext ist kostenlos. Ein C-Compiler muß verfügbar sein. Der kostenlose GNU-C Compiler ist eine preisgünstige Alternative.

3. *Anzahl der Simulationsstudien, die mit der Sprache gemacht werden sollen:* Eine, nämlich IMS.

4. *Typ des simulierten Systems:* Stochastische, diskret-dynamische Simulation eines Versicherungsmarktes mit exogenen Schocks.

5. *Qualität der Dokumentation:* Da der syntaktische Umfang der Sprachergänzung nicht sehr groß und komplex ist (15 Schlüsselworte und einige Systemkonstanten), reicht die dazu vorliegende Dissertation und die Artikel mit Beispielen vollkommen aus. Kenntnisse der Sprache C sind absolut notwendig.

6. *Erlernbarkeit der Sprache:* Wenn Kenntnisse der Sprache C vorhanden sind, können die ESL-Befehle und die Grundstruktur von ESL-Programmen relativ schnell erlernt werden.

7. *Speicheranforderungen für die Sprache:* ESL ist ein C-Präprozessor. Neben dem Präprozessor werden nur zwei zusätzliche Dateien in die späteren Simulationen eingebunden. Der Speicheraufwand ist sehr niedrig.

8. *Recheneffizienz der Sprache:* Da ESL-Programme immer in C-Programme umgewandelt werden, ist die Recheneffizienz nahezu gleichzusetzen mit der Effizienz von C-Programmen. Die Effizienz des C-Übersetzers hat ebenfalls Einfluß auf die Recheneffizienz. Da C als eine der schnellsten Hochsprachen überhaupt gilt, sind ESL-Programme sehr recheneffizient. Compilerbasierte Sprachen wie C, PASCAL oder FORTRAN sind bei interpreterbasierten Sprachen wie BASIC oder LISP in der Recheneffizienz immer überlegen. Nachteilig wirken sich die Übersetzungszeiten nur bei der Programmentwicklung aus.

[92] vgl. Ohlendorf, Stahlecker (1990), S. 187; vgl. Ohlendorf, Stahlecker (1992), S. 105
[93] vgl. Witt, Perske (1982), S. III
[94] vgl. Law, Kelton (1982), S. 133

9. *Effizienz und Flexibilität der Sprache:* Da die gesamte Flexibilität der C-Programmierung in ESL-Programmen zur Verfügung steht, kann die Flexibilität als sehr hoch angesehen werden. Einschränkend auf die Effizienz der Sprache wirkt sich die statische Klassenverwaltung aus, sie kann aber vom Programmierer selbst nachträglich durch eine dynamische Subjektverwaltung ersetzt werden.

Der Vergleich bestehender Simulationssprachen hat die große Vielfalt und das Anwendungsspektrum aufgezeigt. Eine Simulationssprache für alle Simulationen gibt es noch nicht. Die Wahl der Simulationssprache ist dadurch Teil einer Simulationsstudie und kann ihren Erfolg entscheidend beeinflussen. Die Wahl der Sprache ESL für diese Simulation ist durch ihre explizite Ausrichtung zur Simulation stochastischer mikroökonomischer Märkte begründet. Ihre hohe Recheneffizienz und ihre leichte Erlernbarkeit waren weitere Gründe für die Auswahl.

2.7 Simulation in der Versicherungswissenschaft

Zu den Hauptanwendungsgebieten für Simulationen in der Versicherungswissenschaft gehören die Planung und die Ausbildung. Im Planungsbereich können Simulationen das strategische Management unterstützen[95] oder die Versicherungsmathematik ergänzen[96].

Als zu Beginn der 60er Jahre erste Vorschläge zu Unternehmensmodellen von Versicherungen gemacht wurden, stand die Ausbildungsfunktion dieser Simulationen im Vordergrund[97]. Unternehmensspiele[98] hatten nicht die genaue Prognose der Unternehmensentwicklung zum Ziel, sondern die Folgenabschätzung strategischer Entscheidungen. Dem Spieler sollte die Komplexität seines Entscheidungsraums verdeutlicht werden. In diesen Simulationen war der Spieler Leiter einer Versicherung und konnte Strategieparameter des Unternehmens beeinflussen. Die Unternehmensumwelt, wie Konkurrenten oder Nachfrager von Versicherungsschutz, war nicht explizit modelliert. Neben den reinen Simulationen von Versicherungen gibt es auch Optimierungsmodelle. Optimierungsmodelle sind nummerische Ansätze zur Maximierung oder Minimierung einer Unternehmensgröße unter Nebenbedingungen[99].

Simulationen von Versicherungen sind üblicherweise in Teilmodelle untergliedert. *Scharek* unterscheidet für ein Gesamtplanungsmodell eines Lebensversicherungsbetriebs sieben Teilmodelle:[100]"

[95] vgl. Wilde (1983), S. 12
[96] vgl. Pitacco (1986), S. 49
[97] vgl. McGuinness (1960), S. 196
[98] oft auch als *Management Games* bezeichnet
[99] vgl. Gaidamavicius (1985), S. 1250
[100] Schareck (1979), S. 93

1. Absatzplanung
2. Beitragseinnahmenplanung
3. Produktions- und Verwaltungsplanung
4. Vermögenswerte-Planung
5. Reserven-Planung
6. Steuer-Planung
7. Überschuß-Planung"

Um die Komplexität der Erstellung und die Handhabung derartiger Simulationen kontrollieren zu können, sind hohe Anforderungen an die Qualitätssicherung und an die Einarbeitung der Benutzer zu stellen.

Oftmals ist aber die Modellierung größerer operativer Bereiche einer Versicherung für die Anwender in den Vorständen und Stäben nicht wichtig, da hochaggregierte Kennzahlen schneller zu beurteilen sind. Diesem Umstand hat *Wilde* in seinem Unternehmensmodell INSULA Rechnung getragen. Das INSULA-Modell wurde als evolutionäres Modell ausgelegt[101]. Diese unter Programmierern oft verwendete Technik beschränkt den Funktionsumfang am Beginn der Erstellung auf ein Mininum, um möglichst schnell eine lauffähige Version zu bekommen. Die einzelnen vorskizzierten Teilbereiche des Programm werden dann später ausformuliert. Auch in INSULA wird die Umwelt des Unternehmens nicht in Konkurrenten und Nachfrager disaggregiert. Analysen der Unternehmensstrategien im Wettbewerb mit anderen simulierten Versicherern sind nicht möglich.

Zwei frühe Simulationen von Versicherungen waren GIM[102] und SOFI[103]. GIM analysierte die Inflationswirkungen auf einem Kraftfahrzeugversicherungsmarkt[104]. SOFI war zwar flexibler ausgelegt, aber nicht speziell für Versicherungsmärkte konzipiert[105]. Beide Simulationen waren deterministisch.

Das ASIR-Modell[106] ist eine detaillierte Unternehmenssimulation von Versicherungen, die sowohl als Direkt- und als Rückversicherer agieren können. Die Möglichkeiten von ASIR sind weitreichend. Es ist unter anderem in der Lage neun aktive Versicherer oder Rückversicherer zu simulieren, 40 Risikoarten, neun Rückversicherungsvertragsarten und 20 Wechselkurse mit einzubeziehen[107]. ASIR ist in FORTRAN geschrieben. Seine Modellierungsart ist nahe beim System Dynamics-Ansatz anzusiedeln[108]. Wie INSULA ist auch

[101] vgl. Wilde (1983), S. 47
[102] **G**eneva **I**nsurance **M**odell
[103] **S**imulation **of** **F**inancial **I**nstitutions
[104] vgl. Hansen (1982), S. 6
[105] vgl. Galitz, Brown (1981), S. 85
[106] The **A**dvanced **S**imulation Modell of **I**nsurance and **R**e-insurance Operations
[107] vgl. Galitz (1982), S. 283
[108] vgl. Galitz, Brown (1981), S. 89

ASIR eine Versicherungssimulation und keine Versicherungs*markt*simulation. Marktprozesse und Wettbewerb sind entweder gar nicht oder nur indirekt analysierbar.

Das INSGAME von *Hansen* ist zwar als reine Ausbildungssimulation ausgelegt und schon weit über 30 Jahre alt, ist aber in diesen Jahren oft verändert und angepaßt worden. Im INSGAME steuert ein Spieler eine Versicherung. INSGAME kann mehrere Spieler und Versicherungen verwalten[109]. INSGAME simuliert einen Markt mit drei Regionen und drei Sparten[110]. Wie IMS arbeitet auch INSGAME mit künstlichen Daten, die nur wenigen Plausibilitätsanforderungen genügen müssen. Ein Spielleiter kann die Parameter der Marktumgebung für die Spieler verändern. Im INSGAME ist ein Ansatz eines Wettbewerbsmodell enthalten, da sich die Entscheidungen einzelner Spieler zumindest indirekt (z.B. über die Werbeausgaben) untereinander beeinflussen können[111].

Die Fragestellungen, die im Rahmen dieser Arbeit untersucht werden, gehören nicht zu den Hauptanwendungsbereichen von Versicherungssimulationen. Daher ist es auch verständlich, daß eine Simulation, die Analysen dynamischer, stochastischer Versicherungsmärkte mit exogenen Schocks ermöglicht, noch nicht programmiert worden ist. Die bisher verfügbaren Versicherungssimulationen konzentrieren sich entweder auf eine möglichst detailreiche oder aber leicht verständliche Umsetzung der unternehmerischen Entscheidungen eines Einzelversicherers. Wettbewerb und Marktprozesse stehen dabei eher im Hintergrund.

Der Mangel der Verfügbarkeit einer geeigneten Versicherungsmarktsimulation war ausschlaggebend für die Entscheidung eine eigene Simulation zu entwickeln. Eine Versicherungsmarktsimulation, die nicht nur die Anbieter, sondern auch die Nachfrager transparent modelliert und simuliert, stellt eine sinnvolle Ergänzung der bestehenden Programmpalette dar. Das Modell dieser Simulation soll in den folgenden Abschnitten vorgestellt werden.

[109] vgl. Schulenburg, Schattschneider, Knemeyer, Karten, Rautmann, Hipp, Wehrmann (1995), S. 1
[110] vgl. Hansen (1982), S. 8
[111] vgl. Hansen (1979), S. 66

Kapitel 3

Das Modell

3.1 Ziele des Kapitels

In diesem Kapitel wird das Modell zur Untersuchung von exogenen Schocks auf Versicherungsmärkten vorgestellt. Im Rahmen eines drei-phasigen Schemas einer Simulationsstudie[1] entspricht dieses Kapitel der ersten Phase. Die Analyse der Auswirkungen von Schocks auf einen Versicherungsmarkt erfolgt in dem hier formulierten Modellrahmen. Die Eigenschaften des simulierten Marktes werden beschrieben und der Ablauf der Simulation dargestellt. Die Angebots- und Nachfrageseite des simulierten Versicherungsmarktes und die möglichen Schocks werden beschrieben. Der zweite Teil des Kapitels beinhaltet eine Beschreibung der Verhaltenregeln von Angebots- und Nachfrageseite.

3.2 Der Modellrahmen

3.2.1 Das Marktmodell

Eines der Hauptcharakteristika von Marktmodellen ist die Marktform. Das Marktmodell in IMS kann die Marktformen der monopolistischen Konkurrenz und des Oligopols auf der Angebotsseite simulieren. Die Marktform wird dabei sowohl durch die Anzahl der Anbieter als auch durch ihre Verhaltensregeln bestimmt. Neben der Anzahl von Anbietern und Nachfragern ist ihr Verhalten von entscheidender Bedeutung für das Marktergebnis. Die Annahme der traditionellen Theorie, die optimierendes Verhalten auf beiden Marktseiten postuliert, wird im IMS-Modell nicht getroffen.

Für die Anbieterseite von Oligopolmärkten gibt es drei Gleichgewichtsmodelle bei voller Information: Den *Cournot*-Ansatz, den *Bertrand*-Ansatz und den Ansatz von *Edgeworth*[2].

[1] vgl. Kapitel 2.6.2

[2] vgl. Fischer, (1988), S. 8

Im Cournot-Ansatz ist die Ausbringungsmenge die Maximierungsvariable eines Anbieters. Er optimiert sie unter der Annahme, daß sich alle anderen Oligopolisten genauso verhalten. Im Bertrand-Ansatz ist der Preis die Entscheidungsvariable. Ohne Kapazitätsrestriktionen kann nur der preiswerteste Anbieter seine produzierte Menge absetzen. Da die Nachfrager alle Preise kennen und ihre Ausgaben minimieren, wählen sie immer den billigsten Anbieter. *Edgeworth* ergänzt das *Bertrand*-Modell um Kapazitätsrestriktionen. Auch in seinem Ansatz ist der Preis die Entscheidungsvariable der Anbieter.

Diese Gleichgewichte gelten aber nur auf einem Markt bei Sicherheit. Ein Versicherungsmarkt ist ein Markt unter Risiko. Für die Nachfrager sind Schadeneintritt und Schadenhöhe risikobehaftete Größen. Durch das Zahlen einer Prämie an einen Versicherer ist dieser bereit, dieses Risiko zu übernehmen. Das Marktmodell von IMS stellt einen Markt unter Risiko dar.

Unter Risiko wird die Anwendung von Optimierungsregeln durch die Marktteilnehmer sehr komplex. Die analytische Ableitung eines Gleichgewichts in einem dynamischen Marktmodell wird nahezu unmöglich. Das Marktmodell von IMS stellt nicht das Gleichgewicht, sondern den Marktprozeß in den Mittelpunkt der Betrachtung. Es wird davon ausgegangen, daß der Markt dauernd im Ungleichgewicht ist, weil es keine modellinterne Institution oder Regel gibt, die das Erreichen einer gleichgewichtigen Preis-Mengen-Kombination ermöglicht. In traditionellen Marktmodellen kann die Institution des *Auktionators*, der solange Preise ausruft, bis Angebot und Nachfrage im Gleichgewicht sind[3], die Grundlage für die Ableitung eines Gleichgewichts sein. Eine Alternative ist eine Regel, wie das *recontracting*, daß einen Pfad zum Gleichgewicht vorzeichnet. Beim *recontracting* werden Verträge, die nicht zum Gleichgewichtspreis abgeschlossen werden, als ungültig betrachtet. Im IMS-Modell wird keine dieser Regeln und Institutionen postuliert. Es werden auch keine Bedingungen festgelegt, die eine bestimmte Periode der Simulation als Gleichgewicht klassifizieren.

In einem realistischen Ansatz ist davon auszugehen, daß die Marktprozesse ständig wiederholt ablaufen. Das IMS-Marktmodell ist ein dynamisches Ungleichgewichtsmodell eines Versicherungsmarktes. Die Marktteilnehmer optimieren ihre Zielgrößen nicht, sondern handeln nach nicht-optimierenden Verhaltensregeln.

Die Analyse der Dynamik des Marktes wird nicht unter der Prämisse der Suche nach einem Gleichgewicht geführt. Die Schocks auf den simulierten Versicherungsmarkt werden nicht als Störung eines bestehenden Gleichgewichts aufgefaßt. In Marktmodellen mit Gleichgewicht ist ein Schock als Auslöser eines zeitlich begrenzten Ungleichgewichts anzusehen. Ein Schock wird als Test für die Stabilität von Gleichgewichten benutzt. In einem Ungleichgewichtsmarkt nicht-optimierender Verhaltensregeln ist ein Schock eine Ergänzung der Informationsmengen dieser Verhaltensregeln. Er dient zur Überprüfung der Frage, ob diese Regeln in der Lage sind, plötzliche Änderung ihrer Marktumgebung zu verarbeiten. Die Auswirkungen von Schocks auf diesem simulierten Markt werden über die Ergebnisse der Verhaltensregeln von Anbietern und Nachfragern bestimmt. Die Analyse dieser

[3] vgl. Kapitel 2.2.

künstlichen mikroökonomischen Daten läßt Aussagen über die Schockwirkungen zu.

Die Verhaltensregeln der simulierten Wirtschaftssubjekte sind Vorschriften für die sequenzielle Entscheidungsbildung. Jedem Marktteilnehmer ist eine Verhaltensregel zugeordnet. Der Marktprozeß ist eine wiederholte Ausführung der Verhaltensregeln der einzelnen Marktteilnehmer. Jede Verhaltensregel hat eine bestimmte Informationsmenge, nach der sie ihre Zielgrößen setzt.

Weder Versicherer noch Versicherte besitzen eine Nutzenfunktion. Da die Verhaltensregeln der Marktteilnehmer nicht-optimierend sind, ist das Postulat einer (zu optimierenden) Nutzenfunktion nicht notwendig.

Die Entscheidungstheorie postuliert rationales Verhalten aller Marktteilnehmer. Schränkt man die Informationsmenge der Entscheider ein, wird ihr Verhaltens als beschränkt rational bezeichnet. Bestimmend für die beschränkte Rationalität ist die unvollständige Information der Anbieter und Nachfrager. Die neoklassische Theorie geht von vollständiger Information aus, was bedeutet, daß jeder Anbieter und Nachfrager alle Mengen, Preise und Qualitäten kennt. Diese Annahme ist besonders auf einem von Risiko geprägten Versicherungsmarkt unrealistisch. Bei diskreten, dynamischen Modellen gehört die Information über Mengen und Preise der Vorperioden zur vollständigen Informationsmenge hinzu.

Im IMS-Marktmodell hat keiner der Marktteilnehmer vollständige Information. Volle Information würde alle Informationen bis zur Vorperiode über alle Anbieter und Nachfrager umfassen. Es ist nicht anzunehmen, daß eine reale Versicherung über einen derartigen Informationsstand verfügt. Keine Informationsmenge einer Verhaltensregel enthält die Information, in welcher Simulationsperiode ein Schock stattfindet. Dadurch kann kein Marktteilnehmer sein Verhalten bei Schocks im voraus abstimmen. Eine Verhaltensregel kann auf einen Schock in der laufenden Periode reagieren, wenn ihre Informationsmenge die Schockinformation enthält und ihre Verhaltensregel eine Schockreaktion vorsieht. Sie kann in den Perioden nach einem Schock aber auch indirekt, durch die Schockreaktion anderer Verhaltensregeln, auf den Schock reagieren.

Auf dem IMS-Markt wiederholt sich ständig ein Minimalprozeß zwischen verschiedenen Versicherern und Versicherungsnehmern:

1. Ein Versicherungsnehmer zahlt Prämien an seinen Versicherer.
2. Ein Versicherer trägt die Schäden des Versicherten.

Der Marktprozeß wird durch die Nachfrage initiiert. Die Versicherungsnehmer lösen durch ihre Schäden und Prämienzahlungen Änderungen der Prämienfunktionen bei den Versicherern aus. Die Grundentscheidungen, die ein Nachfrager in jeder Periode treffen kann, sind:

1. Versichere ich mich oder trage ich das Risiko selbst ?
2. Wenn ich mich versichere: Welchen Versicherer wähle ich ?

Die Nachfrage nach Versicherungsschutz unterliegt in diesem Marktmodell keiner Rationierung. Jeder Versicherungsnehmer kann sich immer versichern. Es besteht für die Versicherer Kontrahierungszwang. Wenn ein Versicherungsnehmer entscheidet, sich zu versichern, dann werden seine Schäden immer vollständig getragen. Teilschäden werden nicht berücksichtigt.

In jeder Runde erhält der Versicherer Prämien und muß Schäden begleichen. Alle Prämieneinnahmen werden den Reserven zugeführt. Alle Schäden werden aus den Reserven bezahlt. Eine Summenkapazitäts- oder Stückzahlkapazitätsgrenze[4] gibt es nicht.

Da die Nachfrager wie die Anbieter nur über beschränkte Informationen verfügen, sind ihre Verhaltensregeln ebenfalls nicht-optimierend. Die Entscheidung, das Risiko in einer Periode selbst zu tragen, muß auf der Basis eigener Schadenerfahrungen, simulierter subjektiver Wahrscheinlichkeiten oder dem Verhalten anderer Nachfrager getroffen werden. Die Entscheidung, bei welcher Versicherung ein Vertrag abgeschlossen wird, oder ob der bestehende Vertrag aufgelöst wird, kann von verschiedenen Informationen abhängen:

1. Erfahrungen aus einem bestehendem Versicherungsvertrag,
2. dem Ergebnis einer Suche unter den Anbietern und
3. Präferenzen.

Besitzt ein Versicherungsnehmer keine entscheidungsrelevanten Informationen oder ist in seiner Verhaltensregel keine Informationsverarbeitung vorhanden, dann wird er zufällige Entscheidungen treffen. Er kann zufällig entscheiden, ob er sich in einer Periode versichert und den Versicherer durch eine Zufallsauswahl bestimmen.

Die Existenz von Präferenzen bei den Nachfragern ist Grundlage für die Modellierung eines Versicherungsmarktes bei monopolistischer Konkurrenz. Präferenzen stellen für den Nachfrager im IMS-Modell kostenlose Informationen dar. Die Anbieter können darüber entscheiden, ob und wieviel sie für Werbung aufwenden wollen. Werbeaufwendungen können die Präferenzen eines Nachfragers beeinflussen und zu Anbieterwechseln führen. Ein Versicherungsnehmer kann aber auch selbst einen Suchprozeß beginnen. Implizite Annahme der Suchstrategien ist, daß es Prämienunterschiede zwischen den verschiedenen Versicherern gibt. Würden alle Versicherer eine Einheitsprämie verlangen, dann wäre Suchen nach dem billigsten Anbieter überflüssig und die zufällige Wahl eines Versicherers wäre eine optimale Verhaltensregel. Prämiendispersion ist der Auslöser für Suchprozesse der Nachfrager. Im Gegensatz zu präferenzbasierten Wahl- und Wechselentscheidungen, fallen bei Suchprozessen Kosten an. Diese Kosten erhöhen die Gesamtaufwendungen des Nachfragers, die im Laufe des Marktprozesses anfallen. Diese Gesamtaufwendungen eines Versicherungsnehmers für

[4] vgl. Ackermann (1983), S. 88

1. Eigenversicherung,
2. Prämien und
3. Informationsbeschaffung

können zur Bewertung seiner Verhaltensregel herangezogen werden.

Die Prämisse der unendlich hohen Reaktionsgeschwindigkeit[5] kann in einem Marktmodell mit diskreten Prozeßzeitpunkten nicht aufrecht erhalten werden. Im IMS-Marktmodell sind die Nachfrager unvollkommen beweglich[6]. Wäre das nicht der Fall, müßte bei vollkommener Information in jeder Periode immer der billigste Anbieter alle Nachfrager an sich ziehen. Das setzt eine Informiertheit voraus, die in der Realität nicht zu finden ist[7].

3.2.2 Das Angebotsmodell

Das Angebotsmodell des IMS-Marktes unterliegt drei einfachen Regeln:

1. Die Angebotsmenge entspricht der Nachfragemenge.
2. Die Nachfragemenge hängt vom Angebotspreis ab.
3. Der Angebotspreis hängt von der Informationsmenge der Verhaltensregel des Anbieters ab.

Die erste Regel ist auf einem simulierten Markt für Versicherungen notwendig, da Dienstleistungen nicht lagerfähig sind. Die Angebotsmenge einer Versicherung ist immer gleich ihrer Ausbringungs- oder Produktionsmenge. Die Produktionsmenge einer Versicherung ist eine Funktion ihrer Produktionsfaktoren. Die beiden Hauptkomponenten jeder Produktionsfunktion sind die Faktoren Arbeit und Kapital. Werden die Produktionsfaktoren mit ihren Preisen und Mengen bewertet, dann entsteht die Kostenfunktion eines Unternehmens. Der wichtigste Kostenfaktor einer Versicherung sind die Schadenkosten. Da die Anzahl und Höhe der Schäden einer Versicherung von ihren Versicherten abhängt und Versicherungsschäden zufallsbedingt sind, besitzt jeder Anbieter eine eigene Produktionsfunktion. Weil die Schäden streuen, entsteht auf dem IMS-Markt eine natürliche Preis- oder Prämiendispersion. Diese Preisstreuung wird auf realen Versicherungsmärkten durch den Schadenausgleich im Kollektiv begrenzt. Um die Schockwirkungen auf die Anbieter besser untersuchen zu können, wurde die Anzahl der Nachfrager und damit die Möglichkeit des Schadenausgleichs im Kollektiv der Anbieter, bewußt begrenzt. Dadurch können die Auswirkungen von Verhaltensänderungen nach Schocks besser beobachtet werden.

[5] vgl. 2.2.
[6] vgl. Fischer (1988), S. 20
[7] vgl. Schulenburg (1987), S. 699

In Summenversicherungen ergeben sich die Schadenkosten aus dem Produkt aus Schadenanzahl und Versicherungssumme. In Schadenversicherungen wird jeder Schaden entsprechend seiner Höhe reguliert. Das IMS-Marktmodell simuliert einen Markt mit zwei Schadenversicherungssparten. Die Schadenkosten einer Versicherung entsprechen der Summe aller Einzelschäden der bei ihr versicherten Nachfrager. Personal- und Verwaltungskosten werden nicht berücksichtigt. Die Versicherer agieren in beiden Sparten als Prämienanpasser.

Ein Versicherungsnehmer, der einen Versicherer in der Periode t als Vertragspartner gewählt hat, zahlt ihm immer die vom Versicherer in der Periode verlangte Prämie. Diese Prämie kann sich in jeder Periode ändern, der Nachfrager kann in jeder Periode den Vertrag auflösen und den Anbieter wechseln.

Die Verhaltensregeln der Anbieter können im IMS-Modell in jeder Sparte zwei Parameter ändern:

1. Die Prämie und
2. die Höhe der Werbeaufwendungen.

Die Prämien dienen zur Deckung der Schadenkosten. Die Aufwendungen für Werbung sind der zweite Faktor der Kostenfunktion der Anbieter. Werbung ist in der Theorie der monopolistischen Konkurrenz das Mittel, um die Präferenzen der Nachfrager zu beeinflussen[8].

Im IMS-Markt können sowohl Nachfrager mit präferenzgesteuerten Verhaltensregeln als auch Nachfrager, die nicht auf Werbung reagieren, vertreten sein. Die Präferenzen können von den Nachfragern als kostenlose Informationen für Wahl- und Wechselentscheidungen genutzt werden. Der einzelne Anbieter hat keine Informationen darüber, wie groß der Anteil der Nachfrager ist, die auf Werbung reagieren und er hat auch keine Informationen über die Höhe der Werbeaufwendungen anderer Anbieter. Die implizite Annahme der werbenden Anbieter ist, daß Werbung die Nachfrage erhöht, weil durch sie Wahl- und Wechselentscheidungen zugunsten des eigenen Unternehmens beeinflußt werden können.

Die Kostenfunktion der Anbieter ist eine Funktion von Schadenkosten und Werbeaufwendungen. Sie ist unabhängig vom Nachfragevolumen, das eine Versicherung hält. Eine Kostendegression kann also nicht eintreten. Ohne Skalen- und Lerneffekte gibt es keine kostenminimale Produktion und damit keine optimale Größe für eine Versicherung im IMS-Markt. Ein Anbieter kann sein Risiko nicht weiter streuen. Ein Rückversicherungsmarkt existiert nicht und eine Kreditfinanzierung der Schadenkosten ist nicht möglich.

Das Verhalten der Anbieter wird durch Regeln modelliert, die festlegen, welche Informationen zu Reaktionen führen. Eine Verhaltensregel kann als Regel zur Informationsverarbeitung aufgefaßt werden. Die Informationsmenge einer Verhaltensregel ist durch die von ihr verwendeten Informationen definiert. Das Angebotsmodell unterscheidet drei Arten von Verhaltensregeln:

[8] vgl. 2.2.

1. Informationsfreie Verhaltensregeln,
2. Verhaltensregeln auf der Basis anbietereigner Informationen und
3. Verhaltensregeln auf der Basis anbieterfremder Informationen.

Alle oligopolistischen Verhaltensregeln greifen nur auf anbieterfremde Informationen zu, alle monopolistischen Strategien basieren auf anbietereigenen Informationen.

Die Informationsmenge der Verhaltensregeln ist die Basis für die Modellierung der Anbieterseite. Ihre Elemente bestimmen die Entscheidungen des Anbieters über seine Prämienänderungen und die Höhe seiner Werbeaufwendungen. Von der Stufe der Informationsverarbeitung ist auch die Marktform der Anbieterseite abhängig. Je nachdem ob die Verhaltensregeln nur eigene oder auch fremde Informationen für ihre Entscheidungen verwenden, ist die Anbieterseite des IMS-Marktmodells oligopolistisch oder monopolistisch.

3.2.3 Das Nachfragemodell

Die Nachfrage im IMS-Modell ist keine aggregierte, homogene Marktgröße für die Anbieter. Genau wie die Anbieter sind auch die Nachfrager zu mikroökonomischen Einheiten disaggregiert. Das Nachfragemodell des IMS-Marktes wird durch drei Komponenten bestimmt:

1. Der Verhaltensregel für die Entscheidung über Eigen- oder Fremdversicherung,
2. der Verhaltensregel für die Wahl der Versicherung und
3. Dem Schadenmodell der Nachfrager.

Die Verhaltensregeln der Nachfrager sind zweistufig. Auf der ersten Stufe trifft der Nachfrager die Entscheidung, ob er sein Risiko in der Periode t selbst trägt oder ob er sich dagegen versichert. Dafür stehen ihm drei Arten von Verhaltensregeln zur Verfügung:

1. Informationsfreie Verhaltensregeln,
2. Verhaltensregeln auf der Basis von Eigeninformationen und
3. Verhaltensregeln auf der Basis von Fremdinformationen.

Der Nachfrager hat aber nicht nur die Möglichkeit von Eigen- auf Fremdversicherung zu wechseln, sondern kann auch den umgekehrten Wechsel durchführen.

Bei informationsfreien Verhaltensregeln zur Wahl des Versicherungsstatus entscheidet der Nachfrager in jeder Periode zufällig, ob er einen möglichen Schaden selbst trägt oder sich dagegen versichert. Für die Entscheidung auf der Basis von Eigeninformationen steht

ihm ein Vektor mit der subjektiven Schadenwahrscheinlichkeit der aktuellen Periode zur Verfügung. Die subjektive Wahrscheinlichkeit in der Periode t einen Schaden zu erleiden, ist in einem Vektor definiert, dessen Länge der Simulationsdauer entspricht. Der Nachfrager versucht durch eine Schätzung sein Risiko für die laufende Periode zu bestimmen. Dabei vergleicht er eine Zufallsvariable mit seiner subjektiven Schadenwahrscheinlichkeit der laufenden Periode. Der Wert der Zufallsvariable ist die Schätzung für das Risiko einen Schaden in der laufenden Periode zu haben. Ist die Schätzung dieser Schadenwahrscheinlichkeit höher als die individuelle subjektive Risikoeinschätzung, dann entscheidet sich der Nachfrager für die Fremdversicherung. Wird der Schwellenwert aber unterschritten, dann ist das gleichbedeutend mit der Einschätzung des Nachfragers, daß die Wahrscheinlichkeit in dieser Periode einen Schaden zu erleiden relativ klein ist. In diesem Fall trägt er das Risiko selbst.

Die Fremdinformation für die Entscheidung über den Versicherungsstatus basiert auf dem durchschnittlichen Versicherungsgrad eines Risikos. Der Nachfrager vergleicht dazu einen Schwellenwert mit dem durchschnittlichen Versicherungsgrad des betreffenden Risikos in der Periode t. Wird der Schwellenwert überschritten, dann wählt er die Fremdversicherung, wird er unterschritten, trägt er sein Risiko selbst. Die Funktionsweise dieser Verhaltensregel soll ein Beispiel verdeutlichen. Der durchschnittliche Versicherungsgrad wird aus dem Quotienten der Anzahl der fremdversicherten Nachfrager und aller aktivierten Versicherungsnehmer berechnet. Sind in einer Periode t 100 Nachfrager aktiv, von denen 70 ihr Risiko fremdversichern und 30 die Selbstversicherung wählen, dann ist der durchschnittliche Versicherungsgrad dieses Risikos 0.7. Angenommen der Schwellwert eines Nachfragers ist 0.5, dann würde er sich in dieser Periode für die Fremdversicherung entscheiden. Sinkt der durchschnittliche Versicherungsgrad in der nächsten Periode auf 0.4, dann wechselt er von der Fremdversicherung zur Eigenversicherung.

Falls der Nachfrager sich für die Fremdversicherung entschieden hat, wird in der zweiten Stufe der Versicherer, bei dem er für die laufende Periode einen Vertrag abschließt, gewählt. Die Modellierung einer Pflichtversicherung ist ebenfalls möglich. In diesem Fall ist der Verhaltenregel die Möglichkeit des Wechsels von der Fremd- zur Eigenversicherung genommen. Ist der Anbieter gewählt, hat dieser keine Möglichkeit, den Vertrag abzulehnen. Bleibt ein Vertragsverhältnis über mehrere Perioden bestehen, weil der Nachfrager nicht wechselt, hat der Versicherer keine Möglichkeit das Vertragverhältnis zu beenden.

Auch für die Wahl des Anbieters stehen dem Versicherungsnehmer drei Arten von Verhaltensregeln zur Verfügung:

1. Informationsfreie Verhaltensregeln,
2. Verhaltensregeln auf der Basis Eigeninformationen und
3. Verhaltensregeln auf der Basis Fremdinformationen.

Eine Wahl des Versicherers ohne Information ist mit einer Zufallswahl gleichzusetzen. Diese Wahlentscheidung verursacht keine direkten Kosten für den Versicherungsnehmer.

Dieses Verhalten wird auch als naive Suche bezeichnet[9]. Die Hauptquelle nachfragereigener Informationen sind seine Präferenzen[10]. Wahlentscheidungen über die Präferenzen verursachen keine direkten Kosten beim Nachfrager.

In IMS werden die Präferenzen durch den Versicherungsnehmer berechnet. Verhaltensregeln, die Präferenzen zur Wahl ihrer Versicherung benötigen, berechnen sie in jeder Periode neu. Versicherer, die Präferenzen beeinflussen wollen, wählen in jeder Periode einen Betrag für die Präferenzbildung in jeder Sparte. Die Wirkung dieser "Werbeausgaben" hängt einerseits von den Werbeausgaben der anderen aktiven Versicherer ab und andererseits davon, ob die Nachfrager Präferenzen zur Wahl ihrer Versicherung benutzen. Weiterhin ist er vom Anteil der präferenzbildenden Nachfrager an den Gesamtnachfragern abhängig.

Den Wirkungsablauf soll ein Beispiel verdeutlichen. Angenommen in einem Marktmodell sind drei Versicherer aktiv, die alle Werbung betreiben. Es wird eine Sparte in einer beliebigen Periode betrachtet. In diesem Marktmodell ist nur ein Nachfrager aktiv, der seinen Versicherer nach Präferenzen auswählt. Der Versicherungsnehmer sammelt zunächst die Werbeausgaben der aktiven Versicherer: $W_{VU1} = 10, W_{VU2} = 20, W_{VU3} = 70$. Er berechnet nun die Präferenz für einen Versicherer, indem er die Einzelaufwendungen für Werbung eines Versicherers durch die gesamten Präferenzaufwendungen einer Sparte teilt:

$$Pr_{VN1}(\frac{10}{10+20+70}, \frac{20}{10+20+70}, \frac{70}{10+20+70})$$

also $Pr_{VN1}(0.1, 0.2, 0.7)$. Der präferenzbestimmte Nachfrager wählt Versicherer 3. Der Präferenzwert eines Versicherers ist gleich seinem Anteil an den gesamten Werbeausgaben in einer Sparte. Wenn keine Versicherung Werbeaufwendungen tätigt, ist die Maximalpräferenz Null. In diesem Fall wählt der Nachfrager zufällig eine Versicherung.

Für die Anbieterwahl auf der Basis von Fremdinformationen muß der Nachfrager einen Suchvorgang durchführen. Die Suche und der Vergleich von Preisinformationen ist eines der Hauptmerkmale des Nachfragerverhaltens auf Märkten mit unvollkommener Information. Auf einem Versicherungsmarkt mit Einheitsprämie ist die Suche nach Prämieninformationen kaum sinnvoll. Auf dem IMS-Markt sind die Prämien aber nicht einheitlich, weil es keine Regulierungsinstitution gibt, die den Anbietern eine Prämie vorgibt. Durch wechselnde Schäden und unterschiedliche Prämienänderungsverhalten bilden die in einer Periode t von den Anbietern verlangten Prämien eine dem Nachfrager unbekannte Verteilung. Über diese Verteilung besitzt er keine Informationen. Ihm ist weder Maximal-, Minimal- noch Durchschnittsprämie bekannt noch besitzt er Informationen darüber, welcher Versicherer welche Prämie verlangt. Die Beschaffung von Informationen über die Preisverteilung in den beiden Einzelsparten verursacht beim Versicherungsnehmer direkte Kosten. Die Informationen und deren Preise sind für den Nachfrager bei den Anbietern und bei einer Informationsinstitution abrufbar. Die Kosten für diese Informationen bela-

[9] vgl. Crössmann (1982), S. 12
[10] vgl. Finsinger, Schulenburg (1987), S. 247

sten das Vermögen des Versicherungsnehmers zusätzlich zu Prämien und Aufwendungen der Eigenversicherung.

Die Grundlagen für die Teildisziplin der Ökonomie, die sich mit der Suche und der Bewertung von Informationen beschäftigt, wurde von *Stigler* zu Beginn der 60er Jahre entwickelt[11]. In dem von ihm entwickelten Modell zieht der Suchende aus einer ihm bekannten Preisverteilung eine Stichprobe. Jede Ziehung einer Einzelinformation kostet ihn einen konstanten Betrag. Der Nachfrager holt alle Preisangebote gleichzeitig ein. Die Größe der Stichprobe wird zu Beginn des Suchvorgangs durch einen Optimierungsvorgang bestimmt. Dieser Optimierungsvorgang setzt die Grenzkosten der Information in Vergleich zur ihrer Grenzersparnis. Die optimale Stichprobengröße ist erreicht, wenn die Grenzkosten einer weiteren Preisinformation größer sind, als die Grenzersparnis auf der Basis der schon gezogenen Informationen[12]. Weil alle Preisstichproben gleichzeitig vorliegen bezeichnet man diese Regel auch als nicht-sequenzielle Suchregel.

Die Stigler-Regel zur Berechnung der optimalen Größe der Preisstichprobe hat einige Nachteile für reale oder simulierte Versicherungsmärkte. Ihr Informationsbedarf ist sehr hoch. Während die Grenzkosten eines Suchschritts noch relativ einfach bestimmbar sind (z.B. die Kosten für ein Telefonat mit einer Versicherung, um eine Prämie zu erfahren) ist die Preisverteilung für einen bestimmten Versicherungsvertrag in der Realität nur schwer zu beschaffen. Ein Preisvergleich ist trotz allgemeiner Vertragsbedingungen sehr aufwendig, weil dazu eine Homogenisierung der heterogenen Produkte notwendig ist. Preisvergleiche sind aber die Basis für die Erstellung einer Preisverteilung.

Ein weiterer Nachteil der Stigler-Regel ist ihre "Sturheit". Obwohl der Suchende die Preisverteilung kennt (und damit auch ihr Minimum) führt er alle von der Regel geforderten Suchschritte durch. Das liegt an der angenommenen Gleichzeitigkeit des Ziehens der Preisstichprobe.

Bei sequenziellen Suchregeln wird nach jeder Preisstichprobe geprüft, ob es sich überhaupt lohnt weiterzusuchen. Falls ein Anbieter mit minimalem Preis vor der gemäß der Stigler-Regel berechneten optimalen Anzahl an Suchschritten gefunden wird, kann die Suche vorzeitig abgebrochen werden. Das ist bei der nicht-sequenziellen Suchstrategie von Stigler nicht möglich[13]. Weil der Suchende bereits zu Beginn der Suche festlegt, wieviele Preise er vergleicht, kann er keine zusätzliche Stopregel benutzen, die beim vorzeitigen Finden des minimalen Preises die Suche abbricht[14].

Wesentlich für eine Suchregel ist der Rückgriff auf zurückliegende Angebote. Wenn ein Angebot nur zeitlich begrenzt zur Verfügung steht muß der Suchende bereits nach dem Ziehen einer Einzelstichprobe entscheiden, ob er das Angebot annimmt. Ist dagegen ein Rückgriff auf die vergangenen Suchergebnisse möglich, kann er alle gewünschten Stichproben einholen und danach den Anbieter mit dem kleinsten Preis daraus auswählen.

[11] vgl. Stigler (1961), S. 213
[12] vgl. Crössmann (1982), S. 9
[13] vgl. Haid (1984), S. 37
[14] vgl. Le Vaillant de Folleville (1986), S. 62

Die Nachfrager im IMS-Markt können aufgrund fehlender Informationen über die Prämienverteilungen in den einzelnen Sparten keine nicht-sequenziellen Suchstrategien in den einzelnen Perioden anwenden. Sie suchen mit einer Suchstrategie mit exogen vorgegebener Stichprobengröße und Rückgriff auf die gefundenen Prämieninformationen. Angenommen der exogene Such-Stop-Wert des Nachfragers 15 ist vier, dann wird dieser Nachfrager vier Prämienangebote von zufällig ausgewählen Versicherern einholen und im zweiten Schritt aus diesen vier Angeboten den Versicherer mit der kleinsten Prämie auswählen und einen Vertrag für die laufende Periode kaufen. Die Kosten pro Prämieninformation eines Versicherers mindern sein Vermögen.

Die dritte Komponente der IMS-Nachfrageseite ist das Schadenmodell. Die Schadenhöhe der Versicherungsnehmer bewegen sich in einem Bereich zwischen Null und Hundert. Die Schadenwahrscheinlichkeit ändert sich nicht durch den Abschluß eines Vertrags, d.h. Moral Hazard ist ausgeschlossen. Um die Einflüsse unterschiedlicher Schadenfunktionen besser untersuchen zu können, hat jeder Versicherungsnehmer zwei Risiken in jeder Periode: $S_1(t), S_2(t)$. Die Schadenhöhe dieser Risiken ist eine linear transformierte normalverteilte Zufallsvariable: $\mu_1, \mu_2 \sim N(0,1)$.

$$\begin{aligned} S_1(t) &= a_1 + b_1 * \mu_1 \\ S_2(t) &= a_2 + b_2 * \mu_2 \end{aligned} \tag{3.1}$$

Die Parameter der Schadenfunktionen eines einzelnen Nachfragers sind für die Versicherer nicht beobachtbar. Der Versicherungsnehmer kennt seine Schadenwahrscheinlichkeit ebenfalls nicht. Seine eigene Risikoeinschätzung basiert auf dem Vektor der subjektiven Wahrscheinlichkeiten. Es liegt keine asymmetrische Information[15] vor, da auch der Versicherungsnehmer nicht weiß, ob es ein "gutes" oder ein "schlechtes" Risiko ist. Weil jeder Nachfrager andere Parameterwerte in der Schadenfunktion haben kann, sind die Risiken auf dem IMS-Markt als heterogen zu bezeichnen.

3.2.4 Schocks

Im Mittelpunkt dieser Arbeit steht die Analyse der Auswirkungen exogener Schocks auf Versicherungsmärkte. Exogene Schocks auf dem IMS-Markt sind sprunghafte Änderungen von Informationen, die das Verhalten der Anbieter und Nachfrager beeinflussen. Ein Schock im IMS-Modell ist definiert als die Änderung einer verhaltensrelevanten Information von einer Periode t zur Periode $t+1$. Das IMS-Modell unterscheidet drei Arten von Schocks:

1. Aktivierungsschocks,
2. Änderungsschocks und
3. indirekte Schocks.

[15] vgl. Strassl (1980), S. 124

Die Anbieter und Nachfrager im IMS-Modell können in jeder Simulationsperiode aktiviert werden. Wird ein Marktteilnehmer in einer späteren als der ersten Periode aktiviert, dann bezeichnet das einen Aktivierungsschock. Die Aktivierung führt bei einer Versicherung dazu, daß sie in allen Perioden nach ihrer Aktivierung durch Nachfrager gewählt werden kann. Bei konstanter Anzahl der Nachfrager kann es durch einen Aktivierungsschock auf der Anbieterseite zu einer Verschiebung der Marktanteile und Wechselbewegungen kommen. Bei einem Versicherungsnehmer wird durch die Aktivierung sein Schadengenerierungsprozeß initialisiert. Er bestimmt seinen Versicherungsstatus und sorgt im Falle der Wahl der Fremdversicherung für eine Nachfrageerhöhung.

Das Kernkonzept des Änderungsschocks ist der Wechsel des Parametervektors einer Verhaltensregel. Dieser Wechsel wird durch einen Schockindikator gesteuert. Zeigt er einen Änderungsschock an, dann wechselt der Marktteilnehmer seine Verhaltensparameter. Jeder Anbieter und Nachfrager besitzt eigene Parametervektoren, so daß eine es differenzierte Modellierung des Änderungsschocks möglich ist. Innerhalb eines Simulationslaufs kann immer nur ein Änderungsschock, aber beliebig viele Aktivierungsschocks stattfinden. Exogene Schocks können auch aus der Kombination von Aktivierungs- und Änderungsschocks erzeugt werden.

Neben den Änderungen der Verhaltensparameter von Nachfragern und Anbietern können auch die allgemeinen Marktparameter geändert werden. Diese Parameter gelten für alle Anbieter und Nachfrager unabhängig von ihrer Verhaltensregel. Dazu gehören:

- Zinsen für die Verzinsung der Reserven und
- Kosten pro Prämieninformation für die Nachfrager.

Die Reserven aller Versicherer werden in jeder Periode verzinst. Wenn die Reserven eines Anbieters negativ sind, werden sie mit dem gleichen Satz verzinst. Die Kosten pro Prämieninformation werden von den Nachfragern benötigt, deren Anbieterwahl auf Fremdinformationen basiert.

Während die beiden ersten Schockarten direkt wirken, lösen die überwiegende Anzahl der Parameteränderungen indirekte Schocks aus. Wenn in einem Markt zunächst alle Versicherer eine monopolistische Verhaltensregel haben, ab einer bestimmten Periode aber oligopolistische Anbieter aktiviert werden, dann liegt ein kombinierter Aktivierungs- und Änderungsschock vor, der indirekt auch auf die Nachfrageseite wirkt. Indirekt ändert sich auch die Marktform von der reinen monopolistischen Anbieterkonkurrenz zu einer Mischung von Oligopol und monopolistischer Konkurrenz. Diese Schockform kann mit einem Aktivierungsschock auf der Nachfrageseite kombiniert werden, so daß komplexe Schockmodelle aufgestellt werden können.

Die Möglichkeiten der Schockmodellierung sollen an einem kurzen Beispiel verdeutlicht werden. In einem IMS-Markt, der über 50 Modellperioden simuliert wird, sind 100 Versicherungsnehmer mit der Verhaltensregel A ab Periode $t = 1$ aktiv. Die Anbieterseite ist durch fünf Versicherer mit einer monopolistischen Verhaltensregel modelliert. In Periode

20 werden 100 weitere Nachfrager aktiviert, was gleichbedeutend mit einem Aktivierungsschock auf der Nachfragerseite ist. In der gleichen Periode werden zusätzlich fünf Versicherer mit einer oligopolistischen Verhaltensregel aktiviert, was einen Aktivierungsschock auf der Angebotsseite darstellt. In Periode 25 wird der Indikator des Änderungsschocks gesetzt und alle Marktteilnehmer wechseln den Parametervektor ihrer Verhaltensregel. Damit ist ein indirekter Schock ausgelöst worden, der die Marktform der Anbieterseite von monopolistischer Konkurrenz hin zu einer Mischung von Oligopol und monopolistischer Konkurrenz geändert hat.

3.3 Die Modellbausteine

3.3.1 Die Verhaltensregeln der Anbieter

3.3.1.1 Informationsfreie Verhaltensregeln

Die Verhaltensregeln von Anbietern im IMS-Modell bestehen aus den Formeln für die vier Zielgrößen, die jeder Versicherer beeinflussen kann:

1. Die Prämie in Sparte 1 und Sparte 2 und
2. die Werbeaufwendungen in Sparte 1 und Sparte 2.

Prämien- und Werbeaufwandsbestimmung basieren aus Gründen der Komplexitätsbegrenzung auf der gleichen Informationsmenge. Die Berechnung dieser Zielgrößen nimmt im Programm jede Versicherung seperat vor. Von den drei Schockarten im IMS-Modell

1. Aktivierungsschocks,
2. Änderungsschocks und
3. indirekte Schocks

führen Schocks der zweiten Kategorie zu direkten Verhaltensänderungen der Anbieter, weil durch sie die Parameter beeinflußt werden. Die Schockeinflüsse wirken immer auf die gleiche Art. Wenn in einer Periode t ein Änderungsschock modelliert worden ist, wird ein Element der Informationsmenge der Anbieter verändert. Die Verhaltensregel der Versicherer bemerkt diese Änderung und reagiert darauf mit einer Parameteränderung. Das schockempfindliche Element in der Informationsmenge der Anbieter sei K. Durch den Änderungsschock wird es von 0 auf 1 erhöht. Das führt dazu, daß der Verhaltensparameter X, der vor dem Änderungsschock einen Wert von X^0 hatte, von der Verhaltensregel des Anbieters zu X^* verändert wird.

$$X = \begin{cases} K = 0 & : \quad X^0 \\ K = 1 & : \quad X^* \end{cases} \tag{3.2}$$

Die ersten beiden Verhaltensregeln sind informationsfreie Vorschriften für die Anbieter. Die Regel **ZUFALL I** wählt in jeder Simulationsperiode Prämien und Werbeaufwendungen aus gleichverteilten Zufallszahlen $\mu \sim U(0,1)$:

$$\begin{aligned} P_1(t) &= a * \mu_1 \\ P_2(t) &= b * \mu_2 \end{aligned} \tag{3.3}$$

$$\begin{aligned} W_1(t) &= c * \mu_3 \\ W_2(t) &= d * \mu_4 \end{aligned} \tag{3.4}$$

Bezieht man nun die Parameteränderungen bei einem Änderungsschock mit ein, sind für diese Verhaltensregel acht Parameter zu spezifizieren: $(a, a^*, b, b^*, c, c^*, d, d^*)$

Die zweite informationsfreie Verhaltensregel (**Zufall II**) läßt sich durch die Hinzunahme von Verteilungsinformationen über die Schadenhöhenverteilung bei der Zielgrößenbildung ableiten. Dazu wird die Kenntnis der Schadenhöhenfunktions*form* hinzugezogen. Die Versicherer besitzen weiterhin keinerlei Kenntnis über die Parameter der Schadenfunktion der Versicherungsnehmer, aber sie kennen die Verteilung[16], die linear zur Schadenhöhenfunktion der Versicherungsnehmer transformiert wird.

$$\begin{aligned} P_1(t) &= a + b * \mu_1 \\ P_2(t) &= c + d * \mu_2 \end{aligned} \tag{3.5}$$

$$\begin{aligned} W_1(t) &= e + f * \mu_3 \\ W_2(t) &= g + h * \mu_4 \end{aligned} \tag{3.6}$$

Wie schon in der Verhaltensregel **Zufall I** reagieren alle Parameter direkt schockempfindlich auf Änderungsschocks. Daraus ergibt sich ein Parametervektor mit 16 Elementen: $(a, a^*, b, b^*, c, c^*, d, d^*, e, e^*, f, f^*, g, g^*, h, h^*)$.

Die Verhaltensregeln **Zufall I** und **Zufall II** stellen im Rahmen des IMS-Modells die Referenz für die beschränkte Rationalität dar[17]. Die Informationsmenge der Versicherer, die sich nach diesen Regeln verhalten, ist leer. Da die Anbieter aber dem Zwang der sequenziellen Entscheidung unterliegen, also in jeder Periode die Prämie festsetzen müssen, ist die zufällige Wahl der Prämie die angemessene Entscheidungsregel. Im Verlauf der späteren Analysen wird überprüft, ob Verhaltensregeln, die mehr Informationen verarbeiten, in einer Marktumgebung bei Schocks generell überlegen sind. Es muß weiterhin untersucht werden, ob die fehlende Informationsverarbeitung in jeder Marktumgebung ein Nachteil ist und wie andere Versicherer mit komplexeren Verhaltensregeln auf das Vorhandensein von "dummen" Konkurrenten im Markt reagieren.

3.3.1.2 Verhaltensregeln mit Eigeninformation

Die zweite Klasse von Verhaltensregeln der Anbieterseite ist in der behavioristischen Unternehmenstheorie verwurzelt. Diese Theorie stellt prozeßorientierte Modelle von Unter-

[16] Die Schadenhöhenfunktion ist eine Standardnormalverteilung: $\mu \sim N(0,1)$

[17] vgl. Fischer (1988), S. 31

nehmen auf, wobei die Entscheidungsmenge der Unternehmen auf eine kleine Anzahl von Schlüsselentscheidungen begrenzt ist[18]. Optimierung und vollständige Information werden durch psychologische Annahmen über das Verhalten ganzer Organisationen ersetzt. Statt einer Zieloptimierung wird postuliert, daß das Zielniveau aus Erfahrungen und Erwartungen abgeleitet wird und das sich eine Organisation keine Maximalziele, sondern objektiv erreichbare Anspruchsniveaus setzt[19]. Die verhaltensorientierte Wirtschaftstheorie stellt dieses Satisficing-Prinzip dem Maximierungsprinzip der klassischen und neo-klassischen Theorie gegenüber. Zentrales Konzept im Satisficing-Prinzip ist der sequenzielle Vergleich aller Entscheidungsalternativen mit einem exogenen Kriterium: Dem Anspruchsniveau. Vier Anspruchsniveaugrößen sind im IMS-Modell definiert:

1. Die Reserven einer Sparte in der Vorperiode,
2. die Anzahl der Nettowechsel in einer Sparte in der Vorperiode,
3. der Marktanteil an Versicherten in der Vorperiode und
4. durchschnittliche Schadenhöhe auf der Basis eigener Schäden in der Vorperiode.

Die Versicherungsnehmer können in jeder Periode den Versicherer wechseln. Die Anzahl der Versicherten, die ein Unternehmen im Vergleich zur Vorperiode zusätzlich unter Vertrag nehmen konnte, ist ein Maßstab für ihren Erfolg im Neugeschäft[20]. Das Neugeschäft ist auf realen Versicherungsmärkten das eigentliche Feld des dynamischen Wettbewerbs[21]. Die Erhöhung des Marktanteils gilt als klassisches Unternehmensziel, während die durchschnittliche Schadenhöhe der Vorperiode die Umsetzung der Vollkostenhypothese ist[22].

Um die Entscheidungsbildung der Anbieter nicht durch die Stellung des einzelnen Anbieters in der seriellen Berechnungsreihenfolge zu beeinträchtigen, muß die Informationsmenge eines Anbieters immer auf die Informationen einer abgeschlossenen Periode zugreifen. Sonst könnte theoretisch ein Anbieter y der nach einem Anbieter x seine Prämien- und Werbeentscheidung trifft, aus seiner Stellung in der Berechnung einen Vorteil ziehen. Versetzt man sich in einen simulierten Nachfrager im IMS-Modell, dann treffen aus seiner Perspektive alle Versicherer gleichzeitig ihre Entscheidungen. Aus diesem Grund können die Anbieter nur auf Informationen aus bereits abgeschlossenen Perioden zugreifen. Die Informationen aus der Periode $t-1$ sind somit die neusten Informationen, die einem Anbieter zur Verfügung stehen. Es liegt daher nahe, in einem Modell mit beschränkter Rationalität der Marktteilnehmer auf die neusten verfügbaren Informationen zur Entscheidungsbildung zuzugreifen.

[18] vgl. Cyert, March (1995), S. 1
[19] vgl. Klopstech (1980), S. 10
[20] vgl. Schulenburg (1987), S. 703
[21] vgl. Farny (1979), S. 66
[22] vgl. Wanninger (1981), S. 17

Die erste Verhaltensregel mit Eigeninformationen **Mark-Up I** bestimmt die Prämien- und Werbekostenreaktion auf der Anspruchsbasis der Reserven der Vorperiode in den jeweiligen Sparten: $R_1(t-1)$ und $R_2(t-1)$. Werden die Anspruchsniveaus R_1^*, R_2^* unterschritten, dann ändert sich die Reaktionsrichtung der Prämien.

$$P_1(t) = \begin{cases} R_1(t-1) \leq R_1^* & : \quad a * P_1(t-1) \\ R_1(t-1) > R_1^* & : \quad b * P_1(t-1) \end{cases} \tag{3.7}$$

$$P_2(t) = \begin{cases} R_2(t-1) \leq R_2^* & : \quad c * P_2(t-1) \\ R_2(t-1) > R_2^* & : \quad d * P_2(t-1) \end{cases} \tag{3.8}$$

$$W_1(t) = \begin{cases} R_1(t-1) \leq R_1^* & : \quad e * W_1(t-1) \\ R_1(t-1) > R_1^* & : \quad f * W_1(t-1) \end{cases} \tag{3.9}$$

$$W_2(t) = \begin{cases} R_2(t-1) \leq R_2^* & : \quad g * W_2(t-1) \\ R_2(t-1) > R_2^* & : \quad h * W_2(t-1) \end{cases} \tag{3.10}$$

Diese Verhaltensregel stellt die Reserven wegen ihrer Bedeutung für das globale Unternehmenziel "Langfristiges Überleben" in den Mittelpunkt. Wird das Anspruchsniveau der Reserven in einer Sparte unterschritten, dann berechnet sich die neue Prämie aus einen Aufschlag auf die Prämie der Vorperiode. Eine rationale Parameterwahl muß demnach die Parameter $a, a^*, c, c^* > 1$ festlegen. Bei negativen Reserven war die bisherige Prämie zu niedrig angesetzt, so daß die Reserven überlastet und das Ziel des langfristigen Überlebens der Versicherung in Gefahr war. Sind dagegen die Reserven positiv, versucht diese Verhaltensregel durch Prämiensenkungen mehr Versicherungsnachfrager an sich zu ziehen. Die Parameter b, b^*, d, d^* sind danach kleiner als Eins zu wählen.

Die Werbeaufwendungen unterliegen dem gleichen Anspruchsniveau. Sind die Reserven negativ, dann kann sich der Anbieter keine Werbung mehr leisten, obwohl er durch einen erhöhten Werbeaufwand unter Umständen mehr Nachfrager an sich ziehen und damit seine Reserven erhöhen kann. Es wird daher zu untersuchen sein, welches Werbeverhalten zu besseren Ergebnissen führt und wie sich das Werbeverhalten bei Schocks auswirkt.

Neben dem Parametervektor $(a, a^*, b, b^*, c, c^*, d, d^*, e, e^*, f, f^*, g, g^*, h, h^*)$ sind für die Periode 1 die Startwerte $P_1(1), P_2(1), W_1(1), W_2(1)$ und die Anspruchsniveaus R_1^*, R_2^* festzulegen. Der Einfluß der Startparameter auf das Ergebnis der Verhaltensregel **Mark-Up I** ist ebenso zu berücksichtigen, wie die Auswirkungen unterschiedlicher Anspruchsniveaus auf das Ergebnis bei Schocks.

Die Verhaltensregel **Mark-Up II** besitzt als Anspruchsniveaugröße für Prämien- und Werbeaufswandsänderungen die Anzahl der Nettowechsler der Vorperiode $Nw(t-1)$ in der jeweiligen Sparte. Über- oder unterschreitet diese Größe das Anspruchsniveau Nw^*, dann wird eine Prämienreaktion ausgelöst. Die Nettowechsler berechnen sich aus der Differenz der Anzahl von Versicherten der Vorperiode $Azvn(t-1)$ und der Vor-Vorperiode $Azvn(t-2)$: $Nw(t-1) = Azvn(t-1) - Azvn(t-2)$.

In einer Simulation mit fünf Perioden sei die Anzahl der Versicherten in einem Vektor gespeichert: $(20, 25, 15, 20, 10)$. Aufgrund des zwei-periodigen Rückgriffs sind für die ersten

beiden Perioden der Wert der Nettowechsel nicht definiert. Aus diesem Grund ist für die ersten beiden Perioden die Prämie für beide Sparten als Startwerte vorzugeben. Als Vektor der Nettowechsel ergibt sich in diesem Beispiel: $(n.d., n.d., 5, -10, 5)$. Nimmt man wie *Farny*[23] das Neugeschäft als eigentliches Feld des Wettbewerbs auf dynamischen Versicherungsmärkten an, dann liegt die Wahl der Nettowechsler als Anspruchsniveaugröße nahe. Die Versicherungsnehmer im IMS-Modell können in jeder Periode ihre Verträge lösen. Bestandspflege oder -wettbewerb ist daher überflüssig, da das gesamte Marktvolumen in jeder Periode neu verteilt werden kann. Die Nettowechsler sind daher als ein Indikator für den Erfolg der Verhaltenregel im Kampf um das Neugeschäft zu interpretieren. Die Höhe der Prämien- und Werbeaufwendungen läßt sich danach beschreiben als:

$$P_1(t) = \begin{cases} Nw_1(t-1) \leq Nw_1^* & : \quad a * P_1(t-1) \\ Nw_1(t-1) > Nw_1^* & : \quad b * P_1(t-1) \end{cases} \tag{3.11}$$

$$P_2(t) = \begin{cases} Nw_2(t-1) \leq Nw_2^* & : \quad c * P_2(t-1) \\ Nw_2(t-1) > Nw_2^* & : \quad d * P_2(t-1) \end{cases} \tag{3.12}$$

$$W_1(t) = \begin{cases} Nw_1(t-1) \leq Nw_1^* & : \quad e * W_1(t-1) \\ Nw_1(t-1) > Nw_1^* & : \quad f * W_1(t-1) \end{cases} \tag{3.13}$$

$$W_2(t) = \begin{cases} Nw_2(t-1) \leq Nw_2^* & : \quad g * W_2(t-1) \\ Nw_2(t-1) > Nw_2^* & : \quad h * W_2(t-1) \end{cases} \tag{3.14}$$

Wie in **Mark-Up I** besteht auch hier der Parametervektor aus 16 Elementen: $(a, a^*, b, b^*, c, c^*, d, d^*, e, e^*, f, f^*, g, g^*, h, h^*)$. Weiterhin wird das Verhalten dieser Regel durch die Startwerte der Prämien und Werbeaufwendungen für die erste Periode bestimmt. Zur Vereinfachung und zur besseren Vergleichbarkeit werden diese Startwerte auch in der zweiten Periode verwendet. Die Anspruchniveauhöhe der Nettowechsler in beiden Sparten: Nw_1^*, Nw_2^* ist ebenfalls festzulegen.

Die Verhaltenshypothese dieser Regel besagt, daß es bei sinkendem Neugeschäft, also negativen Nettowechslern, notwendig ist, die Prämien zu senken, um das Unternehmen für die Nachfrager wieder interessanter zu machen. Ob es günstig ist, bei rückläufigem Neugeschäft die Werbung zu erhöhen um Präferenzen zugunsten des Unternehmens zu beeinflussen, ist in Einzelanalysen zu untersuchen.

Die Verhaltensregel **Mark-Up III** basiert auf dem gleichen Prinzip wie ihre beiden Vorgänger. Ihre Anspruchsniveaugröße ist die Höhe des relativen Marktanteils der Versicherung in der jeweiligen Sparte: M^*. Diese Größe berechnet sich als Quotient aus der Anzahl der Versicherten, die bei dem Anbieter in der Vorperiode versichert waren $Azvn(t-1)$ und der in der Vorperiode insgesamt in der Sparte aktiven Nachfrager $Vn(t-1)$: $M^* = \frac{Azvn(t-1)}{Vn(t-1)}$. Die Höhe der Prämien- und Werbeaufwendungen ergibt sich dann aus:

$$P_1(t) = \begin{cases} M_1(t-1) \leq M_1^* & : \quad a * P_1(t-1) \\ M_1(t-1) > M_1^* & : \quad b * P_1(t-1) \end{cases} \tag{3.15}$$

[23] vgl. Farny (1979), S. 66

$$P_2(t) = \begin{cases} M_2(t-1) \leq M_2^* & : \quad c * P_2(t-1) \\ M_2(t-1) > M_2^* & : \quad d * P_2(t-1) \end{cases} \tag{3.16}$$

$$W_1(t) = \begin{cases} M_1(t-1) \leq M_1^* & : \quad e * W_1(t-1) \\ M_1(t-1) > M_1^* & : \quad f * W_1(t-1) \end{cases} \tag{3.17}$$

$$W_2(t) = \begin{cases} M_2(t-1) \leq M_2^* & : \quad g * W_2(t-1) \\ M_2(t-1) > M_2^* & : \quad h * W_2(t-1) \end{cases} \tag{3.18}$$

Der Parametervektor und die Startwerte entsprechen denen der Regeln **Mark-Up I** und **Mark-Up II**.

Die Verhaltensregel **Erwartungsschaden** hat als Anspruchsniveaugröße die durchschnittliche Schadenhöhe pro Versicherungsfall der Vorperiode. Die Prämie der laufenden Periode wird aus der Prämie der Vorperiode berechnet, die um einen Aufschlag angepaßt wird, falls die Prämie der Vorperiode niedriger war als die durchschnittliche Schadenhöhe der Vorperiode. Die durchschnittlichen Schadenhöhe bezieht sich aber nur auf die Schadenstichprobe der Versicherung und nicht auf die gesamte Sparte. Die durchschnittliche Schadenhöhe der Vorperiode $\bar{S}(t-1)$ berechnet sich als Quotient aus der Schadensumme der Einzelschäden $S(t-1)$, die der Versicherer in der Vorperiode regulieren mußte und der Anzahl der Versicherten in der Vorperiode $Azvn(t-1)$: $\bar{S}(t-1) = \frac{S(t-1)}{Azvn(t-1)}$. Die Höhe der Prämien und Werbeaufwendungen ergibt sich dann aus:

$$P_1(t) = \begin{cases} P_1(t-1) \leq \bar{S}_1(t-1) & : \quad a * P_1(t-1) \\ P_1(t-1) > \bar{S}_1(t-1) & : \quad b * P_1(t-1) \end{cases} \tag{3.19}$$

$$P_2(t) = \begin{cases} P_2(t-1) \leq \bar{S}_2(t-1) & : \quad c * P_2(t-1) \\ P_2(t-1) > \bar{S}_2(t-1) & : \quad d * P_2(t-1) \end{cases} \tag{3.20}$$

$$W_1(t) = \begin{cases} P_1(t-1) \leq \bar{S}_1(t-1) & : \quad e * W_1(t-1) \\ P_1(t-1) > \bar{S}_1(t-1) & : \quad f * W_1(t-1) \end{cases} \tag{3.21}$$

$$W_2(t) = \begin{cases} P_2(t-1) \leq \bar{S}_2(t-1) & : \quad g * W_2(t-1) \\ P_2(t-1) > \bar{S}_2(t-1) & : \quad h * W_2(t-1) \end{cases} \tag{3.22}$$

Diese Form der Prämienreaktion entspricht der "Full Cost Pricing"-Hypothese, die auf ein Versicherungsunternehmen angewendet wird[24]. Die Höhe der Werbeaufwendungen der Versicherung werden aus den Überschüssen aus der Einzelschäden abgeleitet. Der Parametervektor und die Startwerte entsprechen denen der anderen Verhaltensregeln, die auf Eigeninformation basieren.

3.3.1.3 Verhaltensregeln mit Fremdinformation

Während die Verhaltensregeln des letzten Abschnitts dem Verhalten eines Versicherers in einem Markt mit monopolistischer Konkurrenz entsprechen, repräsentieren die Verhaltensregeln mit Fremdinformation die Verhaltensformen für oligopolistische Märkte im

[24] vgl. Wanninger (1981), S. 17

IMS-Modell. Bestimmend für ihre Prämien- und Werbereaktionen sind Informationen über die Reaktionen anderer Anbieter in den Vorperioden. Im Gegensatz zu den klassischen Oligopolansätzen von Cournot, von Stackelberg[25] und der Spieltheorie wird im IMS-Modell die Annahme der vollständigen Information fallengelassen. Eine oligopolistische Verhaltensregel kann aber ihre eigene optimale Prämiehöhe nur festlegen, wenn sie die Prämien aller anderen Oligopolisten kennt. Auf dem IMS-Markt sind zwei Annahmen der klassischen Oligopoltheorie **nicht** getroffen:

1. Nicht alle anderen Marktteilnehmer verhalten sich oligopolistisch und sind sich der oligopolistischen Interdependenz ihrer Entscheidungen bewußt. Auf dem IMS-Markt können gleichzeitig sowohl Versicherer mit Verhaltensregeln auf der Basis von Eigeninformationen als auch Versicherer mit oligopolistischen Verhaltensregeln im Wettbewerb stehen.

2. Weil die Nachfrage für den einzelnen Anbieter stochastisch ist und er auf die nachgefragte Menge immer nur reagieren kann, wird die Annahme der statischen Gewinnmaximierung durch die Annahme der dynamischen Anspruchsmotivierung ersetzt.

War bei den Verhaltensregeln auf der Basis von Eigeninformation der Satisficing-Ansatz die Lösung des Verhaltensproblems bei unvollständiger Information und beschränkter Rationalität, so ist der Adaptionsansatz die Lösung bei Verhaltensregeln auf der Basis von Fremdinformationen. "Adaptives Verhalten der Oligopolisten bedeutet hiernach, daß sie sich (in einem zu präzisierenden Sinn) dem Verhalten der Mitkonkurrenten anpassen."[26] Auf der Basis unvollständiger Information ist die Annahme, daß sich ein Anbieter genauso oder ähnlich wie ein oder mehrere andere Anbieter verhält, realistisch. Im IMS-Modell stehen dem einzelnen Versicherer dazu drei Möglichkeiten zur Verfügung:

1. Prämiendumping,

2. Durchschnittsverhalten und

3. direkter Angriff des Marktführers.

Die erste Verhaltensregel **Dumping** basiert auf der Annahme, daß auf einem Markt mit wenig informierten Nachfragern ein Anbieter den Preiskampf in einer Sparte durch eine Hochprämienstrategie in der anderen Sparte ungestraft finanzieren kann und damit eine Quersubventionierung zwischen den Sparten stattfinden kann. Diese oligopolistische Vehaltensregel richtet sich nicht gegen einen bestimmtem Versicherer (z.B. Anbieter Nr. 3), sondern gegen den Anbieter mit der niedrigsten Prämie in der Vorperiode $P^{min}(t-1)$. Diese Prämie kann dann unterboten, gleichzeitig das Ziel des langfristigen Überlebens durch ein Überbieten der Prämie in der zweiten Sparte gesichert werden. Diese aggressive

[25] vgl. 3.2.1.

[26] Klopstech (1980), S. 23

Prämienpolitik wird durch eine aggressive Werbung unterstützt. Bei der Höhe der Werbeausgaben orientiert sich diese Verhaltensregel an den maximalen Werbeausgaben einer Sparte in der Vorperiode: $W^{max}(t-1)$. Die Höhe der Prämien und Werbeaufwendungen ergibt sich dann aus:

$$P_1(t) = a + b * P_1^{min}(t-1) \tag{3.23}$$

$$P_2(t) = c + d * P_2^{min}(t-1) \tag{3.24}$$

$$W_1(t) = e + f * W_1^{max}(t-1) \tag{3.25}$$

$$W_2(t) = g + h * W_2^{max}(t-1) \tag{3.26}$$

Der Parametervektor und die benötigten Startwerte entsprechen den Anforderungen der Verhaltensregeln mit Eigeninformation.

Die Verhaltensregel **Durchschnitt** besitzt das gleiche Reaktionsmuster wie die Regel **Dumping**, jedoch mit dem Unterschied, daß sie die durchschnittlichen Prämien $\bar{P}(t-1)$ und die durchschnittlichen Werbeaufwendungen $\bar{W}(t-1)$ der gesamten Sparte in der Vorperiode als Fremdinformation benötigt. Die Höhe der Prämien und Werbeaufwendungen ergibt sich dann aus:

$$P_1(t) = a + b * \bar{P}_1(t-1) \tag{3.27}$$

$$P_2(t) = c + d * \bar{P}_2(t-1) \tag{3.28}$$

$$W_1(t) = e + f * \bar{W}_1(t-1) \tag{3.29}$$

$$W_2(t) = g + h * \bar{W}_2(t-1) \tag{3.30}$$

Mit dieser Verhaltensregel kann untersucht werden, ob auf einem Markt mit Schocks und aggressiveren Oligopolregeln das Durchschnittsverhalten immer von Vorteil ist oder ob auch für den IMS-Markt die alte Unternehmensregel gilt: "Never stuck in the middle!".

In der dritten und letzten Verhaltensregel mit Fremdinformation wird kein passives, adaptives Verhalten modelliert, sondern eine aktive Form. Die Regel **Angriff** adaptiert in jeder Periode die Prämien- und Werbehöhe des Marktführers in der Vorperiode: $P^{MF}(t-1), W^{MF}(t-1)$. Der Marktführer einer Sparte im IMS-Modell ist die Versicherung mit den höchsten Reserven, weil die Reserven als Indikator für die langfristige Überlebensfähigkeit des Unternehmens gelten. Die Höhe der Prämien und Werbeaufwendungen ergibt sich somit aus:

$$P_1(t) = a + b * P_1^{MF}(t-1) \tag{3.31}$$

$$P_2(t) = c + d * P_2^{MF}(t-1) \tag{3.32}$$

$$W_1(t) = e + f * W_1^{MF}(t-1) \tag{3.33}$$

$$W_2(t) = g + h * W_2^{MF}(t-1) \tag{3.34}$$

Die Anforderungen an den Parametervektor dieser Verhaltensregel und die Startwerte entsprechen denen der anderen Verhaltensregeln mit Fremdinformation.

Bei der Analyse der Auswirkungen von Schocks wird, neben der Messung an einzelnen Verhaltensregeln, auch der Erfolg einer ganzen Regelart zu untersuchen sein. Es ist zu

bestimmen, ob Verhaltensregeln mit Eigeninformation bei Aktivierungsschocks auf der Nachfragerseite generell schlechter abschneiden als Verhaltensregeln mit Fremdinformation. Weiterhin ist von Interesse, ob bei einem Aktivierungsschock auf der Anbieterseite Regeln mit Fremdinformation auch nach dem Schock bessere Ergebnisse erzielen als Regeln mit Eigeninformation.

3.3.2 Die Verhaltensregeln der Nachfrager

3.3.2.1 Informationsfreie Verhaltensregeln

Die Verhaltensregeln der Nachfrager im IMS-Modellen bestehen aus drei Teilen:

1. Dem Schadenmodell,
2. der Statuswahlfunktion und
3. der Anbieterwahlfunktion.

Die Bestimmung der Schadenhöhe ist bei jedem Versicherungsnachfrager gleich[27]:

$$\begin{aligned} S_1(t) &= a + b * \mu_1 \\ S_2(t) &= c + d * \mu_2 \end{aligned} \tag{3.35}$$

μ_1 und μ_2 sind standardnormalverteilte Pseudo-Zufallszahlen. Neben der Bestimmung der Schadenhöhe, wird die Eintrittswahrscheinlichkeit in den beiden Risiken $w_1(t), w_2(t)$ durch zwei Parameter festgelegt.

$$w_1(t) = \begin{cases} P_1(t) \leq \mu_3 & : \ 0 \\ P_1(t) > \mu_3 & : \ 1 \end{cases} \tag{3.36}$$

$$w_2(t) = \begin{cases} P_2(t) \leq \mu_4 & : \ 0 \\ P_2(t) > \mu_4 & : \ 1 \end{cases} \tag{3.37}$$

Wenn $w_1(t) = 1$, dann hat der Versicherungsnehmer in der Periode t einen Schaden in der Höhe von $S_1(t)$. $P_1(t)$ und $P_2(t)$ sind die individuellen Schwellenwerte der Schadeneintrittswahrscheinlichkeiten der Periode eines Nachfragers. Ist dieser Schwellenwert größer als eine normalverteilte Zufallsvariable, dann wird $w(t)$ auf Eins gesetzt, und in einem zweiten Schritt die Schadenhöhe bestimmt.

Die Parameter der Schadenfunktionen können für jeder Nachfrager frei festgelegt werden. $\mu_1 \ldots \mu_4$ sind normalverteilte Zufallsvariablen, die durch einen Pseudo-Zufallszahlengenerator erzeugt werden: $\mu_1 \ldots \mu_4 \sim N(0,1)$. Die Schockreaktion ist die gleiche wie bei den

[27] Die Nicht-Negativität der Schadenhöhe wird durch die programmtechnische Stutzung der Normalverteilung erreicht. Die Parameter der Schadenhöhenfunktion werden bei der Eingabe auf Plausibilität geprüft.

Anbietern. Hat ein Schock stattgefunden, wird das schockempfindliche Element der Informationsmenge K von 0 auf 1 geändert. Diese Änderung führt zu einem Wechsel aller Parameter dieser Verhaltensregel:

$$X = \begin{cases} K = 0 & : \quad X^0 \\ K = 1 & : \quad X^* \end{cases} \tag{3.38}$$

Jeder Versicherungsnehmer muß in jeder Periode für beide Risiken entscheiden, ob er einen eventuellen Schaden selbst trägt oder ihn gegen die Zahlung einer Prämie von einer Versicherung tragen läßt. Der Versicherungsstatus $V(t)$ kann nur die Werte Null und Eins annehmen:

$$V(t) = \begin{cases} 0 & : \quad Eigenversicherung \\ 1 & : \quad Fremdversicherung \end{cases} \tag{3.39}$$

Während das Verhalten der Versicherer auf das langfristige Überleben des Unternehmens ausgerichtet ist, verhalten sich die Versicherungsnehmer vermögensoriertiert. Bei Eigenversicherung wird das Vermögen durch eventuelle Schäden in der Periode gemindert, bei Fremdversicherung durch die in t vom Versicherer verlangte Prämie.

Der dritte Teil der Verhaltensregel eines IMS-Nachfragers ist die Anbieterwahlfunktion: $A(t)$. Diese Funktion wählt, falls der Nachfrager sich für die Fremdversicherung entscheidet, in der Periode t aus den in dieser Periode aktiven Anbietern auf der Basis der Informationsmenge des Nachfragers einen Versicherer aus. Der Wert von $A(t)$ ist somit der Index oder die Nummer des Versicherers, an den der Nachfrager seine Prämie überweist und der im Bedarfsfall seine Schäden reguliert. Wenn $A(t) = 0$ ist, dann hat der Nachfrager keinen Anbieter in t gewählt.

Die Verhaltensregel **Zufall I** ist eine informationsfreie Verhaltensregel. Die Statusfunktionen entscheiden sich in jeder Periode für Fremdversicherung. Dieses Verhalten entspricht somit einer Versicherungspflicht.

$$\begin{aligned} V_1(t) &= 1 \\ V_2(t) &= 1 \end{aligned} \tag{3.40}$$

Die Anbieterwahlfunktion von **Zufall I** wählt in jeder Periode aus den aktiven Versicherern zufällig einen aus. Die Zufallsvariable werden von einem Pseudo-Zufallsgenerator gleichverteilt erzeugt: $\mu_5, \mu_6 \sim U(0, maxvu)$.

$$\begin{aligned} A_1(t) &= \mu_5 \\ A_2(t) &= \mu_6 \end{aligned} \tag{3.41}$$

Ist die gewählte Versicherung in t nicht aktiv, dann wird die Zufallswahl solange wiederholt, bis eine aktive Versicherung gefunden ist. Der Parametervektor dieser Verhaltensregel besteht nur aus den Parametern der Schadenfunktion und deren Werte nach einem Änderungsschock: $(a, a^*, b, b^*, c, c^*, d, d^*)$.

Die Verhaltensregel **Zufall II** ist zweistufig ausgelegt. Erst nachdem der Status festgelegt ist, entscheidet sich, ob der Nachfrager einen Versicherer wählen muß. Entscheidet er

sich für die Eigenversicherung, dann ist $A(t) = 0$. Dazu wird im Gegensatz zu **Zufall I** keine Versicherungspflicht modelliert, sondern durch eine Zufallszahl entschieden, ob Eigen- oder Fremdversicherung gewählt wird.

$$V_1(t) = \begin{cases} e \leq \mu_3 & : \ 0 \\ e > \mu_3 & : \ 1 \end{cases} \tag{3.42}$$

$$V_2(t) = \begin{cases} f \leq \mu_4 & : \ 0 \\ f > \mu_4 & : \ 1 \end{cases} \tag{3.43}$$

μ_3, μ_4 sind im Intervall $[0 \ldots 1]$ gleichverteile Pseudo-Zufallszahlen. Die Parameter e und f legen eine maximale Schadeneintrittswahrscheinlichkeit fest, bis zu der ein Nachfrager bereit ist, die eintretenden Schäden selbst zu tragen. Sind e und f 0.5, dann werden Fremd- und Eigenversicherung in beiden Risiken gleichhäufig gewählt. Ist e größer als 0.5, dann wird das Ereignis $e > \mu_3$ seltener eintreten und damit im Risiko 1 seltener die Fremdversicherung und häufiger die Eigenversicherung gewählt werden.

Die Anbieterwahl erfolgt nun in Abhängigkeit von der Statusentscheidung zufällig:

$$A_1(t) = \begin{cases} V_1(t) = 0 & : \ 0 \\ V_1(t) = 1 & : \ \mu_5 \end{cases} \tag{3.44}$$

$$A_2(t) = \begin{cases} V_2(t) = 0 & : \ 0 \\ V_2(t) = 1 & : \ \mu_6 \end{cases} \tag{3.45}$$

Wie auch bei **Zufall I** wird mittels einer gleichverteilten Pseudo-Zufallszahl aus dem Intervall der möglichen Anbieternummern: $[1 \ldots maxvu]$ solange Nummern gezogen, bis ein aktiver Anbieter gefunden wurde. Der Parametervektor von **Zufall II** umfaßt die zwölf Elemente: $(a, a^*, b, b^*, c, c^*, d, d^*, e, e^*, f, f^*)$.

Die Verhaltensregeln **Zufall I** und **Zufall II** stellen im Rahmen des IMS-Modells die Referenz für die beschränkte Rationalität der Nachfrager dar[28]. Sie können für Referenznachfrager stehen, die sich in einer neuen Marktumgebung ohne Vorinformationen versichern wollen. Die einzig rationale Entscheidung aus ihrer Sicht ist die Zufallsentscheidung.

3.3.2.2 Verhaltensregeln mit Eigeninformation

Die zweite Klasse der Verhaltensregeln der Nachfragerseite basiert auf den Eigeninformationen der Versicherungsnehmer. Drei Informationen werden dabei verwendet:

1. Die subjektive Wahrscheinlichkeit des Nachfragers in der Periode t einen Schaden zu haben.

2. Die Präferenzeinschätzung für die Anbieter.

[28] vgl. Fischer (1988), S. 31

3. Die Prämien, die in jeder Vorperiode an einen Versicherer gezahlt wurden.

Die subjektive Wahrscheinlichkeit ist ein Vektor $P(t)$, dem in jeder Periode ein Wert zwischen Null und Eins zugeordnet wird. Gleichzeitig stellt er die individuelle Schadeneintrittswahrscheinlichkeit eines Risikos dar. Diese Zahl stellt die individuelle Einschätzung des Versicherungsnachfragers bezüglich seines Risikos dar. Ist der Wert höher als eine bestimmte Sicherheitsgrenze, dann wählt der Versicherungsnehmer die Fremdversicherung. Die subjektive Wahrscheinlichkeit steuert bei beiden Risiken nur die Statusentscheidung. Die Anbieterwahl greift entweder auf die Präferenzen oder auf die eigenen historischen Prämieninformationen zu.

$$V_1(t) = \begin{cases} P_1(t) \leq e & : \ 0 \\ P_1(t) > e & : \ 1 \end{cases} \tag{3.46}$$

$$V_2(t) = \begin{cases} P_2(t) \leq f & : \ 0 \\ P_2(t) > f & : \ 1 \end{cases} \tag{3.47}$$

Die Mindestsicherheitsniveaus e, f können durch einen Änderungsschock beeinflußt werden.

Das Anbieterwahlverhalten wird durch die Präferenzen des Nachfragers bestimmt. Die Präferenzen wiederum hängen von der Höhe der Werbeaufwendungen der einzelnen Versicherer ab. Die Präferenzinformationen stehen dem Nachfrager kostenlos zur Verfügung. Es wird angenommen, daß der Nachfrager im IMS-Modell der Werbung, wenn er sie wahrnimmt, vollkommen vertraut. Deshalb wählt er den Anbieter, für den er die maximalen Präferenzen hat, also den Versicherer, der die meiste Werbung betreibt. Die Präferenzen des Versicherungsnehmers sind im Vektor $F(t)$ gespeichert und werden in jeder Runde auf der Basis der Werbeausgaben der Vorperiode neu berechnet. In Periode 1 kann deshalb noch keine Präferenzwahl stattfinden, darum findet dort eine Zufallsauswahl statt. Gibt es in späteren Perioden mehrere Versicherer mit gleich hohen Präferenzen, dann wird unter ihnen eine Zufallsauswahl durchgeführt. $max(F(t))$ ist das Maximum des Präferenzvektors, $A(max(F(t)))$ ist die Anbieternummer oder Index des Versicherers, für den der Nachfrager die höchsten Präferenzen hat. Dieser Anbieter wird gewählt:

$$A_1(t) = \begin{cases} V_1(t) = 0 & : \ 0 \\ V_1(t) = 1 & : \ A(max(F_1(t))) \end{cases} \tag{3.48}$$

$$A_2(t) = \begin{cases} V_2(t) = 0 & : \ 0 \\ V_2(t) = 1 & : \ A(max(F_2(t))) \end{cases} \tag{3.49}$$

Die Regel **Totale Erinnerung** besitzt die gleiche Schadenfunktion wie alle Nachfragerregeln und die gleiche Statusfunktion wie die Verhaltensregel **Präferenz**. Im Gegensatz zu **Präferenz** läßt sie sich aber nicht durch Werbung beeindrucken, sondern führt über alle in den Vorperioden gezahlten Prämien eine Kartei: Pr^-. Sie wählt aus den ihr bekannten Versicherern denjenigen aus, der ihr die bislang geringste Prämie abverlangt hat, gleichgültig wie lange das schon her ist. Die Verhaltensannahme hinter dieser Regel ist,

daß sich die Prämien eher langsam ändern und das ein billiger Versicherer über längere Zeit ein billiger Versicherer bleibt. $min(Pr^-)$ ist das Minimum der bislang vom Nachfrager verlangten Prämien. $A(min(Pr^-))$ ist die Anbieternummer dieses Versicherers.

$$A_1(t) = \begin{cases} V_1(t) = 0 & : \quad 0 \\ V_1(t) = 1 & : \quad A(min(Pr_1^-)) \end{cases} \tag{3.50}$$

$$A_2(t) = \begin{cases} V_2(t) = 0 & : \quad 0 \\ V_2(t) = 1 & : \quad A(min(Pr_2^-) \end{cases} \tag{3.51}$$

Der Parametervektor dieser Verhaltensregel enthält nur die Parameter des Schadenmodells.

3.3.2.3 Verhaltensregeln mit Fremdinformation

Wie bei den Anbietern, basiert die dritte Art von Verhaltensregeln der Nachfrager auf Fremdinformationen. Im Gegensatz zu den Versicherern, die die Informationen über ihre Konkurrenten kostenfrei beziehen, werden bei den Versicherungsnachfragern Informationskosten angenommen. Die Kostensätze sind für den einzelnen Nachfrager exogen. Teilinformationen sind nicht verfügbar. Die Kosten für eine Information werden vom Vermögen des Nachfragers abgezogen. Drei Fremdinformationen werden verwendet:

1. Der durchschnittliche Versicherungsgrad eines Risikos in der Vorperiode,
2. eine Stichprobe aus den Prämien aller Anbieter in der Vorperiode und
3. die kleinste von einem Versicherer in der Vorperiode verlangte Prämie.

Der durchschnittliche Versicherungsgrad $\bar{G}(t-1)$ eines Risikos ist der Prozentsatz von Nachfragern, die sich in der Vorperiode für die Fremdversicherung entschieden haben. Diese Information ist kostenlos verfügbar und wird in den beiden Verhaltensregeln mit Fremdinformation für die Entscheidung über den Versicherungsstatus benötigt.

Die Auswahlfunktion der Verhaltensregel **Suche** basiert auf einer nicht-sequentiellen Suchregel. Der Nachfrager zieht aus den Prämieninformationen der Periode in einer Sparte eine Stichprobe, deren Umfang durch einen Verhaltensparameter bestimmt wird: $H_g(Pr_1(t))$ ist eine Stichprobe mit dem Umfang g aus den Prämien aller Versicherer der Sparte 1 in der Periode t. Der Nachfrager bestimmt aus dieser Stichprobe das Minimum: $min(H_g(Pr_1(t)))$ und in einem zweiten Schritt die Nummer des Anbieters, der diese Prämie verlangt: $A(min(H_g(Pr_1(t))))$. Für die Verhaltensregel **Suche** ergeben sich danach folgende Status- und Anbieterwahlfunktionen:

$$V_1(t) = \begin{cases} \bar{G}_1(t-1) \leq e & : \quad 0 \\ \bar{G}_1(t-1) > e & : \quad 1 \end{cases} \tag{3.52}$$

$$V_2(t) = \begin{cases} \bar{G}_2(t-1) \leq f & : \ 0 \\ \bar{G}_2(t-1) > f & : \ 1 \end{cases} \tag{3.53}$$

$$A_1(t) = \begin{cases} V_1(t) = 0 & : \ 0 \\ V_1(t) = 1 & : \ A(min(H_g(Pr_1(t)))) \end{cases} \tag{3.54}$$

$$A_2(t) = \begin{cases} V_2(t) = 0 & : \ 0 \\ V_2(t) = 1 & : \ A(min(H_h(Pr_2(t)))) \end{cases} \tag{3.55}$$

Der Parametervektor dieser Verhaltensregel hat 12 Elemente, die das Schadenmodell spezifizieren und vier Parameter, die die Stichprobenumfänge der Suche vor und nach einem Änderungsschock festlegen: $(a, a^*, b, b^*, c, c^*, d, d^*, e, e^*, f, f^*, g, g^*, h, h^*)$. Da in der ersten Periode einer Simulation keine Stichprobenziehung möglich ist, werden Startwerte für Status und Anbieter benutzt.

Die Verhaltensregel **Beste Information** besitzt die gleiche Statusfunktion wie die Regel **Suche**. Im Gegensatz zu ihr greift **Beste Information** aber nicht auf eine Stichprobe von Prämien der Vorperiode zu, sondern wählt immer den Anbieter, der in der Periode die niedrigste Prämie verlangt hat: $A(Pr^{min}(t))$. Der Preis für diese Information ist das Produkt aus den Kosten pro Suchschritt und der Anzahl der in t aktiven Versicherer. Als Auswahlfunktion ergibt sich somit:

$$A_1(t) = \begin{cases} V_1(t) = 0 & : \ 0 \\ V_1(t) = 1 & : \ A(Pr_1^{min}(t)) \end{cases} \tag{3.56}$$

$$A_2(t) = \begin{cases} V_2(t) = 0 & : \ 0 \\ V_2(t) = 1 & : \ A(Pr_2^{min}(t)) \end{cases} \tag{3.57}$$

Die Regel **Beste Information** kommt im IMS-Modell dem idealen, vollständig informierten Nachfrager der klassischen Gleichgewichtstheorie am nähsten.

Die Modellspezifikation ist in folgender Tabelle zusammengefaßt dargestellt:

Marktteilnehmer	**Verhaltensregeln**	
200 Versicherungsnehmer	Versicherer	Versicherungsnehmer
25 Versicherungen	Zufall I	Zufall I
Beliebige Regelzuordnung	Zufall II	Zufall II
Beliebige Parameterwahl	Mark-Up I	Präferenz
	Mark-Up II	Totale Erinnerung
	Mark-Up III	Suche
	Erwartungsschaden	Beste Information
	Dumping	
	Durchschnitt	
	Angriff	
Schocks	**Simulation**	
Aktivierungsschocks	Marktprozeß wird in diskreten Perioden simuliert	
Änderungsschocks	Maximal 100 Perioden pro Simulationslauf	
Indirekte Schocks	Maximal 100 Wiederholungen pro Simulation	

Tabelle 3.1: Synopsis der Modellspezifikation

Kapitel 4

Das Programm

4.1 Ziele des Kapitels

Das vierte Kapitel hat zwei Ziele: die Beschreibung der Programmbedienung und die Erläuterung des Quelltextaufbaus. Der erste Teil beschreibt die Benutzerschnittstelle und die Auswertungsmöglichkeiten und ist an den Programmanwender gerichtet. Dabei wird das Hauptmenü des Programms als Ausgangspunkt für einen typischen Anwendungszyklus gewählt und dabei die wichtigsten Elemente der Programmbedienung und der anschließenden Datenauswertung erläutert.

Der zweite Abschnitt befaßt sich mit den einzelnen Programmteilen und dem Aufbau des Quelltextes. Er ist auf das tiefere Verständnis und die Programmweiterentwicklung ausgerichtet. Voraussetzung dafür sind grundlegende Kenntnisse der höheren Programmiersprache C. Die IMS-spezifischen Erweiterungen der Syntaxmenge um die ESL-Befehle werden beschrieben. Dabei wird auch auf Probleme der Programmentwicklung mit ESL eingegangen. Daran schließt sich eine Beschreibung der beiden Hauptkomponenten des Programms an:

- Die Datenstrukturen und
- die Unterprogramme.

Die Umsetzung der Modellvariablen in Datenstrukturen ist die Grundlage der gesamten Programmentwicklung. Fehler bei der Modellierung der Datenstrukturen können weitreichende Auswirkungen auf die daran anschließende Formulierung der Unterprogramme haben.

4.2 Programmbedienung

4.2.1 Das Hauptmenü

Über das Hauptmenü von IMS wird der gesamte Simulationsablauf gesteuert. IMS verfügt über keine grafische Benutzeroberfläche. Die Befehle zur Steuerung des Bildschirm setzen ein ANSI-Terminal oder dessen Emulation voraus. Diese Wahl beschränkt zwar die Gestaltungsmöglichkeiten der Benutzeroberfläche, erhöht aber die Portabilität des Programm erheblich. Auf nahezu allen gängigen Betriebssytemen sind ANSI-Emulationen verfügbar, so daß die Umsetzung des Programms auf andere Betriebssysteme nicht an inkompatiblen Befehlen der Benutzerschnittstelle scheitern kann.

Das Hauptmenü bietet sieben Wahlmöglichkeiten:

1. Modell laden
2. Modell speichern
3. Modell auswählen und bearbeiten
4. Modell simulieren
5. Modellergebnis analysieren
6. Programminformation
7. Programm beenden

Mit *Modell laden* wird ein bereits gespeichertes IMS-Modell wieder in den Hauptspeicher geladen. Danach kann es sofort simuliert werden. Der zweite Auswahlpunkt überträgt das im Hauptspeicher definierte Modell auf einen externen Massenspeicher. Dieser Punkt ist besonders wichtig, wenn der Anwender ein eigenes Modell parametrisiert hat. Der sechste Programmpunkt zeigt einige Programminformationen und mit der Wahl des siebten Menüpunkts wird das Programm verlassen. Die eigentliche Simulation wird mit dem vierten Auswahlpunkt gestartet.

Ein typischer Anwendungslauf mit IMS kann zwei Ausgangspunkte haben: Ein vordefiniertes Modell oder ein durch den Anwender definiertes Modell. Aufgrund des hohen Aufwands sind sechs Modelle bereits vordefiniert. Nach der Wahl des Punktes *[3] Modell auswählen und bearbeiten* erhält der Benutzer die Möglichkeit eines dieser sechs Modells auszuwählen, ein eigenes Modell zu definieren oder zum Hauptmenü zurückzukehren. Da die Definition eines großen IMS-Modells bis zu 5700 Parametereingaben erfordern kann, sind die vordefinierten Modelle aus Gründen der Zeitersparnis programmiert worden. Der zweite Ausgangspunkt eines typischen Anwendungslaufs ist ein Modell selbst zu definieren.

Die vordefinierten Modelle zeigen einen Ausschnitt des Anwendungsspektrums von IMS. Es sind sowohl Modelle mit als auch ohne Schocks vordefiniert. Die Modelle sind bezüglich ihrer Komplexität aufsteigend angeordnet. Im Rahmen dieser Arbeit haben sie zwei Aufgaben erfüllt:

- Das Kennenlernen der Modelldynamik zu unterstützen und
- das Modell zu kalibrieren.

Besonders der letzte Punkt ist hervorzuheben, weil ohne Kalibrierung die Schockanalyse durch "explodierende" Modelle und arithmetische Überläufe überdeckt oder erschwert worden wäre.

Nachdem eines der vordefinierten Modelle ausgewählt und ein Informationsbildschirm die Modellkurzbeschreibung gezeigt hat, befindet man sich wieder im Hauptmenü. Da nun alle Modellparameter festliegen, kann die Simulation gestartet werden. Dazu wird der Menüpunkt *Modell simulieren* gewählt. Je nachdem, ob das vordefinierte Modell für wiederholte Simulationsläufe parametrisiert worden ist, werden dann ein oder zwei Parameter abgefragt:

- Die Anzahl der Perioden innerhalb eines Simulationslaufs und
- die Anzahl der Simulationsläufe.

Nach der Eingabe dieser beiden Parameter startet die Simulation und ihr Fortgang wird auf dem Bildschirm angezeigt. Nachdem alle Perioden berechnet wurden, liegen die Simulationsdaten im Startverzeichnis von IMS vor. Die Auswertung dieser Daten kann nun entweder programmintern oder aber mit externen Programmen erfolgen. Falls die weitere Auswertung mit externen Programmen erfolgen soll, dann kann das Programm über den letzten Punkt des Hauptmenüs an dieser Stelle verlassen werden.

4.2.2 Die Modellkonfiguration

Der zweite mögliche Ausgangspunkt für einen typischen Anwendungslauf ist ein selbstdefiniertes Modell. Dazu wird nach dem Punkt *Modell auswählen und bearbeiten* das Menü *Modell erstellen* gewählt. Danach sind vier Auswahlmöglichkeiten gegeben:

1. Versicherer konfigurieren
2. Versicherungsnehmer konfigurieren
3. Allgemeine Marktparameter konfigurieren
4. Zum Menü [Marktkonfiguration bearbeiten]

Nach der Wahl des ersten Menüpunktes erscheint die folgende Eingabemaske:

```
************************[ Versicherer konfigurieren ]*************************
*        Versicherer          :[  12  ]*         Startwerte d. Sparten:       *
*******************************************************************************
*        VR[Mark-Up III      ]:[      ]*         Praemie, Sparte 1    : 40.0  *
*        Verhaltensregelklasse:   2    *         Werbung, Sparte 1    : 0.0   *
*        Aggregatdatei(0=nein):   0    *         Praemie, Sparte 2    : 40.0  *
*                                      *         Werbung, Sparte 2    : 0.0   *
*******************************************************************************
*        Aktivierungsschockparameter:  *         Anspruchsniveaus     :       *
*******************************************************************************
*        Aktivierungslauf     :   1    *         Fuer Sparte 1        : 0.06  *
*        Aktivierungsperiode  :   1    *         Fuer Sparte 2        : 0.06  *
*                                      *                                      *
*                                      *                                      *
*                                      *                                      *
*******************************************************************************
*        Vor Aenderungsschock :        *         Nach Aenderungsschock:       *
*******************************************************************************
*        a  : 1.01         e  : 0.0    *         a* : 0.0          e* : 0.0   *
*        b  : 0.99         f  : 0.0    *         b* : 0.0          f* : 0.0   *
*        c  : 1.02         g  : 0.0    *         c* : 0.0          g* : 0.0   *
*        d  : 0.98         h  : 0.0    *         d* : 0.0          h* : 0.0   *
*                                      *                                      *
*******************************************************************************
```

Abbildung 4.1: Beispiel der Eingabemaske für einen Versicherer

Zunächst wird die Nummer des Versicherers abgefragt, dessen Parameter eingegeben werden sollen. Danach werden die bisherigen Parameterwerte dieser Versicherung angezeigt. Der Anwender kann nun nacheinander alle Parameter, deren Anzeigefeld mit eckigen Klammern überschrieben werden, neu setzen. Zuerst wird die Verhaltensregel abgefragt, die diesem Versicherer zugewiesen werden soll. Wird eine Aggregatdatei dieses Anbieters gewünscht, dann ist der entsprechende Parameter auf Eins zu setzen. Danach werden die Startwerte der beiden Sparten abgefragt. Bei den Aktivierungsschockparametern wird die Aktivierungsperiode und der Aktivierungslauf eingegeben. Ist die Aktivierungsperiode Eins, dann wird der Versicherer von Beginn der Simulation an aktiv. Soll er dagegen einen Aktivierungsschock auslösen, dann muß dieser Parameter größer als Eins gewählt werden. Falls der Versicherung eine der **Mark-Up**-Regeln zugewiesen wurde, müssen im vierten Eingabefenster die Anspruchsniveaus der beiden Sparten festgelegt werden. Die beiden unteren Eingabefenster dienen der Festlegung des Parametervektors der Versicherung. Das linke Fenster umfaßt alle Parameter vor einen Änderungsschock, das rechte Fenster alle Parameter, die nach einen Änderungsschock gelten sollen.

Der Menüpunkt *Versicherungsnehmer konfigurieren* öffnet eine Dialogmaske, die wie die Eingabemaske der Versicherer aufgebaut ist:

```
********************[ Versicherungsnehmer konfigurieren ]**********************
*        Versicherungsnehmer  :[  99  ]*         Startwerte d. Risiken:       *
*******************************************************************************
*        VR[ Zufall II       ]:[      ]*         Status     , Risiko 1: 0     *
*        Verhaltensregelklasse:   1    *         Versicherer, Risiko 1: 0     *
*        Aggregatdatei(0=nein):   1    *         Status     , Risiko 2: 0     *
*                                      *         Versicherer, Risiko 2: 0     *
*******************************************************************************
*        Aktivierungsschockparameter:  *         Anfangsausstattung   :       *
*******************************************************************************
*        Aktivierungslauf     :   1    *         Gesamtvermoegen      : 0.0   *
*        Aktivierungsperiode  :   1    *                                      *
*                                      *                                      *
*                                      *                                      *
*                                      *                                      *
*******************************************************************************
*        Vor Aenderungsschock :        *         Nach Aenderungsschock:       *
*******************************************************************************
*        a  : 40.00        e  : 0.0    *         a* : 50.00        e* : 0.0   *
*        b  : 5.00         f  : 0.0    *         b* : 10.00        f* : 0.0   *
*        c  : 10.00        g  : 0.0    *         c* : 30.00        g* : 0.0   *
*        d  : 1.00         h  : 0.0    *         d* : 20.00        h* : 0.0   *
*                                      *                                      *
*******************************************************************************
```

Abbildung 4.2: Beispiel der Eingabemaske für einen Versicherungsnehmer

Der einzige wesentliche Unterschied zur Eingabemaske der Versicherer ist im Eingabefenster der Anfangsausstattung zu finden. Hier wird der Startwert des Gesamtvermögens abgefragt, aus dem die Prämien und eigenversicherten Schäden bezahlt werden.

Im Untermenü *Allgemeine Marktparameter konfigurieren* werden fünf Parameter abgefragt:

1. Die Periode, in der ein Änderungsschock stattfinden soll,
2. der Simulationslauf, ab dem Änderungsschocks stattfinden sollen,
3. die Kosten pro Prämieninformation für die Nachfragerregeln **Suche** und **Beste Information**,
4. der Periodenzinssatz für die Verzinsung der Reserven der Versicherer und
5. der Parameter, um Wiederholungen eines Simulationslaufs zuzulassen.

Der erste Wert legt fest, in welcher Periode innerhalb eines Simulationslaufs ein Änderungsschock stattfindet. Soll ein Modell wiederholt simuliert werden, dann kann zusätzlich bestimmt werden, ab welchem Lauf die Änderungsschocks in der gewählten Periode stattfinden sollen. Die Parameter drei und vier sind allgemeine Modellparameter. Der letzte Wert muß auf Eins gesetzt werden, wenn ein Modell wiederholt simuliert werden soll.

4.2.3 Die Modellauswertung

Der letzte und wichtigste Schritt nach dem Berechnungsvorgang der Simulation ist die Modellauswertung. Grundlage diese Analyse bilden die von IMS erzeugten Simulationsrohdaten. Die Daten sind in Aggregatdateien zusammengefaßt und besitzen eine einheitliche Namenskonvention. Der Name einer Aggregatdatei besteht aus 12 Zeichen: *imsxxxxx.dat*. Die ersten drei und die letzten vier Zeichen sind bei allen Aggregatdateien identisch. An den fünf inneren Zeichen kann der Inhalt der Aggregatdatei identifiziert werden. Für Versicherer und Versicherungsnehmer sind jeweils vier verschiedene Typen von Aggregatdateien definiert:

1. Alle Daten eines einzelnen Versicherers oder Versicherungsnehmers: *imsvu001.dat* bis *imsvu025.dat* für die Versicherer und *imsvn001.dat* bis *imsvn200.dat* für die Versicherungsnehmer,

2. alle Daten der Versicherer oder Versicherungsnehmer mit der gleichen Verhaltensregel: *imsvur01.dat* bis *imsvur10.dat* für die Versicherer und *imsvnr01.dat* bis *imsvnr07.dat* für die Versicherungsnehmer,

3. alle Daten der Versicherer oder Versicherungsnehmer in der gleichen Verhaltensregelklasse:

   - Informationsfreie Verhaltensregeln,
   - Verhaltensregeln mit Eigeninformation und
   - Verhalensregeln mit Fremdinformation.

   imsvuvk1.dat bis *imsvuvk3.dat* für die Versicherer und *imsvnvk1.dat* bis *imsvnvk3.dat* für die Versicherungsnehmer und

4. alle Versicherer oder Versicherungsnehmer: *imsvusk1.dat* für die Versicherer und *imsvnsk1.dat* für die Versicherungsnehmer.

Werden alle Aggregatdateien auf allen Stufen angefordert, dann generiert IMS 250 Dateien. Hinzu kommt noch die Datei *imsrepor.dat*, die allgemeine Protokolleinträge aufnimmt und immer erzeugt wird. In den Dateien sind die Attribute der jeweiligen Aggregatstufe in Spalten angeordnet. Die linke Spalte in jeder Aggregatdatei beinhaltet die Periodennummer. Die simulierten Perioden sind fortlaufend nummeriert. Wird ein Simulationslauf wiederholt, setzt sich die Nummerierung fort und beginnt nicht wieder bei Eins. Die erste Zeile jeder Aggregatdatei enthält einen Kopf, der den Inhalt der jeweiligen Sparte beschreibt. Diese Namen können von externen Auswertungsprogrammen als Variablenbezeicher genutzt werden.

```
#t   Pr1   Wa1    Rs1       Vn1 Sa1 Sh1     Pr2   Wa2    Rs2       Vn2 Sa2 Sh2
0001 040.0 010.0 +000000.0 000 000 00000.0 040.0 010.0 +000000.0 000 000 00000.0
0002 039.2 010.3 +000009.9 001 001 00029.3 039.2 010.3 +000038.6 004 004 00118.2
0003 038.4 010.6 +000018.8 001 001 00029.7 038.4 010.6 +000066.0 003 003 00088.6
```

Bei den Aggregatdateien von Versicherern sind in der linken Hälfte der Spalten die Attribute der Sparte 1 und in der rechten Sparte die der Sparte 2 gespeichert. Die Reihenfolge der Attribute von links nach rechts ist: Periode, Prämie Sparte 1, Werbeausgaben Sparte 1, Reserven Sparte 1, Versicherte Sparte 1, Schadenanzahl Sparte 1, Schadensumme Sparte 1. Diese Attributreihenfolge wiederholt sich dann für die zweite Sparte der Versicherung.

```
#t   Vu1 Vs1  Vp1  Ev1   Sh1   Vu2 Vs2  Vp2  Ev2   Sh2    Vm
0001 000 0.00 00.0 021.6 021.6 000 0.00 00.0 016.4 016.4 -000038.0
0002 002 0.81 10.0 006.7 031.1 002 0.76 17.4 006.4 029.1 -000125.6
0003 001 0.65 09.5 011.6 031.1 001 0.79 23.5 005.4 029.1 -000218.7
```

Die Aggregatdatei eines Versicherungsnehmers hat einen ähnlichen Aufbau. Die linke Hälfte enthält die Attributwerte des Risikos 1, die rechte Hälfte die Attribute des Risikos 2. Die Reihenfolge lautet: Periode, (häufigster) gewählter Versicherer im Risiko 1, gezahlte Prämie für Risiko 1, eigenversicherte Schäden im Risiko 1, Schadenhöhe im Risiko 1. Diese Reihenfolge wiederholt sich danach für Risiko 2.

Der Anwender ist in der Wahl des externen Analyseprogramms völlig frei. Das gewählte Format der Aggregatdateien ist so allgemein gehalten, daß es von den meisten Grafik und Statistikprogrammen gelesen werden sollte. Die Auswertung selbst unterteilt sich in zwei Bereiche: die grafische Auswertung und die statistische Auswertung.

Für die MSDOS-Version in IMS haben sich für die grafische Auswertung gnuplot von der Free Software Foundation und SIMSTAT 3.5e (Autor: Normand Péladeau von Provalis Research) für die statistische Auswertung bewährt. Beide Programme sind über Skripts steuerbar und erlauben so die automatische Bearbeitung großer Datenmengen.

Innerhalb von IMS ist nach Abschluß einer Simulation eine grafische Datenauswertung möglich. Nach Berechnung aller Perioden ist der Anwender wieder im Hauptmenü. Wenn der Punkt *Modellergebnis analysieren* gewählt wird, öffnet sich ein Untermenü mit vier Auswahlmöglichkeiten:

1. Grafische Analyse der Versicherer
2. Grafische Analyse der Versicherungsnehmer
3. Grafiken interaktiv erstellen
4. Zurück zum Hauptmenü

Durch Wahl des letzten Punktes gelangt der Anwender wieder zum Hauptmenü. Der dritte Punkt startet das Grafikprogramm *gnuplot* im interaktiven Befehlsmodus. Ein versierter Anbieter kann nun durch manuelle Eingabe der entsprechenden *gnuplot*-Kommandos Grafiken erzeugen. In den beiden anderen Submenüs werden aus Benutzereingaben und den Aggragatdateien temporäre *gnuplot*-Steuerdateien erzeugt und ausgeführt.

Die grafische Auswertung der Simulationsdaten basiert auf dem System der Aggregatdateien und deren Aufbau. Durch die Benutzereingaben werden Teile einzelnen Spalten aus verschiedenen Aggregatdateien nacheinander oder übereinander grafisch dargestellt. Der x-Achsenabschnitt ist dabei immer die Periode. Nach der Auswahl des Punktes *Grafische Analyse der Versicherer* wird zunächst die Aggregatstufe erfragt, aus deren Menge eine Aggregatdatei gewählt werden soll. Danach wird eine Liste der Aggregatdateien angezeigt, die im Rahmen der letzten Simulation erzeugt wurden. Zu jeder der erzeugten Dateien wird nun gefragt, ob der Anwender *nicht plotten*, *plotten* oder *replotten* will. Will er sie *plotten*, dann erzeugt er damit eine Einzelgrafik, auf der nur die Daten dieses Einzelaggregats dargestellt werden. Will man dagegen zwei oder mehrere Aggregate miteinander vergleichen, dann muß beim ersten Aggregat *plotten* und bei allen nachfolgenden Aggregaten *replotten* gewählt werden. Die gewünschten Aggregatdateien werden dann in einer Grafik übereinander dargestellt. Nach der Wahl des oder der Aggregate müssen noch zwei weitere Fragen beantwortet werden:

1. Welches Attribut dargestellt werden soll und
2. aus welchem Simulationslauf die Aggregatdaten gewählt werden.

Für die erste Frage wird eine Liste der Attribute dargestellt, aus denen der Anwender wählen kann. Falls ein Modell wiederholt simuliert wurde, kann der er aus den berechneten Läufen auswählen. Nach dieser Eingabe wird das Grafikprogramm gestartet. Sind mehrere Grafiken angefordert oder werden Aggregate mit *replot* übereinander dargestellt, dann muß zwischen den Einzelgrafiken die *RETURN*-Taste betätigt werden. Nach der Darstellung der Grafiken wird gefragt, ob weitere Grafiken verlangt sind, oder ob dieses Unterprogramm verlassen werden soll. Falls *ENDE* gewählt wird, befindet man sich wieder im Untermenu *Modellergebnis analysieren*. Wenn weitere Grafiken dargestellt werden sollen beginnt der besschriebene Dialog von vorn. Die bisher erzeugten Steuerbefehle bleiben erhalten, so daß bei einem erneuten Start von *gnuplot* die im ersten Dialog angeforderten Grafiken vor den im zweiten Dialog gewählten Grafiken erneut angezeigt werden.

4.3 Programmaufbau

4.3.1 Die Programmiersprache

IMS ist in ESL[1] programmiert. Die Programmiersprache stellt eine Ergänzung der allgemeinen Hochsprache C dar. Diese Spracherweiterung wird durch einen zusätzlichen C-Präcompiler bereitgestellt. Dieses Programm liest den Quelltext des ESL-Programms, wandelt ESL-Syntax in C-Syntax um und übernimmt C-Syntax unverändert. ESL enthält einige zusätzliche Datentypen und Funktionen, die den Simulationsablauf steuern. Weil

[1] vgl. 2.6.4.

es speziell für die Erstellung zeitdiskreter Simulationen mikroökomomischer Einheiten entwickelt wurde, ist die Simulationszeit in einzelne Perioden unterteilt. Die einzelnen Perioden sind ihrerseits in logische Zeitpunkte untergliedert. Auf die aktuelle Periode kann mittels der Systemvariable *period*, auf den logischen Zeitpunkt mittels der Systemvariable *logtime*, zugegriffen werden. Die Anzahl der logischen Zeitpunkte einer Periode werden im Kopf des ESL-Programms festgelegt.

Gleichartige mikroökonomische Subjekte sind in einem ESL-eigenen Datentyp zusammengefaßt: Der *class*. Alle Klassen werden in der Reihenfolge ihres Auftretens im Quelltext abgearbeitet. Innerhalb der Klasse kann auf das Einzelsubjekt mittels seines Index im Klassenvektor zugegriffen werden. Die Anzahl der Subjekte pro Klasse ist beliebig. In IMS gibt es fünf Klassen: BAV, VU, StratdVU, VN und StratdVN. Die Klasse BAV ist für die Bereitstellung der Fremdinformationen und der Aggregatdateien zuständig. StratdVU und StratdVN stellen die Verhaltensregeln für die Subjektklassen der Versicherer (VU) und der Versicherungsnehmer (VN) zur Verfügung. In den Klassen BAV, StratdVU und StratdVN gibt es jeweils nur ein Subjekt, in den Klasse VU 25 und in der Klasse VN 200. Im Gegensatz zu C, wo ein Vektorindex immer von 0 zu zählen beginnt, startet der Vektorindex einer ESL-Subjektklasse immer bei 1. Jedes ESL-Subjekt besitzt Attribute. Diese Attribute entsprechen den Variablendefinitionen in C. Die Variablennamen und die Klassennamen müssen mit einem Großbuchstaben beginnen. Neben den Subjektvariablen können globale oder lokale C-Variable generiert werden. Deren Namen haben mit einem Kleinbuchstaben zu beginnen. Der Zugriff auf ein Attribut eines Subjekts erfolgt durch *Klasse[Index].Attribut=Wert;*. Die ESL-Funktion *Ownnr()* liefert die Nummer des Subjekts, daß gerade bearbeitet wird.

Eine Klassendefinition besteht aus der Definition der Attribute und der Definition des Aktionsplans der Klasse. Im Aktionsplan werden die Unterprogramme der Klasse festgelegt. Diese Unterprogramme werden in jeder Periode zu einem bestimmten logischen Zeitpunkt ausgeführt. Neben der regelmäßigen Ausführung dieser Unterprogramme kann auch eine einmalige Ausführung vereinbart werden. In IMS wird nur eine Teilmenge der ESL-Schlüsselworte verwendet[2]:

- **class** definiert eine ESL-Klasse
- **subj** faßt die Befehle xact, act, mess und val für ein Subjekt einer Klasse zusammen
- **val** weist einem Subjektattribut einen Wert zu
- **act** definiert ein Unterprogramm als regelmäßige Aktion eines Subjektes
- **xact** definiert ein Unterprogramm, daß nicht in jeder Periode ausgeführt wird
- **func** definiert ein subjektunabhängiges Unterprogramm
- **pact** ESL-Variablentyp: Zeiger auf eine Aktion (act)

[2] vgl. Ohlendorf (1987), S. 32

- **Systemkonstanten** GO, STOP, n_log
- **Systemvariable** period, logtime
- **Systemfunktionen** Init(), Fini(), Ownnr()

Von besonderer Bedeutung sind die Systemfunktionen **Fini()** und **Init()**. Die Funktion **Init** wird vor der ersten Simulationsperiode einmalig aufgerufen. Die Funktion **Fini()** wird nach jeder Periode ausgeführt. Diese beiden Funktionen können durch den Programmierer überschrieben werden. In IMS wird von der **Init()**-Funktion aus die gesamte Benutzeroberfläche gesteuert. Die **Fini()**-Funktion steuert den Simulationslauf. Ihr Grundgerüst lautet:

```
func Fini() {
 if ( period <= sl )
  {
   return(GO);
  }
  else
  {
   if ( restal < al )
    {
     period  = 0; logtime = 0; restal++;
     return(GO);
    }
    else
    {
     period  = 0; logtime = 0; jumpback=1;
     return(STOP);
    }
  }
}
```

Die **Fini**-Funktion führt eine IMS-Simulation nicht einmalig durch, sondern wiederholt den Lauf. Die Anzahl der Wiederholungen ist in der globalen Variable *al* gespeichert. Dieser Ablauf ist in der Originalfassung von ESL nicht vorgesehen und wurde erst durch eine Modifikation des ESL-Basismoduls *ess.c* möglich. In *ess.c* wird in der C-Funktion *main()* der einmalige Simulationslauf durch eine *for()*-Schleife gesteuert. Danach wird die *main()*-Funktion verlassen und die Simulation ist damit beendet. Durch eine zusätzliche Endlosschleife um die beschriebene Hauptschleife herum wird die Simulation nach dem Abschluß eines Simulationslaufs nicht verlassen, sondern beginnt wieder mit Periode 1. Nachdem alle Läufe abgeschlossen sind, wird wieder die Funktion **Init()** aufgerufen. IMS wird von dort aus durch die C-Funktion *exit()* verlassen.

Die Funktion **Init()** wird in IMS dazu benutzt, um Speicherplatz für die dynamischen Subjekte anzufordern, die Protokoldatei zu öffnen und das Hauptmenü aufzurufen.

Zwei Punkte bei der ESL-Programmierung sind besonders hervorzuheben: die Suche syntaktischer Fehler in ESL-Programmen und betriebssystembedingte Besonderheiten. Der Weg vom ESL-Quelltext bis zum ausführbaren Programm umfaßt drei Schritte:

1. C-Präcompiler-Lauf: *file*.e $\Longrightarrow$ *tmpfile*.e
2. ESL-Präcompiler-Lauf: *tmpfile*.e $\Longrightarrow$ *tmpfile*.c
3. C-Compilerlauf: *tmpfile*.c ess.c $\Longrightarrow$ *file*.exe

Innerhalb jeder dieser Schritte können Fehler auftreten, die eine erfolgreiche Programmerstellung verhindern. Das Problem bei der Fehlerfindung in ESL-Programmen ist, daß direkt kein Quelltext-Debugger eingesetzt werden kann. Da dieser nur C-Code verarbeiten kann, ist man gezwungen, mögliche ESL-Fehler im vom ESL-Präprozessor erzeugten C-Programm zu finden.

Für die Erstellung von IMS wurde der GNU C-Compiler verwendet. Er hat den Vorteil, daß er auf den verschiedensten Betriebssystemen frei verfügbar ist und so hilft, betriebsystembedingte Unterschiede möglichst aus der Programmentwicklung auszuklammern. Zu den zusätzlichen Entwicklungswerkzeugen des GNU C-Compilers gehört ein Debugger, der hilft Programmfehler zu entdecken[3].

Der ESL-Präprozessor kann auch selbst Fehler anzeigen. Weil vor dem Lauf des ESL-Präprozessor aber schon ein Lauf des C-Präprozessors stattgefunden hat, zeigt ESL bei der Verwendung des *include*-Befehls falsche Zeilennummern bei den Fehlerhinweisen an. Durch das *include* werden Definitionsdateien in den Quelltext kopiert, der diesen um M Zeilen verlängert. ESL-Fehler sind dadurch immer um M Zeilen verschoben.

ESL- und C-Fehler sind unter Umständen schwer zu unterscheiden. Ein Fehler, der relativ einfach entdeckt werden kann, ist ein falscher Attribut- oder Aktionsname. Beide müssen als ersten Buchstaben im Namen einen Großbuchstaben haben. Ist das nicht der Fall, dann zeigt nicht der ESL-Präcompiler den Fehler an, sondern der Linker im abschließenden C-Compilerlauf. Dieser hält die falsch benannte Aktion für eine C-Funktion, die er in externen Bibliotheken sucht und dort nicht findet. Falls ein fehlerfreies ESL-Programm neu übersetzt wird und plötzlich Fehler auftreten, gibt es zwei Möglichkeiten:

1. Der Fehler kann in einer *act* eingegrenzt werden, weil nur dort Änderungen vorgenommen wurden.
2. Der erste C-Präprozessor-Lauf muß umgangen, die *include*- Anweisungen auskommentiert und ein weiterer Übersetzungslauf durchgeführt werden. Die dann angezeigten Fehlermeldungen verweisen auf die richtigen Zeilennummern.

[3] bug: englisches Wort für Wanze

Der Übersetzungsvorgang eines ESL-Quelltextes wird durch ein Programm gesteuert. IMS läuft bislang unter drei verschiedenen Betriebssystemen: MS-DOS ab Version 6.0 aufwärts, LINUX[4] ab Kernel 0.99pl16 und Sun SOLARIS 2.1. Die Steuerprogramme unter MS-DOS und LINUX unterscheiden sich nur unwesentlich. Unter SOLARIS ist zusätzlich ein AWK[5]-Filter notwendig. AWK ist ein Standard-Unix Werkzeug zur automatischen, programmierten Bearbeitung großer Textmengen. Durch den AWK-Filter werden Identifikationszeichenketten aus den Definitionsdateien des C-Compilers ausgefiltert, die sonst ESL-Fehler verursachen würden.

4.3.2 Die Datenstrukturen

In IMS gibt es drei große Datenstrukturen, die von zentraler Bedeutung sind:

- Die Datenstruktur der Versicherer,
- die Datenstruktur der Versicherungsnehmer und
- die Datenstruktur des Marktinformationanbieters.

Zusätzlich sind eine Reihe von globalen und lokalen Variablen definiert, die den Ablauf des Programms steuern. Die IMS-Datenstrukuren sind in der Datei *imsdata.c*[6] zusammengefaßt.

Zunächst sind einige Konstanten zu nennen, die die Grenzen von IMS festlegen. Jeder einzelne Simulationslauf darf maximal 100 Perioden umfassen, was in der Konstante *SIMLAENGE* festgelegt ist. Die maximale Anzahl aufeinanderfolgender Simulationsläufe beträgt ebenfalls 100. Die maximale Anzahl der aktivierbaren Versicherungen und Versicherungsnehmer ist in den Konstanten *MAXVU, MAXVN* festgelegt. Danach folgen einige Konstanten, die den Speicherverbrauch der Versicherer und der Versicherungsnehmer kontrollieren. Die Ganzzahlvariable *sl, al* und *rl* werden für die Steuerung der Anzahl der Simulationsläufe verwendet und werden in der ESL-Funktion **func Fini()** ausgelesen. Ihre Werte können vom Anwender in der Modellkonfiguration gesetzt werden. Die ESL-Systemvariable *period, logtime* und der Rückgabewert der Funktion **Ownnr()**, mit der sich das aktuelle Subjekt ansprechen läßt, werden in den Planungsfunktionen *act PlanVU* und *act PlanVN* in globale Variable kopiert, um die indirekten Aktionsaufrufe zu ermöglichen. In einer weiteren Ganzzahlvariable *gmem* wird der aktuell angeforderte dynamische Speicher aufsummiert, um eine Prüfung der Verwaltung der dynamischen ESL-Subjekte zu ermöglichen. Die beiden Vektoren *aktiVU[]* und *aktiVN[]* verwalten die aktivierten Versicherer und Versicherungsnehmer. Die Verhaltensregeln benutzen diese Vektoren um festzustellen, ob eine bestimmte Versicherung oder ein Versicherungsnehmer aktiv ist.

[4] LINUX ist ein Unix-Betriebssystem für INTEL-basierte Rechner
[5] vgl. Wielsch (1994), S. 288
[6] siehe Anhang

Diese Information wird benötigt, wenn z.B. ein Versicherungsnehmer eine Versicherung als Anbieter wählt. Dabei muß gewährleistet sein, daß keine inaktiven Versicherer gewählt werden.

Die Datenstrukturen von Versicherern und Versicherungsnehmern weisen strukturelle Übereinstimmungen auf. Beide besitzen Ganzzahlvariablen, die bestimmen, in welcher Periode und in welchem Simulationslauf ein Subjekt aktiviert wird: *Ap, Al[]*. Weiterhin besitzen diese Strukturen jeweils drei Variablen, die die von ihr verwendete Verhaltenregel bestimmen: *Vr, Vk und Ac[]*. Jeder Versicherung und jedem Versicherungsnehmer kann eine andere Verhaltensregel zugewiesen werden. Um diese Flexibilität zu erreichen, muß das ESL-Konzept der Aktionspläne zum Teil erheblich umstrukturiert werden. Die beiden Ganzzahlvariablen *Vr* und *Vk* dienen der Identifikation und Zuordnung des Subjekts zu den unterschiedlichen Datenaggregationsstufen: Einzelsubjekt, Verhaltensregel, Verhaltensregelklasse und Subjektklasse. Der Vektor aus Strukturvariablen *Action Ac[]* speichert für jeden der 10 möglichen logischen Zeitpunkte, an der ein Subjekt aktiv sein kann, ein Tupel aus logischem Zeitpunkt und Nummer der Verhaltensregel, die ausgeführt wird. Der *class VU* und der *class VN* ist jeweils eine Klasse zur Verwaltung der Verhaltensregeln zugeordnet: die Klassen *StratdVN* und *StratdVU*. Diese Klassen stellen einen Vektor von Zeigern auf Aktionen zur Verfügung. Der Index einer Verhaltensregel in diesem Vektor korrespondiert mit der im Aktionsvektor *Ak[]* gespeicherten Verhaltensregelnummer. In den Aktionen *PlanVU* und *PlanVN* prüfen die Subjekte, ob der Zeitpunkt der Ausführung ihrer Verhaltensregel gekommen ist und führen dann über einen indirekten Funktionsaufruf die entsprechende Verhaltensregel aus. Der Aufbau der Aktionspläne ist bei Versicherern und Versicherungsnehmern identisch:

```
{ xact: (1,1,0) Myinitvn
   act: 1 Newinivn PlanVN   /* Aktionsplan eines der 200 */
        2 PlanVN            /* Versicherungsnehmer       */
        3 PlanVN
        /* ... */
       10 PlanVN  }
```

In den Aktionsplänen der VU und VN wird in jedem der 10 möglichen logischen Zeitpunkte die Planungsfunktion der jeweiligen Subjektklasse ausgeführt.

```
act PlanVN  {
int i;
gOwnnr   = Ownnr();             /* Ueber alle logischen Zeitpunkt    */
gperiod  = period;              /* wird geprueft, ob der VN dafuer in*/
glogtime = logtime;             /* seinem Aktionsvektor eine VR vor- */
for( i = 1; i <= MAXNLOG; i++ ) /* gesehen hat; dann wird diese VR   */
 {                              /* INDIREKT aufgerufen.              */
  if ( ( VN[ gOwnnr ].DatenVN->Ac[i].st != 0        ) &&
       ( VN[ gOwnnr ].DatenVN->Ac[i].lt == glogtime )    )
```

```
   {
    (StratdVN[1].S[ VN[ gOwnnr ].DatenVN->Ac[i].st  ] )(); /* AUFRUF!*/
   } /* END: if */
 } /* END: for */
} /* END: act PlanVN */
```

In diesem Unterprogramm wird anhand des Vektors *Ac[]* geprüft, ob und welche Verhaltensregel dieses Subjekt im aktuellen logischen Zeitpunkt ausführen muß. Neben *Ac[]* besitzen beide Subjektklassen einen Parametervektor *Pv[]*.

Die eigentlichen Attribute, die für die Modellsimulation benötigt werden, sind bei den Versicherern in den Strukturvariablen *RISIKO Rk[]* und bei den Versicherungsnehmern in den Strukturvariablen *SPARTE Sp[]* abgelegt. Hier sind für alle Modellgrößen Vektoren mit dem Umfang *SIMLAENGE+1* deklariert, in denen die Daten eines Simulationslaufs Platz haben. Werden mehrere Simulationsläufe durchgeführt, dann sind in diesen Variablen immer die Daten des aktuellen Laufs gespeichert. Alle Daten der vorherigen Läufe werden in den externen Aggregatdateien abgelegt.

Die gesamte Datenstruktur der Subjektklassen *classVU* und *classVN* sind in den Strukturvariablen *DatenVU* und *DatenVN* zusammengefaßt. Der Grund dafür lag in dem Wunsch, den für diese beiden Subjektklassen benötigten Hauptspeicher nicht statisch sondern dynamisch anzufordern. Dynamischer Speicher wird erst zur Laufzeit angefordert, während statischer Speicher bereits zur Übersetzung an das ausführbare Programm angehängt wird und es dadurch unnötigt aufbläht. IMS fordert während der Ausführung für die Daten der classVU und classVN ungefähr 2 Megabyte Hauptspeicher an. Der Speicher wird vor Beginn der Simulation komplett angefordert und bei ihrem Ende vollständig freigegeben. Durch die dynamische Speicherverwaltung wird der Zugriff auf die einzelnen Subjektattribute verlängert: *Klasse[Index].DatenVx→Attribut=wert;*. Für die Klasse BAV ist diese Lösung nicht praktikabel, da diese Klasse nur ein Subjekt enthält und beijeder Simulation aktiviert ist. Die dynamische Verwaltung der Subjekte hat für die weitere Programmentwicklung eine unscheinbare, aber entscheidende Auswirkung: die Variablennamen werden länger und damit unübersichtlicher. Je länger die Variablennamen sind, um so größer ist die Wahrscheinlichkeit der Tippfehler und damit der Übersetzungsfehler. Mit diesem Nachteil wird die höhere Flexibilität der Modellumsetzung erkauft.

Die fünfte und letzte Datenstruktur ist der Klasse *BAV* zugeordnet. Diese Klasse steuert die exogenen Schocks, stellt die Fremdinformationen[7] zur Verfügung und schreibt die Simulationsergebnisse in jeder Periode in die dafür vorgesehenen Aggregatdateien. Der Vektor *As[]* steuert zusammen mit dem Vektor *Ar[]* den Änderungsschockindikator, auf den die einzelnen Verhaltensregeln mit dem Wechsel ihrer Verhaltensparameter reagieren. *As[]* bestimmt die Periode innerhalb eines Simulationslaufs, in der der Schockindikator ausgelöst wird, und *Ar[]* bestimmt die Läufe, in denen Änderungsschocks stattfinden. Dadurch kann z.B. ein Modell konfiguriert werden, daß aus 10 einzelnen Simulationsläufen

[7] vgl. 3.3.1.3. und 3.3.2.3.

besteht und bei dem in den letzten fünf Läufen jeweils in der zwanzigsten Periode ein Änderungsschock bei Anbietern und Nachfragern auslöst wird. Die Elemente von *As[], Ar[]* können nur die Werte 0 oder 1 annehmen. Der Index des Elements korrespondiert mit der Nummer der Periode und des Laufs. Wenn das Produkt aus $As[i] * Ar[j]$ den Wert Eins annimmt, dann wird in den Verhaltensregeln der Änderungsschock ausgelöst. Die Datenstrukturen, die die Aktierungsschocks der Einzelsubjekte steuern, sind in deren eigenen Datenstrukturen abgelegt. Somit ist gesichert, das Änderungs- und Aktivierungsschocks unabhängig voneinder stattfinden können.

Neben der Steuerung der Änderungsschocks stellt die class BAV für die Verhaltensregeln, die auf Fremdinformationen zugreifen, die notwendigen Informationen zur Verfügung. Sie sind in den Vektoren *Pm[], Wm[], Dp[], Dw[], Mp[], Mw[]* und *Dg[]* abgelegt. Die dritte Aufgabe der Klasse BAV ist das Speichern der Simulationsergebnisse in externen Dateien. Wenn eine Simulation mehr als einen Lauf umfaßt, dann werden ihre Daten aus Effizienzgründen nicht mehr im Hauptspeicher gehalten. Die Daten werden gesammelt, aggregiert und dann in Dateien mit festdefinierten Namen geschrieben. Es gibt vier Aggregatstufen, deren Aggregatdateien nacheinander beschrieben werden. So ist z.B. *imsvu005.dat* der Name der Aggregatdatei der Versicherung Nummer 5 und *imsvnvk2.dat* die Aggregatdatei der Klasse der Verhaltensregeln mit Eigeninformation der Versicherungsnehmer. Jede Aggregatdatei kann vom Anwender angefordert werden. Die Vektoren *agvu1[], agvu2[], agvu3[], agvu4[], agvn1[], agvn2[], agvn3[]* und *agvn4[]* bestimmen, welche Aggregatdateien erzeugt werden. Die Klasse *BAV* berechnet in jeder Periode die Häufigkeitsverteilung der Verhaltensregeln in den einzelnen Aggregaten und speichert sie in den Vektoren *Vuag1[], Vuag2[], Vuag3[], Vnag1[], Vnag2[]* und *Vnag3[].*

Als letzte Information stellt die *class BAV* den Zinsvektor und den Suchkostenvektor zur Verfügung. Die Zinsreihe wird von den Versicherern in jeder Periode zur Berechnung der einperiodigen Verzinsung ihrer Reserven herangezogen. Der Vektor der Suchkosten wird von den Nachfragerregeln **Suche** und **Beste Information** zur Berechnung der Informationsbeschaffungskosten benötigt.

4.3.3 Die Unterprogramme

Um den Quelltext von IMS besser überblicken zu können, muß ein schneller Zugriff auf die einzelnen Unterprogramme gewährleistet sein. Dazu dient die folgende Tabelle, die jedes Unterprogramm und die Zeilennummer seiner ersten Zeile auflistet und damit ein Inhaltsverzeichnis für den Programmtext darstellt:

Inhaltsverzeichnis der Unterprogramme

Zeile	Unterprogramm	Zeile	Unterprogramm	Zeile	Unterprogramm
0034	act **Myinitbv**	2111	act **Newinivn**	4566	func **Vdefmd6**
0060	act **Newinibv**	2147	act **PlanVN**	4713	func **Erstvu**
0092	act **Frmdinf**	2176	act **Leer**	4978	func **Erstvn**
0402	act **Agrsich**	2185	act **Vrvn01**	5153	func **Erstmkt**
0980	act **Myinitvu**	2406	act **Vrvn02**	5197	func **Moderst**
1014	act **Newinivu**	2627	act **Vrvn03**	5233	func **Mkbear**
1042	act **Leer**	2948	act **Vrvn04**	5288	func **Starts**
1049	act **PlanVU**	3229	act **Vrvn05**	5338	func **Analyvu**
1081	act **Leer**	3521	act **Vrvn06**	5611	func **Analyvn**
1087	act **Vrvu01**	3793	act **Vrvn07**	5890	func **Analy**
1176	act **Vrvu02**	4009	func **Mkload**	5929	func **Pginfo**
1266	act **Vrvu03**	4066	func **Mksave**	5954	func **Pgende**
1365	act **Vrvu04**	4102	func **Bavauin**	5969	func **Mainmenu**
1471	act **Vrvu05**	4137	func **Vuauini**	6022	func **Opendata**
1582	act **Vrvu06**	4191	func **Vnauini**	6052	func **Freemem**
1695	act **Vrvu07**	4240	func **Vdefmd1**	6091	func **Getmem**
1788	act **Vrvu08**	4301	func **Vdefmd2**	6131	func **Init**
1881	act **Vrvu09**	4367	func **Vdefmd3**	6147	func **Fini**
1974	act **Vrvu10**	4441	func **Vdefmd4**		
2075	act **Myinitvn**	4500	func **Vdefmd5**		

Tabelle 4.1: Inhaltsverzeichnis der Unterprogramme im Quelltext von *ims.e*

Die Unterprogramme in IMS teilen sich in die Umsetzung der Verhaltensregeln und Serviceunterprogramme auf. Die Verhaltensregeln sind den beiden Hauptklassen der Versicherer und den Versicherungsnehmern zugeordnet. Die Klasse BAV besitzt keine eigenen Verhaltensregeln und nimmt nur Serviceaufgaben wahr. Die Unterprogramme der Klasse BAV heißen **act Myinitbv**, **act Newinibv**, **act Frmdinf** und **act Agrsich**. Die beiden ersten Unterprogramme dienen der Initialisierung der Subjektattribute. In der ersten Periode des ersten Simulationslaufs wird **act Myinitbv** ausgeführt, in den ersten Perioden aller nachfolgenden Läufe **act Newinibv**. In **act Frmdinf** werden die von den Verhaltensregeln benötigten Fremdinformationen berechnet. Dabei müssen eventuelle Aktivierungsschocks in der Berechnungsreihenfolge berücksichtigt werden.

act Frmdinf wird in jeder Periode als zweites Unterprogramm ausgeführt. Die Berechnung aller Verhaltensregeln von Versicherern und Versicherungsnehmern erfolgt immer nach **act Frmdinf**. Dadurch muß dieses Unterprogramm immer auf die Attribute der Verhaltensregeln in der Vorperiode zurückgreifen. In der ersten Periode eines Laufs können dadurch keine Fremdinformationen berechnet werden. Für die Berechnung der Durchschnittswerte wird die Anzahl der in der Vorperiode aktiven Anbieter und Nachfrager

gebraucht. Diese Werte werden aus den Vektoren *aktiVU[]* und *aktiVN[]* bestimmt. Die Neuberechnung dieser beiden Vektoren darf in **act Frmdinf** erst nach der Berechnung der verschiedenen Durchschnittswerte erfolgen. Falls in t ein Aktivierungsschock stattfindet und dadurch mehr Subjekte aktiviert werden, würden sonst falsche Werte bestimmt werden. Nach der Berechnung der Durchschnittsprämien und -werbeaufwendungen der Sparten 1 und 2 wird der durchschnittliche Versicherungsgrad in beiden Risiken bestimmt. Jeder Versicherungsnehmer besitzt einen Statusvektor für jedes Risiko. Wählt er Eigenversicherung in der Perode i, dann ist das i-te Element 0 sonst 1. Summiert man nun über die Statusvektoren aller in der Vorperiode aktiven Versicherungsnehmer und dividiert die Summe durch ihre Anzahl, dann erhält man den durchschnittlichen Versicherungsgrad für ein Risiko.

Nachdem die Anzahl der aktiven Anbieter und Nachfrager neu bestimmt wurde, werden die Minimalprämien und Maximalprämien für die Anbieterverhaltensregel **Angriff** berechnet. Dazu muß ein bestimmtes Element innerhalb eines Vektors gefunden werden. Der Algorithmus dazu ist immer dergleiche. Wird ein Maximum gesucht, dann wird ein absolutes Minimum als Startwert vorgegeben. Danach wird nacheinander jedes Element des Vektors mit dem Startwert als Referenzwert verglichen. Wird ein größeres Element gefunden, dann wird dieses das neue Referenzelement und die Suche geht bis zum Ende des Vektors weiter. Für die Anbieterverhaltensregel **Angriff** wird in jeder Periode der Anbieter mit den größten Reserven gesucht und dessen Prämienentscheidungen und Werbeaufwendungen gespeichert.

Das BAV-Unterprogramm **act Agrsich** wird in jeder Periode im logischen Zeitpunkt 10 ausgeführt. Zu diesem Zeitpunkt haben alle Subjekte der Klasse ihre Verhaltensregeln durchgeführt. Nun werden die vom Anwender gewünschten Aggregatdateien beschrieben. Es sind vier Aggregatstufen definiert:

- Aggregat I: Einzelsubjekt Versicherer oder Versicherungsnehmer,
- Aggregat II: Alle Subjekte mit gleicher Verhaltensregel,
- Aggregat III: Alle Subjekte der gleichen Verhaltensregelklasse:
  - Informationsfreie Verhaltensregeln,
  - Verhaltensregeln mit Eigeninformation,
  - Verhaltensregeln mit Fremdinformation,
- Aggregat IV: Subjektklasse, d.h. alle Versicherer und alle Versicherungsnehmer.

Für jedes Aggregat existiert ein Vektor, dessen Elementanzahl mit der Anzahl der möglichen Aggregate diese Stufe übereinstimmt. Auf der ersten Stufe gibt es 200 mögliche Aggregatdateien für die Versicherungsnehmer und 25 für die Versicherer. Auf der zweiten Stufe können 10 Dateien für die Versicherer und sieben für die Verhaltensregeln der

Versicherungsnehmer beschrieben werden. Auf der dritten Stufe sind es für Anbieter und Nachfrager jeweils drei und auf der vierten Stufe jeweils eine Aggregatdatei.

Die Aggregatdateien haben auf jeder Aggregatstufe das gleiche Format. Sie sind in Tabellenformat angelegt und in der ersten Spalte steht die Nummer der Periode. In den anderen Spalten stehen die Periodenwerte der Ergebnis-, Ziel- und Folgevektoren. In der Aggregatdatei eines Einzelsubjekts werden die Werte aus dessen Attributvektoren geschrieben. Für die Aggregate höherer Stufen müssen die Attributwerte zunächst nach dem Selektionskriterium zusammengefaßt werden. Wenn z.B. die Aggregatdatei aller Versicherer mit der Verhaltensregel **Mark Up II** beschrieben werden soll, dann müssen dazu die Daten aller Versicherer mit dieser Verhaltensregel addiert werden und durch die Anzahl der aktiven Versicherer mit der Verhaltensregel **Mark Up II** geteilt werden. Die einzelnen Spalten der Aggregatdatei enthalten danach die Durchschnittswerte der Prämien, Werbeausgaben, Reserven, Versicherten, Schadenanzahl und Schadenhöhen in beiden Sparten.

Nicht bei allen Aggregaten kann der Durchschnittsoperator angewendet werden. Bei den Versicherungsnehmern werden die gewählte Versicherung, der Status, die gezahlte Prämie, der eigenversicherte Schaden, die Schäden und das Vermögen gespeichert. Bei dem gewählten Versicherer handelt es sich um die Nummer des Anbieters. Würde man z.B. über alle Versicherungsnehmer mit der Verhaltensregel **Suche** aggregieren und die Durchschnittsnummer des Anbieters speichern, dann könnte diese Information kaum für weitere Analysen herangezogen werden. Interessanter ist der am häufigsten gewählte Versicherer. An ihm kann gemessen werden, wie erfolgreich die Verhaltenregel **Suche** war, wenn die Prämienvektoren dieses Versicherers dazugenommen werden.

Die Klasse der Versicherer *class VU* hat den gleichen Ablauf der Initialisierung wie die Klasse BAV. **act Myinitvu** wird einmalig in der ersten Periode des ersten Laufs ausgeführt. In jedem weiteren Lauf wird zur Reinitialisierung **act Newinivn** aufgerufen. Danach wird in jedem logischem Zeitpunkt die Aktion **act PlanVU** ausgeführt, die prüft, ob und welche Verhaltensregel vom aktuellen Subjekt abgearbeitet werden muß. Nach der Definition der Klasse *StratdVU*, die die Verhaltensregeln verwaltet, folgt die eigentliche Umsetzung der im Abschnitt 3.3.1. formulierten Verhaltensregeln. Alle Verhaltensregeln der Versicherer besitzen den gleichen Aufbau:

- Vereinbarung der lokalen Hilfsvariablen,
- Umsetzung der langen Attributenamen der dynamischen Subjektverwaltung in kurze lokale Hilfsvariablen, um die Umsetzung der Verhaltensregel zu erleichtern,
- Prüfung, ob der Versicherer in dieser Periode aktiv sein soll,
- Anpassen der globalen Aktivitätsvektoren, die die Zuordnung dieses Subjekts zu den unterschiedlichen Aggregatstufen sichern und die Verzinsung der Reserven,
- Prüfung, ob der Änderungsschockindikator gesetzt ist,
- Ausführung der Verhaltensregeln mit Berücksichtigung der Startwerte und die

- Rückspeicherung der lokalen Hilfsvariablen in die dynamischen Datenstrukturen der Versicherung.

Die Verhaltensregeln der Versicherer sind in den **act Vrvu01** bis **act Vrvu10** umgesetzt. Die Verhaltensregel **act Vrvu10** setzt keine der formulierten Verhaltensregeln um, sondern kann durch den versierten Anwender nach den eigenen Bedürfnissen angepaßt werden. Um die Zuordnung dieser frei definierbaren Verhaltensregel zu den unterschiedlichen Aggregaten sicherzustellen, muß zusätzlich das zehnte Element des Zuordnungsvektors *int vkrvu[11] = {0,1,1,2,2,2,2,3,3,3,0}*; auf eine Zahl zwischen 1 und 3 gesetzt werden, die der Verhaltensregelklasse entspricht.

Der Initialisierungsablauf der Klasse der Versicherungsnehmer *class VN* entspricht dem der *class BAV* und dem der *class VU*. Er besteht aus den Unterprogrammen **act Myinitvn** und **act Newinivn**. Der Aufruf der Verhaltensregeln wird durch das Unterprogramm **act PlanVN** gesteuert. **act PlanVN** ruft bei Bedarf eine der sieben Verhaltensregeln **Vrvn01** bis **Vrvn07** auf. **Vrvn07** ist eine frei definierbare Verhaltensregel, die eigenen Bedürfnissen angepaßt werden kann. Die Umsetzung der im Abschnitt 3.3.2. definierten Verhaltenregeln basiert auf einem gleichartigen Aufbau der Verhaltensregeln der Versicherungsnehmer:

- Vereinbarung der lokalen Hilfsvariablen,
- Umsetzung der langen Attributenamen der dynamischen Subjektverwaltung in kurze lokale Hilfsvariablen um die Umsetzung der Verhaltensregel zu erleichtern,
- Prüfung, ob der Nachfrager in dieser Periode aktiv sein soll,
- Anpassen der globalen Aktivitätsvektoren, die die Zuordnung dieses Subjekts zu den unterschiedlichen Aggregatstufen sichern,
- Prüfung, ob der Änderungsschockindikator gesetzt ist,
- Ausführung der Verhaltensregeln mit Berücksichtigung der Startwerte,
- Rückspeicherung der lokalen Hilfsvariablen in die dynamischen Datenstrukturen der Versicherungsnehmer und die
- Abrechnung von eventuellen Schäden und Anpassung der Datenstrukturen der dynamischen Datenstrukturen der Versicherer.

Die Abrechung der Schadenfälle wurde den Versicherungsnehmern zugerechnet, weil diese auch die Schäden erzeugen und dadurch von ihnen aus eine einfachere Schadenabrechnung möglich ist.

Auf die Umsetzung der Verhaltensregeln folgen alle Unterprogramme, die nicht an eine bestimmte Klasse gebunden sind. Der Ursprung ihres Aufrufs liegt in der ESL-Funktion

func Init(). Die Funktion **func Getmem()** fordert zum Programmbeginn den Speicher für die dynamischen ESL-Subjekte an. Dieser Speicher wird in **func Freemem()** wieder freigegeben. Eventuelle Fehler werden in der IMS-Protokolldatei *imsrepor.dat* verzeichnet.

Die Abhängigkeiten der Unterprogramme kann anhand folgender Grafik verdeutlicht werden:

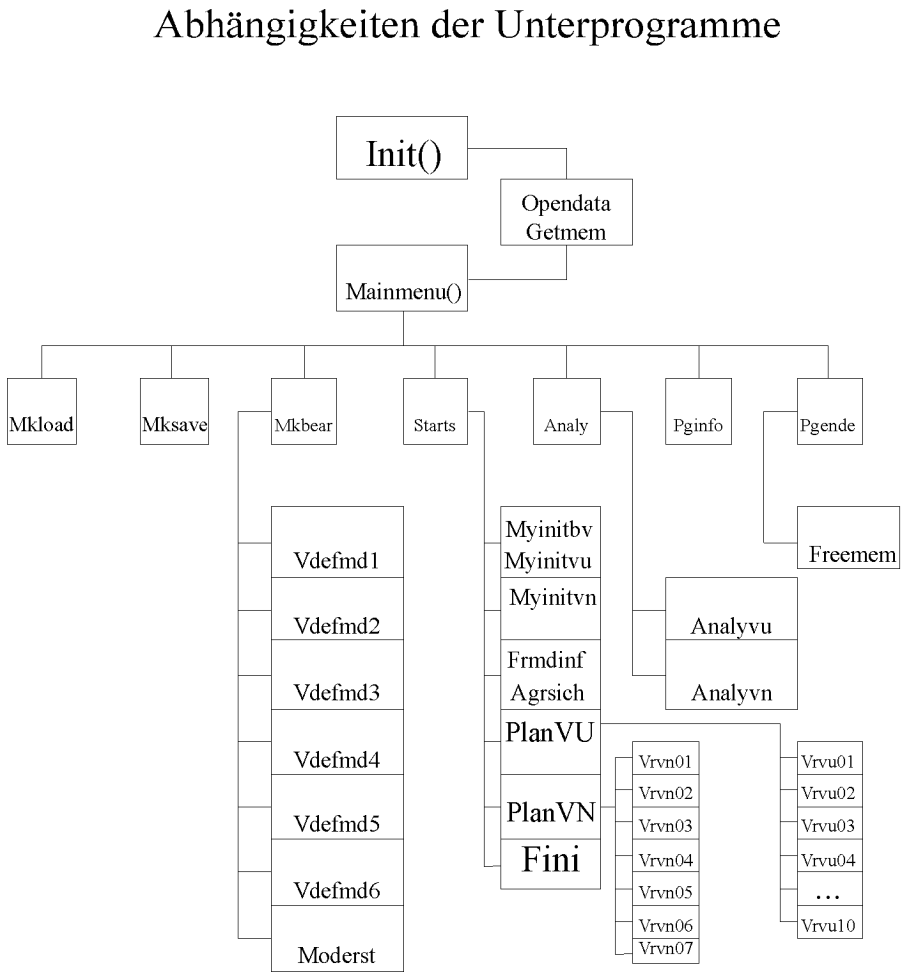

Abbildung 4.3: Abhängigkeit der Unterprogramme

Ausgangspunkt der Aufrufe aller Unterprogramme ist die **func Init()**. Alle für den Benutzer sichtbaren Aktionen werden vom Unterprogramm **func Mainmenu()** aus gesteuert.

Kapitel 5

Die Programmanwendung

5.1 Ziele des Kapitels

Voraussetzungen für eine Simulation sind die Existenz eines Modells und die Umsetzung des Modells in ein Programm. Diese beiden Bedingungen sind nun erfüllt. In diesem Kapitel wird der Versuchsaufbau, die Analysemethoden und die Ergebnisse der Simulation dargestellt. Dabei wird zunächst auf die Grundlagen zur Bewertung der Simulationsergebnisse eingegangen. Danach werden die einzelnen Phasen des Untersuchungsablaufs beschrieben. Diese Phasen bilden das Ablaufgerüst für den anschließenden Simulationslauf.

Bei der Ausführung von IMS werden die Annahmen an das Modell der Wiedervereinigung in Parametervektoren für das Programm umgesetzt und untersucht, wie sich die Hypothesen über die Schocks der Wiedervereinigung auf die Ausgabedaten auswirken. Mittels statistischer Verfahren wird nach der Simulation geprüft, ob die Schocks zu signifikanten Änderungen der Ergebnisse führen. Das Modell der Wiedervereinigung wird in zwei Varianten untersucht. Das erste Modell verwendet alle Verhaltensregeln und untersucht einen kombinierten Aktivierungs- und Änderungsschock. In einer einfacheren Modellvariante sollen die Auswirkungen eines Aktivierungsschocks auf einen Markt mit homogenem Verhalten untersucht werden.

5.2 Grundlagen zur Bewertung der Simulationsergebnisse

Die Bewertung der Ergebnisse einer Simulation sind immer in Beziehung zu ihren Zielen zu sehen. Wenn das Ziel die Prognose von Entwicklungen ist, dann sind die Ergebnisse des simulierten Systems an realen Daten zu bewerten. Die Realität ist aber nicht die einzige Meßlatte für Simulationen. Das Verständnis der Abläufe von Prozessen ist eine weitere

Möglichkeit für diese Bewertung. Das Ziel des IMS-Modells ist nicht hohe Ergebnisrealität, sondern hohe Prozeßrealität[1]. Würden die Daten realer Marktteilnehmer zum Vergleich benutzt, dann müßte die implizite Annahme gemacht werden, daß sich diese Subjekte nach den hier postulierten Regeln verhalten. Nur dann wären Entwicklungsprognosen auf der Basis des IMS-Modells gerechtfertigt. Diese Extrapolation ist aber nicht Thema dieser Arbeit.

Die Wahl von Start- und Vergleichswerten auf der Basis realer Daten würde voraussetzen, daß die Realität und die Simulation die gleiche funktionale Struktur besitzen. Wenn das der Fall ist, dann wären nur noch die Funktionsparameter durch Schätzungen anzupassen, um reale und simulierte Daten in Übereinstimmung zu bringen. Die eigentliche Stärke des IMS-Modells liegt aber in der Abstraktion von realen Daten und Strukturen. Sie läßt den Blick frei auf die Abläufe und Prozesse, focusiert nicht auf Daten- oder Parameterübereinstimmung und ist nicht einfach ein erweitertes ökonometrisches Modell. Darin liegt aber auch der Hauptunterschied zwischen einem ökonometrischem Modell und einer Simulation: Die ökonometrische Schätzung testet mittels Daten eine Hypothese **über** die Struktur der Realität, die Simulation ist eine Hypothese der Struktur der Realität. Statt der Daten wären die Strukturen von Modell und Realität zu vergleichen. Im IMS-Modell müßte also durch geeignete Verfahren geprüft werden, inwieweit die postulierten Verhaltensregeln realem Verhalten von Anbietern und Nachfragern auf Versicherungsmärkten entsprechen. Wenn das der Fall wäre, müßte in einem zweiten Schritt die Verteilung und Zuordnung dieser Verhaltensregeln zu den realen Anbietern und Nachfragern bestimmt werden.

Jedes nummerische Modell benötigt Startwerte. Diesen Ausgangsdaten kommt daher eine besondere Bedeutung zu. Im IMS-Modell sind zwei Arten von Startwerten zu bestimmen: Die Startwerte für die Generatoren der Pseudo-Zufallszahlen und die Startwerte der Zielgrößen der Verhaltensregeln. Die erste Kategorie wird durch das Programm bestimmt. Die Startwerte der Zufallsgeneratoren werden durch eine aus Uhrzeit und Datum abgeleitete Variable initialisiert. Die Startwerte der Verhaltensregeln wie z.B. die Prämie einer Versicherung in einer Sparte, können durch den Anwender festgelegt werden. Diese Werte müssen grundlegenden Plausibilitätsanforderungen genügen. Da die Schäden z.B. nur in einem Bereich zwischen Null und Hundert liegen, ist es unplausibel den Startwert der Prämie auf Tausend zu setzen, außer man will die Güte der Verhaltensregeln der Nachfrager testen. Zu Testzwecken ist jede mögliche Startwertkombination geeignet, einerseits um die Stabilität des Programms, aber auch um die logische Richtigkeit des Modells zu prüfen. Würden alle Nachfrager immer zum teuersten Versicherer wechseln, wäre das ein starker Hinweis auf logische Fehler im Modell oder im Programm.

Tests mit unplausiblen Startwerten sind nur eine Möglichkeit Fehler zu finden. Bei Fehlern in einer Computersimulation muß zwischen Fehlern des Modells und Fehlern des Programms getrennt werden. Beide Arten führen zu fehlerhaftem Programmverhalten. Wird ein richtiges Modell falsch in ein Programm umgesetzt, dann sind die Ergebnisse des

[1] vgl. Fischer (1988), S. 2

Programms nicht mit den Modellvorgaben in Übereinstimmung zu bringen. Durch diese fehlende Übereinstimmung lassen sich logische Programmfehler feststellen. Ihre Lokalisation innerhalb des Programms ist dann ein zusätzliches Problem. Wird ein klassisches, optimierendes Modell in eine Simulation umgesetzt, dann können die Ergebnisse mit den Optimalwerten des analytischen Modells verglichen werden. Abweichungen sind ein klarer Hinweis auf logische Programmfehler. Sind durch das Modell der Simulation keine optimalen, analytischen Sollwerte vorgegeben, kann der Test der Simulation nur durch den Test aller ihrer Einzelteile erfolgen. Das IMS-Modell macht keine Aussagen über optimales Verhalten oder Gleichgewichte. Ein Test der Simulation durch den Vergleich mit analytischen Modellvorgaben ist dadurch nicht möglich.

Die Validierung mikroanalytischer Simulationen, denen IMS nahe steht, ist ein "bis heute noch nicht befriedigend gelöstes Problem"[2]. Die Bewertung der Ergebnisse von Simulationen ist von den Zielen, aber auch von der Richtigkeit des Programms abhängig. In IMS muß der Validierungsansatz auf einem umfassenden Test aller Programmeinzelteile und einer Überprüfung der Ergebnisse auf ihre modellinterne Plausibilität basieren. Ein Vergleich mit analytischen Optimal- oder Erwartungswerten oder ein Vergleich mit realen Daten ist nicht möglich. Fehlende Sollvorgaben der Theorie und der Realität stellen aber nur ein Problem dar, wenn Ergebnisrealität ein Ziel der Simulation ist. IMS stellt aber die Prozeßrealität in den Mittelpunkt der Zielvorgaben. Die Überprüfung des Programms auf logische Richtigkeit und modellintern plausible Startwerte sind daher ausreichende Anforderungen für korrekte, prozeßreale Endergebnisse.

Zwischen der Validierung und dem eigentlichen Simulationsexperiment besteht ein fließender Übergang[3]. Die Bewertung von Simulationsergebnissen ist daher ein Prozeß, an dessen Ende ein Ergebnis steht, daß den Zielanforderungen der Simulation entspricht. Weil die Startwerte nicht aus realen Daten abgeleitet und die Endwerte nicht mit realen Daten oder analytischen Ergebnissen verglichen werden können, sind an die Bewertungsmethoden zielorientierte Anforderungen zu stellen. Die Datenanalyse des IMS-Modells der Wiedervereinigung basiert auf zwei Bewertungsarten: Der visuellen Bewertung der grafischen Simulationsergebnisse und der statistischen Prüfung von Testhypothesen.

5.3 Untersuchungsablauf

Wie in einem Laborexperiment ist auch bei einer Simulation ein Ablauf der Untersuchung festzulegen. Dieser Ablauf gliedert sich in vier Phasen:

1. Modellkonfiguration,
2. Meßgrößenwahl,

[2] Vetterle (1986), S. 184
[3] vgl. Vetterle (1986), S. 212

3. Meßmethoden und
4. Aussagenableitung.

Die Phase der Modellkonfiguration umfaßt drei Arbeitsschritte:

- Die Zuordnung der Verhaltensregeln zu den Subjekten,
- die Festlegung der Startwerte und Parametervektoren und
- die Definition der Schocks.

Jeder Versicherung und jedem Versicherungsnehmer kann eine andere Verhaltensregel zugeordnet werden. Die Wahl der verwendeten Regeln und ihre Aufteilung innerhalb einer Subjektklasse ist Teil der Konfiguration. Bei den Startwerten ist zwischen Versicherern und Versicherungsnachfragern zu unterscheiden. Die wichtigsten Startwerte der Versicherer sind die Prämien in der Periode Eins in beiden Sparten, die Höhe der Werbeaufwendungen und die Höhe der Anspruchsniveaus, falls es die Verhaltensregel erfordert. Bei den Versicherungsnehmern ist der anfängliche Versicherungsstatus zu wählen. Die Parametervektoren sind in zwei Teilvektoren getrennt. Der erste Teilvektor beinhaltet alle Verhaltensparameter, die bis zu einem Änderungsschock gelten. Der zweite Teilvektor bestimmt das Verhalten nach einem Änderungsschock. Die Bestimmung des ersten Teilvektors gehört zur zweiten Phase der Modellkonfiguration, während die Festlegung des zweiten Teilvektors zur Definition der Schocks gehört. Die Definition der Schocks untergliedert sich in die Festlegung der Zeitpunkte der Aktivierung einzelner Subjekte oder Gruppen von Subjekten und die Wahl der Parameter, die das Verhalten der Subjekte nach einem Änderungsschock bestimmen.

Die Meßgrößenwahl ist durch die Datenstrukturen der simulierten Marktteilnehmer einerseits und die Schockdefinition der Modellkonfiguration andererseits bestimmt. In einer diskreten Simulation sind die Daten immer in Vektoren $V(x_1, \ldots, x_T)$ gespeichert. Jedes Einzelelement stellt den Wert einer Variablen in einer bestimmten Periode t dar. Wird der Zeitpunkt eines Schocks t^* mit berücksichtigt, dann ist der Vektor in zwei Untervektoren teilbar: $V^*(x_1, \ldots, x_{t^*}, x_{t^*+1}, \ldots, x_T)$. Die Teilvektoren vor und nach dem Schock können dadurch getrennt untersucht und verglichen werden.

Die Datenvektoren der Versicherer und der Versicherungsnehmer sind in die Kategorien Ergebnis-, Ziel- und Folgegrößenvektoren unterteilt. Die Ergebnisvektoren stellen die quantifizierten Konzequenzen ihrer Entscheidungen dar. Die Zielgrößenvektoren beschreiben die sequenziellen Entscheidungen in den Perioden. Folgegrößenvektoren fassen die Daten zusammen, auf die die Marktteilnehmer keinen Einfluß haben.

- Ergebnisvektoren
  - Der Versicherer
    - Die Reserven in beiden Sparten
    - Die Anzahl der Versicherungsnehmer in beiden Sparten
  - Der Versicherungsnehmer
    - Die gezahlten Prämien in beiden Risiken
    - Die selbstgetragenen Schäden in beiden Risiken
- Zielgrößenvektoren
  - Der Versicherer
    - Die Prämien in beiden Sparten
    - Die Werbeaufwendungen in beiden Sparten
  - Der Versicherungsnehmer
    - Der Versicherungsstatus in beiden Risiken
    - Der gewählte Anbieter in beiden Risiken
- Folgegrößenvektoren
  - Der Versicherer
    - Schadenanzahl in beiden Sparten
    - Schadenhöhe in beiden Sparten
  - Der Versicherungsnehmer
    - Schadenhöhe in beiden Risiken
    - Endvermögen

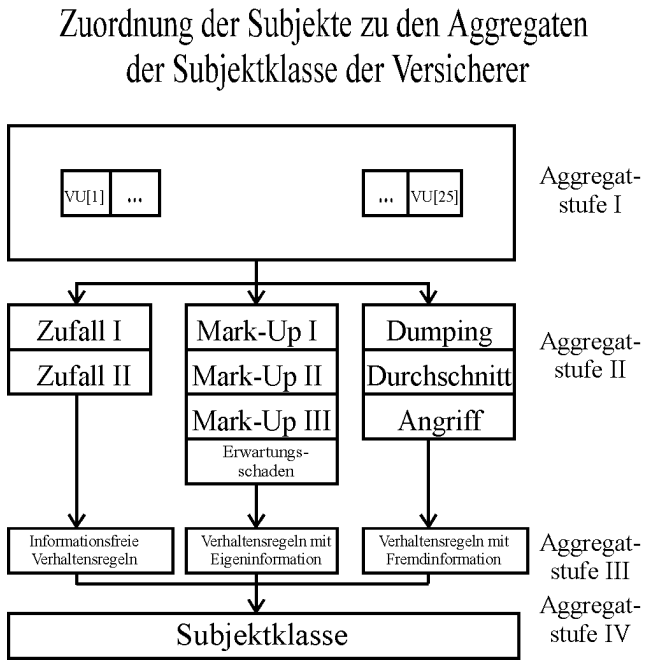

Abbildung 5.1: Aggregatzuordnung der Versicherer

Diese Meßgrößen können für unterschiedliche Hypothesen in verschiedenen Aggregaten zusammengefaßt werden: Nachdem der Anwender jedem Marktteilnehmer eine Verhaltensregel zugewiesen hat, setzt IMS diese Zuordnung für die Aggregatstufen II bis IV fort. Die Aggregate sich vergleichbar mit den Vergrößerungsstufen eines Mikroskops. Aggregatstufe IV ist die niedrigste, Stufe I die höchste Auflösung. Effekte, die auf der niedrigsten Aggregatstufe nicht nachweisbar sind, können es unter Umständen auf einer höheren Stufe sein. Dem Anwender stehen mit den Aggregatdateien die Rohdaten zur Verfügung, deren weitere Analyse mit externen Statistikprogammen zu erfolgen hat. Die einzelnen Aggregatstufen fassen die oben beschriebenen Datenvektoren zusammen. Die verwendeten Aggregatoperatoren sind Mittelwerte und maximale Häufigkeiten. Die Reserven aller Verhaltensregeln mit Eigeninformation werden z.B. mittels einer Durchschnittsbildung aggregiert. Der Versicherer der Verhaltensregel **Präferenz** ist durch das Maximum der Häufigkeitsverteilung der Versicherer der Nachfrager mit dieser Verhaltensregel bestimmt. Wenn diese Versicherungsnehmer z.B. den Versicherer *VU[19]* in der Periode 58 am häufigsten gewählt haben, dann ist 19 der Wert dieses Aggregats in der betreffenden Periode.

Die Meßmethoden sind am Ziel der Simulation auszurichten. Der Nachweis von Schockwirkungen kann an den unterschiedlichen Aggregatstufen mit grafischen statistischen Methoden erfolgen. Beide Methoden sollen im weiteren Verlauf Anwendung finden. In den Verhaltensregeln werden Gleichungen der Form $x(t) = a + b * x(t-1)$ verwendet, wobei $x(t-1)$ eine Zufallsvariable ist. Durch diese Funktionsform hängen alle Glieder der daraus erzeugten Folge vom Startwert $x(0)$ ab. Wenn $x(0)$ eine Zufallszahl ist, dann hängen alle

Folgeglieder vom Startwert des Zufallszahlengenerators ab. Diese Korrelation sinkt mit zunehmendem Abstand vom Startwert. Eine solche Folge bezeichnet man als stochastischen Prozeß[4]. Alle von IMS erzeugten Datenreihen sind stochastische Prozesse. Man unterscheidet stationäre und nicht-stationäre stochastische Prozesse. Ein stationärer stochastischer Prozeß pendelt sich nach einer bestimmten, endlichen Anzahl von Perioden auf einen Wert ein, um den er nur noch in einem Intervall schwankt. Wenn nach einer Übergangsphase die Stationarität des Prozesses erreicht ist, dann sind die daran anschließenden Folgeglieder stochastisch unabhängig vom Startwert der Folge. Die Abhängigkeit einer Zufallsvariable in einem stochastischen Prozeß von ihren Vorgängern wird durch die Autokorrelationsfunktion $AKF(\tau)$ gemessen.

$$AKF(\tau) = \frac{\rho(\tau)}{\rho(0)} \tag{5.1}$$

Mit $\rho(\tau) = E[x(t) - E(x(t))](x(t-\tau) - E[x(t)])$ und τ als Laglänge. Die Auswertung der Autokorrelationsfunktionen wird in der Analyse des Modells der Wiedervereinigung nur grafisch erfolgen.

Die letzte Phase des Untersuchungsablaufs beeinhaltet die Aussagenableitung aus den gefundenen Ergebnissen. Dabei werden die grafischen Auswertungen und Ergebnisse statistischer Tests interpretiert und in verbale Aussagen über die Wirkung der Schocks umgesetzt.

5.4 Die Simulation

5.4.1 Modellkonfiguration

Der erste Schritt der Modellkonfiguration ist die Zuweisung von Verhaltensregeln an die einzelnen Subjekte. Die Nachfrageseite des IMS-Modells der Wiedervereinigung verwendet alle definierten Verhaltensregeln. Die Nachfrager `VN[151]` .. `VN[200]` werden durch einen Aktivierungsschock in der fünfzigsten Periode des einzelnen Simulationslaufs aktiviert.

[4] vgl. Grams (1992), S. 99

Versicherungsnehmer	Verhaltensregel
1 – 15	Zufall I
16 – 30	Zufall II
31 – 60	Präferenz
61 – 90	Totale Erinnerung
91 – 120	Suche
121 – 150	Beste Information
151 – 190	Präferenz
191 – 200	Zufall II

Tabelle 5.1: Zuordnung der Verhaltensregeln zu den Versicherungsnehmern

Jeder Lauf umfaßt 100 Perioden. Die Aktivierung von 25 Prozent neuer Nachfrager entspricht ungefähr dem Anteil, der ostdeutschen Nachfrager am gesamtdeutschen Markt[5]. Die Nachfrager `VN[1]..VU[150]` haben die gleichen Schadenfunktionsparameter[6]:

$$\begin{aligned} S_1(t) &= 30.0 + 5.0 * \mu_1 \\ S_2(t) &= 30.0 + 5.0 * \mu_2 \end{aligned} \tag{5.2}$$

Die Schadenfunktion der ostdeutschen Versicherungsnehmer ist so parametrisiert, daß diese Nachfrager höhere Schäden generieren[7]:

$$\begin{aligned} S_1(t) &= 50.0 + 15.0 * \mu_1 \\ S_2(t) &= 50.0 + 15.0 * \mu_2 \end{aligned} \tag{5.3}$$

Die Erwartungswerte der Schäden sind innerhalb der ost- und westdeutschen Nachfragergruppen gleich. Die Schadeneintrittswahrscheinlichkeiten werden durch eine gleichverteilte Zufallsvariable für jeden Nachfrager einzeln initialisiert. Die Parameter dieser Verteilung sind für alle Nachfrager gleich.

Die Anbieterseite des IMS-Modells der Wiedervereinigung stellt eine Hypothese der Marktform des Versicherungsmarkts dar. Der Anteil der Versicherer mit Verhaltensregeln, die Eigeninformation benutzen, liegt bei 48 %. Da diese Regeln dem Verhalten von monopolistischen Konkurrenten entspricht, ist dieser simulierte Versicherungsmarkt ein Markt *überwiegender* monopolistischer Konkurrenz. Die Konzeption des IMS-Modells erlaubt es dem Anwender, die von der Marktformenlehre vorgegebene starre Einteilung der Märkte zu verlassen und zu untersuchen, welchen Einfluß die Mischung von oligopolistischem und monopolistischem Verhalten in einem Markt auf die Ergebnisse hat. Der Anteil der Verhaltensregeln mit Fremdinformation beträgt 36 %, der Anteil der informationsfreien Verhaltensregeln 16 %.

[5] vgl. GDV (Hrsg.) (1991), S. 13
[6] vgl. 3.2.3
[7] vgl. GDV (Hrsg.) (1992), S. 7; vgl. BAV (Hrsg.) (1992), S. 20

Versicherer	Verhaltensregel
1 – 2	Zufall I
3 – 4	Zufall II
5 – 7	Mark-Up I
8 – 10	Mark-Up II
11 – 13	Mark-Up III
14 – 16	Erwartungsschaden
17 – 19	Dumping
20 – 22	Durchschnitt
23 – 25	Angriff

Tabelle 5.2: Zuordnung der Verhaltensregeln zu den Versicherern

Die Startwerte der Prämien sind bei allen Versicherern gleich. Sie haben alle in der ersten Periode den Wert 40 für beide Sparten. Die Versicherer mit den Verhaltensregeln **Zufall I**, **Dumping** und **Angriff** starten mit Werbeausgaben von 0.0, während alle anderen Versicherer mit Werbeausgaben von 10.0 beginnen.

Der Versicherer `VU[14]` besitzt eine Sonderrolle im Modell der Wiedervereinigung. Er stellt einen Anbieter dar, der durch den Schock der Wiedervereinigung besonders begünstigt ist und soll für den Allianz-Konzern im IMS-Modell stehen[8]. Durch die Kombination eines Aktivierungs- und Änderungsschocks wird der exogene Schock der Wiedervereinigung erzeugt. Teil dieses Schocks ist der *Allianz-Effekt*. Der Allianz-Effekt ist die durch das IMS-Modell erzeugte Bindung der ostdeutschen Nachfrager an den Versicherer `VU[14]`. Die nach dem Schock aktivierten ostdeutschen Nachfrager wählen immer den Anbieter mit den größten Werbeausgaben. `VU[14]` ist der Anbieter, dessen Werbeausgaben nach dem Schock schneller steigen, als die Werbeausgaben aller anderen Anbieter. Ein wichtiger Bestandteil der Analyse der Simulationsdaten ist die Untersuchung der Auswirkungen des Allianz-Effekts. Der `VU[14]` ist die Verhaltensregel **Erwartungsschaden** zugeordnet[9]. Die `VN[151]..VN[190]` benutzen die Nachfrager-Verhaltensregel **Präferenz**. Die Parameter der Verhaltensregel `VU[14]` vor dem Aktivierungsschock der Wiedervereinigung lauten:

$$P_1(t) = \begin{cases} P_1(t-1) \leq \bar{S}_1(t-1) & : \quad 1.02 * P_1(t-1) \\ P_1(t-1) > \bar{S}_1(t-1) & : \quad 0.98 * P_1(t-1) \end{cases} \tag{5.4}$$

$$P_2(t) = \begin{cases} P_2(t-1) \leq \bar{S}_2(t-1) & : \quad 1.02 * P_2(t-1) \\ P_2(t-1) > \bar{S}_2(t-1) & : \quad 0.98 * P_2(t-1) \end{cases} \tag{5.5}$$

$$W_1(t) = \begin{cases} P_1(t-1) \leq \bar{S}_1(t-1) & : \quad 1.00 * W_1(t-1) \\ P_1(t-1) > \bar{S}_1(t-1) & : \quad 1.03 * W_1(t-1) \end{cases} \tag{5.6}$$

[8] vgl. 1.1

[9] vgl. Formeln 3.19 bis 3.22

$$W_2(t) = \begin{cases} P_2(t-1) \leq \bar{S}_2(t-1) & : \quad 1.00 * W_2(t-1) \\ P_2(t-1) > \bar{S}_2(t-1) & : \quad 1.03 * W_2(t-1) \end{cases} \tag{5.7}$$

Auf die Wiedervereinigung reagiert der `VU[14]` mit folgendem Änderungsschock:

$$P_1(t) = \begin{cases} P_1(t-1) \leq \bar{S}_1(t-1) & : \quad 1.03 * P_1(t-1) \\ P_1(t-1) > \bar{S}_1(t-1) & : \quad 0.99 * P_1(t-1) \end{cases} \tag{5.8}$$

$$P_2(t) = \begin{cases} P_2(t-1) \leq \bar{S}_2(t-1) & : \quad 1.01 * P_2(t-1) \\ P_2(t-1) > \bar{S}_2(t-1) & : \quad 0.99 * P_2(t-1) \end{cases} \tag{5.9}$$

$$W_1(t) = \begin{cases} P_1(t-1) \leq \bar{S}_1(t-1) & : \quad 1.06 * W_1(t-1) \\ P_1(t-1) > \bar{S}_1(t-1) & : \quad 1.06 * W_1(t-1) \end{cases} \tag{5.10}$$

$$W_2(t) = \begin{cases} P_2(t-1) \leq \bar{S}_2(t-1) & : \quad 1.06 * W_2(t-1) \\ P_2(t-1) > \bar{S}_2(t-1) & : \quad 1.06 * W_2(t-1) \end{cases} \tag{5.11}$$

Besonders zu beachten sind die Reaktionen der Werbeausgaben vor und nach dem Wiedervereinigungschock. Während der `VU[14]` bereits vor der Wiedervereinigung bei nicht ausreichenden Prämien nicht mit einer Senkung der Werbeausgaben reagiert hat, erhöht er diese Ausgaben nach der Wiedervereinigung unabhängig von der Prämiensituation immer um 6 %. Die nach der Wiedervereinigung aktivierten ostdeutschen Nachfrager wählen ihren Anbieter nach den maximalen Werbeausgaben. Da die anderen Versicherer nach der Wiedervereinigung nicht mit einen gleich hohen Änderungsschock in den Parametern der Funktion der Werbeausgaben reagieren, hat die `VU[14]` nach der Wiedervereinigung immer die höchsten Werbeausgaben und wird von allen Nachfragern, deren Versichererwahl auf Präferenzen basieren, gewählt.

Die vor dem Schock der Wiedervereinigung aktiven Nachfrager mit der Verhaltensregel **Präferenz** besitzen andere Parameter in der Statusfunktion. Die Statusfunktion bestimmt, ob ein Nachfrager in der Periode Eigen- oder Fremdversicherung wählt. Die Statusfunktion der westdeutschen Präferenz-Nachfrager lautet:

$$V_1(t) = \begin{cases} 0.3 \leq \mu_3 & : \quad 0 \\ 0.3 > \mu_3 & : \quad 1 \end{cases} \tag{5.12}$$

$$V_2(t) = \begin{cases} 0.3 \leq \mu_4 & : \quad 0 \\ 0.3 > \mu_4 & : \quad 1 \end{cases} \tag{5.13}$$

Die ostdeutschen Versicherungsnehmer haben in beiden Risiken einen Parameterwert von 0.9, was bedeutet, daß sie sich seltener für die Fremd- und häufiger für die Eigenversicherung entscheiden. Nach dem Schock der Wiedervereinigung ist **Präferenz** die am häufigsten vorkommene Verhaltensregel.

5.4.2 Datenauswertung

Um Einflüsse der Startwerte der Verhaltensregeln und der Zufallsgeneratoren ausschließen zu können, ist die Simulation des IMS-Modells der Wiedervereinigung wiederholt mit identischen Startwerten durchgeführt worden. Die folgenden Untersuchungsergebnisse basieren auf einem Experiment von 30 Simulationsläufen mit jeweils 100 Perioden. In jeder der erzeugten Aggregatdateien lagen 3000 Periodenergebnisse vor.

Zunächst wird die Autokorrelationsfunktion der Ergebnisgrößenvektoren *Reserven* und *Versicherte* in der Sparte 1 des `VU[14]` betrachtet:

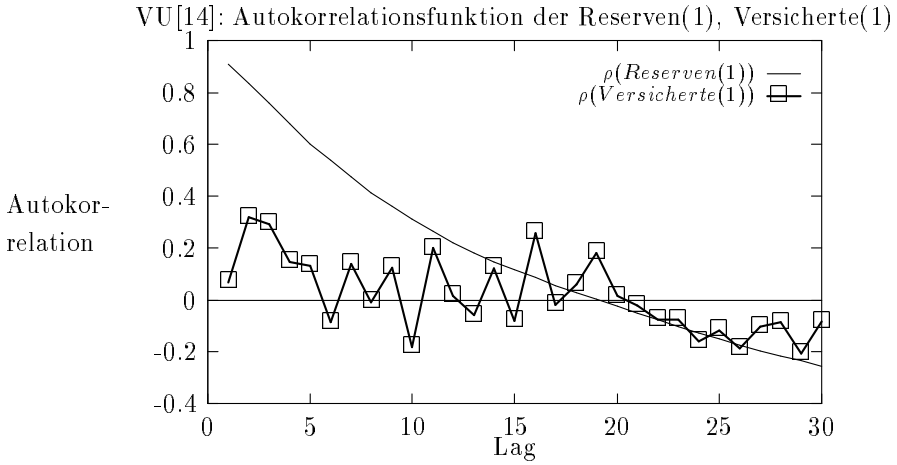

Abbildung 5.2: Autokorrelationsfunktionen der `VU[14]`

Die Autokorrelationsfunktion zeigt an, wie sehr ein Zeitreihenelement der Periode t mit einem Element $t - a$ korreliert ist. Wird a als Laufindex von Eins beginnend erhöht, erhält man eine Funktion, die diese Korrelation mit immer weiter entfernten Vergangenheitswerten darstellt. Die Reserven sind bei einem Lag von Eins in jeder Periode nahezu vollständig mit der Vorperiode korreliert. Ihre Autokorrelationsfunktion besitzt einen stetig abnehmenden Verlauf. Der stochastische Prozeß, der den Reserven unterliegt, ist kaum also statinonär zu bezeichnen. Berücksichtigt man nun, daß die `VU[14]` nicht auf Reserven der Vorperiode reagiert, überrascht das Ergebnis auf den ersten Blick. Beachtet man aber, daß $Pr(t) = f(Pr(t-1), S(t-1))$ ist, also von den Prämien und Schäden der Vorperiode abhängt und das die Reserven $R(t) = g(R(t-1), Pr(t))$ von den Reserven der Vorperiode und den Prämien abhängen, dann wird die stochastische Verknüpfung klar. Die Anzahl der Versicherten der `VU[14]` ist dagegen nur mäßig mit den Werten der Vorperioden korreliert. Ihre Korrelation schwankt relativ konstant im Bereich $-0.2 \ldots 0.3$ und scheint ab einer Verzögerung von mehr als 25 Perioden noch abzunehmen.

Wechselt man nun die Aggregatsstufe von der höchsten Auflösung des Einzelsubjekts zur gröbsten Auslösung der Subjektklasse der Versicherer zeigt sich ein nahezu identisches Bild:

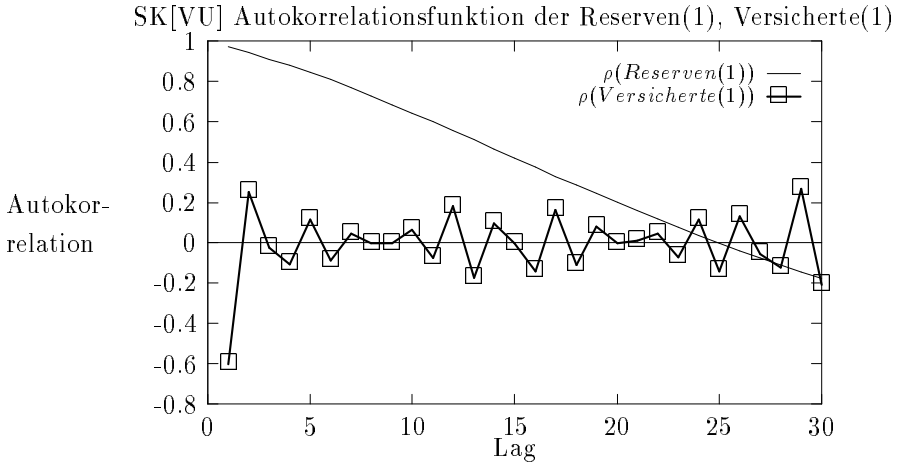

Abbildung 5.3: Autokorrelationsfunktionen der Subjektklasse `VU`

Bis auf einen einmaligen Ausschlag am Anfang schwankt die Autokorrelationsfunktion der Versicherten im Bereich $-0.2 \ldots 0.2$. Die Annahme eines stationären stochastischen Prozesses für die durchschnittliche Anzahl der Versicherten pro Versicherer kann daher als gerechtfertigt angesehen werden. Die Autokorrelation der durchschnittlichen Reserven aller Versicherer pro Periode zeigt fast den gleichen Verlauf wie auf der feineren Aggregationsstufe des Einzelsubjekts und stellt einen nicht-stationären stochastischen Prozeß dar. Eine der grundlegenden dynamischen Eigenschaften von IMS-Modellen ist damit gefunden: Da die stochastische Zusammenhänge $Pr(t) = f(Pr(t-1))$ und $R(t) = g(Pr(t))$ auf der höchsten und niedrigsten Aggregatstufe gelten, stellen die Reserven auf jeder Aggregatstufe einen nicht-stationären Prozeß dar, während die Versicherten pro Anbieter einem stationären Prozeß unterliegen. Damit sind die Endwerte der Reserven in einem Experiment stark von den (möglicherweise unplausiblen) Startwerten des Modells abhängig. Um Effekte, die sich aus diesem Umstand ergeben können zu umgehen, haben alle Versicherer im IMS-Modell der Wiedervereinigung die gleichen Startwerte der Reserven.

Als nächstes sollen die Wirkungen des Allianz-Effektes untersucht werden. Der Versicherer `VU[14]` erhöht nach dem Schock der Wiedervereinigung durch seine spezielle Parametrisierung der Verhaltensregeln seine Werbung stärker als die anderen Anbieter.

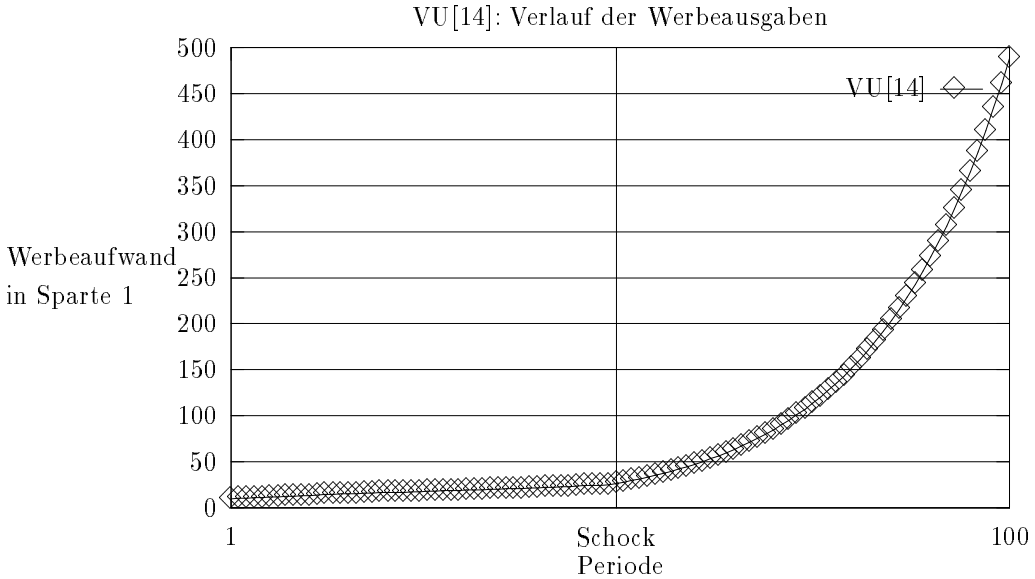

Abbildung 5.4: Werbeausgaben der `VU[14]`

Weil die ostdeutschen Nachfrager aber höhere Schäden als die westdeutschen Versicherungsnehmer haben, muß der Versicherer `VU[14]` gemäß seiner Verhaltensregel **Erwartungsschaden** seine Prämie erhöhen.

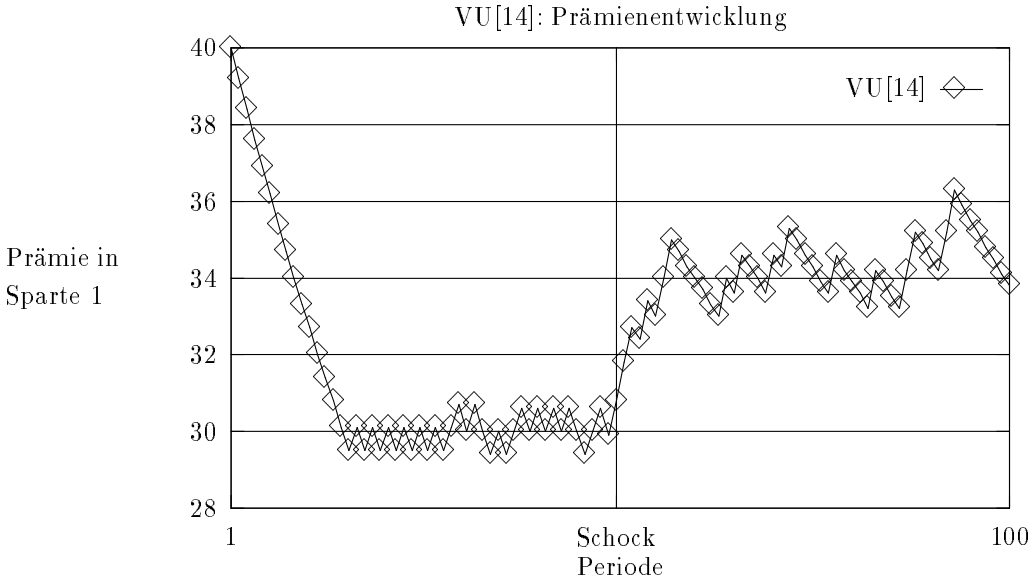

Abbildung 5.5: Prämie der `VU[14]`

Dieses Unternehmensverhalten führt dazu, daß sich sein Marktanteil auf einem höheren Niveau als vor der Wiedervereinigung bei kleineren Schwankungen einpegelt. Diese Marktanteilsentwicklung läßt sich auch in unterschiedlichen Simulationsläufen bestätigen.

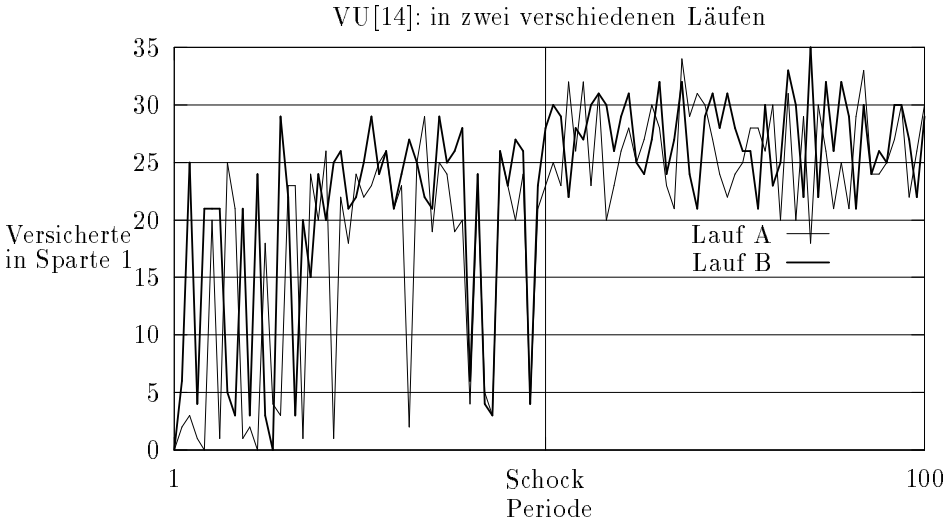

Abbildung 5.6: Marktanteil der `VU[14]` in Sparte 1

In beiden Läufen sinkt die Varianz des Marktanteils nach dem Schock der Wiedervereinigung und er stabilisiert sich auf einem höheren Niveau.

Auch in der Sparte 2 kann diese Entwicklung beobachtet werden:

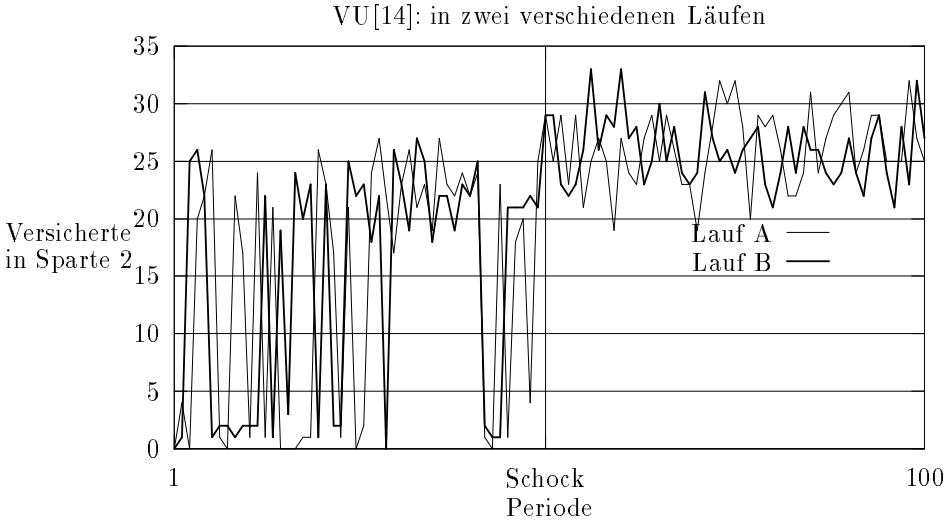

Abbildung 5.7: Marktanteil der `VU[14]` in Sparte 2

Wird eine Reihe des Marktanteils in einer der beiden Sparten aus einem beliebigen Simulationslauf ausgewählt und zusätzlich die gleitenden Durchschnitte über den Kurvenverlauf

gelegt, dann ergibt sich immer ein ähnliches Bild, daß hier an einem Beispiel verdeutlicht werden soll[10]:

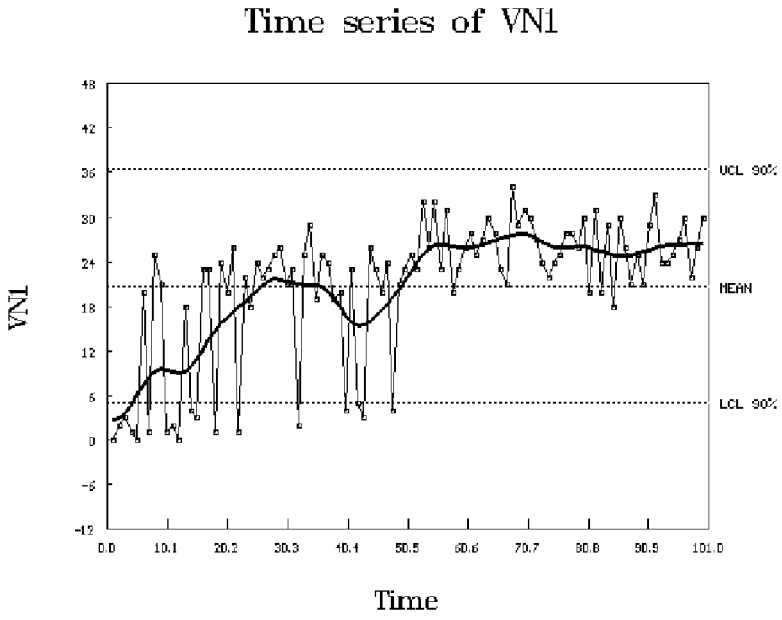

Abbildung 5.8: Marktanteil und gleitende Durchschnitte des Marktanteils `VU[14]`

Die Anzahl der Versicherten stabilisiert sich nach dem exogenen Schock der Wiedervereinigung deutlich über dem Durchschnitt der Gesamtreihe. Die Varianz sinkt, was an dem deutlich glatteren Verlauf der Kurve der gleitenden Durchschnitte abzulesen ist.

Neben dem intuitiven Nachweis an den Periodenreihen, kann die Wirkung des Schocks auf die durchschnittliche Anzahl der Versicherten der `VU[14]` auch statistisch nachgewiesen werden. Dazu wird die Periodenreihe in zwei Teilreihen aufgespalten. Die erste Reihe umfaßt alle Periodenwerte bis zur Periode 50, dem Zeitpunkt des Schocks, die zweite Reihe alle Werte danach. Nun vergleicht man die Mittelwerte der Anzahl der Versicherten vor und nach dem Schock.

[10]Die statistischen Auswertungen wurden mit dem Programm SIMSTAT 3.5 durchgeführt, daß keine deutschen Beschriftungen vorsieht. Die Betitelung dieser Grafiken erfolgt deshalb in englischer Sprache.

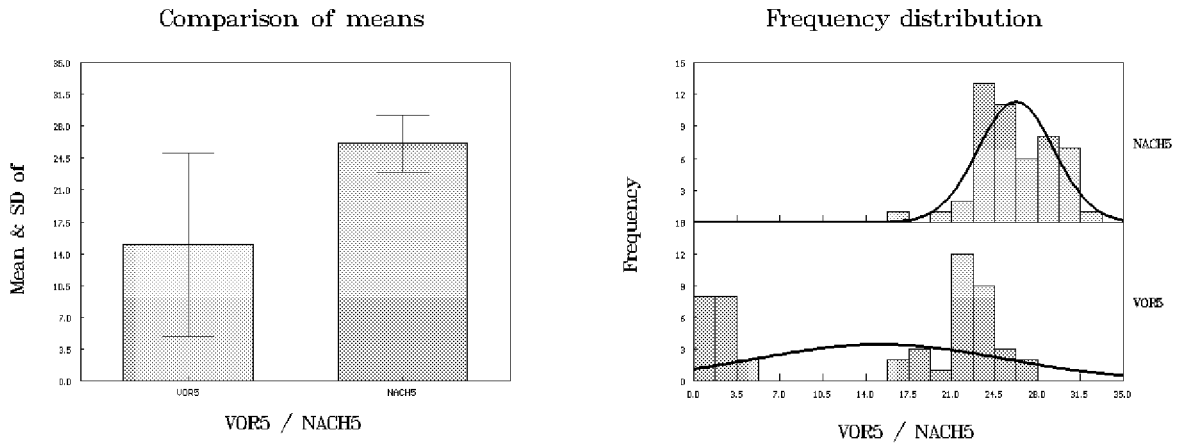

Abbildung 5.9: `VU[14]`: Signifikanz der Schockwirkung auf den Marktanteil

Die linke Grafik zeigt die Höhe des Mittelwertes und seine Standardabweichung vor und nach dem Schock der Wiedervereinigung. Die Differenz der Mittelwerte ist in diesem Beispiel 11.1 und der t-Wert 7.19, was einen hochsignifikanten Unterschied belegt. Die linke Grafik zeigt die Häufigkeitsverteilung der Anzahl der Versicherten der `VU[14]` vor und nach dem Schock. Das Maximum der Verteilung der Stichprobe nach dem Schock ist deutlich größer als das Maximum der Häufigkeitsverteilung vor dem Schock.

Auch am Ergebnisvektor *Reserven* in Sparte 1 lassen sich die verschiedenen Wirkungen der Schockauswirkungen auf unterschiedlichen Aggregatstufen verdeutlichen. Dazu vergleicht man die Reihe der Reserven von `VU[14]` mit der Reihe der durchschnittlichen Reserven der Anbieter-Verhaltensregel **Angriff** und der Reihe der Reserven in Sparte 1 aller Versicherungen:

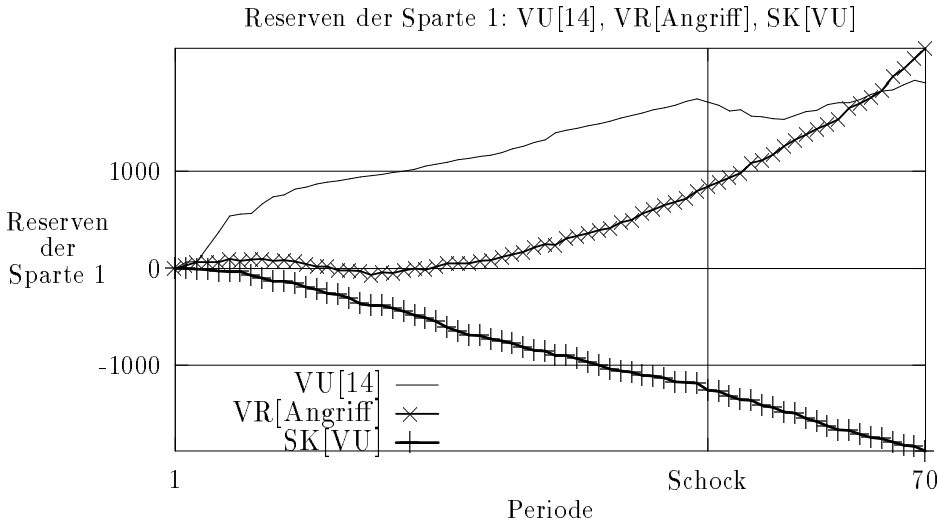

Abbildung 5.10: Vergleich der Reserven auf unterschiedlichen Aggregatstufen

Das Aggregat des Einzelsubjekts `VU[14]` weist nach dem Schock einen deutlichen Einbruch auf, während die beiden anderen Reihen keinen eindeutigen Hinweis auf Schockwirkungen geben.

Aber nicht nur auf der Angebotsseite lassen sich Schockwirkungen nachweisen. Auch bei den Ergebnisvektoren der Nachfrager sind sie feststellbar. Dazu vergleicht man die durchschnittlich gezahlte Prämie unterschiedlicher Aggregate. Als Datenbasis werden alle Aggregate der Stufe drei und das Aggregat der Stufe zwei der Nachfragerregel **Präferenz** herangezogen.

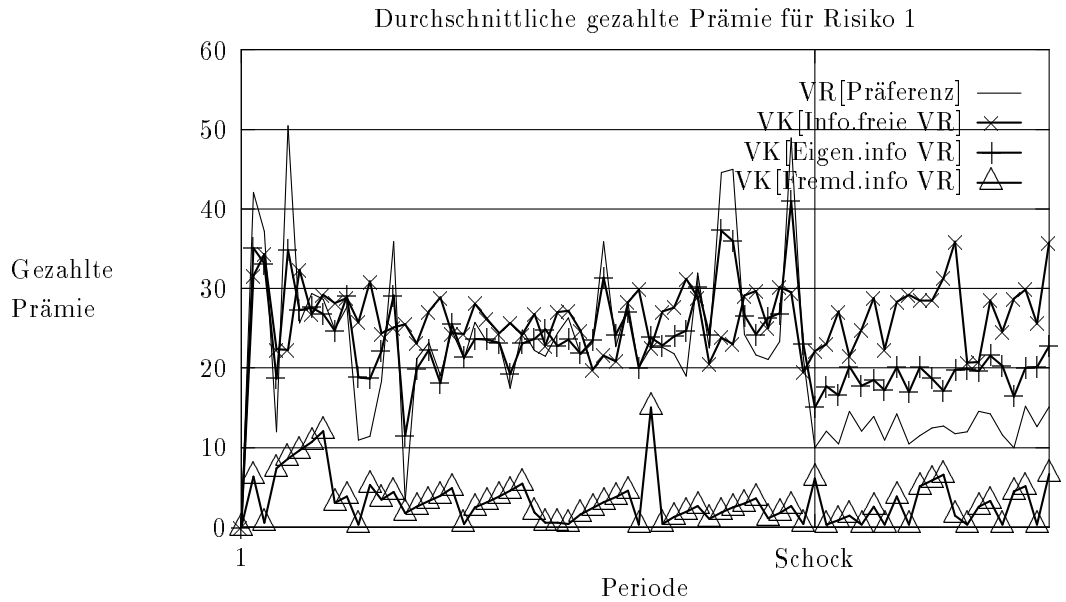

Abbildung 5.11: Vergleich der gezahlten Prämien unterschiedlicher Aggregate

Die Aggregate der Stufe drei fassen die Nachfrager-Verhaltensregeln mit Eigen-, Fremdinformation und die informationsfreien Verhaltensregeln zusammen. Zunächst fällt auf, daß die durchschnittlich gezahlte Prämie aller Nachfrager mit Verhaltensregeln auf der Basis von Fremdinformationen (**Suche** und **Beste Information**) immer die geringste Prämie im Durchschnitt zahlen. Sie wählen durchgängig einen Versicherer mit der Verhaltensregel **Zufall I**, dessen Prämie die größte Spannweite hat und dementsprechend häufig die kleinste Prämie in der Periode hat. Die Aggregate der informationsfreien Verhaltensregeln, der Verhaltensregeln mit Eigeninformation und das Aggregat aller Nachfrager mit der Verhaltensregel **Präferenz** weisen vor dem Schock keine eindeutige Rangfolge auf. Das ändert sich nach dem Schock. Nachfrager mit der Verhaltensregel **Präferenz** zahlen eine deutlich niedrigere Prämie als die anderen beiden Aggregate. Auf dem zweiten Platz liegt das Aggregat der Verhaltensregeln mit Eigeninformationen. Das ist durch den hohen Anteil der Verhaltensregel **Präferenz** an diesem Aggregat zu begründen. Die informationsfreien Verhaltensregeln bleiben vom Schock unberührt.

Neben der grafischen Ableitung dieses Ergebnisses, läßt sich auf der Basis der statistischen

Analyse der Simulationsdaten diese Schockwirkung begründen. Dazu vergleicht man wieder die Mittelwerte der durchschnittlich gezahlten Prämie vor und nach dem Schock und stellt deren Häufigkeitsverteilungen auf.

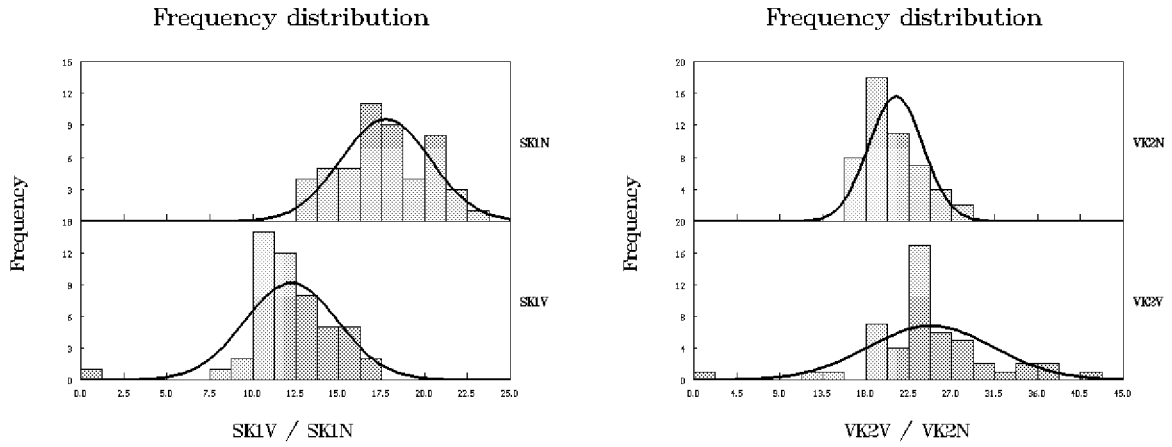

Abbildung 5.12: Signifikanzvergleich der Schockwirkung auf die gezahlte Prämie

Die linke Grafik zeigt die Verteilung der durchschnittlichen gezahlte Prämie vor und nach dem Schock für die Aggregatstufe Subjektklasse, die rechte Grafik stellt diese Verteilung für alle Nachfrager mit den Verhaltensregeln **Präferenz** und **Totale Erinnerung** dar. Die Verteilung der linken Grafik zeigt, daß alle Versicherten nach dem Schock der Wiedervereinigung eine höhere Prämie zahlen als vorher. In diesem Simulationslauf beträgt die Differenz 5.5 und der t-Wert des Mittelwertvergleichs 10.9. Für die Verhaltensregeln **Zufall I** und **Zufall II** ist diese Differenz noch etwas größer (absolut 5.7) und noch signifikanter (t-Wert: 11.4).

Das Aggregat der Verhaltensregeln **Präferenz** und **Totale Erinnerung** dagegen zahlt nach dem Schock der Wiedervereinigung eine um absolut 3.6 niedrigere Prämie (t-Wert: 3.8). Die Nachfragerregeln, die auf Fremdinformation basieren weisen dagegen keinen signifikanten Unterschied der durchschnittlich gezahlten Prämie vor und nach dem Schock auf.

5.4.3 Ableitung der Modellaussagen

Das IMS-Modell der Wiedervereinigung ermöglicht durch die unterschiedlichen Perspektiven der Aggregate eine differenzierte Untersuchung der Angebots- und Nachfrageseite des simulierten Versicherungsmarktes. Durch die getrennte Untersuchung der Simulationsdaten vor und nach dem exogenen Schock der Wiedervereinigung können die qualitativen und quantitativen Schockwirkungen getrennt analysiert werden. Pauschale, undifferenzierte Deskriptivaussagen werden so vermieden.

Die Analyse der Autokorrelationsfunktionen auf unterschiedlichen Aggregatstufen zeigt, daß die grundlegenden dynamischen Eigenschaften des IMS-Modells auch für das Modell der Wiedervereinigung gelten. Die generelle Abhängigkeit der Prämie von der Prämie

der Vorperiode stellt aber auch ein reales Phänomen dar, so daß das Modell hier eine Eigenschaft realer Versicherungsmärkte abbildet. Die verwendeten Verhaltensregeln auf der Anbieter- und Nachfragerseite zeigen ein modellkonformes Verhalten. So zahlen z.B. die Nachfrager mit den Regeln **Suche** und **Beste Information** auch im Modell der Wiedervereinigung signifikant niedrigere Prämien als alle anderen Nachfragerregeln, was mit den Modellerwartungen übereinstimmt.

Als besonders hilfreich für die Analyse von Schockwirkungen haben sich die unterschiedlichen Aggregatstufen für die Angebots- und Nachfragerregeln erwiesen. Festzustellen war, daß sich die Auswirkungen von Schocks durch die Disaggregation viel genauer untersuchen ließen als es mit hochaggregierten Daten möglich gewesen wäre. Diese Ergebnis bestätigt auch die Entscheidung, die Simulationsdaten nicht mit realen Daten zu mischen oder ganz auf hochaggregierten realen Statistiken basieren zu lassen. Eine differenzierte Analyse der Marktprozesse bei Schocks ist nur mit disaggregierten Daten möglich. Trotz dieses Verzichts zeigt das Verhalten der simulierten Wirtschaftssubjekte und die erhaltenen Ergebnisse ein hohes Maß an Übereinstimmung mit den realen Prozessen auf Versicherungsmärkten.

Der Modellierungsansatz des IMS-Modells der Wiedervereinigung basiert auf zwei Schlüsselfaktoren:

- Dem kombinierten Aktivierungs- und Änderungsschock in der Mitte der Simulationsläufe und
- dem Allianz-Effekt.

Der Aktivierungsschock ließ 25 % mehr Nachfrager in den Markt eintreten, der Änderungsschock modellierte die Änderungen der Verhaltensparameter auf der Angebots- und Nachfrageseite. Der Allianz-Effekt führte dazu, daß ein Versicherer einen besonders hohen Anteil an dem zusätzlichen Marktvolumen errang.

Aus der Perspektive aller Nachfrager ist eine schockbedingte Prämienerhöhung für den gesamten Versicherungsmarkt feststellbar. Untersucht man aber die disaggreagierten Daten der einzelnen Verhaltensregeln und -regelklassen, dann ergibt sich ein anderes Bild. Die Prämienerhöhung wird zum größten Teil von den Nachfragern getragen, deren Verhalten auf informationsfreien Regeln basiert. Dagegen ist bei Versicherungsnehmern die Suchverhalten aufweisen keine Erhöhung der durchschnittlich gezahlten Prämie nach der Wiedervereinigung feststellbar. Für Nachfrager mit der Verhaltensregel **Präferenz** ist sogar ein Prämienrückgang nachzuweisen. Dieser Rückgang ist eine mittelbare Auswirkung des Allianz-Effekts, da diese Nachfrager nach dem Schock ausschließlich den Allianz-Versicherer wählen. Obwohl auch dieser Anbieter nach dem Schock seine Prämien erhöht, ist seine Anhebung im Vergleich zu anderen Versicherern niedriger, so daß Nachfrager, die ihn wählen von ihrem Verhalten nach der Wiedervereinigung profitieren.

Die nach der Wiedervereinigung viel diskutierte Frage, ob durch den Allianz-Effekt das Marktgefüge der deutschen Versicherungsmärkte nachhaltig beeinflußt wird, kann im Rah-

men dieser Simulationsstudie verneint werden. Der höhere Marktanteil, den der Allianz-Versicherer nach der Wiedervereinigung erhält, führt nicht automatisch dazu, daß andere Marktteilnehmer beeinträchtigt werden. Die Analyse der Entwicklung der Reserven zeigt, daß der Schock der Wiedervereinigung nur kurzfristige Ausschläge bedingt, der langfristige Markterfolg eines Anbieters aber weniger von seinen Verhaltensparametern und mehr von der von ihm angewendeten Verhaltensregel abhängt. Die Analyse wiederholter Simulationsläufe bestätigt die gefundenen Ergebnisse und läßt dadurch den Schluß zu, daß sie modellstabil sind und nicht zufallsbedingt.

5.5 Die Variation

5.5.1 Modellkonfiguration

In der Variation des IMS-Modells der Wiedervereinigung wird das Abstraktionsniveau noch weiter erhöht. Auf der Anbieterseite wird homogenes Verhalten angenommen. Alle 25 simulierten Versicherer verhalten sich nach der Regeln **Erwartungsschaden**[11]. Die Parameter ihrer Verhaltensregeln sind identisch. Kein Anbieter betreibt Werbung. Die Prämienreaktionen und die Startprämien sind in den beiden Sparten unterschiedlich. In Sparte 1 verlangt jeder Versicherer in der ersten Periode $P_1(1) = 40$ und $P_2(1) = 35$ in Sparte 2. Die Prämienreaktionen in den Sparten sind verschieden:

$$P_1(t) = \begin{cases} P_1(t-1) \leq \bar{S}_1(t-1) & : \quad 1.03 * P_1(t-1) \\ P_1(t-1) > \bar{S}_1(t-1) & : \quad 0.97 * P_1(t-1) \end{cases} \tag{5.14}$$

$$P_2(t) = \begin{cases} P_2(t-1) \leq \bar{S}_2(t-1) & : \quad 1.05 * P_2(t-1) \\ P_2(t-1) > \bar{S}_2(t-1) & : \quad 0.95 * P_2(t-1) \end{cases} \tag{5.15}$$

Die Verhaltensregel **Erwartungsschaden** erhöht die Prämie der Vorperiode um einen bestimmten Prozentsatz, wenn die durchschnittliche Schadenhöhe größer war als die verlangte Prämie der Vorperiode. War die durchschnittliche Schadenhöhe dagegen kleiner, wird die Prämie gesenkt. Die Regeln reagieren in beiden Sparten symmetrisch. Wenn die Prämie in Sparte 1 zu gering angesetzt war, wird sie um 3 % erhöht und um 3 % gesenkt, falls sie zu hoch war. In Sparte zwei beträgt die Prämienreaktion in beiden Richtungen 5 %. Die Anbieter können in Sparte 2 also schneller auf Änderungen reagieren.

Auf der Nachfragerseite werden nur informationsfreie Verhaltensregeln eingesetzt. In den ersten 50 Perioden sind 50 % der Nachfrager aktiv. Sie verhalten sich nach der Regeln **Zufall II**. Sie generieren in beiden Sparten gleich hohe Schäden und wählen in jeder Periode mit einer Wahrscheinlichkeit von 0.5 den Status Eigenversicherung. Ab Periode 51 werden weitere 50 % der Nachfrager marktaktiv. Sie verwenden die Verhaltensregel **Zufall I** und generieren durchschnittlich größere Schäden als die Nachfrager der ersten Gruppe. Sie wählen in jeder Periode die Fremdversicherung.

[11] vgl. Formeln 3.19–3.22

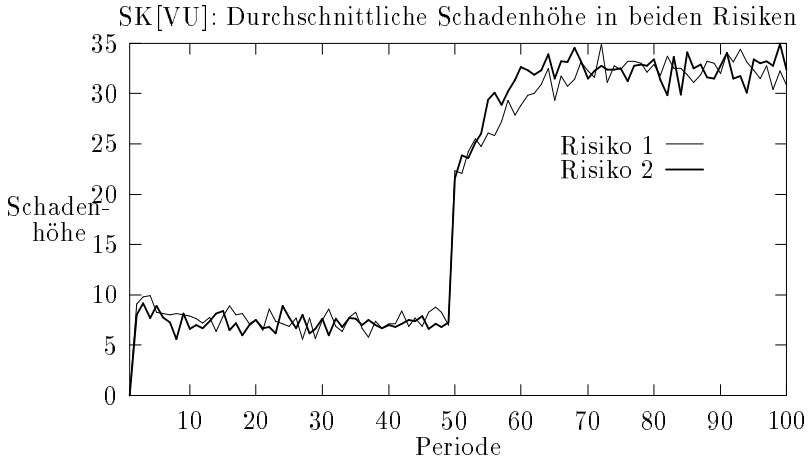

Abbildung 5.13: Durchschnittliche Schadenhöhe in beiden Risiken

Die durchschnittliche Schadenhöhe in beiden Risiken steigt nach dem Aktivierungsschock von ungefähr 9 auf circa 33.

Die unterschiedlichen Startwerte der Prämien und die verschieden hohen Reaktionsparameter der Prämienfunktionen finden auch auf der Aggregationsstufe der Subjektklasse ihren Niederschlag:

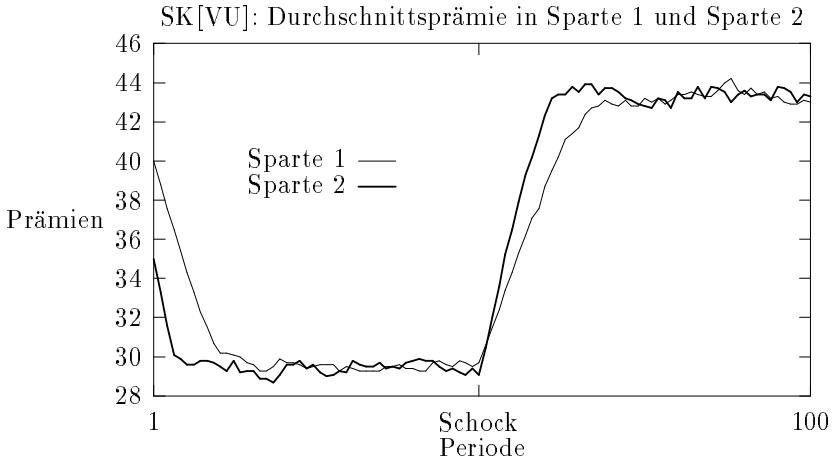

Abbildung 5.14: Durchschnittsprämie in beiden Sparten

In Sparte 2 beginnen alle Versicherer mit einer Prämie von 35. Nach wenigen Perioden hat sie sich der durchschnittlichen Schadenhöhe angepaßt und schwankt nur noch wenig. Die durchschnittliche Prämie in Sparte 1 braucht aufgrund der geringeren Prämienreaktionshöhe länger für die Anpassung an die durchschnittliche Schadenhöhe.

5.5.2 Datenauswertung

Betrachtet man zunächst die durchschnittlichen Reserven auf der Aggregatstufe der Subjektklasse, dann lassen sich für beide Sparten unterschiedliche Verläufe feststellen.

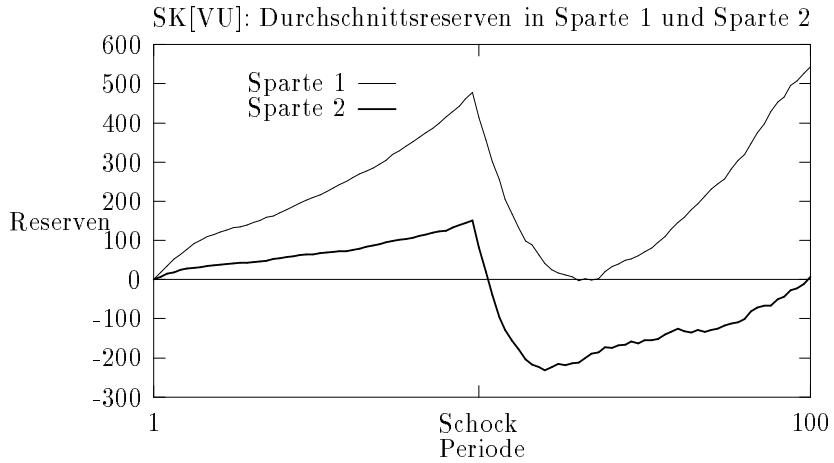

Abbildung 5.15: Durchschnittsreserven in beiden Sparten

Bis zur Periode des Aktivierungsschocks auf der Nachfrageseite steigen die durchschnittlichen Reserven zunächst an und erreichen ihr Maximum in Periode 50. Durch die Aktivierung zusätzlicher Nachfrager mit anderen Schadenfunktionen müssen alle Versicherer zunächst einen Anpassungsprozeß an die neue durchschnittliche Schadenhöhe durchlaufen. Diese Anpassung dauert in der ersten Sparte länger als in der zweiten Sparte. Weil die Anpassung an die richtige Prämie in der zweiten Sparte schneller erfolgt, können weniger zusätzlichen Reserven durch überhöhte Prämien angesammelt werden. Das Maximum der durchschnittlichen Reserven in Sparte 2 ist deshalb auch deutlich niedriger als in Sparte 1.

Nach dem Schock auf der Nachfrageseite erweist sich die höhere Reaktionsgewindigkeit in der zweiten Sparte aber als Vorteil. Der Anpassungsprozeß an die höhere Prämie erfolgt schneller. Das Ergebnis dieser Anpassung spiegelt sich auch in den durchschnittlichen Reserven wider. Das Minimum der Reserven liegt in Sparte 2 zeitlich vor dem Reservenminimum der ersten Sparte. Nach einer kürzeren Anpassungsphase steigen die durchschnittlichen Reserven in Sparte zwei langsamer als in Sparte 1.

Wechseln man nun die Beobachtungsperspektive von der Aggregatstufe der Subjektklasse auf die Aggregatstufe der Einzelversicherung dann stellen sich die Verläufe der Reserven in Sparte 1 wie folgt dar:

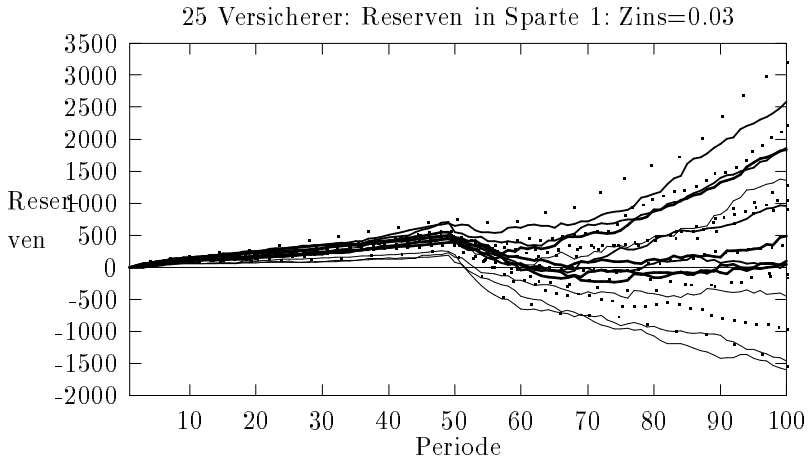

Abbildung 5.16: Reserven der Einzelversicherer in Sparte 1, Zins 3 %

Bis zum Aktivierungsschock steigen die Reserven gleichförmig. Ihre Schwankungsbreite ist relativ gering ist sie erhöht sich bis zur Periode 50 kaum. Alle Einzelversicherer haben positive Reserven. Nach dem Aktivierungsschock fächern sich die Reserven stark auf. In der letzten Periode beträgt die Spannbreite fast 5000, in der letzten Periode vor dem Schock circa 500. Ähnliche Verläufe lassen sich auch in der zweiten Sparte beobachten.

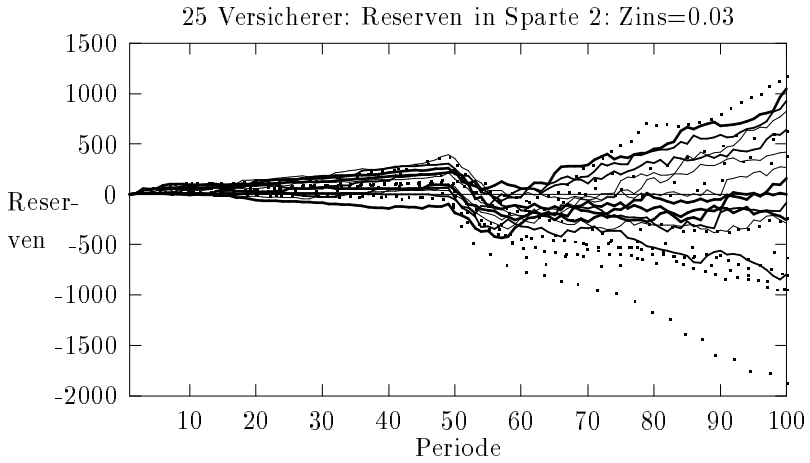

Abbildung 5.17: Reserven der Einzelversicherer in Sparte 2, Zins 3 %

Aufgrund der kürzen Anpassungsdauer an die richtige Prämienhöhe und der Schadenschwankungen sind bereits vor dem Schock die Reserven einiger Anbieter negativ. Trotz der schnelleren Prämienreaktiv schaffen es nur wenige Versicherer in der zweiten Sparte nach dem Schock wieder positive Reserven zu erreichen. Auch in dieser Sparte führt der Schock zu einem sehr heterogenen Reservenbestand.

Wird der Zins, mit dem die Reserven diskontiert werden, von bislang 3 % auf 0 % gesenkt, dann treten die Auswirkungen des Schocks in beiden Sparten noch deutlicher hervor:

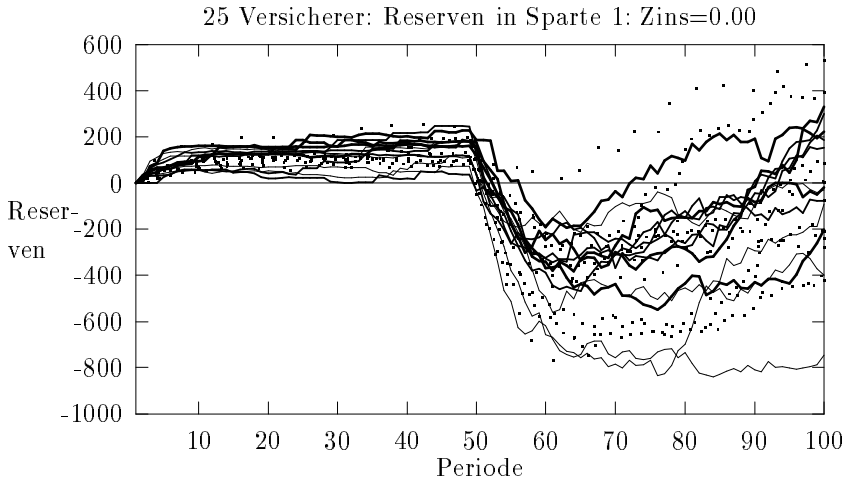

Abbildung 5.18: Reserven der Einzelversicherer in Sparte 1, Zins 0 %

Weil durch die fehlende Verzinsung vor dem Schock nur wenig Reserven angesammelt wurden, sind die Schockwirkungen in beiden Sparten besser zu beobachten.

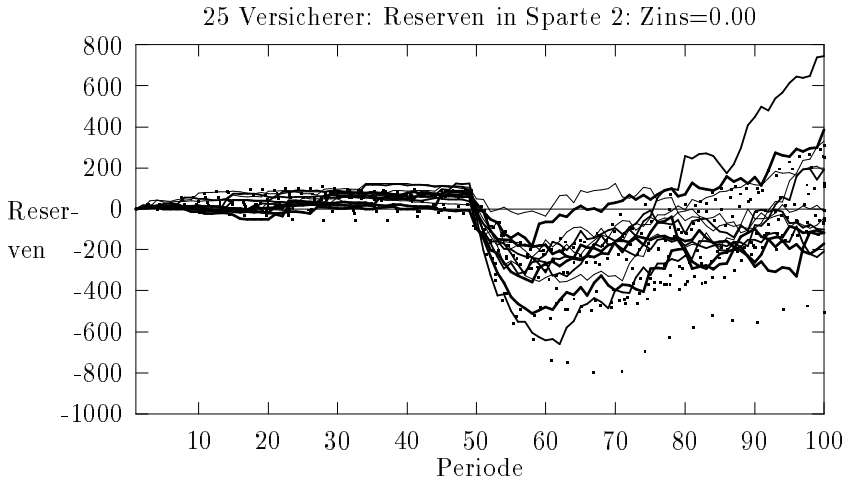

Abbildung 5.19: Reserven der Einzelversicherer in Sparte 2, Zins 0 %

Durch die langsamere Reaktionsgeschwindigkeit bleiben in der ersten Sparte viele Versicherer länger im Bereich negativer Reserven. In der zweiten Sparte ist bereits wenige Perioden nach dem Schock eine allgemeine Erholung der Reserven festzustellen.

5.5.3 Ableitung der Modellaussagen

Im Mittelpunkt der Ableitung von Modellaussagen dieser Modellvariation steht nicht eine bestimmte Versicherung, sondern die Gesamtmarkteffekte. Von zentraler Bedeutung ist die unterschiedliche Reaktionsgeschwindigkeit der Prämienfunktionen in Sparte 1 und 2. Es zeigte sich, daß die niedrigere Reaktionsstärke in Sparte 1 zum Aufbau größerer Reserven führte, die sich auch als "Trägheitsgewinne" bezeichnen lassen. Nach dem Aktivierungsschock erweißt sich die größere Reaktionsgeschwindigkeit in Sparte 2 als Vorteil, weil nun die neue, richtige Prämienhöhe schneller erreicht wird. In der ersten Sparte werden, weil dieser Anpassungsprozeß länger dauert, nach dem Schock "Trägheitsverluste" gemacht.

Auf der Ebene der Einzelversicherer führt der Aktivierungsschock zu einer höheren Streuung der Reservenverläufe in beiden Sparten. Wird der Zinseffekt der Reserven ausgeblendet, indem man den Zins auf Null setzt, dann tritt der Vorteil der schnelleren Reaktionsgeschwindigkeit der Prämie in Sparte 2 noch deutlicher hervor. Dreht man diese Argumentation um, dann muß es einen Zinssatz geben, der den Nachteil einer langsameren Prämienreaktion nach einem Schock aufhebt.

Generell zeigt sich, daß "träge" Versicherer, die Prämienänderungen nur langsam an ihre Kunden weitergeben, nicht generell im Vorteil gegenüber Anbietern sind, die Prämienänderungen schnell weitergeben. Bei homogenem Verhalten auf der Anbieterseite läßt sich nicht a priori sagen, ob oder welcher Anbieter nach dem Schock positive Reserven hat. Unabhängig von der Höhe der Verzinsung der Reserven kommt es immer zu einer Heterogenierierung der Reservenverläufe nach dem Schock.

Kapitel 6

Die Ergebnisse

6.1 Die Ergebnisse in Thesen

- Die Wiedervereinigung der deutschen Versicherungsmärkte wird als ein exogener Schock aufgefaßt, dessen Auswirkungen mit Hilfe eines Computermodells untersucht werden sollen.
- Die Gliederung dieser Arbeit orientiert sich an den Entwicklungsphasen einer Simulationsstudie:
  1. Modellierung: Kapitel 2 und 3,
  2. Programmierung: Kapitel 4,
  3. Experiment: Kapitel 5.
- Schocks und ihre Auswirkungen werden in Marktprozessen sichtbar. Eine einfache komparativ-statische Analyse kann der Untersuchung der Auswirkungen von Schocks nicht gerecht werden. Deshalb werden sie in einem dynamischen Versicherungsmarktmodell mit expliziter Modellierung des Marktprozesses untersucht.
- Aus Mangel geeigneter analytischer Instrumente zur Schockanalyse und aufgrund fehlender Aussagen mikroökonomischer Modelle zu diesem Bereich, wurde bei der Untersuchung einer nummerischen Speziallösung (Simulation) mit realistischeren Annahmen der Vorzug gegenüber einer allgemeineren analytischen Lösung mit unrealistischeren Annahmen gegeben.
- Das Marktmodell enthält keinen internen Koordinationsmechanismus, der einen Pfad zum Marktgleichgewicht vorzeichnet. Es wird von einem permanenten Ungleichgewicht ausgegangen.

- Das Verhalten der simulierten Marktteilnehmer wird durch ihre Informationsmenge bestimmt. Diese Informationen werden in den ihnen zugewiesenen Verhaltensregeln verarbeitet.
- Die Verhaltensregeln der Anbieter und Nachfrager sind aus zwei Teiltheorien der behavioristischen Wirtschaftstheorie abgeleitet:
  1. Der Anspruchsniveautheorie
  2. Der Adaptionstheorie
- Das IMS-Modell sieht drei Arten von Schocks vor
  1. Aktivierungsschocks:
     Wann tritt ein Marktteilnehmer in den Marktprozeß ein ?
  2. Änderungsschocks:
     Wie ändern sich die Verhaltensparameter eines Marktteilnehmers nach einem Schock ?
  3. Indirekte Schocks:
     Wie wirken Schocks der Nachfrager auf die Anbieter und umgekehrt ?
- Es gibt drei Arten von Regeln, die das Anbieterverhalten steuern:
  - Informationsfreie Verhaltensregeln
    1. **Zufall I** bestimmt Werbeaufwendungen und Prämienhöhe durch gleichverteilte Pseudo-Zufallsvariablen.
    2. **Zufall II** besitzt Verteilungsinformationen die in die Prämien- und Werbeaufwandsbestimmung einfließen. Die Parameter der Schadenfunktionen kennt diese Regel nicht. Prämie und Werbeverhalten basieren auf den gleichen Informationsmengen.
  - Verhaltensregeln mit Eigeninformationen
    1. **Mark-Up I** ändert Prämien und Werbung durch linearen Zu- oder Abschlag auf der Basis der Prämien- und Werbeaufwendungen der Vorperiode. Änderungsauslöser sind unterschrittene Anspruchsniveaus der Reserven.
    2. **Mark-Up II** reagiert wie **Mark-Up I**, aber der Änderungsauslöser sind Anspruchsniveaus der Nettowechsler.
    3. **Mark-Up III** reagiert wie **Mark-Up I**, aber der Änderungsauslöser sind Anspruchsniveaus der Mindestmarktanteile.
    4. **Erwartungsschaden** reagiert wie die Mark-Up-Regeln. Änderungsauslöser sind die durchschnittlichen Schadenhöhen.

- Verhaltensregeln mit Fremdinformationen
  1. **Dumping** orientiert sich an der kleinsten Prämie und an den höchsten Werbeaufwendungen anderer Anbieter in der Vorperiode.
  2. **Durchschnitt** wählt durchschnittliche Prämienhöhen und durchschnittliche Werbeaufwendungen der Vorperiode in beiden Sparten.
  3. **Angriff** orientiert seine Entscheidungen an dem Anbieter, der in der Vorperiode die höchsten Reserven hatte und als Marktführer bezeichnet wird.

Die Verhaltensregeln mit Eigeninformationen basieren auf der Anspruchsniveautheorie der verhaltenswissenschaftlichen Unternehmenstheorie und stellen das Verhalten der monopolistischen Konkurrenz dar. Die Verhaltensregeln mit Fremdinformation modellieren oligopolistisches Verhalten und basieren auf der adaptiven Oligopoltheorie.

- In den drei Arten der Nachfragerregeln sind jeweils zwei Regeln definiert:
  1. Informationsfreie Verhaltensregeln:
     1.1. **Zufall I** wählt immer die Fremdversicherung und einen Versicherer zufällig.
     1.2. **Zufall II** wählt zufällig zwischen Eigen- und Fremdversicherung und wählt in Abhängigkeit davon einen Versicherer zufällig.
  2. Verhaltensregeln mit Eigeninformation
     2.1. **Präferenz** bestimmt den Versicherungsstatus aus den eigenen subjektiven Schadeneintrittswahrscheinlichkeiten und wählt den Anbieter nach seinen maximalen Präferenzen aus. Die Präferenzen werden durch die Werbeaufwendungen der Anbieter bestimmt.
     2.2. **Totale Erinnerung** bestimmt den Versicherungsstatus aus den eigenen subjektiven Wahrscheinlichkeiten und wählt den Anbieter, der in der Vergangenheit die niedrigste Prämie verlangt hat.
  3. Verhaltensregeln mit Fremdinformation
     3.1. **Suche** wählt nach dem Versicherungsgrad der jeweiligen Sparte den eigenen Versicherungsstatus. Zur Anbieterwahl muß eine Stichprobe von Prämieninformationen gezogen werden. Aus dieser Stichprobe wird der Anbieter gewählt, der die niedrigste Prämie in der jeweiligen Sparte verlangt.
     3.2. **Beste Information** wählt nach dem Versicherungsgrad der jeweiligen Sparte den eigenen Versicherungsstatus. Bei der Statuswahl der Fremdversicherung, wird immer der Versicherer gewählt, der in der Vorperiode die niedrigste Prämie verlangt hat.

- Die diskrete Simulation IMS wurde mit ESL programmiert. ESL ist eine speziell für die Erstellung ökonomischer Simulationen konzipierte Sprachergänzung der Hochsprache C. Durch diese Kombination konnte der Geschwindigkeitsvorteil von C bei der Programmierung erhalten werden.

- Die Simulationsdaten sind in Aggregationsdateien gespeichert. Diese Dateien stellen die Rohdaten der Simulation dar. Programmintern sind grafische Analysen der Simulationsdaten möglich. Die statistischen Datenauswertungen werden von externen Programmen ausgeführt.

- Ein Schwerpunkt der Programmentwicklung war die Portabilität. IMS ist für die Betriebssysteme MS-DOS, LINUX und Solaris verfügbar.

- Das IMS-Modell der Wiedervereinigung der deutschen Versicherungsmärkte besitzt zwei Schlüsselfaktoren:

  - Den Aktivierungschock 25 % zusätzlicher Nachfrager,
  - Den Allianz-Effekt, bei dem ein Versicherer nach dem Schock der Wiedervereinigung seine Werbeausgaben stärker als alle anderen Anbieter erhöht und dadurch die neuen präferenzbestimmten Nachfrager an sich binden kann.

- Die wichtigsten Ergebnisse der Analyse des IMS-Modells der Wiedervereinigung sind:

  - Aus der Perspektive aller Nachfrager erhöht sich die durchschnittliche Prämie nach dem Schock der Wiedervereinigung.
  - Nachfrager mit informationsfreien Verhaltensregeln tragen die Hauptlast der Prämienerhöhung, während Nachfrager mit Suchverhalten und mit voller Prämieninformation von diesem Effekt nicht betroffen sind. Die nach dem Schock aktivierten Nachfrager zahlen eine signifikant niedrigere Prämie als die Versicherungsnehmer mit der gleichen Verhaltensregel vor der Wiedervereinigung. Die Erklärung dafür liegt in der Verhaltensregel, die der Allianz-Versicherer benutzt. Seine Prämienerhöhung nach dem Schock der Wiedervereinigung ist niedriger als die Anhebungen der meisten anderen Versicherer.
  - Der Marktanteil der simulierten Allianzversicherung stabilisiert sich nach der Wiedervereinigung auf einem höheren Niveau. Durch die gegebene Verhaltensverteilung auf der Nachfragerseite kommt es aber nicht zu generellen Konzentrationserscheinungen, da die präferenzgesteuerten Nachfrager, die den Marktanteil der Allianz-Versicherung begründen, zwar einen hohen, aber keinen überragenden Anteil an der Gesamtmenge der Nachfrager haben.
  - Die Reserven des Allianz-Versicherers als Maßstab der Überlebensfähigkeit einer Versicherung im IMS-Modell werden durch den Schock nicht nachhaltig beeinflußt, so daß nicht von einem besonderen Marktvorteil des simulierten Allianz-Versicherers durch die zusätzliche Nachfrage ausgegangen werden kann.

- Die wichtigsten Ergebnisse der Analyse der Modellvariation sind:

  - Vor einem Aktivierungsschock führt eine geringere Reaktionsgeschwindigkeit der Prämienfunktion zum Aufbau von Trägheitsgewinnen. Nach einem Schock

werden diese Gewinne durch einen längeren Anpassungsprozeß an eine höhere durchschnittliche Schadenhöhe wieder gemindert.

- Ein Aktivierungsschock führt zu einer starken Heterogenisierung der Reservenverläufe in beiden Sparten.
- Diese Heterogenisierung ist unabhängig von der Höhe der Verzinsung der Reserven.

6.2 Ausblick und Schluß

Der Titel dieser Arbeit offenbart bereits, daß die hier erstellte Simulation nicht nur zur Analyse eines ganz bestimmten Schocks verwendet werden kann. Das Programm ist durchaus nicht nur für den einmaligen Gebrauch im Rahmen dieser Arbeit entwickelt worden. Der Ausblick dieser Arbeit wird sich daher nicht mit allgemeinen Betrachtungen über zukünftige Entwicklungen der Ökonomie im allgemeinen, sondern mit dem konkreten Entwicklungspotential des hier vorgestellten Programms im besonderen befassen.

Die Entwicklungsmöglichkeiten dieser Arbeiten können in zwei Felder unterschieden werden:

1. Die Weiterentwicklung des Programms und
2. die Weiterentwicklung des Modells.

Die Weiterentwicklung des Programms setzt an dem bereits vorhandenen Quelltext an. Die vorhandenen vordefinierten Modelle können verändert oder neue vordefinierte Modelle programmiert werden. Ein weites Betätigungsfeld bieten die Auswertungsroutinen. In der vorliegenden Version von IMS werden die Simulationsdaten größtenteils mit externen Programmen weiterverarbeitet. IMS selbst wird fast ausschließlich nur zur Eingabe der Parameter und zur Berechnung der Simulationsrohdaten benötigt. Die Unterprogramme zur einfachen Darstellung der Periodendaten benutzen ein externes Programm zum Daten plotten. Die statistische Auswertung der Daten erfolgt komplett mit externen Statistikprogrammen. Die gesamte Benutzeroberfläche, die sich bislang auf sehr einfache, aber portable Routinen stützt, könnte auf eine grafische Benutzeroberfläche umgestellt werden. Ein System von Hilfstexten, die innerhalb von IMS den Anwender in die Möglichkeiten des Programms einführen, wäre eine weitere denkbare Erweiterung.

Neben den Weiterentwicklungen des Programms könnte auch das Modell weiter ausgebaut werden. Die erste Möglichkeit dazu bieten die vorhandenen Verhaltensregeln, die verändert werden können. Zusätzlich ist jeweils eine leere Verhaltensregel für Nachfrager und Anbieter programmiert: `Vrvu10` und `Vrvn07`. Diese Regeln enthalten nur die notwendigen Verwaltungsroutinen und können die Grundlage für eigene Verhaltensregeln bilden. Tiefergehende Programmänderungen sind mit der Änderung der Datenstruktur verbunden, weil sie die Basis des Programms bildet.

Eine interessante Entwicklungsmöglichkeit wäre die Modifikation einer Verhaltensregel, die bei einem Schock nicht nur die Parameter ändert, sondern die Verhaltensregel selbst wechselt. Verhaltensänderung und Verhaltenswechsel sind beides real beobachtbare Verhaltensweisen von Wirtschaftsubjekten, die auf diese Art in das IMS-Modell übernommen werden könnten.

Die hier gegebenen Entwicklungsmöglichkeiten zeigen, daß eine wissenschaftliche Arbeit an ihrem Ende nicht notwendigerweise abgeschlossen sein muß. Sie sollte oder muß Perspektiven aufzeigen, die ihr Entwicklungspotential belegen, wenn sie über die reine Deskription hinaus gelangen will.

In dieser Arbeit wurden die Auswirkungen von exogenen Schocks auf Versicherungsmärkte untersucht. Da die Versicherungstheorie und die allgemeine Markttheorie dafür keine befriedigenden Ansätze boten, wurde ein eigener, spezieller Ansatz entwickelt. Die Wurzeln des Modells liegen in:

- Der Marktprozeßtheorie,
- der behavioristischen Theorie,
- der adapativen Oligopoltheorie und
- der Ungleichgewichtstheorie.

Die Grundannahme des Modell ist, daß das Verhalten eines Wirtschaftssubjekts durch die ihm zur Verfügung stehende Informationsmenge bestimmt ist. Die komplexe Dynamik eines Marktprozesses entsteht durch die Interaktion vieler, sehr einfacher Verhaltensregeln.

Durch die Wahl eines eigenen Modellrahmens konnte die starre Trennung ökonomischer Größen in Makro- und Mikroebene aufgegeben werden. Das Modell war in der Lage, die Daten von Anbieter und Nachfrager auf unterschiedlichen Aggregationsebenen zu analysieren. Diese unterschiedlichen Vergrößerungsstufen der Datenanalyse erlaubten eine differenziertere Untersuchung der Schockwirkungen als es eine reine Makro- oder Mikrobetrachtung zugelassen hätte.

Aufgrund der Beschränkungen mathematischer Analysen wurde der nummerischen Speziallösung einer Simulation der Vorzug gegenüber einer allgemeingültigen, algebraischen Lösung gegeben. Durch die frühzeitige Entscheidung für ein Simulationsmodell konnten die Vorzüge der Computersimulation bereits bei der Modellkonstruktion berücksichtigt werden. Das erstellte Computerprogramm ist nicht speziell für die Analyse des Schocks der deutschen Wiedervereinigung programmiert, sondern ist für weitere Marktprozeßanalysen von Versicherungsmärkten ausgelegt.

Als Beispiel für die Programmanwendung wurde der Schock der deutschen Wiedervereinigung gewählt. Grundannahme des Modells der Wiedervereinigung war, daß zu einem bestimmen Zeitpunkt des Marktprozesses zusätzliche Nachfrager in den Markt eintreten und ein bestimmter Versicherer diese zusätzliche Marktnachfrage an sich binden kann.

Basierend auf diesen Annahmen wurden die Auswirkungen dieses exogenen Schocks auf die unterschiedlichen Aggregatstufen untersucht. Die Analyse bestätigte die Vermutung, daß ein Schock auf den verschiedenen Aggregatstufen unterschiedlich wirkt. Schockwirkungen hatten auf der Nachfragerseite bei verschiedenen Aggregaten der gleichen Stufe unterschiedliche Wirkungsrichtungen. Waren die Auswirkungen auf der Ebene der höchsten Aggregation kaum nachweisbar, konnten auf Stufen stärker disaggregierten Daten hohe Signifikanz festgestellt werden.

Die Grundannahme der Verifikation des Modells war, daß bei der Untersuchung von Marktprozessen der Prozeßrealität eine höhere Bedeutung als der Ergebnisrealität zukommen muß. Deshalb wurde auf eine Ausrichtung des Modells an realen Daten oder eine Mischung aus realen und künstlichen Daten verzichtet. An den rein künstlichen Daten wurde geprüft, ob die einzelnen Verhaltensregeln den Modellerwartungen entsprachen und ob die globale Dynamik des Modells mit den Modellspezifikationen in Einklang zu bringen war. Es zeigte sich, daß das Prozeßverhalten der Simulation eine hohe Übereinstimmung mit den postulierten dynamischen Eigenschaften besaß und das die Simulation in der Lage war, reale Prozeßphänomene zu reproduzieren.

Zufassenfassend läßt sich feststellen, daß die Verhaltenstheorie eine bessere Modellbasis für Marktprozeßsimulationen bildet als die viel häufiger verwendete Entscheidungstheorie. Die hier durchgeführte Simulation der Auswirkungen von exogenen Schock auf Versicherungsmärkte zeigt weiterhin, daß die Störung eines Gleichgewichts nicht zwingend der Ausgangspunkt für eine Schockanalyse sein muß. Das Fehlen eines Preiskoordinationsmechnismus führt dazu, daß die Schockanalyse nicht durch Konformismus der Gleichgewichtstheorie bestimmt wird.

Schocks sind die Motoren der Marktprozesse. Ihre Analyse wird eine angemessene Position innerhalb der ökonomischen Theorie erhalten, wenn die Erkenntnis, daß Gleichgewichte immer nur das Ergebnis von Marktprozessen sein können, stärker etabliert ist.

Anhänge

```
0001 /* ************************************************************** */
0002 /*                                                                */
0003 /*   Datenstrukturen, Deklarationen und Konstanten von IMS        */
0004 /*                                                                */
0005 /* ************************************************************** */
0006 #if defined(PC386)
0007  #define version   "MSDOS v1.0"
0008 #elif defined(UNIX)
0009  #define version   "UNIX  v1.0"
0010 #else
0011  #define version   "UNDE. v1.0"
0012 #endif
0013 #include <stdio.h>
0014 #define SIMLAENGE  100 /* maximale Perioden pro Simulationslauf     */
0015 #define MAXVU      25  /* maximale Anzahl aktiver Versicherer       */
0016 #define MAXVN      200 /* maximale Anzahl aktiver Versicherungsnehmer*/
0017 #define MAXSPARTEN 2   /* maximale Anzahl der Sparten einer VU      */
0018 #define MAXRISKS   2   /* maximale Anzahl von Risiken eines VN      */
0019 #define MAXDATEI   10  /* maximale Anzahl gleichzeitig offener Files */
0020 #define MAXGNUDT   4   /* maximale Anzahl GNUplot Steuerfiles       */
0021 /* ************************************************************** */
0022 /* Deklaration der Dateivariablen fuer die Datein, in die die       */
0023 /* Ergebnis, Ziel und Folgevektoren und deren Aggregate geschrieben  */
0024 /* werden                                                          */
0025 /* ************************************************************** */
0026 typedef FILE file;     /* fuer ESL                                 */
0027 file *runreport;       /* Programmablaufskontrolldatei             */
0028 file *konfig;          /* Modellkonfigurationsdatei                */
0029 /* ************************************************************** */
0030 /*                                                                */
0031 /*  Globale Konstanten und Variable fuer den Simulationsablauf      */
0032 /*                                                                */
0033 /* ************************************************************** */
0034 int sl; /* Laenge der Einzelsimulation (RUN) in Perioden           */
0035 int al; /* Anzahl der sukzessiven Simulationslaeufe (SET)          */
0036 int rl = 1  ; /* Zaehlvariable fuer den laufenden SET              */
0037 /* jumpback wird in Init() nach dem Oeffnen der Protokoldateien auf  */
0038 /* Null gesetzt und in Fini() vor dem Ruecksprung in Init() auf Eins */
0039 /* gesetzt um ein wiederholtes Oeffnen der Protokoldateien zu       */
0040 /* verhindern                                                      */
0041 int jumpback=0;
0042 int gOwnnr;/* fuer Uebergabe von Ownnr() an indirekt aufgerufene acts*/
0043 int gperiod;/* Globale ESL-Variablen der diskreten Simulationsuhrzeit*/
0044 int glogtime;/* IMMER benutzt, statt period und logtime von ESL     */
0045 long int gmem; /* Bytesanzahl des dynamischen angeforderten Speichers*/
```

```
0046 /* Wird gesetzt, wenn Konfiguration aus Datei geladen sind          */
0047 int load=0; /* [0] = nicht geladen, [1] = geladen                   */
0048 /* ************************************************************** */
0049 /*                                                                  */
0050 /* Definitionen und Strukturen fuer die dynamische Subjektverwaltung */
0051 /*                                                                  */
0052 /* ************************************************************** */
0053 int akvu; /* Anzahl der aktivierten VU                              */
0054 int akvn; /* Anzahl der aktivierten VN                              */
0055 int aktiVU[MAXVU+1]; /* Vektor der aktivierten VUs                   */
0056 int aktiVN[MAXVN+1]; /* Vektor der aktivierten VN                    */
0057 /* 0 : nicht aktiviert, 1 : aktiviert und vordefiniert              */
0058 /* 2 = nicht aktiviert und vordefiniert,d.h. werden spaeter aktiviert*/
0059 /* 3 = aktiviert und editierbar                                     */
0060 /* ACHTUNG: NUR wenn aktiV_[_] <> 0 ist darf zu Beginn der Simulation*/
0061 /*          Speicher fuer das Subjekt angefordert werden            */
0062 /* ************************************************************** */
0063 /* Datenstruktur fuer die Zuordnung einer Verhaltenregel zu einem   */
0064 /* ESL-Subjekt. Wird von den Strategiedaemonen der dynamischen      */
0065 /* Subjektklassen VU und VN benoetigt.                              */
0066 /* ************************************************************** */
0067 typedef struct s0
0068 {
0069   int  st;    /* Nummer der Verhaltensregel,  die ausgefuehrt wird  */
0070   int  lt;    /* Logischer Zeitpunkt der Ausfuehrung                */
0071 } ACTION;
0072 /* ************************************************************** */
0073 /* Zuordnungstabellen fuer die Verhaltensregeln (Aggregat II) zu den */
0074 /* Regelklassen (Aggregat III)  und die Zuordnung der Verhaltens-   */
0075 /* regeln zu ihren Namen                                            */
0076 /* ************************************************************** */
0077 int vkrvu[11] = {0,1,1,2,2,2,2,3,3,3,0}; /* Zuordnung von Verhaltens-*/
0078 int vkrvn[8]  = {0,1,1,2,2,3,3,0}; /* regel und Verhaltensklasse    */
0079 static char *vrvunm[] = { "VR(0): unbesetzt ", "Zufall I         ",
0080                      "Zufall II        ", "Mark Up I        ",
0081                      "Mark Up II       ", "Mark Up III      ",
0082                      "Erwartungsschaden", "Dumping          ",
0083                      "Durchschnitt     ", "Angriff          ",
0084                      "VR(10): unbesetzt" };
0085 static char *vrvnnm[] = { "VR(0): unbesetzt ", "Zufall I         ",
0086                      "Zufall II        ", "Praeferenz       ",
0087                      "Totale Erinnerung", "Suche            ",
0088                      "Beste Information", "VR(7): unbesetzt " } ;
0089 /* ************************************************************** */
0090 /* Zuordnungstabellen der Dateinamen der Aggregatdateien und        */
```

```
0091 /* Initialisierung der Aggregattabellen. Ist ein Element dieser      */
0092 /* Tabellen=1, dann wird die zugehoerige Aggregatdatei erzeugt.      */
0093 /* ***************************************************************** */
0094 static char *agvu1nm[] = { /*Datei-Namenstabelle Aggregat I der VUs  */
0095 "imsvu000.dat",
0096 "imsvu001.dat","imsvu002.dat","imsvu003.dat","imsvu004.dat",
0097 "imsvu005.dat","imsvu006.dat","imsvu007.dat","imsvu008.dat",
0098 "imsvu009.dat","imsvu010.dat","imsvu011.dat","imsvu012.dat",
0099 "imsvu013.dat","imsvu014.dat","imsvu015.dat","imsvu016.dat",
0100 "imsvu017.dat","imsvu018.dat","imsvu019.dat","imsvu020.dat",
0101 "imsvu021.dat","imsvu022.dat","imsvu023.dat","imsvu024.dat",
0102 "imsvu025.dat"
0103 };
0104 int agvu1[26] = {0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0};
0105 static char *agvu2nm[] = { /*Datei-Namenstabelle Aggregat II der VUs */
0106 "imsvur00.dat",
0107 "imsvur01.dat","imsvur02.dat","imsvur03.dat","imsvur04.dat",
0108 "imsvur05.dat","imsvur06.dat","imsvur07.dat","imsvur08.dat",
0109 "imsvur09.dat","imsvur10.dat"
0110 };
0111 int agvu2[11] = {0,0,0,0,0,0,0,0,0,0,0};
0112 static char *agvu3nm[] = { /*Datei-Namenstabelle Aggregat III der VUs*/
0113 "imsvuvk0.dat","imsvuvk1.dat","imsvuvk2.dat","imsvuvk3.dat"
0114 };
0115 int agvu3[4] = {0,0,0,0};
0116 static char *agvu4nm[] = { /*Datei-Namenstabelle Aggregat IV der VUs */
0117 "imsvusk0.dat","imsvusk1.dat"
0118 };
0119 int agvu4[2] = {0,0};
0120 static char *agvn1nm[] = { /*Datei-Namenstabelle Aggregat I der VNs  */
0121 "imsvn000.dat",
0122 "imsvn001.dat","imsvn002.dat","imsvn003.dat","imsvn004.dat",
0123 "imsvn005.dat","imsvn006.dat","imsvn007.dat","imsvn008.dat",
0124 "imsvn009.dat","imsvn010.dat","imsvn011.dat","imsvn012.dat",
0125 "imsvn013.dat","imsvn014.dat","imsvn015.dat","imsvn016.dat",
0126 "imsvn017.dat","imsvn018.dat","imsvn019.dat","imsvn020.dat",
0127 "imsvn021.dat","imsvn022.dat","imsvn023.dat","imsvn024.dat",
0128 "imsvn025.dat","imsvn026.dat","imsvn027.dat","imsvn028.dat",
0129 "imsvn029.dat","imsvn030.dat","imsvn031.dat","imsvn032.dat",
0130 "imsvn033.dat","imsvn034.dat","imsvn035.dat","imsvn036.dat",
0131 "imsvn037.dat","imsvn038.dat","imsvn039.dat","imsvn040.dat",
0132 "imsvn041.dat","imsvn042.dat","imsvn043.dat","imsvn044.dat",
0133 "imsvn045.dat","imsvn046.dat","imsvn047.dat","imsvn048.dat",
0134 "imsvn049.dat","imsvn050.dat","imsvn051.dat","imsvn052.dat",
0135 "imsvn053.dat","imsvn054.dat","imsvn055.dat","imsvn056.dat",
```

```
0136 "imsvn057.dat","imsvn058.dat","imsvn059.dat","imsvn060.dat",
0137 "imsvn061.dat","imsvn062.dat","imsvn063.dat","imsvn064.dat",
0138 "imsvn065.dat","imsvn066.dat","imsvn067.dat","imsvn068.dat",
0139 "imsvn069.dat","imsvn070.dat","imsvn071.dat","imsvn072.dat",
0140 "imsvn073.dat","imsvn074.dat","imsvn075.dat","imsvn076.dat",
0141 "imsvn077.dat","imsvn078.dat","imsvn079.dat","imsvn080.dat",
0142 "imsvn081.dat","imsvn082.dat","imsvn083.dat","imsvn084.dat",
0143 "imsvn085.dat","imsvn086.dat","imsvn087.dat","imsvn088.dat",
0144 "imsvn089.dat","imsvn090.dat","imsvn091.dat","imsvn092.dat",
0145 "imsvn093.dat","imsvn094.dat","imsvn095.dat","imsvn096.dat",
0146 "imsvn097.dat","imsvn098.dat","imsvn099.dat","imsvn100.dat",
0147 "imsvn101.dat","imsvn102.dat","imsvn103.dat","imsvn104.dat",
0148 "imsvn105.dat","imsvn106.dat","imsvn107.dat","imsvn108.dat",
0149 "imsvn109.dat","imsvn110.dat","imsvn111.dat","imsvn112.dat",
0150 "imsvn113.dat","imsvn114.dat","imsvn115.dat","imsvn116.dat",
0151 "imsvn117.dat","imsvn118.dat","imsvn119.dat","imsvn120.dat",
0152 "imsvn121.dat","imsvn122.dat","imsvn123.dat","imsvn124.dat",
0153 "imsvn125.dat","imsvn126.dat","imsvn127.dat","imsvn128.dat",
0154 "imsvn129.dat","imsvn130.dat","imsvn131.dat","imsvn132.dat",
0155 "imsvn133.dat","imsvn134.dat","imsvn135.dat","imsvn136.dat",
0156 "imsvn137.dat","imsvn138.dat","imsvn139.dat","imsvn140.dat",
0157 "imsvn141.dat","imsvn142.dat","imsvn143.dat","imsvn144.dat",
0158 "imsvn145.dat","imsvn146.dat","imsvn147.dat","imsvn148.dat",
0159 "imsvn149.dat","imsvn150.dat","imsvn151.dat","imsvn152.dat",
0160 "imsvn153.dat","imsvn154.dat","imsvn155.dat","imsvn156.dat",
0161 "imsvn157.dat","imsvn158.dat","imsvn159.dat","imsvn160.dat",
0162 "imsvn161.dat","imsvn162.dat","imsvn163.dat","imsvn164.dat",
0163 "imsvn165.dat","imsvn166.dat","imsvn167.dat","imsvn168.dat",
0164 "imsvn169.dat","imsvn170.dat","imsvn171.dat","imsvn172.dat",
0165 "imsvn173.dat","imsvn174.dat","imsvn175.dat","imsvn176.dat",
0166 "imsvn177.dat","imsvn178.dat","imsvn179.dat","imsvn180.dat",
0167 "imsvn181.dat","imsvn182.dat","imsvn183.dat","imsvn184.dat",
0168 "imsvn185.dat","imsvn186.dat","imsvn187.dat","imsvn188.dat",
0169 "imsvn189.dat","imsvn190.dat","imsvn191.dat","imsvn192.dat",
0170 "imsvn193.dat","imsvn194.dat","imsvn195.dat","imsvn196.dat",
0171 "imsvn197.dat","imsvn198.dat","imsvn199.dat","imsvn200.dat"
0172 };
0173 int agvn1[201] = {0,
0174 0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
0175 0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
0176 0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
0177 0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
0178 0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
0179 0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0
0180 };
```

```
0181 static char *agvn2nm[] = { /*Datei-Namenstabelle Aggregat II der VNs */
0182 "imsvnr00.dat",
0183 "imsvnr01.dat","imsvnr02.dat","imsvnr03.dat","imsvnr04.dat",
0184 "imsvnr05.dat","imsvnr06.dat","imsvnr07.dat"
0185 };
0186 int agvn2[8] = {0,0,0,0,0,0,0,0};
0187 static char *agvn3nm[] = { /*Datei-Namenstabelle Aggregat III der VNs*/
0188 "imsvnvk0.dat","imsvnvk1.dat","imsvnvk2.dat","imsvnvk3.dat"
0189 };
0190 int agvn3[4] = {0,0,0,0};
0191 static char *agvn4nm[] = { /*Datei-Namenstabelle Aggregat IV der VNs */
0192 "imsvnsk0.dat","imsvnsk1.dat"
0193 };
0194 int agvn4[2] = {0,0};
0195 /* ************************************************************** */
0196 /* class VU: Definition der Sub-Datenstrukturen der beiden Sparten.  */
0197 /*           Die Umsetzung der Variable erfolgt nach der Definition: */
0198 /*           Zielvektor, Ergebnisvektor, Folgevektor.               */
0199 /* ************************************************************** */
0200 typedef struct s11        /* Spartendefinition: Sparte 1            */
0201 {
0202   float  Pr[SIMLAENGE+1]; /* Zielvektor: Praemie in Periode t       */
0203   float  Wa[SIMLAENGE+1]; /* Zielvektor: Werbeaufwand in Periode t  */
0204   float  Rs[SIMLAENGE+1]; /* Ergebnisv.: Schadenreserven in Periode t*/
0205   int    Vn[SIMLAENGE+1]; /* Ergebnisv.: Anzahl der Versicherten    */
0206   int    Sa[SIMLAENGE+1]; /* Folgevektr: Schadenanzahl in Periode t */
0207   float  Sh[SIMLAENGE+1]; /* Folgevektr: Schadensumme in Periode t  */
0208 } LV;
0209 typedef struct s12        /* Spartendefinition: Sparte 2            */
0210 {
0211   float  Pr[SIMLAENGE+1]; /* Zielvektor: Praemie in Periode t       */
0212   float  Wa[SIMLAENGE+1]; /* Zielvektor: Werbeaufwand in Periode t  */
0213   float  Rs[SIMLAENGE+1]; /* Ergebnisv.: Schadenreserven in Periode t*/
0214   int    Vn[SIMLAENGE+1]; /* Ergebnisv.: Anzahl der Versicherten    */
0215   int    Sa[SIMLAENGE+1]; /* Folgevektr: Schadenanzahl in Periode t */
0216   float  Sh[SIMLAENGE+1]; /* Folgevektr: Schadensumme in Periode t  */
0217 } KV;
0218 typedef union u1    /* Strukturvariablen der Gesamtspartenstruktur  */
0219 {
0220  int   type; /* fuer Identifierung bei Speichern/Laden der Daten    */
0221  LV    l; /* Sparte 1 */
0222  KV    k; /* Sparte 2 */
0223 } SPARTE;
0224 /* ************************************************************** */
0225 /*                                                                */
```

```
0226 /*                  classVU: Die Versicherer                        */
0227 /*                                                                  */
0228 /* ***************************************************************** */
0229 typedef struct s1
0230 {
0231   int    Ap;              /* Aktivierungsperiode                    */
0232   int    Al[SIMLAENGE+1]; /* Aktivierungslauf                       */
0233   float  A1[4];           /* Anspruchniveaus: Sparte 1              */
0234   float  A2[4];           /* Anspruchniveaus: Sparte 2              */
0235   float  Pv[16];          /* IMS-Paramtervektor der Versicherungen  */
0236                           /*(a,b,c,d,e,f,g,h,a*,b*,c*,d*,e*,f*,g*,h*)*/
0237   int    Pa[24];          /* =1, wenn Pv[i] in der Parametermaske   */
0238                           /* abgefragt werden soll. [0..15] fuer PV, */
0239                           /* [16..19] fuer A1[], [20..23] fuer A2[] */
0240   int    Vr;              /* Nummer der Verhaltensregel: VU: 1..9   */
0241   int    Vk;              /* Regelklasse fuer Aggregatbildung:1,2,3 */
0242   ACTION Ac[MAXNLOG+1];   /* Zuordnung der Verhaltensregeln         */
0243   SPARTE Sp[MAXSPARTEN+1];/* Sparte 1 und 2 zusammengefasst         */
0244 } classVU;
0245 /* ***************************************************************** */
0246 /* class VN: Definition der Sub-Datenstrukturen der beiden Sparten. */
0247 /*           Die Umsetzung der Variable erfolgt nach der Definition: */
0248 /*           Zielvektor, Ergebnisvektor, Folgevektor.               */
0249 /* ***************************************************************** */
0250 typedef struct s21       /* Risiko 1                               */
0251 {
0252  int    Vu[SIMLAENGE+1]; /* Zielvektor: Anbieternummer, 0=eigenvers. */
0253  int    Vs[SIMLAENGE+1]; /* Zielvektor: Versicherungsstatus in t    */
0254  float  Vp[SIMLAENGE+1]; /* Ergebnisv.: Gezahlte Praemie in t       */
0255  float  Ev[SIMLAENGE+1]; /* Ergebnisv.: Eigenversichert.Schaden in t */
0256  float  Sw[SIMLAENGE+1]; /* Folgevektr: Schadenwahrscheinlichk. in t */
0257  float  Sh[SIMLAENGE+1]; /* Folgevektr: Schadenhoehe in t           */
0258  float  Pf[MAXVU+1];     /* Hilfsvektr: Anbieterpraeferenz in t     */
0259 } RISKLV;
0260 typedef struct s22       /* Risiko 2                               */
0261 {
0262  int    Vu[SIMLAENGE+1]; /* Zielvektor: Anbieternummer, 0=eigenvers. */
0263  int    Vs[SIMLAENGE+1]; /* Zielvektor: Versicherungsstatus in t    */
0264  float  Vp[SIMLAENGE+1]; /* Ergebnisv.: Gezahlte Praemie in t       */
0265  float  Ev[SIMLAENGE+1]; /* Ergebnisv.: Eigenversichert.Schaden in t */
0266  float  Sw[SIMLAENGE+1]; /* Folgevektr: Schadenwahrscheinlichk. in t */
0267  float  Sh[SIMLAENGE+1]; /* Folgevektr: Schadenhoehe in t           */
0268  float  Pf[MAXVU+1];     /* Hilfsvektr: Anbieterpraeferenz in t     */
0269 } RISKKV;
0270 typedef union u2      /* Strukturvariable der Gesamtrisikostruktur  */
```

```
0271 {
0272  int     type; /* wichtig fuer laden und speichern                  */
0273  RISKLV  l;
0274  RISKKV  k;
0275 } RISIKO;
0276 /* ************************************************************** */
0277 /*                                                                */
0278 /*              classVN: Die Versicherungsnehmer                  */
0279 /*                                                                */
0280 /* ************************************************************** */
0281 typedef struct s2
0282 {
0283  int    Ap;                 /* Aktivierungsperiode                */
0284  int    Al[SIMLAENGE+1];    /* Aktivierungslauf                   */
0285  int    Vr;                 /* Nummer der Verhaltensregel: 1..9,VN  */
0286  int    Vk;                 /* Regelklasse: Aggregatbildung: 1,2,3  */
0287  float  Vm[SIMLAENGE+1];    /* Vermoegen des Versicherten in t    */
0288  float  Pv[16];             /* IMS-Paramtervektor der             */
0289     /* Versicherungsnehmer: (a,b,c,d,e,f,g,h,a*,b*,c*,d*,e*,f*,g*,h*)*/
0290  int    Pa[16];             /* =1, wenn Pv[i] in der Parametermaske  */
0291                             /* abgefragt werden soll. [0..15] fuer PV*/
0292  ACTION Ac[MAXNLOG+1];      /* Zuordnung der Verhaltensregeln     */
0293  RISIKO Rk[MAXRISKS+1];     /* Zusammengefasste Risiken           */
0294 } classVN;
0295 /* ************************************************************** */
0296 /*                                                                */
0297 /*               classBAV: Der Informationsverteiler              */
0298 /*                                                                */
0299 /* ************************************************************** */
0300 typedef struct s7
0301 {
0302  int   As[SIMLAENGE+1];/* Aenderungsschockindikator:0,1=Schock in t  */
0303  int   Ar[SIMLAENGE+1];/*  " fuer die Sim.runs, d.h. Ar[50]=0,Ar[51] */
0304                       /* = 1: kein Aenderungsschock im 50.ten Lauf  */
0305                       /* aber Aenderungsschock im 51.ten Lauf       */
0306                       /* -> Aenderungsschock ist: wenn              */
0307                       /* Ar[gperiod] * As[sl] = 1 ist !             */
0308  int   Rp;            /* = 1, wenn Wiederholungen zugelassen sind   */
0309                       /* wird in func Starts() abgefragt, sonst 0 ! */
0310  float Zs[SIMLAENGE+1];/* Zur Verzinsung der VU-Reserven            */
0311  float Sk[SIMLAENGE+1];/* Kosten fuer EINE Praemieninfo. eines VUs   */
0312                       /* **** Vektoren fuer die Aggregatverwaltung* */
0313  int   Vuag1[MAXVU+1]; /* = 1, wenn VUxy in der Periode t aktiv war  */
0314                       /* -> akvu = Anzahl in t aktiver VUs          */
0315  int   Vuag2[11];      /* wieviel VUs mit Regel i in t aktiv waren   */
```

```
0316  int   Vuag3[4];       /* wieviel in Regelklasse j in t aktiv waren  */
0317  int   Vnag1[MAXVN+1]; /* = 1, wenn VNxy in Periode t aktiv war      */
0318                        /* -> akvn = Anzahl in t aktiver VNs          */
0319  int   Vnag2[8];       /* wieviele VNs mit Regel i in t aktiv waren  */
0320  int   Vnag3[4];       /* wieviele in Regelklasse j in t aktiv waren */
0321                        /* Subjektvektoren fuer die Aggregatbildung   */
0322                        /* **** Vektoren fuer die Fremdinformation ** */
0323                        /* VU: VR DUMPING                             */
0324  float Pm[MAXSPARTEN]; /* Minimale Praemie in Sparte i in t-1        */
0325  float Wm[MAXSPARTEN]; /* Maximale Werbung in Sparte i in t-1        */
0326                        /* VU: VR: DURCHSCHNITT                       */
0327  float Dp[MAXSPARTEN]; /* Durchschnittspraemie der Sparten in t      */
0328  float Dw[MAXSPARTEN]; /* Durchschnittlicher Werbeaufwand, Sparte,t-1*/
0329                        /* VU: VR ANGRIFF                             */
0330  float Mp[MAXSPARTEN]; /* Praemien des Marktfuehrers(max. Reserven)  */
0331  float Mw[MAXSPARTEN]; /* Werbeaufwendungen des Marktfuehrers        */
0332                        /* VN: alle VR:                               */
0333  float Dg[MAXRISKS];   /* Durchschnittlicher Versicherungsgrad in t-1*/
0334 } classBAV;
0335
```

```
0001 /* ************************************************************** */
0002 /*               IMS, The Insurance Market Simulation             */
0003 /*                       (c) Martin Koehler                       */
0004 /*                             1995                               */
0005 /* ************************************************************** */
0006 #define MAXNLOG   10
0007 ims (n_log=MAXNLOG)      /* ESL-Programmkopf                      */
0008 #include <time.h>        /* Einbinden von Standard-C  Header-Files   */
0009 #include <math.h>
0010 #include <malloc.h>
0011 #include <stdlib.h>
0012 #include "imsrnd.c"      /* Einbinden der IMS-internen Programmteile */
0013 #include "imssurfe.c"
0014 #include "imsdata.c"
0015
0016 /* ************************************************************** */
0017 /* class BAV: Definition der ESL-class BAV und ihrer Aktionen. BAV   */
0018 /*            verwaltet die Fremdinformationen, die von den class VU */
0019 /*            und VN benoetigt werden.                              */
0020 /* ************************************************************** */
0021 class BAV {
0022            classBAV DatenBAV;
0023            } subj[1] =
0024            {
0025            xact: (1,1,1) Myinitbv
0026             act: 1 Newinibv Frmdinf
0027                 10 Agrsich
0028            };
0029 /* ************************************************************** */
0030 /* Initialisierungsfunktion der class BAV. Sie wird NUR in der ersten*/
0031 /* Periode des ersten Simulationslaufs ausgefuehrt. Die Neu-Initia-  */
0032 /* lisierung in den Laeufen > 1 erfolgt in der act <Newinibv>        */
0033 /* ************************************************************** */
0034 act Myinitbv {
0035 int i;                                /* Schleifenvariable         */
0036 if ( ( rl == 1 ) &&  (period == 1) )
0037  {
0038   for(i = 0 ; i < MAXSPARTEN ; i++ )
0039    {
0040     BAV[Ownnr()].DatenBAV.Pm[i] = 0.0;/* Minimalpraemie,Sparte   :{0}*/
0041     BAV[Ownnr()].DatenBAV.Wm[i] = 0.0;/* Maximalwerbung,Sparte   :{0}*/
0042     BAV[Ownnr()].DatenBAV.Dp[i] = 0.0;/* Durchschn.praemie,Spart.:{0}*/
0043     BAV[Ownnr()].DatenBAV.Dw[i] = 0.0;/* Durchschn.werbung,Spart.:{0}*/
0044     BAV[Ownnr()].DatenBAV.Mp[i] = 0.0;/* Marktfueh.praemie,Spart.:{0}*/
0045     BAV[Ownnr()].DatenBAV.Mw[i] = 0.0;/* Marktfueh.werbung,Spart.:{0}*/
```

```
0046    }
0047   for( i = 0  ; i < MAXRISKS  ; i++  )
0048    {
0049     BAV[Ownnr()].DatenBAV.Dg[i] = 0.0;/* Durchschnl. Versich.grad {0}*/
0050    }
0051    fprintf(runreport, "Myinitbv:[%d,%d]\n",logtime,period );
0052  } /* END: if ( ( rl == 1 ) &&  (period == 1) ) */
0053 } /* END: act Myinitbv */
0054
0055 /* ************************************************************** */
0056 /* Re-Initialisierungsfunktion der class BAV. Sie wird in aller      */
0057 /* Laeufen NACH dem ersten Lauf immer in der ersten Periode aus -    */
0058 /* gefuehrt.                                                         */
0059 /* ************************************************************** */
0060 act Newinibv {
0061 int i;                                /* Wird nur AB dem 2. Lauf in */
0062 if ( (rl >= 2) && ( period == 1) ) /* der 1. Periode ausgefuehrt */
0063  {
0064   for(i = 0 ; i < MAXSPARTEN ; i++ )
0065    {
0066     BAV[Ownnr()].DatenBAV.Pm[i] = 0.0;/* Minimalpraemie,Sparte   :{0}*/
0067     BAV[Ownnr()].DatenBAV.Wm[i] = 0.0;/* Maximalwerbung,Sparte   :{0}*/
0068     BAV[Ownnr()].DatenBAV.Dp[i] = 0.0;/* Durchschn.praemie,Spart.:{0}*/
0069     BAV[Ownnr()].DatenBAV.Dw[i] = 0.0;/* Durchschn.werbung,Spart.:{0}*/
0070     BAV[Ownnr()].DatenBAV.Mp[i] = 0.0;/* Marktfueh.praemie,Spart.:{0}*/
0071     BAV[Ownnr()].DatenBAV.Mw[i] = 0.0;/* Marktfueh.werbung,Spart.:{0}*/
0072    }
0073   for( i = 0  ; i < MAXRISKS  ; i++  )
0074    {
0075     BAV[Ownnr()].DatenBAV.Dg[i] = 0.0;/* Durchschnl. Versich.grad {0}*/
0076    }
0077   fprintf(runreport, "Newinibv:[%d,%d]\n",logtime,period );
0078  } /* END: if ( (rl >= 2) && ( period == 1) ) */
0079 } /* END: act Newinibv */
0080
0081 /* ************************************************************** */
0082 /* act Frmdinf: Zentrale Berechnung der FREMDINFORMATIONEN          */
0083 /* VUs: VR DUMPING: braucht Minimalpraemie in beiden Sparten in t-1  */
0084 /*                         Maximalwerbung in beiden Sparten in t-1  */
0085 /*         DURCHSCHNITT:   Durchschnittspraemie in b. Sparten in t-1*/
0086 /*                         Durchschnittswerbung in b. Sparten in t-1*/
0087 /*         ANGRIFF:        Praemie der VU mit den hoechsten Reserven*/
0088 /*                         Werbung der VU ( = Marktfuehrer )        */
0089 /* VNs: VR SUCHE:          Durchschnittl. Versichg.grad in Risiko i */
0090 /*         BESTE INFOS.    Durchschhnittl. Versich.grad in Risiko i */
```

```
0091 /* ************************************************************** */
0092 act Frmdinf {
0093 int i, j;                      /* Schleifenzaehler                  */
0094 float r;                       /* Hilfsvariable                     */
0095 if ( period > 1 )              /* Durchschnittspraemie in Sparte 1  */
0096  {
0097   BAV[Ownnr()].DatenBAV.Dp[0] = 0.0;
0098   for( i = 1; i <= MAXVU ; i++)/* ACHTUNG: muss VOR der NEUBerechnung*/
0099    {                           /* von akvu passieren, sonst wird in  */
0100     BAV[Ownnr()].DatenBAV.Dp[0] =  /* Perioden mit Aktivierungsschock*/
0101     BAV[Ownnr()].DatenBAV.Dp[0] +  /* mit Praemien aus t-1 und der   */
0102     VU[i].DatenVU->Sp[1].l.Pr[(period-1)]; /* Versichereranzahl aus  */
0103    }                               /* t gerechnet !                  */
0104   if ( akvu != 0)              /* Division durch Null vermeiden...   */
0105    {
0106     BAV[Ownnr()].DatenBAV.Dp[0] = BAV[Ownnr()].DatenBAV.Dp[0] / akvu ;
0107    }
0108    else /* akvu == 0 */
0109     {
0110      BAV[Ownnr()].DatenBAV.Dp[0] = 0.0;
0111     }
0112  } /* END: if ( period > 1 ) */
0113  else /* period == 0 */
0114   {
0115   BAV[Ownnr()].DatenBAV.Dp[0] = 0.0; /* .. zur Vollstaendigkeit...   */
0116   }
0117 if ( period > 1 )              /* Durchschnittspraemie in Sparte 2   */
0118  {
0119   BAV[Ownnr()].DatenBAV.Dp[1] = 0.0;
0120   for( i = 1; i <= MAXVU ; i++)/* ACHTUNG: muss VOR der NEUBerechnung*/
0121    {                           /* von akvu passieren, sonst wird in  */
0122     BAV[Ownnr()].DatenBAV.Dp[1] =  /* Perioden mit Aktivierungsschock*/
0123     BAV[Ownnr()].DatenBAV.Dp[1] +  /* mit Praemien aus t-1 und der   */
0124     VU[i].DatenVU->Sp[2].k.Pr[(period-1)]; /* Versichereranzahl aus  */
0125    }                               /* t gerechnet !                  */
0126   if ( akvu != 0)              /* Division durch Null vermeiden...   */
0127    {
0128     BAV[Ownnr()].DatenBAV.Dp[1] = BAV[Ownnr()].DatenBAV.Dp[1] / akvu ;
0129    }
0130    else /* akvu == 0 */
0131     {
0132      BAV[Ownnr()].DatenBAV.Dp[1] = 0.0;
0133     }
0134  } /* END: if ( period > 1 ) */
0135  else /* period = 0 */
```

```
0136   {
0137   BAV[Ownnr()].DatenBAV.Dp[1] = 0.0; /* .. zur Vollstaendigkeit...   */
0138   }
0139 if ( period > 1 )              /* Durchschnittswerbung in Sparte 1   */
0140  {
0141   BAV[Ownnr()].DatenBAV.Dw[0] = 0.0;
0142   for( i = 1; i <= MAXVU ; i++)/* ACHTUNG: muss VOR der NEUBerechnung*/
0143    {                           /* von akvu passieren, sonst wird in  */
0144     BAV[Ownnr()].DatenBAV.Dw[0] =  /* Perioden mit Aktivierungsschock*/
0145     BAV[Ownnr()].DatenBAV.Dw[0] +  /* mit Werbung  aus t-1 und der   */
0146     VU[i].DatenVU->Sp[1].l.Wa[(period-1)]; /* Versichereranzahl aus  */
0147    }                               /* t gerechnet !                  */
0148   if ( akvu != 0)              /* Division durch Null vermeiden...   */
0149    {
0150     BAV[Ownnr()].DatenBAV.Dw[0] = BAV[Ownnr()].DatenBAV.Dw[0] / akvu ;
0151    }
0152    else /* akvu == 0 */
0153     {
0154      BAV[Ownnr()].DatenBAV.Dw[0] = 0.0;
0155     }
0156  } /* END: if ( period > 1 ) */
0157  else /* period == 0 */
0158   {
0159    BAV[Ownnr()].DatenBAV.Dw[0] = 0.0; /* .. zur Vollstaendigkeit...   */
0160   }
0161 if ( period > 1 )               /* Durchschnittswerbung in Sparte 2   */
0162  {
0163   BAV[Ownnr()].DatenBAV.Dw[1] = 0.0;
0164   for( i = 1; i <= MAXVU ; i++)/* ACHTUNG: muss VOR der NEUBerechnung*/
0165    {                           /* von akvu passieren, sonst wird in  */
0166     BAV[Ownnr()].DatenBAV.Dw[1] =  /* Perioden mit Aktivierungsschock*/
0167     BAV[Ownnr()].DatenBAV.Dw[1] +  /* mit Werbung  aus t-1 und der   */
0168     VU[i].DatenVU->Sp[2].k.Wa[(period-1)]; /* Versichereranzahl aus  */
0169    }                               /* t gerechnet !                  */
0170   if ( akvu != 0)              /* Division durch Null vermeiden...   */
0171    {
0172     BAV[Ownnr()].DatenBAV.Dw[1] = BAV[Ownnr()].DatenBAV.Dw[1] / akvu ;
0173    }
0174    else /* akvu == 0 */
0175     {
0176      BAV[Ownnr()].DatenBAV.Dw[1] = 0.0;
0177     }
0178  } /* END: if ( period > 1 ) */
0179  else /* period == 0 */
0180   {
```

```
0181    BAV[Ownnr()].DatenBAV.Dw[1] = 0.0; /* .. zur Vollstaendigkeit...  */
0182   }
0183 j = 0;                          /* Durchschnittlicher Vs.grad Risiko 1*/
0184 if ( period > 1 )
0185  {
0186   BAV[Ownnr()].DatenBAV.Dg[0] = 0.0;
0187   for(i = 1; i <= MAXVN ; i++ )/* Versicherungsstatus der Vorperiode */
0188    {                            /* auslesen und aufaddieren ...       */
0189     if ( VN[i].DatenVN->Rk[1].l.Vs[(period-1)] == 1 )
0190      {                           /* j ist die Anzahl der Versicherter,*/
0191       j++;                       /* die in der Vorperiode fremdver-   */
0192      }                           /* sichert waren ...                 */
0193     if ( akvn != 0 )             /* Division durch Null vermeiden ... */
0194      {
0195       BAV[Ownnr()].DatenBAV.Dg[0] = (float) j / (float) akvn ;
0196      }
0197      else /* akvn == 0 */
0198       {
0199        BAV[Ownnr()].DatenBAV.Dg[0] = 0.0;
0200       }
0201    }
0202  } /* END: if ( period > 1 ) */
0203  else /* period == 0 */
0204  {                              /* In der 1. Periode ist der Versiche-*/
0205   BAV[Ownnr()].DatenBAV.Dg[0] = 0.0; /* rungsgrad auf Null gesetzt...*/
0206  }
0207 j = 0;                          /* Durchschnittlicher Vs.grad Risiko 2*/
0208 if ( period > 1 )
0209  {
0210   BAV[Ownnr()].DatenBAV.Dg[1] = 0.0;
0211   for(i = 1; i <= MAXVN ; i++ )/* Versicherungsstatus der Vorperiode */
0212    {                            /* auslesen und aufaddieren ...       */
0213     if ( VN[i].DatenVN->Rk[2].k.Vs[(period-1)] == 1 )
0214      {                           /* j ist die Anzahl der Versicherter,*/
0215       j++;                       /* die in der Vorperiode fremdver-   */
0216      }                           /* sichert waren ...                 */
0217     if ( akvn != 0 )             /* Division durch Null vermeiden ... */
0218      {
0219       BAV[Ownnr()].DatenBAV.Dg[1] = (float) j / (float) akvn ;
0220      }
0221      else /* akvn == 0 */
0222       {
0223        BAV[Ownnr()].DatenBAV.Dg[1] = 0.0;
0224       }
0225    }
```

```
0226  } /* END: if ( period > 1 ) */
0227  else /* period == 0 */
0228  {                              /* In der 1. Periode ist der Versiche-*/
0229   BAV[Ownnr()].DatenBAV.Dg[1] = 0.0; /* rungsgrad auf Null gesetzt...*/
0230  }
0231 for ( i = 1; i <= MAXVU; i++) { aktiVU[i] = 0; } /* ..erst alle auf  */
0232 for ( i = 1; i <= MAXVN; i++) { aktiVN[i] = 0; } /* Null setzen ...  */
0233 akvu = 0;                      /* Berechnen von aktiVU[.], akvu      */
0234 for( i = 1; i <= MAXVU ; i++)  /* aktiVU[i] zeigt, ob VU[i] in t ge- */
0235  {                             /* waehlt werden kann und wieviel VUs */
0236   if ( (VU[i].DatenVU->Ap * VU[i].DatenVU->Al[rl] ) != 0 )
0237    {                           /* in t aktiv sind ...                */
0238     aktiVU[i] = 1;             /* Das wird von den VNs gebraucht,    */
0239     akvu++;                    /* sie einen aktiven VU waehlen wollen*/
0240    }
0241   else
0242    {
0243     aktiVU[i] = 0;
0244    }
0245  }
0246 akvn = 0;                      /* Berechnen von aktiVN[.], akvn      */
0247 for( i = 1; i <= MAXVN ; i++)  /* Wird u.a. von VR Mark-Up III ge-   */
0248  {                             /* braucht...                         */
0249   if ( ( VN[i].DatenVN->Ap * VN[i].DatenVN->Al[rl] ) != 0 )
0250    {
0251     aktiVN[i] = 1;
0252     akvn++;
0253    }
0254   else
0255    {
0256     aktiVN[i] = 0;
0257    }
0258  }
0259 if ( period > 1 ) /* Minimalpraemien berechnen: Sparte 1             */
0260  {                /* Wird von der VR DUMPING benutzt                 */
0261   BAV[Ownnr()].DatenBAV.Pm[0] = 1000.0; /* ... auf fiktives Maximum  */
0262   for( i = 1; i <= MAXVU ; i++ )        /* initialisieren...         */
0263    {
0264     if ( ( VU[i].DatenVU->Sp[1].l.Pr[(period-1)] <
0265           BAV[Ownnr()].DatenBAV.Pm[0]  )                  &&
0266          ( VU[i].DatenVU->Sp[1].l.Pr[(period-1)] > 0.0 ) )
0267      {
0268       BAV[Ownnr()].DatenBAV.Pm[0]=VU[i].DatenVU->Sp[1].l.Pr[(period-1)];
0269      }
0270    }
```

```
0271  } /* END: if ( period > 1 ) */
0272  else /* period == 0 */
0273  {
0274   BAV[Ownnr()].DatenBAV.Pm[0] = 0.0;
0275  }
0276 if ( period > 1 ) /* Minimalpraemien berechnen: Sparte 2            */
0277  {                /* Wird von der VR DUMPING benutzt                */
0278   BAV[Ownnr()].DatenBAV.Pm[1] = 1000.0; /* ... auf fiktives Maximum  */
0279   for( i = 1; i <= MAXVU ; i++ )        /* initialisieren...         */
0280    {
0281     if ( ( VU[i].DatenVU->Sp[2].k.Pr[(period-1)] <
0282            BAV[Ownnr()].DatenBAV.Pm[1] )                &&
0283          ( VU[i].DatenVU->Sp[2].k.Pr[(period-1)] > 0.0 ) )
0284      {
0285       BAV[Ownnr()].DatenBAV.Pm[1]=VU[i].DatenVU->Sp[2].k.Pr[(period-1)];
0286      }
0287    }
0288  } /* END: if ( period > 1 ) */
0289  else /* period == 0 */
0290  {
0291   BAV[Ownnr()].DatenBAV.Pm[1] = 0.0;
0292  }
0293 if ( period > 1 ) /* Maximalwerbung berechnen: Sparte 1             */
0294  {
0295   BAV[Ownnr()].DatenBAV.Wm[0] = 0.0;
0296   for( i = 1; i <= MAXVU ; i++)
0297    {
0298     if ( VU[i].DatenVU->Sp[1].l.Wa[(period-1)] >
0299          BAV[Ownnr()].DatenBAV.Wm[0]            )

0300      {
0301       BAV[Ownnr()].DatenBAV.Wm[0]=VU[i].DatenVU->Sp[1].l.Wa[(period-1)];
0302      }
0303    }
0304  } /* END: if ( period > 1 ) */
0305  else /* period == 0 */
0306  {
0307   BAV[Ownnr()].DatenBAV.Wm[0] = 0.0;
0308  }
0309 if ( period > 1 ) /* Maximalwerbung berechnen: Sparte 2             */
0310  {
0311   BAV[Ownnr()].DatenBAV.Wm[1] = 0.0;
0312   for( i = 1; i <= MAXVU ; i++)
0313    {
0314     if ( VU[i].DatenVU->Sp[2].k.Wa[(period-1)] >
0315          BAV[Ownnr()].DatenBAV.Wm[1]            )
```

```
0316      {
0317       BAV[Ownnr()].DatenBAV.Wm[1]=VU[i].DatenVU->Sp[2].k.Wa[(period-1)];
0318      }
0319    }
0320  } /* END: if ( period > 1 ) */
0321  else /* period == 0 */
0322  {
0323   BAV[Ownnr()].DatenBAV.Wm[1] = 0.0;
0324  }
0325 r = 0.0;                        /* Marktfuehrerpraemie Sparte 1       */
0326 if ( period > 1 )
0327  {
0328   BAV[Ownnr()].DatenBAV.Mp[0] = 0.0; /* Versicherer in Sparte 1 mit  */
0329   for( i = 1; i <= MAXVU ; i++)      /* den groessten Reserven in der*/
0330    {                                 /* Vorperiode finden ...        */
0331     if ( VU[i].DatenVU->Sp[1].l.Rs[(period-1)] > r  )
0332      {                               /* ... und merken ...           */
0333       r = VU[i].DatenVU->Sp[1].l.Rs[(period-1)];
0334       BAV[Ownnr()].DatenBAV.Mp[0]=VU[i].DatenVU->Sp[1].l.Pr[(period-1)];
0335      }
0336    }
0337  }
0338  else /* period == 1 */
0339  {
0340   BAV[Ownnr()].DatenBAV.Mp[0] = 0.0; /* ... zur Vollstaendigkeit...  */
0341  }
0342 r = 0.0;                        /* Marktfuehrerpraemie Sparte 2       */
0343 if ( period > 1 )
0344  {
0345   BAV[Ownnr()].DatenBAV.Mp[1] = 0.0; /* Versicherer in Sparte 2 mit  */
0346   for( i = 1; i <= MAXVU ; i++)      /* den groessten Reserven in der*/
0347    {                                 /* Vorperiode finden ...        */
0348     if ( VU[i].DatenVU->Sp[2].k.Rs[(period-1)] > r  )
0349      {                               /* ... und merken ...           */
0350       r = VU[i].DatenVU->Sp[2].k.Rs[(period-1)];
0351       BAV[Ownnr()].DatenBAV.Mp[1]=VU[i].DatenVU->Sp[2].k.Pr[(period-1)];
0352      }
0353    }
0354  }
0355  else /* period == 1 */
0356  {
0357   BAV[Ownnr()].DatenBAV.Mp[1] = 0.0; /* ... zur Vollstaendigkeit...  */
0358  }
0359 r = 0.0;                        /* Marktfuehrerwerbung Sparte 1       */
0360 if ( period > 1 )
```

```
0361  {
0362   BAV[Ownnr()].DatenBAV.Mw[0] = 0.0; /* Versicherer in Sparte 1 mit  */
0363   for( i = 1; i <= MAXVU ; i++)      /* den groessten Reserven in der*/
0364    {                                 /* Vorperiode finden ...        */
0365     if ( VU[i].DatenVU->Sp[1].l.Rs[(period-1)] > r  )
0366      {                                /* ... und merken ...           */
0367       r = VU[i].DatenVU->Sp[1].l.Rs[(period-1)];
0368       BAV[Ownnr()].DatenBAV.Mw[0]=VU[i].DatenVU->Sp[1].l.Wa[(period-1)];
0369      }
0370    }
0371  }
0372  else /* period == 1 */
0373  {
0374   BAV[Ownnr()].DatenBAV.Mw[0] = 0.0; /* ... zur Vollstaendigkeit...  */
0375  }
0376 r = 0.0;                        /* Marktfuehrerwerbung Sparte 2       */
0377 if ( period > 1 )
0378  {
0379   BAV[Ownnr()].DatenBAV.Mw[1] = 0.0; /* Versicherer in Sparte 2 mit  */
0380   for( i = 1; i <= MAXVU ; i++)      /* den groessten Reserven in der*/
0381    {                                 /* Vorperiode finden ...        */
0382     if ( VU[i].DatenVU->Sp[2].k.Rs[(period-1)] > r  )
0383      {                                /* ... und merken ...           */
0384       r = VU[i].DatenVU->Sp[2].k.Rs[(period-1)];
0385       BAV[Ownnr()].DatenBAV.Mw[1]=VU[i].DatenVU->Sp[2].k.Wa[(period-1)];
0386      }
0387    }
0388  }
0389  else /* period == 1 */
0390  {
0391   BAV[Ownnr()].DatenBAV.Mw[1] = 0.0; /* ... zur Vollstaendigkeit...  */
0392  }
0393  fprintf(runreport, "Frmdinf(%d,%d).akvu(%d),akvn(%d)\n",
0394                     logtime,period,akvu,akvn);
0395 }/* END: act Frmdinf */
0396
0397 /* ************************************************************** */
0398 /* class BAV: ESL-act zur Sicherung der Simulationsdaten in exter-   */
0399 /*            nen Files. Die Daten werden auf den Aggragatstufen     */
0400 /*            I..IV in ACSII-Files geschrieben.                      */
0401 /* ************************************************************** */
0402 act Agrsich {
0403 int i,j,k;                                    /* Schleifenzaehler   */
0404 file *t;         /* Es ist immer nur EINE Aggregatdatei offen ...    */
0405 float pr1,wa1,rs1,sh1,pr2,wa2,rs2,sh2; /* Hilfsvariable fuer         */
```

```
0406 int vn1, sa1, vn2, sa2;                /* Aggregatbildung            */
0407 int vu1,vu2,a1[MAXVU+1],a2[MAXVU+1],l,m;
0408 float vp1,vp2,ev1,ev2,vs1,vs2,vm;
0409 for( i = 1 ; i <= MAXVU ; i++ )/* Aggregat: Versicherer: Stufe I ... */
0410  {                              /*         : Einzelsubjekt            */
0411   if ( agvu1[i] == 1 )         /* WENN Aggregat erzeugt werden soll..*/
0412    {                              /* WENN Datei agvu1nm[i] existiert:*/
0413     t = fopen(agvu1nm[i], "a+t"); /* im Textmodus geoeffnen, SONST   */
0414     if ( t != ( file *) 0 )       /* wird sie neu angelegen ...      */
0415      {
0416       if ( (rl == 1) && ( period == 1) ) /* ERSTE period, ERSTER */
0417        {                          /* Lauf: Dateikopf schreiben ...   */
0418         fprintf(t,"#t  ");
0419         fprintf(t," Pr1   Wa1    Rs1     Vn1 Sa1 Sh1   ");
0420         fprintf(t," Pr2   Wa2    Rs2     Vn2 Sa2 Sh2\n");
0421        } /* END: if ( (rl == 1) && ( period == 1) ) */
0422        fprintf(t,"%04d",(rl-1)*sl+period);
0423        fprintf(t," %05.1f %05.1f %+09.1f %02d %03d %06.1f",
0424                VU[i].DatenVU->Sp[1].l.Pr[period],
0425                VU[i].DatenVU->Sp[1].l.Wa[period],
0426                VU[i].DatenVU->Sp[1].l.Rs[period],
0427                VU[i].DatenVU->Sp[1].l.Vn[period],
0428                VU[i].DatenVU->Sp[1].l.Sa[period],
0429                VU[i].DatenVU->Sp[1].l.Sh[period]
0430               );
0431        fprintf(t," %05.1f %05.1f %+09.1f %03d %03d %06.1f\n",
0432                VU[i].DatenVU->Sp[2].k.Pr[period],
0433                VU[i].DatenVU->Sp[2].k.Wa[period],
0434                VU[i].DatenVU->Sp[2].k.Rs[period],
0435                VU[i].DatenVU->Sp[2].k.Vn[period],
0436                VU[i].DatenVU->Sp[2].k.Sa[period],
0437                VU[i].DatenVU->Sp[2].k.Sh[period]
0438                );
0439        fclose(t);
0440      } /* END: if (t != ( file *) 0) */
0441      else /*  (t == ( file *) 0) */
0442       {
0443        fprintf(runreport, "Kann %12s nicht oeffnen\n", agvu1nm[i]);
0444       }
0445    } /* END: if ( agvu1[i] == 1 ) */
0446  } /* END: for( i = 1 ; i <= MAXVU ; i++ )  */
0447
0448  pr1=0.0;wa1=0.0;rs1=0.0;sh1=0.0;pr2=0.0;wa2=0.0;rs2=0.0;sh2=0.0;
0449  vn1=0  ;sa1=0  ;vn2=0  ;sa2=0  ;
0450  for( j = 1; j < 10 ; j++ )         /* Aggregatberechnung : VU II    */
```

```
0451  {                                  /* ..ueber alle Verhaltenregeln..*/
0452   if ( agvu2[j] == 1 )              /* Wenn Aggregat erzeugt werden  */
0453    {                                /* soll ...                      */
0454     for( i = 1; i <= MAXVU ; i++ ) /* ... ueber alle VUs ...        */
0455      {                              /* Daten aller VUs mit gleicher  */
0456       if (( VU[i].DatenVU->Vr == j ) && ( aktiVU[i] == 1 ))
0457        {                            /* VR zusammenaddieren ....      */
0458         pr1 = pr1 + VU[i].DatenVU->Sp[1].l.Pr[period];
0459         wa1 = wa1 + VU[i].DatenVU->Sp[1].l.Wa[period];
0460         rs1 = rs1 + VU[i].DatenVU->Sp[1].l.Rs[period];
0461         vn1 = vn1 + VU[i].DatenVU->Sp[1].l.Vn[period];
0462         sa1 = sa1 + VU[i].DatenVU->Sp[1].l.Sa[period];
0463         sh1 = sh1 + VU[i].DatenVU->Sp[1].l.Sh[period];
0464         pr2 = pr2 + VU[i].DatenVU->Sp[2].k.Pr[period];
0465         wa2 = wa2 + VU[i].DatenVU->Sp[2].k.Wa[period];
0466         rs2 = rs2 + VU[i].DatenVU->Sp[2].k.Rs[period];
0467         vn2 = vn2 + VU[i].DatenVU->Sp[2].k.Vn[period];
0468         sa2 = sa2 + VU[i].DatenVU->Sp[2].k.Sa[period];
0469         sh2 = sh2 + VU[i].DatenVU->Sp[2].k.Sh[period];
0470        } /* END: (( VU[i].DatenVU->Vr == j ) && ( aktiVU[i] == 1 )) */
0471      } /* END: for( i = 1; i <= MAXVU ; i++ ) */
0472    if ( BAV[Ownnr()].DatenBAV.Vuag2[j] != 0 ) /* Division durch Null */
0473     {                                          /* vermeiden...        */
0474      k  = BAV[Ownnr()].DatenBAV.Vuag2[j];
0475      pr1=pr1/(float)k; wa1=wa1/(float)k; rs1=rs1/(float)k;
0476      vn1=vn1/k       ; sa1=sa1/k       ; sh1=sh1/(float)k;
0477      pr2=pr2/(float)k; wa2=wa2/(float)k; rs2=rs2/(float)k;
0478      vn2=vn2/k       ; sa2=sa2/k       ; sh2=sh2/(float)k;
0479     } /* END: if ( BAV[Ownnr()].DatenBAV.Vuag2[j] != 0 ) */
0480    t = fopen(agvu2nm[j], "a+t");      /* Aggregatdatei beschreiben   */
0481    if ( t != ( file *) 0 )
0482     {
0483      if ( (rl == 1) && ( period == 1) )  /* ERSTE period, ERSTER */
0484       {                            /* Lauf: Dateikopf schreiben ...   */
0485        fprintf(t,"#t  ");
0486        fprintf(t," Pr1   Wa1    Rs1     Vn1 Sa1 Sh1   ");
0487        fprintf(t," Pr2   Wa2    Rs2     Vn2 Sa2 Sh2\n");
0488       } /* END: if ( (rl == 1) && ( period == 1) ) */
0489       fprintf(t,"%04d",(rl-1)*sl+period);
0490       fprintf(t," %05.1f %05.1f %+09.1f %02d %03d %06.1f",
0491                   pr1,   wa1,   rs1,   vn1, sa1, sh1 );
0492       fprintf(t," %05.1f %05.1f %+09.1f %02d %03d %06.1f\n",
0493                   pr2,   wa2,   rs2,   vn2, sa2, sh2 );
0494       fclose(t);
0495     } /* END: if ( t != ( file *) 0 ) */
```

```
0496      else /* ( t == ( file *) 0 ) */
0497       {
0498        fprintf(runreport, "Kann %12s nicht oeffnen\n", agvu2nm[j]);
0499       } /* END: else,if ( t != ( file *) 0 ) */
0500     } /* END: if ( agvu2[j] == 1 ) */
0501   } /* END: for( j = 1; j <= 10 ; j++ ) */
0502  pr1=0.0;wa1=0.0;rs1=0.0;sh1=0.0;pr2=0.0;wa2=0.0;rs2=0.0;sh2=0.0;
0503  vn1=0  ;sa1=0  ;vn2=0  ;sa2=0  ;
0504  for( j = 1; j <= 3 ; j++ )         /* Aggregatberechnung : VU III   */
0505   {                                 /* ..alle Verhaltensregelklassen */
0506    if ( agvu3[j] == 1 )             /* Wenn Aggregat erzeugt werden  */
0507     {                               /* soll ...                      */
0508      for( i = 1; i <= MAXVU ; i++ ) /* ... ueber alle VUs ...        */
0509       {                             /* Daten aller VUs mit gleicher  */
0510        if (( VU[i].DatenVU->Vk == j ) && ( aktiVU[i] == 1 ))
0511         {                           /* VR zusammenaddieren ....      */
0512          pr1 = pr1 + VU[i].DatenVU->Sp[1].l.Pr[period];
0513          wa1 = wa1 + VU[i].DatenVU->Sp[1].l.Wa[period];
0514          rs1 = rs1 + VU[i].DatenVU->Sp[1].l.Rs[period];
0515          vn1 = vn1 + VU[i].DatenVU->Sp[1].l.Vn[period];
0516          sa1 = sa1 + VU[i].DatenVU->Sp[1].l.Sa[period];
0517          sh1 = sh1 + VU[i].DatenVU->Sp[1].l.Sh[period];
0518          pr2 = pr2 + VU[i].DatenVU->Sp[2].k.Pr[period];
0519          wa2 = wa2 + VU[i].DatenVU->Sp[2].k.Wa[period];
0520          rs2 = rs2 + VU[i].DatenVU->Sp[2].k.Rs[period];
0521          vn2 = vn2 + VU[i].DatenVU->Sp[2].k.Vn[period];
0522          sa2 = sa2 + VU[i].DatenVU->Sp[2].k.Sa[period];
0523          sh2 = sh2 + VU[i].DatenVU->Sp[2].k.Sh[period];
0524         } /* END: (( VU[i].DatenVU->Vr == j ) && ( aktiVU[i] == 1 )) */
0525       } /* END: for( i = 1; i <= MAXVU ; i++ ) */
0526
0527     if ( BAV[Ownnr()].DatenBAV.Vuag3[j] != 0 ) /* Division durch Null */
0528      {                                         /* vermeiden...        */
0529       k  = BAV[Ownnr()].DatenBAV.Vuag3[j];
0530       pr1=pr1/(float)k; wa1=wa1/(float)k; rs1=rs1/(float)k;
0531       vn1=vn1/k       ; sa1=sa1/k       ; sh1=sh1/(float)k;
0532       pr2=pr2/(float)k; wa2=wa2/(float)k; rs2=rs2/(float)k;
0533       vn2=vn2/k       ; sa2=sa2/k       ; sh2=sh2/(float)k;
0534      } /* END: if ( BAV[Ownnr()].DatenBAV.Vuag3[j] != 0 ) */
0535     t = fopen(agvu3nm[j], "a+t");      /* Aggregatdatei beschreiben   */
0536     if ( t != ( file *) 0 )
0537      {
0538       if ( (rl == 1) && ( period == 1) )  /* ERSTE period, ERSTER */
0539        {                            /* Lauf: Dateikopf schreiben ...   */
0540         fprintf(t,"#t  ");
```

```
0541        fprintf(t," Pr1   Wa1    Rs1     Vn1 Sa1 Sh1   ");
0542        fprintf(t," Pr2   Wa2    Rs2     Vn2 Sa2 Sh2\n");
0543       } /* END: if ( (rl == 1) && ( period == 1) ) */
0544       fprintf(t,"%04d",(rl-1)*sl+period);
0545       fprintf(t," %05.1f %05.1f %+09.1f %02d %03d %06.1f",
0546                   pr1,   wa1,   rs1,   vn1, sa1, sh1 );
0547       fprintf(t," %05.1f %05.1f %+09.1f %02d %03d %06.1f\n",
0548                   pr2,   wa2,   rs2,   vn2, sa2, sh2 );
0549      fclose(t);
0550     } /* END: if ( t != ( file *) 0 ) */
0551     else /* ( t == ( file *) 0 ) */
0552      {
0553       fprintf(runreport, "Kann %12s nicht oeffnen\n", agvu3nm[j]);
0554      } /* END: else,if ( t != ( file *) 0 ) */
0555    } /* END: if ( agvu3[j] == 1 ) */
0556  } /* END: for( j = 1; j <= 3 ; j++ ) */
0557 pr1=0.0;wa1=0.0;rs1=0.0;sh1=0.0;pr2=0.0;wa2=0.0;rs2=0.0;sh2=0.0;
0558 vn1=0  ;sa1=0  ;vn2=0  ;sa2=0  ;
0559 if ( agvu4[1] == 1 )             /* Aggregatberechnung: VU IV     */
0560  {
0561   for( i = 1; i <= MAXVU ; i++ ) /* ... ueber alle VUs ...        */
0562    {                             /* ... zusammenaddieren ....     */
0563     if ( aktiVU[i] == 1 )
0564      {
0565       pr1 = pr1 + VU[i].DatenVU->Sp[1].l.Pr[period];
0566       wa1 = wa1 + VU[i].DatenVU->Sp[1].l.Wa[period];
0567       rs1 = rs1 + VU[i].DatenVU->Sp[1].l.Rs[period];
0568       vn1 = vn1 + VU[i].DatenVU->Sp[1].l.Vn[period];
0569       sa1 = sa1 + VU[i].DatenVU->Sp[1].l.Sa[period];
0570       sh1 = sh1 + VU[i].DatenVU->Sp[1].l.Sh[period];
0571       pr2 = pr2 + VU[i].DatenVU->Sp[2].k.Pr[period];
0572       wa2 = wa2 + VU[i].DatenVU->Sp[2].k.Wa[period];
0573       rs2 = rs2 + VU[i].DatenVU->Sp[2].k.Rs[period];
0574       vn2 = vn2 + VU[i].DatenVU->Sp[2].k.Vn[period];
0575       sa2 = sa2 + VU[i].DatenVU->Sp[2].k.Sa[period];
0576       sh2 = sh2 + VU[i].DatenVU->Sp[2].k.Sh[period];
0577      } /* END: if ( aktiVU[i] == 1 ) */
0578    } /* END: for( i = 1; i <= MAXVU ; i++ ) */
0579  if ( akvu != 0 )                          /* Division durch Null */
0580   {                                        /* vermeiden...        */
0581    k  = akvu;
0582    pr1=pr1/(float)k; wa1=wa1/(float)k; rs1=rs1/(float)k;
0583    vn1=vn1/k       ; sa1=sa1/k       ; sh1=sh1/(float)k;
0584    pr2=pr2/(float)k; wa2=wa2/(float)k; rs2=rs2/(float)k;
0585    vn2=vn2/k       ; sa2=sa2/k       ; sh2=sh2/(float)k;
```

```
0586    } /* END: if ( akvu != 0 ) */
0587   t = fopen(agvu4nm[1], "a+t");      /* Aggregatdatei beschreiben   */
0588   if ( t != ( file *) 0 )
0589    {
0590     if ( (rl == 1) && ( period == 1) )  /* ERSTE period, ERSTER */
0591      {                            /* Lauf: Dateikopf schreiben ...   */
0592       fprintf(t,"#t  ");
0593       fprintf(t," Pr1   Wa1    Rs1     Vn1 Sa1 Sh1   ");
0594       fprintf(t," Pr2   Wa2    Rs2     Vn2 Sa2 Sh2\n");
0595      } /* END: if ( (rl == 1) && ( period == 1) ) */
0596      fprintf(t,"%04d",(rl-1)*sl+period);
0597      fprintf(t," %05.1f %05.1f %+09.1f %02d %03d %06.1f",
0598                  pr1,   wa1,   rs1,   vn1, sa1, sh1 );
0599      fprintf(t," %05.1f %05.1f %+09.1f %02d %03d %06.1f\n",

0600                    pr2,   wa2,   rs2,   vn2, sa2, sh2 );
0601      fclose(t);
0602    } /* END: if ( t != ( file *) 0 ) */
0603    else /* ( t == ( file *) 0 ) */
0604     {
0605      fprintf(runreport, "Kann %12s nicht oeffnen\n", agvu4nm[1]);
0606     } /* END: else,if ( t != ( file *) 0 ) */
0607   } /* END: if ( agvu4[1] == 1 ) */
0608  for( i = 1 ; i <= MAXVN ; i++ )  /* Aggregatberechnung: VN  I       */
0609   {
0610   if ( agvn1[i] == 1 )            /*WENN Aggregat erzeugt werden soll*/
0611    {                              /* WENN Datei agvn1nm[i] existiert,*/
0612     t = fopen(agvn1nm[i], "a+t"); /* im Textmodus geoeffnen, SONST   */
0613     if ( t != ( file *) 0 )       /* wird sie neu angelegen...       */
0614      {
0615       if ( (rl == 1) && ( period == 1) ) /* ERSTE period, ERSTER */
0616        {                          /* Lauf: Dateikopf schreiben ...   */
0617         fprintf(t,"#t ");
0618         fprintf(t," Vu1 Vs1  Vp1  Ev1   Sh1  ");
0619         fprintf(t," Vu2 Vs2  Vp2  Ev2   Sh2    Vm\n");
0620        } /* END: if ( (rl == 1) && ( period == 1) ) */
0621        fprintf(t,"%04d",(rl-1)*sl+period);
0622        fprintf(t," %03d %03d %04.1f %05.1f %05.1f",
0623                   VN[i].DatenVN->Rk[1].l.Vu[period],
0624                   VN[i].DatenVN->Rk[1].l.Vs[period],
0625                   VN[i].DatenVN->Rk[1].l.Vp[period],
0626                   VN[i].DatenVN->Rk[1].l.Ev[period],
0627                   VN[i].DatenVN->Rk[1].l.Sh[period]
0628               );
0629        fprintf(t," %03d %03d %04.1f %05.1f %05.1f",
0630                   VN[i].DatenVN->Rk[2].k.Vu[period],
```

```
0631                    VN[i].DatenVN->Rk[2].k.Vs[period],
0632                    VN[i].DatenVN->Rk[2].k.Vp[period],
0633                    VN[i].DatenVN->Rk[2].k.Ev[period],
0634                    VN[i].DatenVN->Rk[2].k.Sh[period]
0635                );
0636        fprintf(t," %+09.1f\n", VN[i].DatenVN->Vm[period]);
0637        fclose(t);
0638      } /* END: ( t != ( file *) 0 ) */
0639      else /* ( t == ( file *) 0 ) */
0640      {
0641       fprintf(runreport, "Kann %12s nicht oeffnen\n", agvn1nm[i]);
0642      } /* END: else, if ( t != ( file *) 0 ) */
0643    } /* END: if ( agvn1[i] == 1 )*/
0644   } /* END: for( i = 1 ; i <= MAXVN ; i++ )*/
0645
0646  vu1=0  ; vu2=0  ;   /* Haeufigster von diesem Aggregat gewaehltes VU*/
0647  vs1=0.0; vs2=0.0;   /* Durchschnittlicher Versich.grad d. Aggregats */
0648  vp1=0.0; vp2=0.0;   /* Durchschnittlich gezahlte Praemie d. Aggregts*/
0649  ev1=0.0; ev2=0.0;   /* Durchschnittlich eigenversicherter Schaden " */
0650  sh1=0.0; sh1=0.0;   /* Durchschnittlicher Schaden des Aggregats     */
0651  vm =0.0;            /* Durchschnittsvermoegen des Aggregats         */
0652  for( k = 1; k <= MAXVU; k ++) /* Haeufigkeitsverteilung der vom     */
0653   {                            /* Aggregat gewaehlten VU fuer beide  */
0654    a1[k] = 0;                  /* Risiken...                         */
0655    a2[k] = 0;
0656   }
0657  for( j = 1; j < 7 ; j++ )          /* Aggregatberechnung : VN II    */
0658   {                                  /* ..ueber alle Verhaltenregeln..*/
0659    if ( agvn2[j] == 1 )              /* Wenn Aggregat erzeugt werden  */
0660     {                                /* soll ...                      */
0661      for( i = 1; i <= MAXVN ; i++ ) /* ... ueber alle VNs ...        */
0662       {
0663        if (( VN[i].DatenVN->Vr == j ) && ( aktiVN[i] == 1 ))
0664         {
0665          l = VN[i].DatenVN->Rk[1].l.Vu[period]; /* Gewaehlten VU in  */
0666          if ( ( l >= 0) && ( l <= MAXVU))/* die Haeufigkeitstabelle  */
0667           {                              /* aufnehmen...             */
0668            a1[l]++;
0669           }
0670          vs1 = vs1 + (float) VN[i].DatenVN->Rk[1].l.Vs[period];
0671          vp1 = vp1 + VN[i].DatenVN->Rk[1].l.Vp[period];
0672          ev1 = ev1 + VN[i].DatenVN->Rk[1].l.Ev[period];
0673          sh1 = sh1 + VN[i].DatenVN->Rk[1].l.Sh[period];
0674          l = VN[i].DatenVN->Rk[2].k.Vu[period]; /* Gewaehlten VU in  */
0675          if ( ( l >= 0) && ( l <= MAXVU))/* die Haeufigkeitstabelle  */
```

```
0676          {                              /* aufnehmen...             */
0677           a2[l]++;
0678          }
0679         vs2 = vs2 + (float) VN[i].DatenVN->Rk[2].k.Vs[period];
0680         vp2 = vp2 + VN[i].DatenVN->Rk[2].k.Vp[period];
0681         ev2 = ev2 + VN[i].DatenVN->Rk[2].k.Ev[period];
0682         sh2 = sh2 + VN[i].DatenVN->Rk[2].k.Sh[period];
0683         vm  = vm  + VN[i].DatenVN->Vm[period];
0684        } /* END: if((VN[i].DatenVN->Vr==j)&&(aktiVN[i]==1))*/
0685      } /* END: for( i = 1; i <= MAXVN ; i++ ) */
0686     if ( BAV[Ownnr()].DatenBAV.Vnag2[j] != 0 )
0687      {
0688       l = 0;                        /* Maximum der Haeufigkeitstabelle*/
0689       for( k = 1; k <= MAXVU; k++)  /* des 1. Risikos finden ...      */
0690        {
0691         if ( a1[k] > l )
0692          {
0693           l   = a1[k];
0694           vu1 = k;
0695          }
0696        }
0697       l = 0;                        /* Maximum der Haeufigkeitstabelle*/
0698       for( k = 1; k <= MAXVU; k++)  /* des 2. Risikos finden ...      */
0699        {
0700         if ( a2[k] > l )
0701          {
0702           l   = a2[k];
0703           vu2 = k;
0704          }
0705        }
0706       m = BAV[Ownnr()].DatenBAV.Vnag2[j];
0707       if ( m != 0)
0708       {                             /* Division durch Null vermeiden  */
0709        vs1 = vs1/(float) m; vp1 = vp1/(float) m;
0710        ev1 = ev1/(float) m; sh1 = sh1/(float) m;
0711        vs2 = vs2/(float) m; vp2 = vp2/(float) m;
0712        ev2 = ev2/(float) m; sh2 = sh2/(float) m;
0713        vm  = vm/(float) m;
0714       }
0715      } /* END: BAV[Ownnr()].DatenBAV.Vnag2[j] != 0 )*/
0716     t = fopen(agvn2nm[j], "a+t");     /* Aggregatdatei beschreiben   */
0717     if ( t != ( file *) 0 )
0718     {
0719      if ( (rl == 1) && ( period == 1) )  /* ERSTE period, ERSTER */
0720       {                            /* Lauf: Dateikopf schreiben ...   */
```

```
0721        fprintf(t,"#t  ");
0722        fprintf(t," Vu1 Vs1  Vp1  Ev1   Sh1  ");
0723        fprintf(t," Vu2 Vs2  Vp2  Ev2   Sh2    Vm\n");
0724       } /* END: if ( (rl == 1) && ( period == 1) )*/
0725       fprintf(t,"%04d",(rl-1)*sl+period);
0726       fprintf(t," %03d %1.2f %04.1f %05.1f %05.1f",
0727                   vu1, vs1, vp1,   ev1,   sh1
0728              );
0729       fprintf(t," %03d %1.2f %04.1f %05.1f %05.1f",
0730                    vu2, vs2, vp2,   ev2,   sh2
0731              );
0732       fprintf(t," %+09.1f\n", vm);
0733       fclose(t);
0734     } /* END: if ( t != ( file *) 0 ) */
0735     else /* t == ( file *) 0 ) */
0736      {
0737       fprintf(runreport, "Kann %12s nicht oeffnen\n", agvn2nm[j]);
0738      }
0739    } /* END: if ( agvn2[j] == 1 )*/
0740   } /* END: for( j = 1; j < 7 ; j++ ) */
0741  vu1=0  ; vu2=0  ;   /* Haeufigster von diesem Aggregat gewaehltes VU*/
0742  vs1=0.0; vs2=0.0;   /* Durchschnittlicher Versich.grad d. Aggregats */
0743  vp1=0.0; vp2=0.0;   /* Durchschnittlich gezahlte Praemie d. Aggregts*/
0744  ev1=0.0; ev2=0.0;   /* Durchschnittlich eigenversicherter Schaden " */
0745  sh1=0.0; sh1=0.0;   /* Durchschnittlicher Schaden des Aggregats     */
0746  vm =0.0;            /* Durchschnittsvermoegen des Aggregats         */
0747  for( k = 1; k <= MAXVU; k ++) /* Haeufigkeitsverteilung der vom     */
0748   {                            /* Aggregat gewaehlten VU fuer beide  */
0749    a1[k] = 0;                  /* Risiken...                         */
0750    a2[k] = 0;
0751   }
0752  for( j = 1; j <= 3 ; j++ )         /* Aggregatberechnung : VN III   */
0753   {                                  /* ..ueber alle Verhaltenregeln..*/
0754    if ( agvn3[j] == 1 )              /* Wenn Aggregat erzeugt werden  */
0755     {                                /* soll ...                      */
0756      for( i = 1; i <= MAXVN ; i++ ) /* ... ueber alle VNs ...        */
0757       {
0758        if (( VN[i].DatenVN->Vk == j ) && ( aktiVN[i] == 1 ))
0759         {
0760          l = VN[i].DatenVN->Rk[1].l.Vu[period]; /* Gewaehlten VU in  */
0761          if ( ( l > 0) && ( l <= MAXVU))/* die Haeufigkeitstabelle  */
0762           {                              /* aufnehmen...             */
0763            a1[l]++;
0764           }
0765          vs1 = vs1 + (float) VN[i].DatenVN->Rk[1].l.Vs[period];
```

```
0766          vp1 = vp1 + VN[i].DatenVN->Rk[1].l.Vp[period];
0767          ev1 = ev1 + VN[i].DatenVN->Rk[1].l.Ev[period];
0768          sh1 = sh1 + VN[i].DatenVN->Rk[1].l.Sh[period];
0769          l = VN[i].DatenVN->Rk[2].k.Vu[period]; /* Gewaehlten VU in  */
0770          if ( ( l > 0) && ( l <= MAXVU))/* die Haeufigkeitstabelle  */
0771           {                              /* aufnehmen...             */
0772            a2[l]++;
0773           }
0774          vs2 = vs2 + (float) VN[i].DatenVN->Rk[2].k.Vs[period];
0775          vp2 = vp2 + VN[i].DatenVN->Rk[2].k.Vp[period];
0776          ev2 = ev2 + VN[i].DatenVN->Rk[2].k.Ev[period];
0777          sh2 = sh2 + VN[i].DatenVN->Rk[2].k.Sh[period];
0778          vm  = vm  + VN[i].DatenVN->Vm[period];
0779         } /* END: if((VN[i].DatenVN->Vk==j)&&(aktiVN[i]==1))*/
0780       } /* END: for( i = 1; i <= MAXVN ; i++ ) */
0781
0782     if ( BAV[Ownnr()].DatenBAV.Vnag3[j] != 0 )
0783      {
0784       l = 0;                        /* Maximum der Haeufigkeitstabelle*/
0785       for( k = 1; k <= MAXVU; k++)  /* des 1. Risikos finden ...      */
0786        {
0787         if ( a1[k] > l )
0788          {
0789           l   = a1[k];
0790           vu1 = k;
0791          }
0792        }
0793       l = 0;                        /* Maximum der Haeufigkeitstabelle*/
0794       for( k = 1; k <= MAXVU; k++)  /* des 2. Risikos finden ...      */
0795        {
0796         if ( a2[k] > l )
0797          {
0798           l   = a2[k];
0799           vu2 = k;
0800          }
0801        }
0802       m = BAV[Ownnr()].DatenBAV.Vnag3[j];
0803       if ( m > 0 )                  /* Division durch Null vermeiden  */
0804       {
0805        vs1 = vs1/(float) m; vp1 = vp1/(float) m;
0806        ev1 = ev1/(float) m; sh1 = sh1/(float) m;
0807        vs2 = vs2/(float) m; vp2 = vp2/(float) m;
0808        ev2 = ev2/(float) m; sh2 = sh2/(float) m;
0809        vm  = vm/(float) m;
0810       }
```

```
0811      } /* END: BAV[Ownnr()].DatenBAV.Vnag3[j] != 0 )*/
0812     t = fopen(agvn3nm[j], "a+t");     /* Aggregatdatei beschreiben   */
0813     if ( t != ( file *) 0 )
0814     {
0815      if ( (rl == 1) && ( period == 1) )  /* ERSTE period, ERSTER */
0816       {                            /* Lauf: Dateikopf schreiben ...   */
0817        fprintf(t,"#t  ");
0818        fprintf(t," Vu1 Vs1  Vp1  Ev1   Sh1  ");
0819        fprintf(t," Vu2 Vs2  Vp2  Ev2   Sh2   Vm\n");
0820       } /* END: if ( (rl == 1) && ( period == 1) )*/
0821       fprintf(t,"%04d",(rl-1)*sl+period);
0822       fprintf(t," %03d %1.2f %04.1f %05.1f %05.1f",
0823                   vu1, vs1, vp1,   ev1,   sh1
0824                );
0825       fprintf(t," %03d %1.2f %04.1f %05.1f %05.1f",
0826                   vu2, vs2, vp2,   ev2,   sh2
0827              );
0828       fprintf(t," %+09.1f\n", vm);
0829       fclose(t);
0830     } /* END: if ( t != ( file *) 0 ) */
0831     else /* t == ( file *) 0 ) */
0832      {
0833       fprintf(runreport, "Kann %12s nicht oeffnen\n", agvn3nm[j]);
0834      }
0835    } /* END: if ( agvn3[j] == 1 )*/
0836   } /* END: for( j = 1; j <= 3 ; j++ ) */
0837  vu1=0  ; vu2=0  ;   /* Haeufigster von diesem Aggregat gewaehltes VU*/
0838  vs1=0.0; vs2=0.0;   /* Durchschnittlicher Versich.grad d. Aggregats */
0839  vp1=0.0; vp2=0.0;   /* Durchschnittlich gezahlte Praemie d. Aggregts*/
0840  ev1=0.0; ev2=0.0;   /* Durchschnittlich eigenversicherter Schaden " */
0841  sh1=0.0; sh1=0.0;   /* Durchschnittlicher Schaden des Aggregats     */
0842  vm =0.0;            /* Durchschnittsvermoegen des Aggregats         */
0843  for( k = 1; k <= MAXVU; k ++) /* Haeufigkeitsverteilung der vom     */
0844   {                            /* Aggregat gewaehlten VU fuer beide  */
0845    a1[k] = 0;                  /* Risiken...                         */
0846    a2[k] = 0;
0847   }                            /* Aggregatberechnung : VN IV         */
0848  if ( agvn4[1] == 1 )             /* Wenn Aggregat erzeugt werden    */
0849   {                               /* soll ...                        */
0850    for( i = 1; i <= MAXVN ; i++ ) /* ... ueber alle VNs ...          */
0851     {
0852      if ( aktiVN[i] == 1 )
0853       {
0854        l = VN[i].DatenVN->Rk[1].l.Vu[period];   /* Gewaehlten VU in  */
0855        if ( ( l > 0) && ( l <= MAXVU) )  /* die Haeufigkeitstabelle  */
```

```
0856        {                                /* aufnehmen...             */
0857         a1[l]++;
0858        }
0859       vs1 = vs1 + VN[i].DatenVN->Rk[1].l.Vs[period];
0860       vp1 = vp1 + VN[i].DatenVN->Rk[1].l.Vp[period];
0861       ev1 = ev1 + VN[i].DatenVN->Rk[1].l.Ev[period];
0862       sh1 = sh1 + VN[i].DatenVN->Rk[1].l.Sh[period];
0863       l = VN[i].DatenVN->Rk[2].k.Vu[period];   /* Gewaehlten VU in  */
0864       if ( ( l >= 0) && ( l <= MAXVU) ) /* die Haeufigkeitstabelle  */
0865        {                                /* aufnehmen...             */
0866         a2[l]++;
0867        }
0868       vs2 = vs2 + VN[i].DatenVN->Rk[2].k.Vs[period];
0869       vp2 = vp2 + VN[i].DatenVN->Rk[2].k.Vp[period];
0870       ev2 = ev2 + VN[i].DatenVN->Rk[2].k.Ev[period];
0871       sh2 = sh2 + VN[i].DatenVN->Rk[2].k.Sh[period];
0872       vm  = vm  + VN[i].DatenVN->Vm[period];
0873      } /* END: if( aktiVN[i]==1 )*/
0874    } /* END: for( i = 1; i <= MAXVN ; i++ ) */
0875
0876   if ( akvn != 0 )
0877    {
0878     l = 0;                          /* Maximum der Haeufigkeitstabelle*/
0879     for( k = 1; k <= MAXVU; k++)    /* des 1. Risikos finden ...      */
0880      {
0881       if ( a1[k] > l )
0882        {
0883         l   = a1[k];
0884         vu1 = k;
0885        }
0886      }
0887     l = 0;                          /* Maximum der Haeufigkeitstabelle*/
0888     for( k = 1; k <= MAXVU; k++)    /* des 2. Risikos finden ...      */
0889      {
0890       if ( a2[k] > l )
0891        {
0892         l   = a2[k];
0893         vu2 = k;
0894        }
0895      }
0896     m = akvn;
0897     if ( akvn > 0 )
0898      {
0899       vs1 = vs1/(float) m; vp1 = vp1/(float) m;
```

```
0900        ev1 = ev1/(float) m; sh1 = sh1/(float) m;
0901        vs2 = vs2/(float) m; vp2 = vp2/(float) m;
0902        ev2 = ev2/(float) m; sh2 = sh2/(float) m;
0903        vm  = vm/(float) m;
0904       }
0905     } /* END: akvn != 0 */
0906    t = fopen(agvn4nm[1], "a+t");      /* Aggregatdatei beschreiben   */
0907    if ( t != ( file *) 0 )
0908    {
0909     if ( (rl == 1) && ( period == 1) )   /* ERSTE period, ERSTER */
0910      {                              /* Lauf: Dateikopf schreiben ...   */
0911       fprintf(t,"#t  ");
0912       fprintf(t," Vu1 Vs1  Vp1  Ev1   Sh1  ");
0913       fprintf(t," Vu2 Vs2  Vp2  Ev2   Sh2    Vm\n");
0914      } /* END: if ( (rl == 1) && ( period == 1) )*/
0915      fprintf(t,"%04d",(rl-1)*sl+period);
0916      fprintf(t," %03d %1.2f %04.1f %05.1f %05.1f",
0917                  vu1, vs1, vp1,   ev1,   sh1
0918             );
0919      fprintf(t," %03d %1.2f %04.1f %05.1f %05.1f",
0920                  vu2, vs2, vp2,   ev2,   sh2
0921             );
0922      fprintf(t," %+09.1f\n", vm);
0923      fclose(t);
0924    } /* END: if ( t != ( file *) 0 ) */
0925    else /* t == ( file *) 0 ) */
0926     {
0927      fprintf(runreport, "Kann %12s nicht oeffnen\n", agvn4nm[1]);
0928     }
0929   } /* END: ( agvn4[1] == 1 )*/
0930   fprintf(runreport, "Agrsich(%d,%d)\n", logtime,period);
0931   fprintf(runreport,"Vuag2[.]:{\n");
0932   for(i=1;i<=10;i++)
0933    {
0934     fprintf(runreport," %d ", BAV[Ownnr()].DatenBAV.Vuag2[i] );
0935    }
0936   fprintf(runreport,"}\n");
0937   fprintf(runreport,"Vuag3[.]:{\n");
0938   for(i=1;i<=3;i++)
0939    {
0940     fprintf(runreport," %d ", BAV[Ownnr()].DatenBAV.Vuag3[i] );
0941    }
0942   fprintf(runreport,"}\n");
0943   fprintf(runreport,"Vnag2[.]:{\n");
0944   for(i=1;i<=6;i++)
```

```
0945   {
0946    fprintf(runreport," %d ", BAV[Ownnr()].DatenBAV.Vnag2[i] );
0947   }
0948  fprintf(runreport,"}\n");
0949  fprintf(runreport,"Vnag3[.]:{\n");
0950  for(i=1;i<=3;i++)
0951   {
0952    fprintf(runreport," %d ", BAV[Ownnr()].DatenBAV.Vnag3[i] );
0953   }
0954  fprintf(runreport,"}\n");
0955  /* Alle Aggregatzaehler auf Null setzen fuer naechste Periode      */
0956  /* wieder auf Null setzen ...                                      */
0957  for( i = 0;  i <= MAXVU ; i++ ) { BAV[Ownnr()].DatenBAV.Vuag1[i] = 0;}
0958  for( i = 0;  i <=    10 ; i++ ) { BAV[Ownnr()].DatenBAV.Vuag2[i] = 0;}
0959  for( i = 0;  i <=     3 ; i++ ) { BAV[Ownnr()].DatenBAV.Vuag3[i] = 0;}
0960  for( i = 0;  i <= MAXVN ; i++ ) { BAV[Ownnr()].DatenBAV.Vnag1[i] = 0;}
0961  for( i = 0;  i <=     7 ; i++ ) { BAV[Ownnr()].DatenBAV.Vnag2[i] = 0;}
0962  for( i = 0;  i <=     3 ; i++ ) { BAV[Ownnr()].DatenBAV.Vnag3[i] = 0;}
0963 } /* END: act Agrsich */
0964
0965 /* ************************************************************** */
0966 /* class VU: Definition der ESL-class VU und ihrer Verhaltensregeln. */
0967 /*           Die ESL-Definitionen werden durch einen #include-Befehl */
0968 /*           aus der Datei "imsvu.e" geladen.                        */
0969 /* ************************************************************** */
0970 class VU {
0971            classVU *DatenVU;
0972          } subj[MAXVU]=
0973 #include "imsvu.e"
0974
0975 /* ************************************************************** */
0976 /* class VU: Myinitvu            Initialisierungsfunktion           */
0977 /*           Wird NUR in der ersten Periode des ERSTEN Simulations-  */
0978 /*           laufs ausgefuehrt, DANACH in Periode 1 IMMER <Newinivu> */
0979 /* ************************************************************** */
0980 act Myinitvu {
0981 int i;                                          /* Schleifenzaehler */
0982 gOwnnr   = Ownnr(); /* Sichern der globalen ESL-Steuervariable fuer  */
0983 glogtime = logtime; /* die indirekten Aufrufe der Verhaltensregel-   */
0984 gperiod  = period;  /* aktionen                                      */
0985 if ( ( rl == 1) && ( load == 0 ) ) /* NUR im 1. Lauf wenn KEINE  */
0986  {                                  /* Konfig geladen ist...        */
0987   for( i = 2 ;i <= SIMLAENGE ; i++ )   /* Initialisierung der Sparten*/
0988   {                                        /* Sparte 1 ....         */
0989    VU[gOwnnr].DatenVU->Sp[1].l.Pr[i] = 0.0; /* Praemienvektor       */
```

```
0990    VU[gOwnnr].DatenVU->Sp[1].l.Wa[i] = 0.0; /* Werbeaufwandsvektor   */
0991    VU[gOwnnr].DatenVU->Sp[1].l.Rs[i] = 0.0; /* Reservenvektor        */
0992    VU[gOwnnr].DatenVU->Sp[1].l.Vn[i] = 0  ; /* Versichertenvektor    */
0993    VU[gOwnnr].DatenVU->Sp[1].l.Sa[i] = 0  ; /* Schadenanzahlvektor   */
0994    VU[gOwnnr].DatenVU->Sp[1].l.Sh[i] = 0.0; /* Schadensummenvektor   */
0995                                             /* Sparte 2 ....         */
0996    VU[gOwnnr].DatenVU->Sp[2].k.Pr[i] = 0.0; /* Praemienvektor        */
0997    VU[gOwnnr].DatenVU->Sp[2].k.Wa[i] = 0.0; /* Werbeaufwandsvektor   */
0998    VU[gOwnnr].DatenVU->Sp[2].k.Rs[i] = 0.0; /* Reservenvektor        */
0999    VU[gOwnnr].DatenVU->Sp[2].k.Vn[i] = 0  ; /* Versichertenvektor    */
1000    VU[gOwnnr].DatenVU->Sp[2].k.Sa[i] = 0  ; /* Schadenanzahlvektor   */
1001    VU[gOwnnr].DatenVU->Sp[2].k.Sh[i] = 0.0; /* Schadensummenvektor   */
1002   }
1003     /* VU[gOwnnr].DatenVU->Ac[1].lt=1;VU[gOwnnr].DatenVU->Ac[1].st=1;*/
1004     /* VU[gOwnnr].DatenVU->Vr = 1; VU[gOwnnr].DatenVU->Vk = 1;       */
1005  }                 /* ENDE:.. if ( ( rl == 1) && ( load == 0 ) ) */
1006   /*fprintf(runreport, "Myinitvu VU[%d] im Lauf [%d]\n", gOwnnr, rl);*/
1007 }
1008
1009 /* ************************************************************** */
1010 /* class VU: Newinivu         Re-Initialisierungsfunktion          */
1011 /*           Wird immer in der ersten Periode in alle  Simulations-  */
1012 /*           laeufen NACH dem ersten Lauf durchgefuehrt             */
1013 /* ************************************************************** */
1014 act Newinivu {
1015 int i; /* Schleifenvariable */
1016 gOwnnr   = Ownnr();
1017 glogtime = logtime;
1018 gperiod  = period; /* printf("Newinivu[%d](a)\n",gOwnnr); */
1019 if( (rl >= 2) && (period == 1) )
1020  {
1021   for(i = 2;i <= SIMLAENGE ; i++ ) /* Re-Initialisierung der Sparten */
1022   {                                         /* Sparte 1 .....        */
1023    VU[gOwnnr].DatenVU->Sp[1].l.Pr[i] = 0.0; /* Praemienvektor        */
1024    VU[gOwnnr].DatenVU->Sp[1].l.Wa[i] = 0.0; /* Werbeaufwandsvektor   */
1025    VU[gOwnnr].DatenVU->Sp[1].l.Rs[i] = 0.0; /* Reservenvektor        */
1026    VU[gOwnnr].DatenVU->Sp[1].l.Vn[i] = 0  ; /* Versichertenvektor    */
1027    VU[gOwnnr].DatenVU->Sp[1].l.Sa[i] = 0  ; /* Schadenanzahlvektor   */
1028    VU[gOwnnr].DatenVU->Sp[1].l.Sh[i] = 0.0; /* Schadensummenvektor   */
1029                                             /* Sparte 2 ....         */
1030    VU[gOwnnr].DatenVU->Sp[2].k.Pr[i] = 0.0; /* Praemienvektor        */
1031    VU[gOwnnr].DatenVU->Sp[2].k.Wa[i] = 0.0; /* Werbeaufwandsvektor   */
1032    VU[gOwnnr].DatenVU->Sp[2].k.Rs[i] = 0.0; /* Reservenvektor        */
1033    VU[gOwnnr].DatenVU->Sp[2].k.Vn[i] = 0  ; /* Versichertenvektor    */
1034    VU[gOwnnr].DatenVU->Sp[2].k.Sa[i] = 0  ; /* Schadenanzahlvektor   */
```

```
1035    VU[gOwnnr].DatenVU->Sp[2].k.Sh[i] = 0.0; /* Schadensummenvektor   */
1036   }
1037   /* fprintf(runreport, "Newinivu[%d], Lauf:[%d]\n",gOwnnr, rl); */
1038  }
1039 } /* END: act Newinivu*/
1040
1041 /* Leer Aktion:         wird von der Koordinationsfunktion benoetigt */
1042 act Leer { fprintf(runreport,"VULeer, Logtime %2d\n",glogtime); }
1043
1044 /* ************************************************************** */
1045 /* class VU: PlanVU   Koordinationsfunktion der Verhaltensregeln.    */
1046 /*           Sie nimmt dem Aufruf der Verhaltensregel vor, die dem   */
1047 /*           VU zugeordnet ist. Kriterien: VR und logischer Zeitpunkt*/
1048 /* ************************************************************** */
1049 act PlanVU {
1050 int i;                                          /* Schleifenzaehler */
1051 gOwnnr   = Ownnr();
1052 glogtime = logtime;
1053 gperiod  = period;
1054 for( i=1; i <= MAXNLOG; i++)
1055  {
1056   if ( (VU[ gOwnnr ].DatenVU->Ac[i].st  != 0        ) &&
1057        (VU[ gOwnnr ].DatenVU->Ac[i].lt  == glogtime )
1058      )
1059    {
1060     (StratdVU[1].S[ VU[gOwnnr].DatenVU->Ac[i].st])();
1061    } /* END: if */
1062  } /*  END: for */
1063 } /* END: act PlanVU */
1064
1065 /* ************************************************************** */
1066 /* class StratdVU: ESL-class zur Verwaltung der Verhaltensregeln der */
1067 /*                 class VU. Wird von act <PlanVU> benoetigt, um aus */
1068 /*                 Aktionsvektor die Verhaltensregel auszulesen      */
1069 /* ************************************************************** */
1070 class StratdVU
1071  {
1072   pact S[MAXNLOG+1];
1073  } subj[1] =
1074  {
1075    val:   Leer, Vrvu01, Vrvu02, Vrvu03, Vrvu04, Vrvu05, Vrvu06,
1076                 Vrvu07, Vrvu08, Vrvu09, Vrvu10
1077
1078  };
1079
```

```
1080 /* Leer Aktion:         wird von der Koordinationsfunktion benoetigt */
1081 act Leer { fprintf(runreport,"Leer-Act der class VU "); }
1082 /* ************************************************************** */
1083 /* act Vrvu01: Versicherung:    Verhaltensregel: Zufall I            */
1084 /*                                                                   */
1085 /*                            vgl. Kapitel 3.3.1.1.                  */
1086 /* ************************************************************** */
1087 act Vrvu01 {             /* Vereinbarung der Hilfsvariablen         */
1088 float a0,a1,b0,b1,c0,c1,d0,d1,e0,e1,f0,f1,g0,g1,h0,h1,pr1t0,pr1t1,
1089       wa1t0,wa1t1,rs1t0,rs1t1,sh1t0,pr2t0,pr2t1,wa2t0,wa2t1,rs2t0,
1090       rs2t1,sh2t0,z;
1091 int   vn1t0,sa1t0,vn2t0,sa2t0;
1092 /* LANGE IMS-Variable in KURZE Variable fuer VR uebersetzen         */
1093 if (gperiod > 1)
1094 {                          /* Der Parametervektor der VR            */
1095  a0 = VU[gOwnnr].DatenVU->Pv[0 ]; a1 = VU[gOwnnr].DatenVU->Pv[1 ];
1096  b0 = VU[gOwnnr].DatenVU->Pv[2 ]; b1 = VU[gOwnnr].DatenVU->Pv[3 ];
1097  c0 = VU[gOwnnr].DatenVU->Pv[4 ]; c1 = VU[gOwnnr].DatenVU->Pv[5 ];
1098  d0 = VU[gOwnnr].DatenVU->Pv[6 ]; d1 = VU[gOwnnr].DatenVU->Pv[7 ];
1099  e0 = VU[gOwnnr].DatenVU->Pv[8 ]; e1 = VU[gOwnnr].DatenVU->Pv[9 ];
1100  f0 = VU[gOwnnr].DatenVU->Pv[10]; f1 = VU[gOwnnr].DatenVU->Pv[11];
1101  g0 = VU[gOwnnr].DatenVU->Pv[12]; g1 = VU[gOwnnr].DatenVU->Pv[13];
1102  h0 = VU[gOwnnr].DatenVU->Pv[14]; h1 = VU[gOwnnr].DatenVU->Pv[15];
1103  pr1t0 =  VU[gOwnnr].DatenVU->Sp[1].l.Pr[gperiod-1]; /* Sparte 1     */
1104  wa1t0 =  VU[gOwnnr].DatenVU->Sp[1].l.Wa[gperiod-1];
1105  rs1t0 =  VU[gOwnnr].DatenVU->Sp[1].l.Rs[gperiod-1];
1106  vn1t0 =  VU[gOwnnr].DatenVU->Sp[1].l.Vn[gperiod-1];
1107  sa1t0 =  VU[gOwnnr].DatenVU->Sp[1].l.Sa[gperiod-1];
1108  sh1t0 =  VU[gOwnnr].DatenVU->Sp[1].l.Sh[gperiod-1];
1109  pr2t0 =  VU[gOwnnr].DatenVU->Sp[2].k.Pr[gperiod-1]; /* Sparte 2     */
1110  wa2t0 =  VU[gOwnnr].DatenVU->Sp[2].k.Wa[gperiod-1];
1111  rs2t0 =  VU[gOwnnr].DatenVU->Sp[2].k.Rs[gperiod-1];
1112  vn2t0 =  VU[gOwnnr].DatenVU->Sp[2].k.Vn[gperiod-1];
1113  sa2t0 =  VU[gOwnnr].DatenVU->Sp[2].k.Sa[gperiod-1];
1114  sh2t0 =  VU[gOwnnr].DatenVU->Sp[2].k.Sh[gperiod-1];
1115 }
1116 z = BAV[1].DatenBAV.Zs[gperiod];
1117 if(gperiod>=((VU[gOwnnr].DatenVU->Ap) * (VU[gOwnnr].DatenVU->Al[rl])))
1118  {
1119   /* **************** Verwaltungsroutinen ************************ */
1120   /* Aktivitaetsmeldung an die Aggregattabellen der class BAV: ...   */
1121   BAV[1].DatenBAV.Vuag1[ gOwnnr                 ]++;
1122   BAV[1].DatenBAV.Vuag2[ VU[gOwnnr].DatenVU->Vr ]++;
1123   BAV[1].DatenBAV.Vuag3[ VU[gOwnnr].DatenVU->Vk ]++;
1124   /* Reserven in beiden Sparten fuer eine Periode verzinsen: ....    */
```

```
1125   rs1t1 = ( 1 + z ) * rs1t0;
1126   rs2t1 = ( 1 + z ) * rs2t0;
1127  if ( !( BAV[1].DatenBAV.As[gperiod] * BAV[1].DatenBAV.Ar[rl]))
1128   {/* KEIN Aenderungsschock */
1129    if ( gperiod == 1) /* Erste Periode: Startwerte benutzen */
1130    {
1131     pr1t1 = VU[g0wnnr].DatenVU->Sp[1].l.Pr[1];
1132     pr2t1 = VU[g0wnnr].DatenVU->Sp[2].k.Pr[1];
1133     wa1t1 = VU[g0wnnr].DatenVU->Sp[1].l.Wa[1];
1134     wa2t1 = VU[g0wnnr].DatenVU->Sp[2].k.Wa[1];
1135    }
1136    else /* Periode > 1: Verhaltensregel anwenden */
1137    {
1138     pr1t1 = a0 * myrndf();
1139     pr2t1 = b0 * myrndf();
1140     wa1t1 = c0 * myrndf();
1141     wa2t1 = d0 * myrndf();
1142    }
1143   } /* END !(BAV[1].Daten.As[ ... */
1144  else /* Aenderungsschock */
1145   {
1146    if ( gperiod == 1) /* Erste Periode: Startwerte benutzen */
1147    {
1148     pr1t1 = VU[g0wnnr].DatenVU->Sp[1].l.Pr[1];
1149     pr2t1 = VU[g0wnnr].DatenVU->Sp[2].k.Pr[1];
1150     wa1t1 = VU[g0wnnr].DatenVU->Sp[1].l.Wa[1];
1151     wa2t1 = VU[g0wnnr].DatenVU->Sp[2].k.Wa[1];
1152    }
1153    else /* Periode > 1: Verhaltensregel anwenden */
1154    {
1155     pr1t1 = a1 * myrndf();
1156     pr2t1 = b1 * myrndf();
1157     wa1t1 = c1 * myrndf();
1158     wa2t1 = d1 * myrndf();
1159    }
1160   } /* END: else Aenderungsschock */
1161  } /* END: IF (gperiod>=... */
1162  /* KURZE VR-Variable wieder in LANGE IMS-Variable speichern         */
1163  VU[g0wnnr].DatenVU->Sp[1].l.Pr[gperiod] = pr1t1; /* Zielgroessen in */
1164  VU[g0wnnr].DatenVU->Sp[1].l.Wa[gperiod] = wa1t1; /* die Vektoren    */
1165  VU[g0wnnr].DatenVU->Sp[2].k.Pr[gperiod] = pr2t1; /* zurueckspeichern*/
1166  VU[g0wnnr].DatenVU->Sp[2].k.Wa[gperiod] = wa2t1;
1167  VU[g0wnnr].DatenVU->Sp[1].l.Rs[gperiod] = rs1t1; /* Ergebnisgroessen*/
1168  VU[g0wnnr].DatenVU->Sp[2].k.Rs[gperiod] = rs2t1; /* zurueckspeichern*/
1169 } /* END: Vrvu01 */
```

```
1170
1171 /* ************************************************************** */
1172 /* act Vrvu02: Versicherung:    Verhaltensregel:  Zufall II       */
1173 /*                                                                */
1174 /*                            vgl. Kapitel 3.3.1.1.               */
1175 /* ************************************************************** */
1176 act Vrvu02 {             /* Vereinbarung der Hilfsvariablen       */
1177 float a0,a1,b0,b1,c0,c1,d0,d1,e0,e1,f0,f1,g0,g1,h0,h1,pr1t0,pr1t1,
1178       wa1t0,wa1t1,rs1t0,rs1t1,sh1t0,pr2t0,pr2t1,wa2t0,wa2t1,rs2t0,
1179       rs2t1,sh2t0,z;
1180 int   vn1t0,sa1t0,vn2t0,sa2t0;
1181 /* LANGE IMS-Variable in KURZE Variable fuer VR uebersetzen       */
1182 if (gperiod > 1)
1183 {                          /* Der Parametervektor der VR         */
1184  a0 = VU[gOwnnr].DatenVU->Pv[0 ]; a1 = VU[gOwnnr].DatenVU->Pv[1 ];
1185  b0 = VU[gOwnnr].DatenVU->Pv[2 ]; b1 = VU[gOwnnr].DatenVU->Pv[3 ];
1186  c0 = VU[gOwnnr].DatenVU->Pv[4 ]; c1 = VU[gOwnnr].DatenVU->Pv[5 ];
1187  d0 = VU[gOwnnr].DatenVU->Pv[6 ]; d1 = VU[gOwnnr].DatenVU->Pv[7 ];
1188  e0 = VU[gOwnnr].DatenVU->Pv[8 ]; e1 = VU[gOwnnr].DatenVU->Pv[9 ];
1189  f0 = VU[gOwnnr].DatenVU->Pv[10]; f1 = VU[gOwnnr].DatenVU->Pv[11];
1190  g0 = VU[gOwnnr].DatenVU->Pv[12]; g1 = VU[gOwnnr].DatenVU->Pv[13];
1191  h0 = VU[gOwnnr].DatenVU->Pv[14]; h1 = VU[gOwnnr].DatenVU->Pv[15];
1192  pr1t0 =  VU[gOwnnr].DatenVU->Sp[1].l.Pr[gperiod-1]; /* Sparte 1    */
1193  wa1t0 =  VU[gOwnnr].DatenVU->Sp[1].l.Wa[gperiod-1];
1194  rs1t0 =  VU[gOwnnr].DatenVU->Sp[1].l.Rs[gperiod-1];
1195  vn1t0 =  VU[gOwnnr].DatenVU->Sp[1].l.Vn[gperiod-1];
1196  sa1t0 =  VU[gOwnnr].DatenVU->Sp[1].l.Sa[gperiod-1];
1197  sh1t0 =  VU[gOwnnr].DatenVU->Sp[1].l.Sh[gperiod-1];
1198  pr2t0 =  VU[gOwnnr].DatenVU->Sp[2].k.Pr[gperiod-1]; /* Sparte 2    */
1199  wa2t0 =  VU[gOwnnr].DatenVU->Sp[2].k.Wa[gperiod-1];

1200  rs2t0 =  VU[gOwnnr].DatenVU->Sp[2].k.Rs[gperiod-1];
1201  vn2t0 =  VU[gOwnnr].DatenVU->Sp[2].k.Vn[gperiod-1];
1202  sa2t0 =  VU[gOwnnr].DatenVU->Sp[2].k.Sa[gperiod-1];
1203  sh2t0 =  VU[gOwnnr].DatenVU->Sp[2].k.Sh[gperiod-1];
1204 }
1205 z = BAV[1].DatenBAV.Zs[gperiod];
1206 if(gperiod>=((VU[gOwnnr].DatenVU->Ap) * (VU[gOwnnr].DatenVU->Al[rl])))
1207  {
1208   /* ***************** Verwaltungsroutinen ************************ */
1209   /* Aktivitaetsmeldung an die Aggregattabellen der class BAV: ...   */
1210   BAV[1].DatenBAV.Vuag1[ gOwnnr                 ]++;
1211   BAV[1].DatenBAV.Vuag2[ VU[gOwnnr].DatenVU->Vr ]++;
1212   BAV[1].DatenBAV.Vuag3[ VU[gOwnnr].DatenVU->Vk ]++;
1213   /* Reserven in beiden Sparten fuer eine Periode verzinsen: ....    */
1214   rs1t1 = ( 1 + z ) * rs1t0;
```

```
1215   rs2t1 = ( 1 + z ) * rs2t0;
1216   /* ** Verhaltensregel:                vgl. Kapitel:      ********* */
1217  if ( !( BAV[1].DatenBAV.As[gperiod] * BAV[1].DatenBAV.Ar[rl]))
1218   {/* KEIN Aenderungsschock */
1219    if ( gperiod == 1) /* Erste Periode: Startwerte benutzen */
1220    {
1221     pr1t1 = VU[g0wnnr].DatenVU->Sp[1].l.Pr[1];
1222     pr2t1 = VU[g0wnnr].DatenVU->Sp[2].k.Pr[1];
1223     wa1t1 = VU[g0wnnr].DatenVU->Sp[1].l.Wa[1];
1224     wa2t1 = VU[g0wnnr].DatenVU->Sp[2].k.Wa[1];
1225    }
1226    else /* Periode > 1: Verhaltensregel anwenden */
1227    {
1228     pr1t1 = a0 + b0 * normal();
1229     pr2t1 = c0 + d0 * normal();
1230     wa1t1 = e0 + f0 * normal();
1231     wa2t1 = g0 + h0 * normal();
1232    }
1233   } /* END !(BAV[1].Daten.As[ ... */
1234  else /* Aenderungsschock */
1235   {
1236    if ( gperiod == 1) /* Erste Periode: Startwerte benutzen */
1237    {
1238     pr1t1 = VU[g0wnnr].DatenVU->Sp[1].l.Pr[1];
1239     pr2t1 = VU[g0wnnr].DatenVU->Sp[2].k.Pr[1];
1240     wa1t1 = VU[g0wnnr].DatenVU->Sp[1].l.Wa[1];
1241     wa2t1 = VU[g0wnnr].DatenVU->Sp[2].k.Wa[1];
1242    }
1243    else /* Periode > 1: Verhaltensregel anwenden */
1244    {
1245     pr1t1 = a1 + b1 * normal();
1246     pr2t1 = c1 + d1 * normal();
1247     wa1t1 = e1 + f1 * normal();
1248     wa2t1 = g1 + h1 * normal();
1249    }
1250   } /* END: else Aenderungsschock */
1251  } /* END: IF (gperiod>=... */
1252  /* KURZE VR-Variable wieder in LANGE IMS-Variable speichern         */
1253  VU[g0wnnr].DatenVU->Sp[1].l.Pr[gperiod] = pr1t1; /* Zielgroessen in */
1254  VU[g0wnnr].DatenVU->Sp[1].l.Wa[gperiod] = wa1t1; /* die Vektoren    */
1255  VU[g0wnnr].DatenVU->Sp[2].k.Pr[gperiod] = pr2t1; /* zurueckspeichern*/
1256  VU[g0wnnr].DatenVU->Sp[2].k.Wa[gperiod] = wa2t1;
1257  VU[g0wnnr].DatenVU->Sp[1].l.Rs[gperiod] = rs1t1; /* Ergebnisgroessen*/
1258  VU[g0wnnr].DatenVU->Sp[2].k.Rs[gperiod] = rs2t1; /* zurueckspeichern*/
1259 } /* END: Vrvu02 */
```

```
1260
1261 /* ************************************************************** */
1262 /* act Vrvu03: Versicherung:    Verhaltensregel:  Mark-Up I       */
1263 /*                                                                */
1264 /*                           vgl. 3.3.1.2.                        */
1265 /* ************************************************************** */
1266 act Vrvu03 {             /* Vereinbarung der Hilfsvariablen       */
1267 float a0,a1,b0,b1,c0,c1,d0,d1,e0,e1,f0,f1,g0,g1,h0,h1,pr1t0,pr1t1,
1268       wa1t0,wa1t1,rs1t0,rs1t1,sh1t0,pr2t0,pr2t1,wa2t0,wa2t1,rs2t0,
1269       rs2t1,sh2t0,z,r1,r2;
1270 int   vn1t0,sa1t0,vn2t0,sa2t0;
1271 /* LANGE IMS-Variable in KURZE Variable fuer VR uebersetzen       */
1272 if (gperiod > 1)
1273 {                          /* Der Parametervektor der VR         */
1274  a0 = VU[gOwnnr].DatenVU->Pv[0 ]; a1 = VU[gOwnnr].DatenVU->Pv[1 ];
1275  b0 = VU[gOwnnr].DatenVU->Pv[2 ]; b1 = VU[gOwnnr].DatenVU->Pv[3 ];
1276  c0 = VU[gOwnnr].DatenVU->Pv[4 ]; c1 = VU[gOwnnr].DatenVU->Pv[5 ];
1277  d0 = VU[gOwnnr].DatenVU->Pv[6 ]; d1 = VU[gOwnnr].DatenVU->Pv[7 ];
1278  e0 = VU[gOwnnr].DatenVU->Pv[8 ]; e1 = VU[gOwnnr].DatenVU->Pv[9 ];
1279  f0 = VU[gOwnnr].DatenVU->Pv[10]; f1 = VU[gOwnnr].DatenVU->Pv[11];
1280  g0 = VU[gOwnnr].DatenVU->Pv[12]; g1 = VU[gOwnnr].DatenVU->Pv[13];
1281  h0 = VU[gOwnnr].DatenVU->Pv[14]; h1 = VU[gOwnnr].DatenVU->Pv[15];
1282  pr1t0 =  VU[gOwnnr].DatenVU->Sp[1].l.Pr[gperiod-1]; /* Sparte 1     */
1283  wa1t0 =  VU[gOwnnr].DatenVU->Sp[1].l.Wa[gperiod-1];
1284  rs1t0 =  VU[gOwnnr].DatenVU->Sp[1].l.Rs[gperiod-1];
1285  vn1t0 =  VU[gOwnnr].DatenVU->Sp[1].l.Vn[gperiod-1];
1286  sa1t0 =  VU[gOwnnr].DatenVU->Sp[1].l.Sa[gperiod-1];
1287  sh1t0 =  VU[gOwnnr].DatenVU->Sp[1].l.Sh[gperiod-1];
1288  pr2t0 =  VU[gOwnnr].DatenVU->Sp[2].k.Pr[gperiod-1]; /* Sparte 2     */
1289  wa2t0 =  VU[gOwnnr].DatenVU->Sp[2].k.Wa[gperiod-1];
1290  rs2t0 =  VU[gOwnnr].DatenVU->Sp[2].k.Rs[gperiod-1];
1291  vn2t0 =  VU[gOwnnr].DatenVU->Sp[2].k.Vn[gperiod-1];
1292  sa2t0 =  VU[gOwnnr].DatenVU->Sp[2].k.Sa[gperiod-1];
1293  sh2t0 =  VU[gOwnnr].DatenVU->Sp[2].k.Sh[gperiod-1];
1294 }
1295 z  = BAV[1].DatenBAV.Zs[gperiod];
1296 r1 = VU[gOwnnr].DatenVU->A1[1]; /* Anspruchsniveaus der Reserven: ...*/
1297 r2 = VU[gOwnnr].DatenVU->A2[1];
1298 if(gperiod>=((VU[gOwnnr].DatenVU->Ap) * (VU[gOwnnr].DatenVU->Al[rl])))
1299  {
1300   /* **************** Verwaltungsroutinen ************************ */
1301   /* Aktivitaetsmeldung an die Aggregattabellen der class BAV: ...   */
1302   BAV[1].DatenBAV.Vuag1[ gOwnnr                 ]++;
1303   BAV[1].DatenBAV.Vuag2[ VU[gOwnnr].DatenVU->Vr ]++;
1304   BAV[1].DatenBAV.Vuag3[ VU[gOwnnr].DatenVU->Vk ]++;
```

```
1305   /* Reserven in beiden Sparten fuer eine Periode verzinsen: ....    */
1306   rs1t1 = ( 1 + z ) * rs1t0;
1307   rs2t1 = ( 1 + z ) * rs2t0;
1308  if ( !( BAV[1].DatenBAV.As[gperiod] * BAV[1].DatenBAV.Ar[rl]))
1309   {/* KEIN Aenderungsschock */
1310    if ( gperiod == 1) /* Erste Periode: Startwerte benutzen */
1311    {
1312     pr1t1 = VU[gOwnnr].DatenVU->Sp[1].l.Pr[1];
1313     pr2t1 = VU[gOwnnr].DatenVU->Sp[2].k.Pr[1];
1314     wa1t1 = VU[gOwnnr].DatenVU->Sp[1].l.Wa[1];
1315     wa2t1 = VU[gOwnnr].DatenVU->Sp[2].k.Wa[1];
1316    }
1317    else /* Periode > 1: Verhaltensregel anwenden */
1318    {
1319     if ( rs1t0 <= r1  ) { pr1t1 = a0 * pr1t0; }
1320       else              { pr1t1 = b0 * pr1t0; }
1321     if ( rs2t0 <= r2  ) { pr2t1 = c0 * pr2t0; }
1322       else              { pr2t1 = d0 * pr2t0; }
1323     if ( rs1t0 <= r1  ) { wa1t1 = e0 * wa1t0; }
1324       else              { wa1t1 = f0 * wa1t0; }
1325     if ( rs2t0 <= r2  ) { wa2t1 = g0 * wa2t0; }
1326       else              { wa2t1 = h0 * wa2t0; }
1327    }
1328   } /* END !(BAV[1].Daten.As[ ... */
1329  else /* Aenderungsschock */
1330   {
1331    if ( gperiod == 1) /* Erste Periode: Startwerte benutzen */
1332    {
1333     pr1t1 = VU[gOwnnr].DatenVU->Sp[1].l.Pr[1];
1334     pr2t1 = VU[gOwnnr].DatenVU->Sp[2].k.Pr[1];
1335     wa1t1 = VU[gOwnnr].DatenVU->Sp[1].l.Wa[1];
1336     wa2t1 = VU[gOwnnr].DatenVU->Sp[2].k.Wa[1];
1337    }
1338    else /* Periode > 1: Verhaltensregel anwenden */
1339    {
1340     if ( rs1t0 <= 0.0 ) { pr1t1 = a1 * pr1t0; }
1341       else              { pr1t1 = b1 * pr1t0; }
1342     if ( rs2t0 <= 0.0 ) { pr2t1 = c1 * pr2t0; }
1343       else              { pr2t1 = d1 * pr2t0; }
1344     if ( rs1t0 <= 0.0 ) { wa1t1 = e1 * wa1t0; }
1345       else              { wa1t1 = f1 * wa1t0; }
1346     if ( rs2t0 <= 0.0 ) { wa2t1 = g1 * wa2t0; }
1347       else              { wa2t1 = h1 * wa2t0; }
1348    }
1349   } /* END: else Aenderungsschock */
```

```
1350  } /* END: IF (gperiod>=... */
1351  /* KURZE VR-Variable wieder in LANGE IMS-Variable speichern         */
1352  VU[gOwnnr].DatenVU->Sp[1].l.Pr[gperiod] = pr1t1; /* Zielgroessen in */
1353  VU[gOwnnr].DatenVU->Sp[1].l.Wa[gperiod] = wa1t1; /* die Vektoren    */
1354  VU[gOwnnr].DatenVU->Sp[2].k.Pr[gperiod] = pr2t1; /* zurueckspeichern*/
1355  VU[gOwnnr].DatenVU->Sp[2].k.Wa[gperiod] = wa2t1;
1356  VU[gOwnnr].DatenVU->Sp[1].l.Rs[gperiod] = rs1t1; /* Ergebnisgroessen*/
1357  VU[gOwnnr].DatenVU->Sp[2].k.Rs[gperiod] = rs2t1; /* zurueckspeichern*/
1358 } /* END: Vrvu03 */
1359
1360 /* ***************************************************************** */
1361 /* act Vrvu04: Versicherung:    Verhaltensregel: Mark-Up II          */
1362 /*                                                                   */
1363 /*                            vgl. Kapitel 3.3.1.2.                  */
1364 /* ****************************************************************** */
1365 act Vrvu04 {              /* Vereinbarung der Hilfsvariablen         */
1366 float a0,a1,b0,b1,c0,c1,d0,d1,e0,e1,f0,f1,g0,g1,h0,h1,pr1t0,pr1t1,
1367       wa1t0,wa1t1,rs1t0,rs1t1,sh1t0,pr2t0,pr2t1,wa2t0,wa2t1,rs2t0,
1368       rs2t1,sh2t0,z;
1369 int   vn1t0,sa1t0,vn2t0,sa2t0,vn1t02,vn2t02,nw1,nw2;
1370 /* LANGE IMS-Variable in KURZE Variable fuer VR uebersetzen          */
1371 if (gperiod > 1)
1372 {                          /* Der Parametervektor der VR             */
1373  a0 = VU[gOwnnr].DatenVU->Pv[0 ]; a1 = VU[gOwnnr].DatenVU->Pv[1 ];
1374  b0 = VU[gOwnnr].DatenVU->Pv[2 ]; b1 = VU[gOwnnr].DatenVU->Pv[3 ];
1375  c0 = VU[gOwnnr].DatenVU->Pv[4 ]; c1 = VU[gOwnnr].DatenVU->Pv[5 ];
1376  d0 = VU[gOwnnr].DatenVU->Pv[6 ]; d1 = VU[gOwnnr].DatenVU->Pv[7 ];
1377  e0 = VU[gOwnnr].DatenVU->Pv[8 ]; e1 = VU[gOwnnr].DatenVU->Pv[9 ];
1378  f0 = VU[gOwnnr].DatenVU->Pv[10]; f1 = VU[gOwnnr].DatenVU->Pv[11];
1379  g0 = VU[gOwnnr].DatenVU->Pv[12]; g1 = VU[gOwnnr].DatenVU->Pv[13];
1380  h0 = VU[gOwnnr].DatenVU->Pv[14]; h1 = VU[gOwnnr].DatenVU->Pv[15];
1381  pr1t0 =  VU[gOwnnr].DatenVU->Sp[1].l.Pr[gperiod-1]; /* Sparte 1     */
1382  wa1t0 =  VU[gOwnnr].DatenVU->Sp[1].l.Wa[gperiod-1];
1383  rs1t0 =  VU[gOwnnr].DatenVU->Sp[1].l.Rs[gperiod-1];
1384  vn1t0 =  VU[gOwnnr].DatenVU->Sp[1].l.Vn[gperiod-1];
1385  sa1t0 =  VU[gOwnnr].DatenVU->Sp[1].l.Sa[gperiod-1];
1386  sh1t0 =  VU[gOwnnr].DatenVU->Sp[1].l.Sh[gperiod-1];
1387  pr2t0 =  VU[gOwnnr].DatenVU->Sp[2].k.Pr[gperiod-1]; /* Sparte 2     */
1388  wa2t0 =  VU[gOwnnr].DatenVU->Sp[2].k.Wa[gperiod-1];
1389  rs2t0 =  VU[gOwnnr].DatenVU->Sp[2].k.Rs[gperiod-1];
1390  vn2t0 =  VU[gOwnnr].DatenVU->Sp[2].k.Vn[gperiod-1];
1391  sa2t0 =  VU[gOwnnr].DatenVU->Sp[2].k.Sa[gperiod-1];
1392  sh2t0 =  VU[gOwnnr].DatenVU->Sp[2].k.Sh[gperiod-1];
1393 }
1394 if (gperiod > 2)
```

```
1395 {
1396  vn1t02 =  VU[gOwnnr].DatenVU->Sp[1].l.Vn[gperiod-2];
1397  vn2t02 =  VU[gOwnnr].DatenVU->Sp[2].k.Vn[gperiod-2];
1398 }
1399 z   = BAV[1].DatenBAV.Zs[gperiod];
1400 nw1 = (int) VU[gOwnnr].DatenVU->A1[2]; /* Anspruchsniveaus */
1401 nw2 = (int) VU[gOwnnr].DatenVU->A2[2]; /* der Nettowechsler*/
1402 if(gperiod>=((VU[gOwnnr].DatenVU->Ap) * (VU[gOwnnr].DatenVU->Al[rl])))
1403  {
1404   /* ***************** Verwaltungsroutinen ************************* */
1405   /* Aktivitaetsmeldung an die Aggregattabellen der class BAV: ...   */
1406   BAV[1].DatenBAV.Vuag1[ gOwnnr                 ]++;
1407   BAV[1].DatenBAV.Vuag2[ VU[gOwnnr].DatenVU->Vr ]++;
1408   BAV[1].DatenBAV.Vuag3[ VU[gOwnnr].DatenVU->Vk ]++;
1409   /* Reserven in beiden Sparten fuer eine Periode verzinsen: ....    */
1410   rs1t1 = ( 1 + z ) * rs1t0;
1411   rs2t1 = ( 1 + z ) * rs2t0;
1412  if ( !( BAV[1].DatenBAV.As[gperiod] * BAV[1].DatenBAV.Ar[rl]))
1413   {/* KEIN Aenderungsschock */
1414    if ( gperiod < 3) /* Periode 1 und 2: Startwerte benutzen         */
1415    {
1416     pr1t1 = VU[gOwnnr].DatenVU->Sp[1].l.Pr[1];
1417     pr2t1 = VU[gOwnnr].DatenVU->Sp[2].k.Pr[1];
1418     wa1t1 = VU[gOwnnr].DatenVU->Sp[1].l.Wa[1];
1419     wa2t1 = VU[gOwnnr].DatenVU->Sp[2].k.Wa[1];
1420    }
1421    else /* Periode >= 3: Verhaltensregel anwenden */
1422    {
1423     if ((vn1t0-vn1t02) <= nw1) { pr1t1 = a0 * pr1t0; }
1424       else                     { pr1t1 = b0 * pr1t0; }
1425     if ((vn2t0-vn2t02) <= nw2) { pr2t1 = c0 * pr2t0; }
1426       else                     { pr2t1 = d0 * pr2t0; }
1427     if ((vn1t0-vn1t02) <= nw1) { wa1t1 = e0 * wa1t0; }
1428       else                     { wa1t1 = f0 * wa1t0; }
1429     if ((vn2t0-vn2t02) <= nw2) { wa2t1 = g0 * wa2t0; }
1430       else                     { wa2t1 = h0 * wa2t0; }
1431    }
1432   } /* END !(BAV[1].Daten.As[ ... */
1433  else /* Aenderungsschock */
1434   {
1435    if ( gperiod < 3) /* Erste Periode: Startwerte benutzen */
1436    {
1437     pr1t1 = VU[gOwnnr].DatenVU->Sp[1].l.Pr[1];
1438     pr2t1 = VU[gOwnnr].DatenVU->Sp[2].k.Pr[1];
1439     wa1t1 = VU[gOwnnr].DatenVU->Sp[1].l.Wa[1];
```

```
1440     wa2t1 = VU[g0wnnr].DatenVU->Sp[2].k.Wa[1];
1441    }
1442    else /* Periode >= 3: Verhaltensregel anwenden */
1443    {
1444     if ((vn1t0-vn1t02) <= nw1) { pr1t1 = a1 * pr1t0; }
1445       else                     { pr1t1 = b1 * pr1t0; }
1446     if ((vn2t0-vn2t02) <= nw2) { pr2t1 = c1 * pr2t0; }
1447       else                     { pr2t1 = d1 * pr2t0; }
1448     if ((vn1t0-vn1t02) <= nw1) { wa1t1 = e1 * wa1t0; }
1449       else                     { wa1t1 = f1 * wa1t0; }
1450     if ((vn2t0-vn2t02) <= nw2) { wa2t1 = g1 * wa2t0; }
1451       else                     { wa2t1 = h1 * wa2t0; }
1452    }
1453   } /* END: else Aenderungsschock */
1454   /* fprintf(runreport,"VU[%3d] Wechsler(%d,%d) A.niveau:(%d,%d) \n",
1455            g0wnnr, vn1t0-vn1t02 , vn2t0-vn2t02 ,nw1,nw2 ); */
1456  } /* END: IF (gperiod>=... */
1457  /* KURZE VR-Variable wieder in LANGE IMS-Variable speichern         */
1458  VU[g0wnnr].DatenVU->Sp[1].l.Pr[gperiod] = pr1t1; /* Zielgroessen in */
1459  VU[g0wnnr].DatenVU->Sp[1].l.Wa[gperiod] = wa1t1; /* die Vektoren    */
1460  VU[g0wnnr].DatenVU->Sp[2].k.Pr[gperiod] = pr2t1; /* zurueckspeichern*/
1461  VU[g0wnnr].DatenVU->Sp[2].k.Wa[gperiod] = wa2t1;
1462  VU[g0wnnr].DatenVU->Sp[1].l.Rs[gperiod] = rs1t1; /* Ergebnisgroessen*/
1463  VU[g0wnnr].DatenVU->Sp[2].k.Rs[gperiod] = rs2t1; /* zurueckspeichern*/
1464 } /* END: Vrvu04 */
1465
1466 /* ************************************************************** */
1467 /* act Vrvu05: Versicherung:    Verhaltensregel:  Mark-Up III        */
1468 /*                                                                   */
1469 /*                            vgl. Kapitel 3.3.1.2.                  */
1470 /* ************************************************************** */
1471 act Vrvu05 {             /* Vereinbarung der Hilfsvariablen         */
1472 float a0,a1,b0,b1,c0,c1,d0,d1,e0,e1,f0,f1,g0,g1,h0,h1,pr1t0,pr1t1,
1473       wa1t0,wa1t1,rs1t0,rs1t1,sh1t0,pr2t0,pr2t1,wa2t0,wa2t1,rs2t0,
1474       rs2t1,sh2t0,z,m1,m2,m1a,m2a;
1475 int   vn1t0,sa1t0,vn2t0,sa2t0;
1476 /* LANGE IMS-Variable in KURZE Variable fuer VR uebersetzen         */
1477 if (gperiod > 1)
1478 {                          /* Der Parametervektor der VR            */
1479  a0 = VU[g0wnnr].DatenVU->Pv[0 ]; a1 = VU[g0wnnr].DatenVU->Pv[1 ];
1480  b0 = VU[g0wnnr].DatenVU->Pv[2 ]; b1 = VU[g0wnnr].DatenVU->Pv[3 ];
1481  c0 = VU[g0wnnr].DatenVU->Pv[4 ]; c1 = VU[g0wnnr].DatenVU->Pv[5 ];
1482  d0 = VU[g0wnnr].DatenVU->Pv[6 ]; d1 = VU[g0wnnr].DatenVU->Pv[7 ];
1483  e0 = VU[g0wnnr].DatenVU->Pv[8 ]; e1 = VU[g0wnnr].DatenVU->Pv[9 ];
1484  f0 = VU[g0wnnr].DatenVU->Pv[10]; f1 = VU[g0wnnr].DatenVU->Pv[11];
```

```
1485  g0 = VU[gOwnnr].DatenVU->Pv[12]; g1 = VU[gOwnnr].DatenVU->Pv[13];
1486  h0 = VU[gOwnnr].DatenVU->Pv[14]; h1 = VU[gOwnnr].DatenVU->Pv[15];
1487  pr1t0 =  VU[gOwnnr].DatenVU->Sp[1].l.Pr[gperiod-1]; /* Sparte 1     */
1488  wa1t0 =  VU[gOwnnr].DatenVU->Sp[1].l.Wa[gperiod-1];
1489  rs1t0 =  VU[gOwnnr].DatenVU->Sp[1].l.Rs[gperiod-1];
1490  vn1t0 =  VU[gOwnnr].DatenVU->Sp[1].l.Vn[gperiod-1];
1491  sa1t0 =  VU[gOwnnr].DatenVU->Sp[1].l.Sa[gperiod-1];
1492  sh1t0 =  VU[gOwnnr].DatenVU->Sp[1].l.Sh[gperiod-1];
1493  pr2t0 =  VU[gOwnnr].DatenVU->Sp[2].k.Pr[gperiod-1]; /* Sparte 2     */
1494  wa2t0 =  VU[gOwnnr].DatenVU->Sp[2].k.Wa[gperiod-1];
1495  rs2t0 =  VU[gOwnnr].DatenVU->Sp[2].k.Rs[gperiod-1];
1496  vn2t0 =  VU[gOwnnr].DatenVU->Sp[2].k.Vn[gperiod-1];
1497  sa2t0 =  VU[gOwnnr].DatenVU->Sp[2].k.Sa[gperiod-1];
1498  sh2t0 =  VU[gOwnnr].DatenVU->Sp[2].k.Sh[gperiod-1];
1499 }

1500 z  = BAV[1].DatenBAV.Zs[gperiod];
1501 m1 = VU[gOwnnr].DatenVU->A1[3]; /* Anspruchsniveaus: Marktanteil     */
1502 m2 = VU[gOwnnr].DatenVU->A2[3];
1503 if(gperiod>=((VU[gOwnnr].DatenVU->Ap) * (VU[gOwnnr].DatenVU->Al[rl])))
1504  {
1505   /* ***************** Verwaltungsroutinen ************************* */
1506   /* Aktivitaetsmeldung an die Aggregattabellen der class BAV: ...   */
1507   BAV[1].DatenBAV.Vuag1[ gOwnnr                 ]++;
1508   BAV[1].DatenBAV.Vuag2[ VU[gOwnnr].DatenVU->Vr ]++;
1509   BAV[1].DatenBAV.Vuag3[ VU[gOwnnr].DatenVU->Vk ]++;
1510   /* Reserven in beiden Sparten fuer eine Periode verzinsen: ....    */
1511   rs1t1 = ( 1 + z ) * rs1t0;
1512   rs2t1 = ( 1 + z ) * rs2t0;
1513  if ( !( BAV[1].DatenBAV.As[gperiod] * BAV[1].DatenBAV.Ar[rl]))
1514   {/* KEIN Aenderungsschock */
1515    if ( gperiod == 1) /* Erste Periode: Startwerte benutzen */
1516    {
1517     pr1t1 = VU[gOwnnr].DatenVU->Sp[1].l.Pr[1];
1518     pr2t1 = VU[gOwnnr].DatenVU->Sp[2].k.Pr[1];
1519     wa1t1 = VU[gOwnnr].DatenVU->Sp[1].l.Wa[1];
1520     wa2t1 = VU[gOwnnr].DatenVU->Sp[2].k.Wa[1];
1521    }
1522    else /* Periode > 1: Verhaltensregel anwenden */
1523    {
1524     if ( akvn != 0 ) /* Division durch Null vermeiden               */
1525            { m1a = ((float) vn1t0) / ((float) akvn ) ; }
1526       else  { m1a = 0.0 ; }
1527     if ( akvn != 0 ) /* Division durch Null vermeiden               */
1528            { m2a = ((float) vn2t0) / ((float) akvn ) ; }
1529       else  { m2a = 0.0 ; }
```

```
1530     if ( m1a <= m1 )           { pr1t1 = a0 * pr1t0; }
1531       else                     { pr1t1 = b0 * pr1t0; }
1532     if ( m2a <= m2 )           { pr2t1 = c0 * pr2t0; }
1533       else                     { pr2t1 = d0 * pr2t0; }
1534     if ( m1a <= m1 )           { wa1t1 = e0 * wa1t0; }
1535       else                     { wa1t1 = f0 * wa1t0; }
1536     if ( m2a <= m2 )           { wa2t1 = g0 * wa2t0; }
1537       else                     { wa2t1 = h0 * wa2t0; }
1538    }
1539   } /* END !(BAV[1].Daten.As[ ... */
1540  else /* Aenderungsschock */
1541   {
1542    if ( gperiod == 1) /* Erste Periode: Startwerte benutzen */
1543    {
1544     pr1t1 = VU[gOwnnr].DatenVU->Sp[1].l.Pr[1];
1545     pr2t1 = VU[gOwnnr].DatenVU->Sp[2].k.Pr[1];
1546     wa1t1 = VU[gOwnnr].DatenVU->Sp[1].l.Wa[1];
1547     wa2t1 = VU[gOwnnr].DatenVU->Sp[2].k.Wa[1];
1548    }
1549    else /* Periode > 1: Verhaltensregel anwenden */
1550    {
1551     if ( akvn != 0 ) /* Division durch Null vermeiden               */
1552             { m1a = ((float) vn1t0) / ((float) akvn ) ; }
1553       else  { m1a = 0.0 ; }
1554     if ( akvn != 0 ) /* Division durch Null vermeiden               */
1555             { m2a = ((float) vn2t0) / ((float) akvn ) ; }
1556       else  { m2a = 0.0 ; }
1557     if ( m1a <= m1 )           { pr1t1 = a1 * pr1t0; }
1558       else                     { pr1t1 = b1 * pr1t0; }
1559     if ( m2a <= m2 )           { pr2t1 = c1 * pr2t0; }
1560       else                     { pr2t1 = d1 * pr2t0; }
1561     if ( m1a <= m1 )           { wa1t1 = e1 * wa1t0; }
1562       else                     { wa1t1 = f1 * wa1t0; }
1563     if ( m2a <= m2 )           { wa2t1 = g1 * wa2t0; }
1564       else                     { wa2t1 = h1 * wa2t0; }
1565    }
1566   } /* END: else Aenderungsschock */
1567  } /* END: IF (gperiod>=... */
1568  /* KURZE VR-Variable wieder in LANGE IMS-Variable speichern         */
1569  VU[gOwnnr].DatenVU->Sp[1].l.Pr[gperiod] = pr1t1; /* Zielgroessen in */
1570  VU[gOwnnr].DatenVU->Sp[1].l.Wa[gperiod] = wa1t1; /* die Vektoren    */
1571  VU[gOwnnr].DatenVU->Sp[2].k.Pr[gperiod] = pr2t1; /* zurueckspeichern*/
1572  VU[gOwnnr].DatenVU->Sp[2].k.Wa[gperiod] = wa2t1;
1573  VU[gOwnnr].DatenVU->Sp[1].l.Rs[gperiod] = rs1t1; /* Ergebnisgroessen*/
1574  VU[gOwnnr].DatenVU->Sp[2].k.Rs[gperiod] = rs2t1; /* zurueckspeichern*/
```

```
1575 } /* END: Vrvu05 */
1576
1577 /* ************************************************************** */
1578 /* act Vrvu06: Versicherung:    Verhaltensregel: Erwartungsschaden   */
1579 /*                                                                   */
1580 /*                            vgl. Kapitel 3.3.1.2.                  */
1581 /* ************************************************************** */
1582 act Vrvu06 {             /* Vereinbarung der Hilfsvariablen          */
1583 float a0,a1,b0,b1,c0,c1,d0,d1,e0,e1,f0,f1,g0,g1,h0,h1,pr1t0,pr1t1,
1584       wa1t0,wa1t1,rs1t0,rs1t1,sh1t0,pr2t0,pr2t1,wa2t0,wa2t1,rs2t0,
1585       rs2t1,sh2t0,z,s1,s2;
1586 int   vn1t0,sa1t0,vn2t0,sa2t0;
1587 /* LANGE IMS-Variable in KURZE Variable fuer VR uebersetzen         */
1588 if (gperiod > 1)
1589 {                          /* Der Parametervektor der VR            */
1590  a0 = VU[gOwnnr].DatenVU->Pv[0 ]; a1 = VU[gOwnnr].DatenVU->Pv[1 ];
1591  b0 = VU[gOwnnr].DatenVU->Pv[2 ]; b1 = VU[gOwnnr].DatenVU->Pv[3 ];
1592  c0 = VU[gOwnnr].DatenVU->Pv[4 ]; c1 = VU[gOwnnr].DatenVU->Pv[5 ];
1593  d0 = VU[gOwnnr].DatenVU->Pv[6 ]; d1 = VU[gOwnnr].DatenVU->Pv[7 ];
1594  e0 = VU[gOwnnr].DatenVU->Pv[8 ]; e1 = VU[gOwnnr].DatenVU->Pv[9 ];
1595  f0 = VU[gOwnnr].DatenVU->Pv[10]; f1 = VU[gOwnnr].DatenVU->Pv[11];
1596  g0 = VU[gOwnnr].DatenVU->Pv[12]; g1 = VU[gOwnnr].DatenVU->Pv[13];
1597  h0 = VU[gOwnnr].DatenVU->Pv[14]; h1 = VU[gOwnnr].DatenVU->Pv[15];
1598  pr1t0 =  VU[gOwnnr].DatenVU->Sp[1].l.Pr[gperiod-1]; /* Sparte 1     */
1599  wa1t0 =  VU[gOwnnr].DatenVU->Sp[1].l.Wa[gperiod-1];
1600  rs1t0 =  VU[gOwnnr].DatenVU->Sp[1].l.Rs[gperiod-1];
1601  vn1t0 =  VU[gOwnnr].DatenVU->Sp[1].l.Vn[gperiod-1];
1602  sa1t0 =  VU[gOwnnr].DatenVU->Sp[1].l.Sa[gperiod-1];
1603  sh1t0 =  VU[gOwnnr].DatenVU->Sp[1].l.Sh[gperiod-1];
1604  pr2t0 =  VU[gOwnnr].DatenVU->Sp[2].k.Pr[gperiod-1]; /* Sparte 2     */
1605  wa2t0 =  VU[gOwnnr].DatenVU->Sp[2].k.Wa[gperiod-1];
1606  rs2t0 =  VU[gOwnnr].DatenVU->Sp[2].k.Rs[gperiod-1];
1607  vn2t0 =  VU[gOwnnr].DatenVU->Sp[2].k.Vn[gperiod-1];
1608  sa2t0 =  VU[gOwnnr].DatenVU->Sp[2].k.Sa[gperiod-1];
1609  sh2t0 =  VU[gOwnnr].DatenVU->Sp[2].k.Sh[gperiod-1];
1610 }
1611 z = BAV[1].DatenBAV.Zs[gperiod];
1612 if(gperiod>=((VU[gOwnnr].DatenVU->Ap) * (VU[gOwnnr].DatenVU->Al[rl])))
1613  {
1614   /* **************** Verwaltungsroutinen ************************ */
1615   /* Aktivitaetsmeldung an die Aggregattabellen der class BAV: ...   */
1616   BAV[1].DatenBAV.Vuag1[ gOwnnr                 ]++;
1617   BAV[1].DatenBAV.Vuag2[ VU[gOwnnr].DatenVU->Vr ]++;
1618   BAV[1].DatenBAV.Vuag3[ VU[gOwnnr].DatenVU->Vk ]++;
1619   /* Reserven in beiden Sparten fuer eine Periode verzinsen: ....    */
```

```
1620   rs1t1 = ( 1 + z ) * rs1t0;
1621   rs2t1 = ( 1 + z ) * rs2t0;
1622  if ( !( BAV[1].DatenBAV.As[gperiod] * BAV[1].DatenBAV.Ar[rl]))
1623   {/* KEIN Aenderungsschock */
1624    if ( gperiod == 1) /* Erste Periode: Startwerte benutzen */
1625    {
1626     pr1t1 = VU[gOwnnr].DatenVU->Sp[1].l.Pr[1];
1627     pr2t1 = VU[gOwnnr].DatenVU->Sp[2].k.Pr[1];
1628     wa1t1 = VU[gOwnnr].DatenVU->Sp[1].l.Wa[1];
1629     wa2t1 = VU[gOwnnr].DatenVU->Sp[2].k.Wa[1];
1630    }
1631    else /* Periode > 1: Verhaltensregel anwenden                    */
1632    {
1633     if ( sa1t0 != 0 ) /* Division durch Null vermeiden              */
1634       { s1 = (sh1t0 / ((float) sa1t0)); }
1635      else
1636       { s1 = 0.0; }
1637     if ( sa2t0 != 0 ) /* Division durch Null vermeiden              */
1638      { s2 = (sh2t0 / ((float) sa2t0));  }
1639      else
1640       { s2 = 0.0; }
1641     if ( pr1t0 <= s1 )         { pr1t1 = a0 * pr1t0; }
1642       else                     { pr1t1 = b0 * pr1t0; }
1643     if ( pr2t0 <= s2 )         { pr2t1 = c0 * pr2t0; }
1644       else                     { pr2t1 = d0 * pr2t0; }
1645     if ( pr1t0 <= s1 )         { wa1t1 = e0 * wa1t0; }
1646       else                     { wa1t1 = f0 * wa1t0; }
1647     if ( pr2t0 <= s2 )         { wa2t1 = g0 * wa2t0; }
1648       else                     { wa2t1 = h0 * wa2t0; }
1649    }
1650   } /* END !(BAV[1].Daten.As[ ... */
1651  else /* Aenderungsschock */
1652   {
1653    if ( gperiod == 1) /* Erste Periode: Startwerte benutzen */
1654    {
1655     pr1t1 = VU[gOwnnr].DatenVU->Sp[1].l.Pr[1];
1656     pr2t1 = VU[gOwnnr].DatenVU->Sp[2].k.Pr[1];
1657     wa1t1 = VU[gOwnnr].DatenVU->Sp[1].l.Wa[1];
1658     wa2t1 = VU[gOwnnr].DatenVU->Sp[2].k.Wa[1];
1659    }
1660    else /* Periode > 1: Verhaltensregel anwenden */
1661    {
1662     if ( sa1t0 != 0 ) /* Division durch Null vermeiden              */
1663       { s1 = (sh1t0 / ((float) sa1t0)); }
1664      else
```

```
1665       { s1 = 0.0; }
1666     if ( sa2t0 != 0 ) /* Division durch Null vermeiden              */
1667       { s2 = (sh2t0 / ((float) sa2t0));  }
1668      else
1669       { s2 = 0.0; }
1670     if ( pr1t0 <= s1 )         { pr1t1 = a1 * pr1t0; }
1671       else                     { pr1t1 = b1 * pr1t0; }
1672     if ( pr2t0 <= s2 )         { pr2t1 = c1 * pr2t0; }
1673       else                     { pr2t1 = d1 * pr2t0; }
1674     if ( pr1t0 <= s1 )         { wa1t1 = e1 * wa1t0; }
1675       else                     { wa1t1 = f1 * wa1t0; }
1676     if ( pr2t0 <= s2 )         { wa2t1 = g1 * wa2t0; }
1677       else                     { wa2t1 = h1 * wa2t0; }
1678    }
1679   } /* END: else Aenderungsschock */
1680  } /* END: IF (gperiod>=... */
1681  /* KURZE VR-Variable wieder in LANGE IMS-Variable speichern        */
1682  VU[gOwnnr].DatenVU->Sp[1].l.Pr[gperiod] = pr1t1; /* Zielgroessen in */
1683  VU[gOwnnr].DatenVU->Sp[1].l.Wa[gperiod] = wa1t1; /* die Vektoren    */
1684  VU[gOwnnr].DatenVU->Sp[2].k.Pr[gperiod] = pr2t1; /* zurueckspeichern*/
1685  VU[gOwnnr].DatenVU->Sp[2].k.Wa[gperiod] = wa2t1;
1686  VU[gOwnnr].DatenVU->Sp[1].l.Rs[gperiod] = rs1t1; /* Ergebnisgroessen*/
1687  VU[gOwnnr].DatenVU->Sp[2].k.Rs[gperiod] = rs2t1; /* zurueckspeichern*/
1688 } /* END: Vrvu06 */
1689
1690 /* ************************************************************** */
1691 /* act Vrvu07: Versicherung:    Verhaltensregel:  DUMPING          */
1692 /*                                                                 */
1693 /*                           vgl. Kapitel 3.3.1.3.                 */
1694 /* ************************************************************** */
1695 act Vrvu07 {             /* Vereinbarung der Hilfsvariablen        */
1696 float a0,a1,b0,b1,c0,c1,d0,d1,e0,e1,f0,f1,g0,g1,h0,h1,pr1t0,pr1t1,
1697       wa1t0,wa1t1,rs1t0,rs1t1,sh1t0,pr2t0,pr2t1,wa2t0,wa2t1,rs2t0,
1698       rs2t1,sh2t0,z,pr1fi,wa1fi,pr2fi,wa2fi;
1699 int   vn1t0,sa1t0,vn2t0,sa2t0;
1700 /* LANGE IMS-Variable in KURZE Variable fuer VR uebersetzen        */
1701 if (gperiod > 1)
1702 {                          /* Der Parametervektor der VR           */
1703  a0 = VU[gOwnnr].DatenVU->Pv[0 ]; a1 = VU[gOwnnr].DatenVU->Pv[1 ];
1704  b0 = VU[gOwnnr].DatenVU->Pv[2 ]; b1 = VU[gOwnnr].DatenVU->Pv[3 ];
1705  c0 = VU[gOwnnr].DatenVU->Pv[4 ]; c1 = VU[gOwnnr].DatenVU->Pv[5 ];
1706  d0 = VU[gOwnnr].DatenVU->Pv[6 ]; d1 = VU[gOwnnr].DatenVU->Pv[7 ];
1707  e0 = VU[gOwnnr].DatenVU->Pv[8 ]; e1 = VU[gOwnnr].DatenVU->Pv[9 ];
1708  f0 = VU[gOwnnr].DatenVU->Pv[10]; f1 = VU[gOwnnr].DatenVU->Pv[11];
1709  g0 = VU[gOwnnr].DatenVU->Pv[12]; g1 = VU[gOwnnr].DatenVU->Pv[13];
```

```
1710  h0 = VU[gOwnnr].DatenVU->Pv[14]; h1 = VU[gOwnnr].DatenVU->Pv[15];
1711  pr1t0 =  VU[gOwnnr].DatenVU->Sp[1].l.Pr[gperiod-1]; /* Sparte 1     */
1712  wa1t0 =  VU[gOwnnr].DatenVU->Sp[1].l.Wa[gperiod-1];
1713  rs1t0 =  VU[gOwnnr].DatenVU->Sp[1].l.Rs[gperiod-1];
1714  vn1t0 =  VU[gOwnnr].DatenVU->Sp[1].l.Vn[gperiod-1];
1715  sa1t0 =  VU[gOwnnr].DatenVU->Sp[1].l.Sa[gperiod-1];
1716  sh1t0 =  VU[gOwnnr].DatenVU->Sp[1].l.Sh[gperiod-1];
1717  pr2t0 =  VU[gOwnnr].DatenVU->Sp[2].k.Pr[gperiod-1]; /* Sparte 2     */
1718  wa2t0 =  VU[gOwnnr].DatenVU->Sp[2].k.Wa[gperiod-1];
1719  rs2t0 =  VU[gOwnnr].DatenVU->Sp[2].k.Rs[gperiod-1];
1720  vn2t0 =  VU[gOwnnr].DatenVU->Sp[2].k.Vn[gperiod-1];
1721  sa2t0 =  VU[gOwnnr].DatenVU->Sp[2].k.Sa[gperiod-1];
1722  sh2t0 =  VU[gOwnnr].DatenVU->Sp[2].k.Sh[gperiod-1]; /* Fremdinform. */
1723  pr1fi =  BAV[1].DatenBAV.Pm[0]; /* Kleinste Praemie in t-1, Sparte 1*/
1724  wa1fi =  BAV[1].DatenBAV.Wm[0]; /* Groesste Werbung in t-1, "       */
1725  pr2fi =  BAV[1].DatenBAV.Pm[1]; /* Kleinste Praemie in t-1, Sparte 2*/
1726  wa2fi =  BAV[1].DatenBAV.Wm[1]; /* Groesste Werbung in t-1, "       */
1727 }
1728 z = BAV[1].DatenBAV.Zs[gperiod];
1729 if(gperiod>=((VU[gOwnnr].DatenVU->Ap) * (VU[gOwnnr].DatenVU->Al[rl])))
1730  {
1731   /* ***************** Verwaltungsroutinen ************************* */
1732   /* Aktivitaetsmeldung an die Aggregattabellen der class BAV: ...   */
1733   BAV[1].DatenBAV.Vuag1[ gOwnnr                 ]++;
1734   BAV[1].DatenBAV.Vuag2[ VU[gOwnnr].DatenVU->Vr ]++;
1735   BAV[1].DatenBAV.Vuag3[ VU[gOwnnr].DatenVU->Vk ]++;
1736   /* Reserven in beiden Sparten fuer eine Periode verzinsen: ....    */
1737   rs1t1 = ( 1 + z ) * rs1t0;
1738   rs2t1 = ( 1 + z ) * rs2t0;
1739  if ( !( BAV[1].DatenBAV.As[gperiod] * BAV[1].DatenBAV.Ar[rl]))
1740   {/* KEIN Aenderungsschock */
1741    if ( gperiod == 1) /* Erste Periode: Startwerte benutzen */
1742    {
1743     pr1t1 = VU[gOwnnr].DatenVU->Sp[1].l.Pr[1];
1744     pr2t1 = VU[gOwnnr].DatenVU->Sp[2].k.Pr[1];
1745     wa1t1 = VU[gOwnnr].DatenVU->Sp[1].l.Wa[1];
1746     wa2t1 = VU[gOwnnr].DatenVU->Sp[2].k.Wa[1];
1747    }
1748    else /* Periode > 1: Verhaltensregel anwenden */
1749    {
1750     pr1t1 = a0 + b0 * pr1fi;
1751     pr2t1 = c0 + d0 * pr2fi;
1752     wa1t1 = e0 + f0 * wa1fi;
1753     wa2t1 = g0 + h0 * wa2fi;
1754    }
```

```
1755   } /* END !(BAV[1].Daten.As[ ... */
1756  else /* Aenderungsschock */
1757   {
1758    if ( gperiod == 1) /* Erste Periode: Startwerte benutzen */
1759    {
1760     pr1t1 = VU[g0wnnr].DatenVU->Sp[1].l.Pr[1];
1761     pr2t1 = VU[g0wnnr].DatenVU->Sp[2].k.Pr[1];
1762     wa1t1 = VU[g0wnnr].DatenVU->Sp[1].l.Wa[1];
1763     wa2t1 = VU[g0wnnr].DatenVU->Sp[2].k.Wa[1];
1764    }
1765    else /* Periode > 1: Verhaltensregel anwenden */
1766    {
1767     pr1t1 = a1 + b1 * pr1fi;
1768     pr2t1 = c1 + d1 * pr2fi;
1769     wa1t1 = e1 + f1 * wa1fi;
1770     wa2t1 = g1 + h1 * wa2fi;
1771    }
1772   } /* END: else Aenderungsschock */
1773  } /* END: IF (gperiod>=... */
1774  /* KURZE VR-Variable wieder in LANGE IMS-Variable speichern         */
1775  VU[g0wnnr].DatenVU->Sp[1].l.Pr[gperiod] = pr1t1; /* Zielgroessen in */
1776  VU[g0wnnr].DatenVU->Sp[1].l.Wa[gperiod] = wa1t1; /* die Vektoren    */
1777  VU[g0wnnr].DatenVU->Sp[2].k.Pr[gperiod] = pr2t1; /* zurueckspeichern*/
1778  VU[g0wnnr].DatenVU->Sp[2].k.Wa[gperiod] = wa2t1;
1779  VU[g0wnnr].DatenVU->Sp[1].l.Rs[gperiod] = rs1t1; /* Ergebnisgroessen*/
1780  VU[g0wnnr].DatenVU->Sp[2].k.Rs[gperiod] = rs2t1; /* zurueckspeichern*/
1781 } /* END: Vrvu07 */
1782
1783 /* ************************************************************** */
1784 /* act Vrvu08: Versicherung:    Verhaltensregel:  DURCHSCHNITT       */
1785 /*                                                                   */
1786 /*                            vgl. Kapitel 3.3.1.3.                  */
1787 /* ************************************************************** */
1788 act Vrvu08 {             /* Vereinbarung der Hilfsvariablen          */
1789 float a0,a1,b0,b1,c0,c1,d0,d1,e0,e1,f0,f1,g0,g1,h0,h1,pr1t0,pr1t1,
1790       wa1t0,wa1t1,rs1t0,rs1t1,sh1t0,pr2t0,pr2t1,wa2t0,wa2t1,rs2t0,
1791       rs2t1,sh2t0,z,pr1fi,wa1fi,pr2fi,wa2fi;
1792 int   vn1t0,sa1t0,vn2t0,sa2t0;
1793 /* LANGE IMS-Variable in KURZE Variable fuer VR uebersetzen          */
1794 if (gperiod > 1)
1795 {                          /* Der Parametervektor der VR             */
1796  a0 = VU[g0wnnr].DatenVU->Pv[0 ]; a1 = VU[g0wnnr].DatenVU->Pv[1 ];
1797  b0 = VU[g0wnnr].DatenVU->Pv[2 ]; b1 = VU[g0wnnr].DatenVU->Pv[3 ];
1798  c0 = VU[g0wnnr].DatenVU->Pv[4 ]; c1 = VU[g0wnnr].DatenVU->Pv[5 ];
1799  d0 = VU[g0wnnr].DatenVU->Pv[6 ]; d1 = VU[g0wnnr].DatenVU->Pv[7 ];
```

```
1800  e0 = VU[gOwnnr].DatenVU->Pv[8 ]; e1 = VU[gOwnnr].DatenVU->Pv[9 ];
1801  f0 = VU[gOwnnr].DatenVU->Pv[10]; f1 = VU[gOwnnr].DatenVU->Pv[11];
1802  g0 = VU[gOwnnr].DatenVU->Pv[12]; g1 = VU[gOwnnr].DatenVU->Pv[13];
1803  h0 = VU[gOwnnr].DatenVU->Pv[14]; h1 = VU[gOwnnr].DatenVU->Pv[15];
1804  pr1t0 =  VU[gOwnnr].DatenVU->Sp[1].l.Pr[gperiod-1]; /* Sparte 1     */
1805  wa1t0 =  VU[gOwnnr].DatenVU->Sp[1].l.Wa[gperiod-1];
1806  rs1t0 =  VU[gOwnnr].DatenVU->Sp[1].l.Rs[gperiod-1];
1807  vn1t0 =  VU[gOwnnr].DatenVU->Sp[1].l.Vn[gperiod-1];
1808  sa1t0 =  VU[gOwnnr].DatenVU->Sp[1].l.Sa[gperiod-1];
1809  sh1t0 =  VU[gOwnnr].DatenVU->Sp[1].l.Sh[gperiod-1];
1810  pr2t0 =  VU[gOwnnr].DatenVU->Sp[2].k.Pr[gperiod-1]; /* Sparte 2     */
1811  wa2t0 =  VU[gOwnnr].DatenVU->Sp[2].k.Wa[gperiod-1];
1812  rs2t0 =  VU[gOwnnr].DatenVU->Sp[2].k.Rs[gperiod-1];
1813  vn2t0 =  VU[gOwnnr].DatenVU->Sp[2].k.Vn[gperiod-1];
1814  sa2t0 =  VU[gOwnnr].DatenVU->Sp[2].k.Sa[gperiod-1];
1815  sh2t0 =  VU[gOwnnr].DatenVU->Sp[2].k.Sh[gperiod-1]; /* Fremdinform. */
1816  pr1fi =  BAV[1].DatenBAV.Dp[0]; /* Durchschnittspraemie in Sparte 1 */
1817  wa1fi =  BAV[1].DatenBAV.Dw[0]; /* Durchschnittswerbung in Sparte 1 */
1818  pr2fi =  BAV[1].DatenBAV.Dp[1]; /* Durchschnittspraemie in Sparte 2 */
1819  wa2fi =  BAV[1].DatenBAV.Dw[1]; /* Durchschnittswerbung in Sparte 2 */
1820 }
1821 z = BAV[1].DatenBAV.Zs[gperiod];
1822 if(gperiod>=((VU[gOwnnr].DatenVU->Ap) * (VU[gOwnnr].DatenVU->Al[rl])))
1823  {
1824   /* ***************** Verwaltungsroutinen ************************* */
1825   /* Aktivitaetsmeldung an die Aggregattabellen der class BAV: ...   */
1826   BAV[1].DatenBAV.Vuag1[ gOwnnr                 ]++;
1827   BAV[1].DatenBAV.Vuag2[ VU[gOwnnr].DatenVU->Vr ]++;
1828   BAV[1].DatenBAV.Vuag3[ VU[gOwnnr].DatenVU->Vk ]++;
1829   /* Reserven in beiden Sparten fuer eine Periode verzinsen: ....    */
1830   rs1t1 = ( 1 + z ) * rs1t0;
1831   rs2t1 = ( 1 + z ) * rs2t0;
1832  if ( !( BAV[1].DatenBAV.As[gperiod] * BAV[1].DatenBAV.Ar[rl]))
1833   {/* KEIN Aenderungsschock */
1834    if ( gperiod == 1) /* Erste Periode: Startwerte benutzen */
1835    {
1836     pr1t1 = VU[gOwnnr].DatenVU->Sp[1].l.Pr[1];
1837     pr2t1 = VU[gOwnnr].DatenVU->Sp[2].k.Pr[1];
1838     wa1t1 = VU[gOwnnr].DatenVU->Sp[1].l.Wa[1];
1839     wa2t1 = VU[gOwnnr].DatenVU->Sp[2].k.Wa[1];
1840    }
1841    else /* Periode > 1: Verhaltensregel anwenden */
1842    {
1843     pr1t1 = a0 + b0 * pr1fi;
1844     pr2t1 = c0 + d0 * pr2fi;
```

```
1845     wa1t1 = e0 + f0 * wa1fi;
1846     wa2t1 = g0 + h0 * wa2fi;
1847    }
1848   } /* END !(BAV[1].Daten.As[ ... */
1849  else /* Aenderungsschock */
1850   {
1851    if ( gperiod == 1) /* Erste Periode: Startwerte benutzen */
1852    {
1853     pr1t1 = VU[g0wnnr].DatenVU->Sp[1].l.Pr[1];
1854     pr2t1 = VU[g0wnnr].DatenVU->Sp[2].k.Pr[1];
1855     wa1t1 = VU[g0wnnr].DatenVU->Sp[1].l.Wa[1];
1856     wa2t1 = VU[g0wnnr].DatenVU->Sp[2].k.Wa[1];
1857    }
1858    else /* Periode > 1: Verhaltensregel anwenden */
1859    {
1860     pr1t1 = a1 + b1 * pr1fi;
1861     pr2t1 = c1 + d1 * pr2fi;
1862     wa1t1 = e1 + f1 * wa1fi;
1863     wa2t1 = g1 + h1 * wa2fi;
1864    }
1865   } /* END: else Aenderungsschock */
1866  } /* END: IF (gperiod>=... */
1867  /* KURZE VR-Variable wieder in LANGE IMS-Variable speichern         */
1868  VU[g0wnnr].DatenVU->Sp[1].l.Pr[gperiod] = pr1t1; /* Zielgroessen in */
1869  VU[g0wnnr].DatenVU->Sp[1].l.Wa[gperiod] = wa1t1; /* die Vektoren    */
1870  VU[g0wnnr].DatenVU->Sp[2].k.Pr[gperiod] = pr2t1; /* zurueckspeichern*/
1871  VU[g0wnnr].DatenVU->Sp[2].k.Wa[gperiod] = wa2t1;
1872  VU[g0wnnr].DatenVU->Sp[1].l.Rs[gperiod] = rs1t1; /* Ergebnisgroessen*/
1873  VU[g0wnnr].DatenVU->Sp[2].k.Rs[gperiod] = rs2t1; /* zurueckspeichern*/
1874 } /* END: Vrvu08 */
1875
1876 /* ************************************************************** */
1877 /* act Vrvu09: Versicherung:    Verhaltensregel:  ANGRIFF          */
1878 /*                                                                 */
1879 /*                            vgl. Kapitel 3.3.1.3.                */
1880 /* ************************************************************** */
1881 act Vrvu09 {              /* Vereinbarung der Hilfsvariablen      */
1882 float a0,a1,b0,b1,c0,c1,d0,d1,e0,e1,f0,f1,g0,g1,h0,h1,pr1t0,pr1t1,
1883       wa1t0,wa1t1,rs1t0,rs1t1,sh1t0,pr2t0,pr2t1,wa2t0,wa2t1,rs2t0,
1884       rs2t1,sh2t0,z,pr1fi,wa1fi,pr2fi,wa2fi;
1885 int   vn1t0,sa1t0,vn2t0,sa2t0;
1886 /* LANGE IMS-Variable in KURZE Variable fuer VR uebersetzen         */
1887 if (gperiod > 1)
1888 {                          /* Der Parametervektor der VR           */
1889  a0 = VU[g0wnnr].DatenVU->Pv[0 ]; a1 = VU[g0wnnr].DatenVU->Pv[1 ];
```

```
1890  b0 = VU[gOwnnr].DatenVU->Pv[2 ]; b1 = VU[gOwnnr].DatenVU->Pv[3 ];
1891  c0 = VU[gOwnnr].DatenVU->Pv[4 ]; c1 = VU[gOwnnr].DatenVU->Pv[5 ];
1892  d0 = VU[gOwnnr].DatenVU->Pv[6 ]; d1 = VU[gOwnnr].DatenVU->Pv[7 ];
1893  e0 = VU[gOwnnr].DatenVU->Pv[8 ]; e1 = VU[gOwnnr].DatenVU->Pv[9 ];
1894  f0 = VU[gOwnnr].DatenVU->Pv[10]; f1 = VU[gOwnnr].DatenVU->Pv[11];
1895  g0 = VU[gOwnnr].DatenVU->Pv[12]; g1 = VU[gOwnnr].DatenVU->Pv[13];
1896  h0 = VU[gOwnnr].DatenVU->Pv[14]; h1 = VU[gOwnnr].DatenVU->Pv[15];
1897  pr1t0 =  VU[gOwnnr].DatenVU->Sp[1].l.Pr[gperiod-1]; /* Sparte 1     */
1898  wa1t0 =  VU[gOwnnr].DatenVU->Sp[1].l.Wa[gperiod-1];
1899  rs1t0 =  VU[gOwnnr].DatenVU->Sp[1].l.Rs[gperiod-1];
1900  vn1t0 =  VU[gOwnnr].DatenVU->Sp[1].l.Vn[gperiod-1];
1901  sa1t0 =  VU[gOwnnr].DatenVU->Sp[1].l.Sa[gperiod-1];
1902  sh1t0 =  VU[gOwnnr].DatenVU->Sp[1].l.Sh[gperiod-1];
1903  pr2t0 =  VU[gOwnnr].DatenVU->Sp[2].k.Pr[gperiod-1]; /* Sparte 2     */
1904  wa2t0 =  VU[gOwnnr].DatenVU->Sp[2].k.Wa[gperiod-1];
1905  rs2t0 =  VU[gOwnnr].DatenVU->Sp[2].k.Rs[gperiod-1];
1906  vn2t0 =  VU[gOwnnr].DatenVU->Sp[2].k.Vn[gperiod-1];
1907  sa2t0 =  VU[gOwnnr].DatenVU->Sp[2].k.Sa[gperiod-1];
1908  sh2t0 =  VU[gOwnnr].DatenVU->Sp[2].k.Sh[gperiod-1]; /* Fremdinform. */
1909  pr1fi =  BAV[1].DatenBAV.Mp[0]; /* Marktfuehrerpraemie in Sparte 1  */
1910  wa1fi =  BAV[1].DatenBAV.Mw[0]; /* Marktfuehrerwerbung in Sparte 1  */
1911  pr2fi =  BAV[1].DatenBAV.Mp[1]; /* Marktfuehrerpraemie in Sparte 2  */
1912  wa2fi =  BAV[1].DatenBAV.Mw[1]; /* Marktfuehrerwerbung in Sparte 2  */
1913 }
1914 z = BAV[1].DatenBAV.Zs[gperiod];
1915 if(gperiod>=((VU[gOwnnr].DatenVU->Ap) * (VU[gOwnnr].DatenVU->Al[rl])))
1916  {
1917   /* ***************** Verwaltungsroutinen ************************* */
1918   /* Aktivitaetsmeldung an die Aggregattabellen der class BAV: ...   */
1919   BAV[1].DatenBAV.Vuag1[ gOwnnr                  ]++;
1920   BAV[1].DatenBAV.Vuag2[ VU[gOwnnr].DatenVU->Vr ]++;
1921   BAV[1].DatenBAV.Vuag3[ VU[gOwnnr].DatenVU->Vk ]++;
1922   /* Reserven in beiden Sparten fuer eine Periode verzinsen: ....    */
1923   rs1t1 = ( 1 + z ) * rs1t0;
1924   rs2t1 = ( 1 + z ) * rs2t0;
1925  if ( !( BAV[1].DatenBAV.As[gperiod] * BAV[1].DatenBAV.Ar[rl]))
1926   {/* KEIN Aenderungsschock */
1927    if ( gperiod == 1) /* Erste Periode: Startwerte benutzen */
1928    {
1929     pr1t1 = VU[gOwnnr].DatenVU->Sp[1].l.Pr[1];
1930     pr2t1 = VU[gOwnnr].DatenVU->Sp[2].k.Pr[1];
1931     wa1t1 = VU[gOwnnr].DatenVU->Sp[1].l.Wa[1];
1932     wa2t1 = VU[gOwnnr].DatenVU->Sp[2].k.Wa[1];
1933    }
1934    else /* Periode > 1: Verhaltensregel anwenden */
```

```
1935    {
1936     pr1t1 = a0 + b0 * pr1fi;
1937     pr2t1 = c0 + d0 * pr2fi;
1938     wa1t1 = e0 + f0 * wa1fi;
1939     wa2t1 = g0 + h0 * wa2fi;
1940    }
1941   } /* END !(BAV[1].Daten.As[ ... */
1942  else /* Aenderungsschock */
1943   {
1944    if ( gperiod == 1) /* Erste Periode: Startwerte benutzen */
1945    {
1946     pr1t1 = VU[gOwnnr].DatenVU->Sp[1].l.Pr[1];
1947     pr2t1 = VU[gOwnnr].DatenVU->Sp[2].k.Pr[1];
1948     wa1t1 = VU[gOwnnr].DatenVU->Sp[1].l.Wa[1];
1949     wa2t1 = VU[gOwnnr].DatenVU->Sp[2].k.Wa[1];
1950    }
1951    else /* Periode > 1: Verhaltensregel anwenden */
1952    {
1953     pr1t1 = a1 + b1 * pr1fi;
1954     pr2t1 = c1 + d1 * pr2fi;
1955     wa1t1 = e1 + f1 * wa1fi;
1956     wa2t1 = g1 + h1 * wa2fi;
1957    }
1958   } /* END: else Aenderungsschock */
1959  } /* END: IF (gperiod>=... */
1960  /* KURZE VR-Variable wieder in LANGE IMS-Variable speichern         */
1961  VU[gOwnnr].DatenVU->Sp[1].l.Pr[gperiod] = pr1t1; /* Zielgroessen in */
1962  VU[gOwnnr].DatenVU->Sp[1].l.Wa[gperiod] = wa1t1; /* die Vektoren    */
1963  VU[gOwnnr].DatenVU->Sp[2].k.Pr[gperiod] = pr2t1; /* zurueckspeichern*/
1964  VU[gOwnnr].DatenVU->Sp[2].k.Wa[gperiod] = wa2t1;
1965  VU[gOwnnr].DatenVU->Sp[1].l.Rs[gperiod] = rs1t1; /* Ergebnisgroessen*/
1966  VU[gOwnnr].DatenVU->Sp[2].k.Rs[gperiod] = rs2t1; /* zurueckspeichern*/
1967 } /* END: Vrvu09 */
1968
1969 /* ************************************************************** */
1970 /* act Vrvu10: Versicherung:    Verhaltensregel:  frei definierbar   */
1971 /*                                                                  */
1972 /*                           vgl. Kapitel 3.3.1.3.                  */
1973 /* ************************************************************** */
1974 act Vrvu10 {              /* Vereinbarung der Hilfsvariablen         */
1975 float a0,a1,b0,b1,c0,c1,d0,d1,e0,e1,f0,f1,g0,g1,h0,h1,pr1t0,pr1t1,
1976       wa1t0,wa1t1,rs1t0,rs1t1,sh1t0,pr2t0,pr2t1,wa2t0,wa2t1,rs2t0,
1977       rs2t1,sh2t0,z;
1978 int   vn1t0,sa1t0,vn2t0,sa2t0;
1979 /* LANGE IMS-Variable in KURZE Variable fuer VR uebersetzen         */
```

```
1980 if (gperiod > 1)
1981 {                          /* Der Parametervektor der VR             */
1982  a0 = VU[gOwnnr].DatenVU->Pv[0 ]; a1 = VU[gOwnnr].DatenVU->Pv[1 ];
1983  b0 = VU[gOwnnr].DatenVU->Pv[2 ]; b1 = VU[gOwnnr].DatenVU->Pv[3 ];
1984  c0 = VU[gOwnnr].DatenVU->Pv[4 ]; c1 = VU[gOwnnr].DatenVU->Pv[5 ];
1985  d0 = VU[gOwnnr].DatenVU->Pv[6 ]; d1 = VU[gOwnnr].DatenVU->Pv[7 ];
1986  e0 = VU[gOwnnr].DatenVU->Pv[8 ]; e1 = VU[gOwnnr].DatenVU->Pv[9 ];
1987  f0 = VU[gOwnnr].DatenVU->Pv[10]; f1 = VU[gOwnnr].DatenVU->Pv[11];
1988  g0 = VU[gOwnnr].DatenVU->Pv[12]; g1 = VU[gOwnnr].DatenVU->Pv[13];
1989  h0 = VU[gOwnnr].DatenVU->Pv[14]; h1 = VU[gOwnnr].DatenVU->Pv[15];
1990  pr1t0 =  VU[gOwnnr].DatenVU->Sp[1].l.Pr[gperiod-1]; /* Sparte 1     */
1991  wa1t0 =  VU[gOwnnr].DatenVU->Sp[1].l.Wa[gperiod-1];
1992  rs1t0 =  VU[gOwnnr].DatenVU->Sp[1].l.Rs[gperiod-1];
1993  vn1t0 =  VU[gOwnnr].DatenVU->Sp[1].l.Vn[gperiod-1];
1994  sa1t0 =  VU[gOwnnr].DatenVU->Sp[1].l.Sa[gperiod-1];
1995  sh1t0 =  VU[gOwnnr].DatenVU->Sp[1].l.Sh[gperiod-1];
1996  pr2t0 =  VU[gOwnnr].DatenVU->Sp[2].k.Pr[gperiod-1]; /* Sparte 2     */
1997  wa2t0 =  VU[gOwnnr].DatenVU->Sp[2].k.Wa[gperiod-1];
1998  rs2t0 =  VU[gOwnnr].DatenVU->Sp[2].k.Rs[gperiod-1];
1999  vn2t0 =  VU[gOwnnr].DatenVU->Sp[2].k.Vn[gperiod-1];
2000  sa2t0 =  VU[gOwnnr].DatenVU->Sp[2].k.Sa[gperiod-1];
2001  sh2t0 =  VU[gOwnnr].DatenVU->Sp[2].k.Sh[gperiod-1];
2002 }
2003 z = BAV[1].DatenBAV.Zs[gperiod];
2004 if(gperiod>=((VU[gOwnnr].DatenVU->Ap) * (VU[gOwnnr].DatenVU->Al[rl])))
2005  {
2006   /* ***************** Verwaltungsroutinen ************************ */
2007   /* Aktivitaetsmeldung an die Aggregattabellen der class BAV: ...   */
2008   BAV[1].DatenBAV.Vuag1[ gOwnnr                 ]++;
2009   BAV[1].DatenBAV.Vuag2[ VU[gOwnnr].DatenVU->Vr ]++;
2010   BAV[1].DatenBAV.Vuag3[ VU[gOwnnr].DatenVU->Vk ]++;
2011   /* Reserven in beiden Sparten fuer eine Periode verzinsen: ....    */
2012   rs1t1 = ( 1 + z ) * rs1t0;
2013   rs2t1 = ( 1 + z ) * rs2t0;
2014  if ( !( BAV[1].DatenBAV.As[gperiod] * BAV[1].DatenBAV.Ar[rl]))
2015   {/* KEIN Aenderungsschock */
2016    if ( gperiod == 1) /* Erste Periode: Startwerte benutzen */
2017    {
2018     pr1t1 = VU[gOwnnr].DatenVU->Sp[1].l.Pr[1];
2019     pr2t1 = VU[gOwnnr].DatenVU->Sp[2].k.Pr[1];
2020     wa1t1 = VU[gOwnnr].DatenVU->Sp[1].l.Wa[1];
2021     wa2t1 = VU[gOwnnr].DatenVU->Sp[2].k.Wa[1];
2022    }
2023    else /* Periode > 1: Verhaltensregel anwenden */
2024    {
```

```
2025     pr1t1 = a0 + b0 * pr1t1; /* HIER KOENNEN EIGENE VERHALTENSREGELN */
2026     pr2t1 = c0 + d0 * pr2t1; /* EINGESETZT WERDEN                    */
2027     wa1t1 = e0 + f0 * wa1t1;
2028     wa2t1 = g0 + h0 * wa2t1;
2029    }
2030   } /* END !(BAV[1].Daten.As[ ... */
2031  else /* Aenderungsschock */
2032   {
2033    if ( gperiod == 1) /* Erste Periode: Startwerte benutzen */
2034    {
2035     pr1t1 = VU[g0wnnr].DatenVU->Sp[1].l.Pr[1];
2036     pr2t1 = VU[g0wnnr].DatenVU->Sp[2].k.Pr[1];
2037     wa1t1 = VU[g0wnnr].DatenVU->Sp[1].l.Wa[1];
2038     wa2t1 = VU[g0wnnr].DatenVU->Sp[2].k.Wa[1];
2039    }
2040    else /* Periode > 1: Verhaltensregel anwenden */
2041    {
2042     pr1t1 = a1 + b1 * pr1t1; /* HIER KOENNEN EIGENE VERHALTENSREGELN */
2043     pr2t1 = c1 + d1 * pr2t1; /* EINGESETZT WERDEN                    */
2044     wa1t1 = e1 + f1 * wa1t1;
2045     wa2t1 = g1 + h1 * wa2t1;
2046    }
2047   } /* END: else Aenderungsschock */
2048  } /* END: IF (gperiod>=... */
2049  /* KURZE VR-Variable wieder in LANGE IMS-Variable speichern         */
2050  VU[g0wnnr].DatenVU->Sp[1].l.Pr[gperiod] = pr1t1; /* Zielgroessen in */
2051  VU[g0wnnr].DatenVU->Sp[1].l.Wa[gperiod] = wa1t1; /* die Vektoren    */
2052  VU[g0wnnr].DatenVU->Sp[2].k.Pr[gperiod] = pr2t1; /* zurueckspeichern*/
2053  VU[g0wnnr].DatenVU->Sp[2].k.Wa[gperiod] = wa2t1;
2054  VU[g0wnnr].DatenVU->Sp[1].l.Rs[gperiod] = rs1t1; /* Ergebnisgroessen*/
2055  VU[g0wnnr].DatenVU->Sp[2].k.Rs[gperiod] = rs2t1; /* zurueckspeichern*/
2056  /* fprintf(runreport,"[%d,%d] %3.1f %3.1f %3.1f %3.1f %4.1f %4.1f\n",
2057          g0wnnr,gperiod,pr1t1,wa1t1,pr2t1,wa2t1,rs1t1,rs2t1); */
2058 } /* END: Vrvu10 */
2059
2060 /* ************************************************************** */
2061 /* class VN: Definition der ESL-class VU und ihrer Verhaltensregeln  */
2062 /*           Die ESL-Definitionen werden durch einen #include-Befehl */
2063 /*           aus der Datei "imsvn.e" geladen                         */
2064 /* ************************************************************** */
2065 class VN {
2066          classVN *DatenVN;
2067         } subj[MAXVN]=
2068 #include "imsvn.e"
2069
```

```
2070 /* ************************************************************** */
2071 /* class VN: Initialisierungaktion der class VN. Hier werden die     */
2072 /*           Subjekte der Klasse Versicherungsnehmer in der ERSTEN   */
2073 /*           Periode des ERSTEN Laufes initialisiert                 */
2074 /* ************************************************************** */
2075 act Myinitvn {
2076  int i;                                  /*         Schleifenzaehler */
2077  gOwnnr   = Ownnr();                     /* Globale ESL-Vars fuer in-*/
2078  gperiod  = period;                      /* direkte act-Aufrufe spei-*/
2079  glogtime = logtime;                     /* chern                    */
2080  if ( ( rl == 1 ) && ( load == 0 ) ) /* NUR im 1. Lauf, wenn KEI-*/
2081   {                                      /* NE Konfig geladen ist... */
2082    for( i = 2; i<= SIMLAENGE; i++ )
2083     {                                    /* Gesamtvermoegensvektor   */
2084      VN[gOwnnr].DatenVN->Vm[i] = 0.0;    /* auf {0} initialisieren   */
2085     }
2086    for ( i = 2; i <= SIMLAENGE; i++ )
2087     {                                    /* Risiko 1:                */
2088      VN[gOwnnr].DatenVN->Rk[1].l.Vu[i]=0;/* Anbieternummer = {0}     */
2089      VN[gOwnnr].DatenVN->Rk[1].l.Vs[i]=0;/* Versicherungsstatus = {0}*/
2090      VN[gOwnnr].DatenVN->Rk[1].l.Vp[i]=0.0;/*Gezahlte Praemie = {0}  */
2091      VN[gOwnnr].DatenVN->Rk[1].l.Ev[i]=0.0;/*Eigenversicherte S. ={0}*/
2092      VN[gOwnnr].DatenVN->Rk[1].l.Sw[i]=myrndf();/*Schadenws.=Zufall()*/
2093      VN[gOwnnr].DatenVN->Rk[1].l.Sh[i]=0.0;/*Schadenhoehe = {0}      */
2094                                          /* Risiko 2:                */
2095      VN[gOwnnr].DatenVN->Rk[2].k.Vu[i]=0;/* Anbieternummer = {0}     */
2096      VN[gOwnnr].DatenVN->Rk[2].k.Vs[i]=0;/* Versicherungsstatus = {0}*/
2097      VN[gOwnnr].DatenVN->Rk[2].k.Vp[i]=0.0;/*Gezahlte Praemie = {0}  */
2098      VN[gOwnnr].DatenVN->Rk[2].k.Ev[i]=0.0;/*Eigenversicherte S. ={0}*/
2099      VN[gOwnnr].DatenVN->Rk[2].k.Sw[i]=myrndf();/*Schadenws.=Zufall()*/

2100      VN[gOwnnr].DatenVN->Rk[2].k.Sh[i]=0.0;/*Schadenhoehe = {0}      */
2101     }
2102  }                   /* END: if ( ( rl == 1 ) && ( load == 0 ) )     */
2103  /* fprintf(runreport,"Myinitvn[%d] im Lauf[%d]\n", gOwnnr,rl);      */
2104 } /* END: act Myinitvn */
2105
2106 /* ************************************************************** */
2107 /* class VN: Re-Initialisierungsfunktion der class VN. Jeder VN wird */
2108 /*           vor allen Lauefen NACH dem ersten Lauf neu initiali-    */
2109 /*           siert.                                                  */
2110 /* ************************************************************** */
2111 act Newinivn {
2112 int i;                                        /* Schleifenvariable */
2113 gOwnnr   = Ownnr();
2114 gperiod  = period;
```

```
2115 glogtime = logtime;
2116 if ( (rl >= 2) && (period == 1) )        /* Nur AB dem 2. Lauf immer */
2117  {                                        /* in der ersten Periode... */
2118   for( i = 2; i<= SIMLAENGE; i++ )
2119    {                                      /* Gesamtvermoegensvektor   */
2120     VN[gOwnnr].DatenVN->Vm[i] = 0.0;      /* auf {0} initialisieren   */
2121    }
2122   for ( i = 2; i <= SIMLAENGE; i++ )/* Re-Initialisierung der Risiken*/
2123    {                                      /* Risiko 1:                */
2124     VN[gOwnnr].DatenVN->Rk[1].l.Vu[i]=0; /* Anbieternummer = {0}     */
2125     VN[gOwnnr].DatenVN->Rk[1].l.Vs[i]=0; /* Versicherungsstatus = {0}*/
2126     VN[gOwnnr].DatenVN->Rk[1].l.Vp[i]=0.0;/* Gezahlte Praemie = {0}  */
2127     VN[gOwnnr].DatenVN->Rk[1].l.Ev[i]=0.0;/* Eigenversicherte S. ={0}*/
2128     VN[gOwnnr].DatenVN->Rk[1].l.Sw[i]=myrndf();/* Schadenws.=Zufall()*/
2129     VN[gOwnnr].DatenVN->Rk[1].l.Sh[i]=0.0;/* Schadenhoehe = {0}      */
2130                                           /* Risiko 2:                */
2131     VN[gOwnnr].DatenVN->Rk[2].k.Vu[i]=0; /* Anbieternummer = {0}     */
2132     VN[gOwnnr].DatenVN->Rk[2].k.Vs[i]=0; /* Versicherungsstatus = {0}*/
2133     VN[gOwnnr].DatenVN->Rk[2].k.Vp[i]=0.0;/* Gezahlte Praemie = {0}  */
2134     VN[gOwnnr].DatenVN->Rk[2].k.Ev[i]=0.0;/* Eigenversicherte S. ={0}*/
2135     VN[gOwnnr].DatenVN->Rk[2].k.Sw[i]=myrndf();/* Schadenws.=Zufall()*/
2136     VN[gOwnnr].DatenVN->Rk[2].k.Sh[i]=0.0;/* Schadenhoehe = {0}      */
2137    }
2138    /* fprintf(runreport,"Newinivn[%d] im Lauf[%d]\n",gOwnnr,rl);     */
2139  }                     /* END: if ( (rl >= 2) && (period == 1) )     */
2140 } /* END: Newinivn */
2141
2142 /* ************************************************************** */
2143 /* class VN: PlanVN   Koordinationsfunktion der Verhaltensregeln.    */
2144 /*           Sie nimmt dem Aufruf der Verhaltensregel vor, die dem   */
2145 /*           VN zugeordnet ist. Kriterien: VR und logischer Zeitpunkt*/
2146 /* ************************************************************** */
2147 act PlanVN  {
2148 int i;                                          /* Schleifenzaehler */
2149 gOwnnr   = Ownnr();             /* Ueber alle logischen Zeitpunkt    */
2150 gperiod  = period;              /* wird geprueft, ob der VN dafuer in*/
2151 glogtime = logtime;             /* seinem Aktionsvektor eine VR vor- */
2152 for( i = 1; i <= MAXNLOG; i++ ) /* gesehen hat; dann wird diese VR   */
2153  {                              /* INDIREKT aufgerufen.              */
2154   if ( ( VN[ gOwnnr ].DatenVN->Ac[i].st != 0        ) &&
2155        ( VN[ gOwnnr ].DatenVN->Ac[i].lt == glogtime )
2156      )
2157    {
2158     (StratdVN[1].S[ VN[ gOwnnr ].DatenVN->Ac[i].st  ] )(); /* AUFRUF!*/
2159    } /* END: if */
```

```
2160  } /* END: for */
2161 } /* END: act PlanVN */
2162
2163 /* ************************************************************** */
2164 /* class StratdVN: ESL-class zur Verwaltung der Verhaltensregeln der */
2165 /*                 class VN. Wird von act <PlanVN> benoetigt, um aus */
2166 /*                 Aktionsvektor die Verhaltensregel auszulesen      */
2167 /* ************************************************************** */
2168 class StratdVN
2169 {
2170  pact S[ MAXNLOG+1 ];
2171 } subj[1] =
2172 {
2173  val: Leer, Vrvn01, Vrvn02, Vrvn03, Vrvn04, Vrvn05, Vrvn06, Vrvn07
2174 };
2175
2176 act Leer { fprintf(runreport,"VNLeer[%d](%2d)\n",gOwnnr, glogtime); }
2177 /* ************************************************************** */
2178 /*            Die Verhaltensregeln der Versicherungsnehmer           */
2179 /* ************************************************************** */
2180 /* ************************************************************** */
2181 /* act: Vrvn01:  Versicherungsnehmer   Verhaltensregel:   Zufall I   */
2182 /*                                                                   */
2183 /*                            vgl. Kapitel 3.3.2.1                   */
2184 /* ************************************************************** */
2185 act Vrvn01 {                        /* Vereinbarung der Hilfsvariable */
2186 float s1,s2,pf1[MAXVU+1],pf2[MAXVU+1],sw1,sw2,a0,a1,b0,b1,c0,c1,
2187       d0,d1,e0,e1,f0,f1,g0,g1,h0,h1;
2188 int vr1,vr2,r1,r2;
2189 if(gperiod>=((VN[gOwnnr].DatenVN->Ap)*(VN[gOwnnr].DatenVN->Al[rl])))
2190  {/* LANGE IMS-Variable in KURZE Variable fuer VR uebersetzen        */
2191   a0 = VN[gOwnnr].DatenVN->Pv[0 ]; a1 = VN[gOwnnr].DatenVN->Pv[1 ];
2192   b0 = VN[gOwnnr].DatenVN->Pv[2 ]; b1 = VN[gOwnnr].DatenVN->Pv[3 ];
2193   c0 = VN[gOwnnr].DatenVN->Pv[4 ]; c1 = VN[gOwnnr].DatenVN->Pv[5 ];
2194   d0 = VN[gOwnnr].DatenVN->Pv[6 ]; d1 = VN[gOwnnr].DatenVN->Pv[7 ];
2195   e0 = VN[gOwnnr].DatenVN->Pv[8 ]; e1 = VN[gOwnnr].DatenVN->Pv[9 ];
2196   f0 = VN[gOwnnr].DatenVN->Pv[10]; f1 = VN[gOwnnr].DatenVN->Pv[11];
2197   g0 = VN[gOwnnr].DatenVN->Pv[12]; g1 = VN[gOwnnr].DatenVN->Pv[13];
2198   h0 = VN[gOwnnr].DatenVN->Pv[14]; h1 = VN[gOwnnr].DatenVN->Pv[15];
2199   sw1 = VN[gOwnnr].DatenVN->Rk[1].l.Sw[gperiod];
2200   sw2 = VN[gOwnnr].DatenVN->Rk[2].k.Sw[gperiod];
2201   vr1 = 0; vr2 = 0; r1 = 0; r2 = 0;
2202   /* Aktivitaetsmeldung an die Aggregattabellen der class BAV: ...   */
2203   BAV[1].DatenBAV.Vnag1[ gOwnnr                 ]++;
2204   BAV[1].DatenBAV.Vnag2[ VN[gOwnnr].DatenVN->Vr ]++;
```

```
2205   BAV[1].DatenBAV.Vnag3[ VN[gOwnnr].DatenVN->Vk ]++;
2206  if ( !( BAV[1].DatenBAV.As[gperiod] * BAV[1].DatenBAV.Ar[rl]))
2207   {                                          /* KEIN Aenderungsschock */
2208    if ( gperiod == 1 )         /* Erste Periode: Startwerte benutzen */
2209    {
2210      s1  = ( sw1 > normal() ) * ( a0 + b0 * normal() ); /* Schaeden  */
2211      s2  = ( sw2 > normal() ) * ( c0 + d0 * normal() ); /* erzeugen  */
2212      vr1 = VN[gOwnnr].DatenVN->Rk[1].l.Vs[1];     /* Status in erster*/
2213      vr2 = VN[gOwnnr].DatenVN->Rk[2].k.Vs[1];     /* Periode: ...    */
2214      r1  = VN[gOwnnr].DatenVN->Rk[1].l.Vu[1];     /* Versicherer in  */
2215      r2  = VN[gOwnnr].DatenVN->Rk[2].k.Vu[1];     /* erster Periode  */
2216    }
2217    else                     /* Periode > 1: Verhaltensregel anwenden */
2218    {
2219      s1  = ( sw1 > normal() ) * ( a0 + b0 * normal() ); /* Schaeden  */
2220      s2  = ( sw2 > normal() ) * ( c0 + d0 * normal() ); /* erzeugen  */
2221      vr1 = 1;                                     /* Pflichtversiche-*/
2222      vr2 = 1;                                     /* rund waehlen    */
2223      do {                          /* Waehle Versicherer fuer Risiko */
2224       do {                         /* 1 aus den in der Periode ak-   */
2225        r1 = ( myrnd() % akvu ) + 1;/* tiven VUs aus. aktiVU[.] wird  */
2226       } while ( ( r1 < 1 ) || ( r1 > MAXVU ) ); /* von BAV jede Per- */
2227      } while ( aktiVU[r1] == 0  ); /* iode neu bestimmt !!           */
2228      do {                          /* Waehle Versicherer fuer Risiko */
2229       do {                         /* 2 aus den in der Periode ak-   */
2230        r2 = ( myrnd() % akvu ) + 1;/* tiven VUs aus. aktiVU[.] wird  */
2231       } while ( ( r2 < 1 ) || ( r2 > MAXVU ) ); /* von BAV jede Per- */
2232      } while ( aktiVU[r2] == 0  ); /* iode neu bestimmt !!           */
2233    }                           /* END: else ( gperiod == 1 )         */
2234   }                       /* END: if ( !( BAV[1].Daten.As[gperiod]...*/
2235   else                         /* Aenderungsschock hat stattgefunden */
2236   {
2237    if ( gperiod == 1 )         /* Erste Periode: Startwerte benutzen */
2238    {
2239      s1  = ( sw1 > normal() ) * ( a1 + b1 * normal() ); /* Schaeden  */
2240      s2  = ( sw2 > normal() ) * ( c1 + d1 * normal() ); /* erzeugen  */
2241      vr1 = VN[gOwnnr].DatenVN->Rk[1].l.Vs[1];     /* Status in erster*/
2242      vr2 = VN[gOwnnr].DatenVN->Rk[2].k.Vs[1];     /* Periode: ...    */
2243      r1  = VN[gOwnnr].DatenVN->Rk[1].l.Vu[1];     /* Versicherer in  */
2244      r2  = VN[gOwnnr].DatenVN->Rk[2].k.Vu[1];     /* erster Periode  */
2245    }                                    /* END:   if ( gperiod == 1 )*/
2246    else                     /* Periode > 1: Verhaltensregel anwenden */
2247    {
2248      s1  = ( sw1 > normal() ) * ( a1 + b1 * normal() ); /* Schaeden  */
2249      s2  = ( sw2 > normal() ) * ( c1 + d1 * normal() ); /* erzeugen  */
```

```
2250      vr1 = 1;                                        /* Pflichtversiche-*/
2251      vr2 = 1;                                        /* rund waehlen    */
2252      do {                           /* Waehle Versicherer fuer Risiko */
2253       do {                          /* 1 aus den in der Periode ak-   */
2254        r1 = ( myrnd() % akvu ) + 1;/* tiven VUs aus. aktiVU[.] wird  */
2255       } while ( ( r1 < 1 ) || ( r1 > MAXVU ) ); /* von BAV jede Per- */
2256      } while ( aktiVU[r1] == 0  ); /* iode neu bestimmt !!           */
2257      do {                           /* Waehle Versicherer fuer Risiko */
2258       do {                          /* 2 aus den in der Periode ak-   */
2259        r2 = ( myrnd() % akvu ) + 1;/* tiven VUs aus. aktiVU[.] wird  */
2260       } while ( ( r2 < 1 ) || ( r2 > MAXVU ) ); /* von BAV jede Per- */
2261      } while ( aktiVU[r2] == 0  ); /* iode neu bestimmt !!           */
2262    }                                   /* END: else if (gperiod == 1)*/
2263   }                                    /* END: else Aenderungsschock */
2264   if ( vr1 == 1 )
2265    {                                          /* ABRECHNUNG: Risiko 1 */
2266     if ( s1 > 0.0   )
2267      {                           /* FALL:  Fremdversicherter Schaden: */
2268                                                /* VU-Vektoren:      */
2269        VU[r1].DatenVU->Sp[1].l.Rs[gperiod] =   /* Reserven um       */
2270        VU[r1].DatenVU->Sp[1].l.Rs[gperiod] +   /* Praemie erhoehen  */
2271        VU[r1].DatenVU->Sp[1].l.Pr[gperiod] -   /* und Schaden       */
2272        s1;                                     /* abziehen...       */
2273        VU[r1].DatenVU->Sp[1].l.Sh[gperiod] =   /* Periodenschadens- */
2274        VU[r1].DatenVU->Sp[1].l.Sh[gperiod] +   /* summe um den Scha-*/
2275        s1;                                     /* den erhoehen      */
2276        VU[r1].DatenVU->Sp[1].l.Sa[gperiod]++;  /* Schadenanzahl um  */
2277                                                /* eins erhoehen     */
2278        VU[r1].DatenVU->Sp[1].l.Vn[gperiod]++;  /* Versichertenanzahl*/
2279                                                /* um eins erhoehen  */
2280                                                /* VN-Vektoren :     */
2281        VN[gOwnnr].DatenVN->Rk[1].l.Vu[gperiod]= /* Gehabten Versiche-*/
2282        r1;                                     /* rer merken        */
2283        VN[gOwnnr].DatenVN->Rk[1].l.Vs[gperiod]= /* Versicherungssta- */
2284        vr1;                                    /* tus merken        */
2285        VN[gOwnnr].DatenVN->Rk[1].l.Vp[gperiod]= /* Gezahlte Praemie  */
2286        VU[r1].DatenVU->Sp[1].l.Pr[gperiod];    /* merken            */
2287        VN[gOwnnr].DatenVN->Rk[1].l.Sh[gperiod]= /* Gehabten Schaden  */
2288        s1;                                     /* merken            */
2289      }  /* END: if ( s1 > 0.0   ) */
2290     else
2291      {                       /* FALL: Fremdversicherter Nicht-Schaden */
2292                                                /* VU-Vektoren:      */
2293        VU[r1].DatenVU->Sp[1].l.Rs[gperiod] =   /* Reserven um       */
2294        VU[r1].DatenVU->Sp[1].l.Rs[gperiod] +   /* Praemie erhoehen  */
```

```
2295        VU[r1].DatenVU->Sp[1].l.Pr[gperiod];
2296        VU[r1].DatenVU->Sp[1].l.Vn[gperiod]++;   /* Versichertenanzahl*/
2297                                                 /* um eins erhoehen  */
2298                                                 /* VN-Vektoren :     */
2299        VN[gOwnnr].DatenVN->Rk[1].l.Vu[gperiod]= /* Gehabten Versiche-*/
2300        r1;                                      /* rer merken        */
2301        VN[gOwnnr].DatenVN->Rk[1].l.Vs[gperiod]= /* Versicherungssta- */
2302        vr1;                                     /* tus merken        */
2303        VN[gOwnnr].DatenVN->Rk[1].l.Vp[gperiod]= /* Gezahlte Praemie  */
2304        VU[r1].DatenVU->Sp[1].l.Pr[gperiod];     /* merken            */
2305        VN[gOwnnr].DatenVN->Rk[1].l.Sh[gperiod]= /* Kein Schaden ge-  */
2306        0.0;                                     /* habt              */
2307      }                                       /* END: else (s1 > 0.0  */
2308    }                                         /* END: if ( vr1==1 )   */
2309    else
2310    {
2311     if ( s1 > 0.0   )
2312      {                            /* FALL: Eigenversicherter Schaden: */
2313        VN[gOwnnr].DatenVN->Rk[1].l.Vu[gperiod]= /* Keinen Versicherer*/
2314        0;                                       /* gehabt            */
2315        VN[gOwnnr].DatenVN->Rk[1].l.Vs[gperiod]= /* Versicherungssta- */
2316        vr1;                                     /* tus merken        */
2317        VN[gOwnnr].DatenVN->Rk[1].l.Ev[gperiod]= /* Eigenversicherten */
2318        s1;                                      /* Schaden merken    */
2319        VN[gOwnnr].DatenVN->Rk[1].l.Sh[gperiod]= /* Gehabten Schaden  */
2320        s1;                                      /* merken            */
2321      }                                     /* END: if ( s1 > 0.0   ) */
2322     else
2323      {                       /* FALL: Eigenversicherter Nicht-Schaden */
2324                                               /* NICHTS abzurechnen*/
2325      }                                       /* END: else (s1 > 0.0  */
2326    }                                         /* END: else ( vr1==1 ) */
2327   if ( vr2 == 1 )
2328    {                                        /* ABRECHNUNG: Risiko 2 */
2329     if ( s2 > 0.0   )
2330      {                            /* FALL: Fremdversicherter Schaden: */
2331                                               /* VU-Vektoren:      */
2332        VU[r2].DatenVU->Sp[2].k.Rs[gperiod] =    /* Reserven um       */
2333        VU[r2].DatenVU->Sp[2].k.Rs[gperiod] +    /* Praemie erhoehen  */
2334        VU[r2].DatenVU->Sp[2].k.Pr[gperiod] -    /* und Schaden       */
2335        s2;                                      /* abziehen...       */
2336        VU[r2].DatenVU->Sp[2].k.Sh[gperiod] =    /* Periodenschadens- */
2337        VU[r2].DatenVU->Sp[2].k.Sh[gperiod] +    /* summe um den Scha-*/
2338        s2;                                      /* den erhoehen      */
2339        VU[r2].DatenVU->Sp[2].k.Sa[gperiod]++;   /* Schadenanzahl um  */
```

```
2340                                                /* eins erhoehen     */
2341        VU[r2].DatenVU->Sp[2].k.Vn[gperiod]++;   /* Versichertenanzahl*/
2342                                                /* um eins erhoehen  */
2343                                                /* VN-Vektoren :     */
2344        VN[gOwnnr].DatenVN->Rk[2].k.Vu[gperiod]= /* Gehabten Versiche-*/
2345        r2;                                     /* rer merken        */
2346        VN[gOwnnr].DatenVN->Rk[2].k.Vs[gperiod]= /* Versicherungssta- */
2347        vr2;                                    /* tus merken        */
2348        VN[gOwnnr].DatenVN->Rk[2].k.Vp[gperiod]= /* Gezahlte Praemie  */
2349        VU[r2].DatenVU->Sp[2].k.Pr[gperiod];     /* merken            */
2350        VN[gOwnnr].DatenVN->Rk[2].k.Sh[gperiod]= /* Gehabten Schaden  */
2351        s2;                                     /* merken            */
2352      }                                     /* END: if ( s2 > 0.0   ) */
2353     else
2354      {                       /* FALL: Fremdversicherter Nicht-Schaden */
2355                                                /* VU-Vektoren:      */
2356        VU[r2].DatenVU->Sp[2].k.Rs[gperiod] =    /* Reserven um       */
2357        VU[r2].DatenVU->Sp[2].k.Rs[gperiod] +    /* Praemie erhoehen  */
2358        VU[r2].DatenVU->Sp[2].k.Pr[gperiod];
2359        VU[r2].DatenVU->Sp[2].k.Vn[gperiod]++;   /* Versichertenanzahl*/
2360                                                /* um eins erhoehen  */
2361                                                /* VN-Vektoren :     */
2362        VN[gOwnnr].DatenVN->Rk[2].k.Vu[gperiod]= /* Gehabten Versiche-*/
2363        r2;                                     /* rer merken        */
2364        VN[gOwnnr].DatenVN->Rk[2].k.Vs[gperiod]= /* Versicherungssta- */
2365        vr2;                                    /* tus merken        */
2366        VN[gOwnnr].DatenVN->Rk[2].k.Vp[gperiod]= /* Gezahlte Praemie  */
2367        VU[r2].DatenVU->Sp[2].k.Pr[gperiod];     /* merken            */
2368        VN[gOwnnr].DatenVN->Rk[2].k.Sh[gperiod]= /* Kein Schaden ge-  */
2369        0.0;                                    /* habt              */
2370      }                                     /* END: else (s2 > 0.0 ) */
2371    }                                       /* END: if ( vr2 == 1 )  */
2372    else
2373    {
2374     if ( s2 > 0.0   )
2375      {                           /* FALL: Eigenversicherter Schaden: */
2376        VN[gOwnnr].DatenVN->Rk[2].k.Vu[gperiod]= /* Keinen Versicherer*/
2377        0;                                      /* gehabt            */
2378        VN[gOwnnr].DatenVN->Rk[2].k.Vs[gperiod]= /* Versicherungssta- */
2379        vr2;                                    /* tus merken        */
2380        VN[gOwnnr].DatenVN->Rk[2].k.Ev[gperiod]= /* Eigenversicherten */
2381        s2;                                     /* Schaden merken    */
2382        VN[gOwnnr].DatenVN->Rk[2].k.Sh[gperiod]= /* Gehabten Schaden  */
2383        s2;                                     /* merken            */
2384      }                                     /* END: if ( s2 > 0.0   ) */
```

```
2385     else
2386      {                      /* FALL: Eigenversicherter Nicht-Schaden */
2387                                            /* NICHTS abzurechnen*/
2388      }                                  /* END: else (s2 > 0.0    */
2389    }                                    /* END: else ( vr2 == 1 ) */
2390   VN[gOwnnr].DatenVN->Vm[gperiod]          =    /* Kummuliertes Ver-*/
2391   VN[gOwnnr].DatenVN->Vm[gperiod-1]        -    /* moegen berechnen */
2392   s1                                       -
2393   s2                                       -
2394   VN[gOwnnr].DatenVN->Rk[1].l.Vp[gperiod]  -    /* Falls EigemVS,ist*/
2395   VN[gOwnnr].DatenVN->Rk[2].k.Vp[gperiod]  ;    /* Vp[] = 0.0 ini-  */
2396                                                 /* tialisiert       */
2397    /* Bei VR: SUCHE hier auch die Informationskosten addieren        */
2398  }                                       /* END: IF (gperiod>=... */
2399 }                                            /* END: Vrvn01      */

2400
2401 /* ************************************************************** */
2402 /* act: Vrvn02:  Versicherungsnehmer   Verhaltensregel: Zufall II    */
2403 /*                                                                   */
2404 /*                            vgl. Kapitel 3.3.2.1                   */
2405 /* ************************************************************** */
2406 act Vrvn02 {                        /* Vereinbarung der Hilfsvariable */
2407 float s1,s2,pf1[MAXVU+1],pf2[MAXVU+1],sw1,sw2,a0,a1,b0,b1,c0,c1,
2408       d0,d1,e0,e1,f0,f1,g0,g1,h0,h1;
2409 int vr1,vr2,r1,r2;
2410 if(gperiod>=((VN[gOwnnr].DatenVN->Ap)*(VN[gOwnnr].DatenVN->Al[rl])))
2411  {/* LANGE IMS-Variable in KURZE Variable fuer VR uebersetzen        */
2412   a0 = VN[gOwnnr].DatenVN->Pv[0 ]; a1 = VN[gOwnnr].DatenVN->Pv[1 ];
2413   b0 = VN[gOwnnr].DatenVN->Pv[2 ]; b1 = VN[gOwnnr].DatenVN->Pv[3 ];
2414   c0 = VN[gOwnnr].DatenVN->Pv[4 ]; c1 = VN[gOwnnr].DatenVN->Pv[5 ];
2415   d0 = VN[gOwnnr].DatenVN->Pv[6 ]; d1 = VN[gOwnnr].DatenVN->Pv[7 ];
2416   e0 = VN[gOwnnr].DatenVN->Pv[8 ]; e1 = VN[gOwnnr].DatenVN->Pv[9 ];
2417   f0 = VN[gOwnnr].DatenVN->Pv[10]; f1 = VN[gOwnnr].DatenVN->Pv[11];
2418   g0 = VN[gOwnnr].DatenVN->Pv[12]; g1 = VN[gOwnnr].DatenVN->Pv[13];
2419   h0 = VN[gOwnnr].DatenVN->Pv[14]; h1 = VN[gOwnnr].DatenVN->Pv[15];
2420   sw1 = VN[gOwnnr].DatenVN->Rk[1].l.Sw[gperiod];
2421   sw2 = VN[gOwnnr].DatenVN->Rk[2].k.Sw[gperiod];
2422   vr1 = 0; vr2 = 0; r1=0; r2=0;
2423   /* Aktivitaetsmeldung an die Aggregattabellen der class BAV: ...   */
2424   BAV[1].DatenBAV.Vnag1[ gOwnnr                 ]++;
2425   BAV[1].DatenBAV.Vnag2[ VN[gOwnnr].DatenVN->Vr ]++;
2426   BAV[1].DatenBAV.Vnag3[ VN[gOwnnr].DatenVN->Vk ]++;
2427  if ( !( BAV[1].DatenBAV.As[gperiod] * BAV[1].DatenBAV.Ar[rl]))
2428   {                                        /* KEIN Aenderungsschock */
2429    if ( gperiod == 1 )         /* Erste Periode: Startwerte benutzen */
```

```
2430    {
2431      s1  = ( sw1 > normal() ) * ( a0 + b0 * normal() ); /* Schaeden  */
2432      s2  = ( sw2 > normal() ) * ( c0 + d0 * normal() ); /* erzeugen  */
2433      vr1 = VN[g0wnnr].DatenVN->Rk[1].l.Vs[1];     /* Status in erster*/
2434      vr2 = VN[g0wnnr].DatenVN->Rk[2].k.Vs[1];     /* Periode: ...    */
2435      r1  = VN[g0wnnr].DatenVN->Rk[1].l.Vu[1];     /* Versicherer in  */
2436      r2  = VN[g0wnnr].DatenVN->Rk[2].k.Vu[1];     /* erster Periode  */
2437    }
2438    else                      /* Periode > 1: Verhaltensregel anwenden */
2439    {
2440      s1  = ( sw1 > normal() ) * ( a0 + b0 * normal() ); /* Schaeden  */
2441      s2  = ( sw2 > normal() ) * ( c0 + d0 * normal() ); /* erzeugen  */
2442      vr1 = ( e0 <= myrndf() );          /* Versicherungsstatus fuer  */
2443      vr2 = ( f0 <= myrndf() );          /* beide Risiken bestimmen   */
2444      do {                          /* Waehle Versicherer fuer Risiko */
2445       do {                         /* 1 aus den in der Periode ak-   */
2446        r1 = ( myrnd() % akvu ) + 1;/* tiven VUs aus. aktiVU[.] wird  */
2447       } while ( ( r1 < 1 ) || ( r1 > MAXVU ) ); /* von BAV jede Per- */
2448      } while ( aktiVU[r1] == 0  ); /* iode neu bestimmt !!           */
2449      do {                          /* Waehle Versicherer fuer Risiko */
2450       do {                         /* 2 aus den in der Periode ak-   */
2451        r2 = ( myrnd() % akvu ) + 1;/* tiven VUs aus. aktiVU[.] wird  */
2452       } while ( ( r2 < 1 ) || ( r2 > MAXVU ) ); /* von BAV jede Per- */
2453      } while ( aktiVU[r2] == 0  ); /* iode neu bestimmt !!           */
2454    }                           /* END: else ( gperiod == 1 )         */
2455   }                       /* END: if ( !( BAV[1].Daten.As[gperiod]...*/
2456   else                         /* Aenderungsschock hat stattgefunden */
2457   {
2458    if ( gperiod == 1 )         /* Erste Periode: Startwerte benutzen */
2459    {
2460      s1  = ( sw1 > normal() ) * ( a1 + b1 * normal() ); /* Schaeden  */
2461      s2  = ( sw2 > normal() ) * ( c1 + d1 * normal() ); /* erzeugen  */
2462      vr1 = VN[g0wnnr].DatenVN->Rk[1].l.Vs[1];     /* Status in erster*/
2463      vr2 = VN[g0wnnr].DatenVN->Rk[2].k.Vs[1];     /* Periode: ...    */
2464      r1  = VN[g0wnnr].DatenVN->Rk[1].l.Vu[1];     /* Versicherer in  */
2465      r2  = VN[g0wnnr].DatenVN->Rk[2].k.Vu[1];     /* erster Periode  */
2466    }                                   /* END:   if ( gperiod == 1 )*/
2467    else                      /* Periode > 1: Verhaltensregel anwenden */
2468    {
2469      s1  = ( sw1 > normal() ) * ( a1 + b1 * normal() ); /* Schaeden  */
2470      s2  = ( sw2 > normal() ) * ( c1 + d1 * normal() ); /* erzeugen  */
2471      vr1 = ( e1 <= myrndf() );          /* Versicherungsstatus fuer  */
2472      vr2 = ( f1 <= myrndf() );          /* beide Risiken bestimmen   */
2473      do {                          /* Waehle Versicherer fuer Risiko */
2474       do {                         /* 1 aus den in der Periode ak-   */
```

```
2475        r1 = ( myrnd() % akvu ) + 1;/* tiven VUs aus. aktiVU[.] wird  */
2476       } while ( ( r1 < 1 ) || ( r1 > MAXVU ) ); /* von BAV jede Per- */
2477      } while ( aktiVU[r1] == 0  ); /* iode neu bestimmt !!           */
2478      do {                          /* Waehle Versicherer fuer Risiko */
2479       do {                         /* 2 aus den in der Periode ak-   */
2480        r2 = ( myrnd() % akvu ) + 1;/* tiven VUs aus. aktiVU[.] wird  */
2481       } while ( ( r2 < 1 ) || ( r2 > MAXVU ) ); /* von BAV jede Per- */
2482      } while ( aktiVU[r2] == 0  ); /* iode neu bestimmt !!           */
2483    }                                   /* END: else if (gperiod == 1)*/
2484   }                                    /* END: else Aenderungsschock */
2485   if ( vr1 == 1 )
2486    {                                        /* ABRECHNUNG: Risiko 1 */
2487     if ( s1 > 0.0   )
2488      {                          /* FALL:  Fremdversicherter Schaden: */
2489                                                /* VU-Vektoren:      */
2490        VU[r1].DatenVU->Sp[1].l.Rs[gperiod] =   /* Reserven um       */
2491        VU[r1].DatenVU->Sp[1].l.Rs[gperiod] +   /* Praemie erhoehen  */
2492        VU[r1].DatenVU->Sp[1].l.Pr[gperiod] -   /* und Schaden       */
2493        s1;                                     /* abziehen...       */
2494        VU[r1].DatenVU->Sp[1].l.Sh[gperiod] =   /* Periodenschadens- */
2495        VU[r1].DatenVU->Sp[1].l.Sh[gperiod] +   /* summe um den Scha-*/
2496        s1;                                     /* den erhoehen      */
2497        VU[r1].DatenVU->Sp[1].l.Sa[gperiod]++;  /* Schadenanzahl um  */
2498                                                /* eins erhoehen     */
2499        VU[r1].DatenVU->Sp[1].l.Vn[gperiod]++;  /* Versichertenanzahl*/
2500                                                /* um eins erhoehen  */
2501                                                /* VN-Vektoren :     */
2502        VN[gOwnnr].DatenVN->Rk[1].l.Vu[gperiod]= /* Gehabten Versiche-*/
2503        r1;                                     /* rer merken        */
2504        VN[gOwnnr].DatenVN->Rk[1].l.Vs[gperiod]= /* Versicherungssta- */
2505        vr1;                                    /* tus merken        */
2506        VN[gOwnnr].DatenVN->Rk[1].l.Vp[gperiod]= /* Gezahlte Praemie  */
2507        VU[r1].DatenVU->Sp[1].l.Pr[gperiod];    /* merken            */
2508        VN[gOwnnr].DatenVN->Rk[1].l.Sh[gperiod]= /* Gehabten Schaden  */
2509        s1;                                     /* merken            */
2510      }  /* END: if ( s1 > 0.0   ) */
2511     else
2512      {                       /* FALL: Fremdversicherter Nicht-Schaden */
2513                                                /* VU-Vektoren:      */
2514        VU[r1].DatenVU->Sp[1].l.Rs[gperiod] =   /* Reserven um       */
2515        VU[r1].DatenVU->Sp[1].l.Rs[gperiod] +   /* Praemie erhoehen  */
2516        VU[r1].DatenVU->Sp[1].l.Pr[gperiod];
2517        VU[r1].DatenVU->Sp[1].l.Vn[gperiod]++;  /* Versichertenanzahl*/
2518                                                /* um eins erhoehen  */
2519                                                /* VN-Vektoren :     */
```

```
2520        VN[gOwnnr].DatenVN->Rk[1].l.Vu[gperiod]= /* Gehabten Versiche-*/
2521        r1;                                      /* rer merken        */
2522        VN[gOwnnr].DatenVN->Rk[1].l.Vs[gperiod]= /* Versicherungssta- */
2523        vr1;                                     /* tus merken        */
2524        VN[gOwnnr].DatenVN->Rk[1].l.Vp[gperiod]= /* Gezahlte Praemie  */
2525        VU[r1].DatenVU->Sp[1].l.Pr[gperiod];     /* merken            */
2526        VN[gOwnnr].DatenVN->Rk[1].l.Sh[gperiod]= /* Kein Schaden ge-  */
2527        0.0;                                     /* habt              */
2528      }                                       /* END: else (s1 > 0.0  */
2529    }                                         /* END: if ( vr1==1 )   */
2530    else
2531    {
2532     if ( s1 > 0.0   )
2533      {                            /* FALL: Eigenversicherter Schaden: */
2534        VN[gOwnnr].DatenVN->Rk[1].l.Vu[gperiod]= /* Keinen Versicherer*/
2535        0;                                       /* gehabt            */
2536        VN[gOwnnr].DatenVN->Rk[1].l.Vs[gperiod]= /* Versicherungssta- */
2537        vr1;                                     /* tus merken        */
2538        VN[gOwnnr].DatenVN->Rk[1].l.Ev[gperiod]= /* Eigenversicherten */
2539        s1;                                      /* Schaden merken    */
2540        VN[gOwnnr].DatenVN->Rk[1].l.Sh[gperiod]= /* Gehabten Schaden  */
2541        s1;                                      /* merken            */
2542      }                                     /* END: if ( s1 > 0.0   ) */
2543     else
2544      {                        /* FALL: Eigenversicherter Nicht-Schaden */
2545                                               /* NICHTS abzurechnen*/
2546      }                                       /* END: else (s1 > 0.0  */
2547    }                                         /* END: else ( vr1==1 ) */
2548   if ( vr2 == 1 )
2549    {                                        /* ABRECHNUNG: Risiko 2 */
2550     if ( s2 > 0.0   )
2551      {                            /* FALL: Fremdversicherter Schaden: */
2552                                               /* VU-Vektoren:      */
2553        VU[r2].DatenVU->Sp[2].k.Rs[gperiod] =    /* Reserven um       */
2554        VU[r2].DatenVU->Sp[2].k.Rs[gperiod] +    /* Praemie erhoehen  */
2555        VU[r2].DatenVU->Sp[2].k.Pr[gperiod] -    /* und Schaden       */
2556        s2;                                      /* abziehen...       */
2557        VU[r2].DatenVU->Sp[2].k.Sh[gperiod] =    /* Periodenschadens- */
2558        VU[r2].DatenVU->Sp[2].k.Sh[gperiod] +    /* summe um den Scha-*/
2559        s2;                                      /* den erhoehen      */
2560        VU[r2].DatenVU->Sp[2].k.Sa[gperiod]++;   /* Schadenanzahl um  */
2561                                                 /* eins erhoehen     */
2562        VU[r2].DatenVU->Sp[2].k.Vn[gperiod]++;   /* Versichertenanzahl*/
2563                                                 /* um eins erhoehen  */
2564                                                 /* VN-Vektoren :     */
```

```
2565        VN[gOwnnr].DatenVN->Rk[2].k.Vu[gperiod]= /* Gehabten Versiche-*/
2566        r2;                                      /* rer merken        */
2567        VN[gOwnnr].DatenVN->Rk[2].k.Vs[gperiod]= /* Versicherungssta- */
2568        vr2;                                     /* tus merken        */
2569        VN[gOwnnr].DatenVN->Rk[2].k.Vp[gperiod]= /* Gezahlte Praemie  */
2570        VU[r2].DatenVU->Sp[2].k.Pr[gperiod];     /* merken            */
2571        VN[gOwnnr].DatenVN->Rk[2].k.Sh[gperiod]= /* Gehabten Schaden  */
2572        s2;                                      /* merken            */
2573      }                                     /* END: if ( s2 > 0.0   ) */
2574     else
2575      {                       /* FALL: Fremdversicherter Nicht-Schaden */
2576                                               /* VU-Vektoren:      */
2577        VU[r2].DatenVU->Sp[2].k.Rs[gperiod] =    /* Reserven um       */
2578        VU[r2].DatenVU->Sp[2].k.Rs[gperiod] +    /* Praemie erhoehen  */
2579        VU[r2].DatenVU->Sp[2].k.Pr[gperiod];
2580        VU[r2].DatenVU->Sp[2].k.Vn[gperiod]++;   /* Versichertenanzahl*/
2581                                               /* um eins erhoehen  */
2582                                               /* VN-Vektoren :     */
2583        VN[gOwnnr].DatenVN->Rk[2].k.Vu[gperiod]= /* Gehabten Versiche-*/
2584        r2;                                      /* rer merken        */
2585        VN[gOwnnr].DatenVN->Rk[2].k.Vs[gperiod]= /* Versicherungssta- */
2586        vr2;                                     /* tus merken        */
2587        VN[gOwnnr].DatenVN->Rk[2].k.Vp[gperiod]= /* Gezahlte Praemie  */
2588        VU[r2].DatenVU->Sp[2].k.Pr[gperiod];     /* merken            */
2589        VN[gOwnnr].DatenVN->Rk[2].k.Sh[gperiod]= /* Kein Schaden ge-  */
2590        0.0;                                     /* habt              */
2591      }                                       /* END: else (s2 > 0.0 ) */
2592    }                                         /* END: if ( vr2 == 1 )  */
2593    else
2594    {
2595     if ( s2 > 0.0   )
2596      {                           /* FALL: Eigenversicherter Schaden: */
2597        VN[gOwnnr].DatenVN->Rk[2].k.Vu[gperiod]= /* Keinen Versicherer*/
2598        0;                                       /* gehabt            */
2599        VN[gOwnnr].DatenVN->Rk[2].k.Vs[gperiod]= /* Versicherungssta- */
2600        vr2;                                     /* tus merken        */
2601        VN[gOwnnr].DatenVN->Rk[2].k.Ev[gperiod]= /* Eigenversicherten */
2602        s2;                                      /* Schaden merken    */
2603        VN[gOwnnr].DatenVN->Rk[2].k.Sh[gperiod]= /* Gehabten Schaden  */
2604        s2;                                      /* merken            */
2605      }                                     /* END: if ( s2 > 0.0   ) */
2606     else
2607      {                       /* FALL: Eigenversicherter Nicht-Schaden */
2608                                               /* NICHTS abzurechnen*/
2609      }                                     /* END: else (s2 > 0.0    */
```

```
2610    }                                        /* END: else ( vr2 == 1 ) */
2611   VN[gOwnnr].DatenVN->Vm[gperiod]           =    /* Kummuliertes Ver-*/
2612   VN[gOwnnr].DatenVN->Vm[gperiod-1]         -    /* moegen berechnen */
2613   s1                                        -
2614   s2                                        -
2615   VN[gOwnnr].DatenVN->Rk[1].l.Vp[gperiod]   -    /* Falls EigemVS,ist*/
2616   VN[gOwnnr].DatenVN->Rk[2].k.Vp[gperiod]   ;    /* Vp[] = 0.0 ini-  */
2617                                                  /* tialisiert       */
2618    /* Bei VR: SUCHE hier auch die Informationskosten addieren        */
2619  }                                         /* END: IF (gperiod>=... */
2620 }                                                /* END: Vrvn02      */
2621
2622 /* ************************************************************** */
2623 /* act: Vrvn03:  Versicherungsnehmer   Verhaltensregel: Praeferenz   */
2624 /*                                                                   */
2625 /*                            vgl. Kapitel 3.3.2.2                   */
2626 /* ************************************************************** */
2627 act Vrvn03 {                        /* Vereinbarung der Hilfsvariable */
2628 float s1,s2,pf1[MAXVU+1],pf2[MAXVU+1],sw1,sw2,a0,a1,b0,b1,c0,c1,
2629       d0,d1,e0,e1,f0,f1,g0,g1,h0,h1,m;
2630 int vr1,vr2,i,r1,r2;
2631 if(gperiod>=((VN[gOwnnr].DatenVN->Ap)*(VN[gOwnnr].DatenVN->Al[rl])))
2632  {/* LANGE IMS-Variable in KURZE Variable fuer VR uebersetzen        */
2633   a0 = VN[gOwnnr].DatenVN->Pv[0 ]; a1 = VN[gOwnnr].DatenVN->Pv[1 ];
2634   b0 = VN[gOwnnr].DatenVN->Pv[2 ]; b1 = VN[gOwnnr].DatenVN->Pv[3 ];
2635   c0 = VN[gOwnnr].DatenVN->Pv[4 ]; c1 = VN[gOwnnr].DatenVN->Pv[5 ];
2636   d0 = VN[gOwnnr].DatenVN->Pv[6 ]; d1 = VN[gOwnnr].DatenVN->Pv[7 ];
2637   e0 = VN[gOwnnr].DatenVN->Pv[8 ]; e1 = VN[gOwnnr].DatenVN->Pv[9 ];
2638   f0 = VN[gOwnnr].DatenVN->Pv[10]; f1 = VN[gOwnnr].DatenVN->Pv[11];
2639   g0 = VN[gOwnnr].DatenVN->Pv[12]; g1 = VN[gOwnnr].DatenVN->Pv[13];
2640   h0 = VN[gOwnnr].DatenVN->Pv[14]; h1 = VN[gOwnnr].DatenVN->Pv[15];
2641   sw1 = VN[gOwnnr].DatenVN->Rk[1].l.Sw[gperiod];
2642   sw2 = VN[gOwnnr].DatenVN->Rk[2].k.Sw[gperiod];
2643   vr1 = 0; vr2 = 0; r1 = 0; r2 = 0;
2644   /* Aktivitaetsmeldung an die Aggregattabellen der class BAV: ...   */
2645   BAV[1].DatenBAV.Vnag1[ gOwnnr                 ]++;
2646   BAV[1].DatenBAV.Vnag2[ VN[gOwnnr].DatenVN->Vr ]++;
2647   BAV[1].DatenBAV.Vnag3[ VN[gOwnnr].DatenVN->Vk ]++;
2648  if ( !( BAV[1].DatenBAV.As[gperiod] * BAV[1].DatenBAV.Ar[rl]))
2649   {                                        /* KEIN Aenderungsschock */
2650    if ( gperiod == 1 )         /* Erste Periode: Startwerte benutzen */
2651    {
2652      s1  = ( sw1 > normal() ) * ( a0 + b0 * normal() ); /* Schaeden  */
2653      s2  = ( sw2 > normal() ) * ( c0 + d0 * normal() ); /* erzeugen  */
2654      vr1 = VN[gOwnnr].DatenVN->Rk[1].l.Vs[1];     /* Status in erster*/
```

```
2655      vr2 = VN[gOwnnr].DatenVN->Rk[2].k.Vs[1];     /* Periode: ...    */
2656      r1  = VN[gOwnnr].DatenVN->Rk[1].l.Vu[1];     /* Versicherer in  */
2657      r2  = VN[gOwnnr].DatenVN->Rk[2].k.Vu[1];     /* erster Periode  */
2658    }
2659    else                     /* Periode > 1: Verhaltensregel anwenden */
2660    {
2661      s1  = ( sw1 > normal() ) * ( a0 + b0 * normal() ); /* Schaeden  */
2662      s2  = ( sw2 > normal() ) * ( c0 + d0 * normal() ); /* erzeugen  */
2663      vr1 = ( sw1 > e0       );    /* Versicherungsstatus fuer beide  */
2664      vr2 = ( sw2 > f0       );    /* Risiken aus subj. Wahrschlktn.  */
2665      r1  = 0; m = 0.0;            /* Praeferenzwahl fuer Risiko 1 :  */
2666      for(i=1;i<=MAXVU;i++)        /* Absolute Werbeaufwendungen der  */
2667       {                           /* in t aktiven VUs in pf1[]       */
2668        if ( aktiVU[i] == 1 )      /* sammeln und aufaddieren ...     */
2669         { pf1[i] = VU[i].DatenVU->Sp[1].l.Wa[gperiod];
2670           m = m + pf1[i];
2671         }
2672        else
2673         { pf1[i] = 0.0;  }
2674       }
2675      for(i=0;i<=MAXVU;i++ )       /* Die Praeferenzen aus relativen  */
2676       {                           /* Werbeaufwand in Sparte 1        */
2677        if (m != 0.0) { pf1[i] = pf1[i] / m ; }  /* berechnen         */
2678       }
2679      m = 0.0;
2680      for(i=0;i<=MAXVU;i++ )       /* Maximalen relativen Werbeaufwand*/
2681       {                           /* finden und den dazugehoerenden  */
2682        if ( pf1[i] > m )          /* Versicherer speichern           */
2683         { m  = pf1[i];
2684           r1 = i;
2685         }
2686       }
2687      if ( r1 == 0 )               /* Wenn kein Versicherer mit       */
2688       {                           /* pf1[] > 0.0 gefunden wurde,weil */
2689        do {                       /* kein VU Werbung betreibt, dann  */
2690         do {                      /* muss ein aktiver VU zufaellig   */
2691             r1 = ( myrnd() % akvu ) + 1; /* gewaehlt werden          */
2692           } while ( ( r1 < 1 ) || ( r1 > MAXVU ) );
2693          } while ( aktiVU[r1] == 0  );
2694       }
2695      r2  = 0; m = 0.0;            /* Praeferenzwahl fuer Risiko 2 :  */
2696      for(i=1;i<=MAXVU;i++)        /* Absolute Werbeaufwendungen der  */
2697       {                           /* in t aktiven VUs in pf2[]       */
2698        if ( aktiVU[i] == 1 )      /* sammeln und aufaddieren ...     */
2699         { pf2[i] = VU[i].DatenVU->Sp[2].k.Wa[gperiod];
```

```
2700          m = m + pf1[i];
2701        }
2702       else
2703        { pf2[i] = 0.0;  }
2704      }
2705     for(i=0;i<=MAXVU;i++ )       /* Die Praeferenzen aus relativen  */
2706      {                           /* Werbeaufwand in Sparte 1        */
2707       if (m != 0.0) { pf2[i] = pf2[i] / m ; }  /* berechnen         */
2708      }
2709     m = 0.0;
2710     for(i=0;i<=MAXVU;i++ )       /* Maximalen relativen Werbeaufwand*/
2711      {                           /* finden und den dazugehoerenden  */
2712       if ( pf2[i] > m )          /* Versicherer speichern           */
2713        { m  = pf2[i];
2714          r2 = i;
2715        }
2716      }
2717     if ( r2 == 0 )               /* Wenn kein Versicherer mit       */
2718      {                           /* pf2[] > 0.0 gefunden wurde,weil */
2719       do {                       /* kein VU Werbung betreibt, dann  */
2720        do {                      /* muss ein aktiver VU zufaellig   */
2721            r2 = ( myrnd() % akvu ) + 1; /* gewaehlt werden          */
2722          } while ( ( r2 < 1 ) || ( r2 > MAXVU ) );
2723         } while ( aktiVU[r2] == 0  );
2724      }
2725   }                           /* END: else ( gperiod == 1 )         */
2726  }                       /* END: if ( !( BAV[1].Daten.As[gperiod]...*/
2727  else                         /* Aenderungsschock hat stattgefunden */
2728  {
2729   if ( gperiod == 1 )         /* Erste Periode: Startwerte benutzen */
2730   {
2731     s1  = ( sw1 > normal() ) * ( a1 + b1 * normal() ); /* Schaeden  */
2732     s2  = ( sw2 > normal() ) * ( c1 + d1 * normal() ); /* erzeugen  */
2733     vr1 = VN[gOwnnr].DatenVN->Rk[1].l.Vs[1];     /* Status in erster*/
2734     vr2 = VN[gOwnnr].DatenVN->Rk[2].k.Vs[1];     /* Periode: ...    */
2735     r1  = VN[gOwnnr].DatenVN->Rk[1].l.Vu[1];     /* Versicherer in  */
2736     r2  = VN[gOwnnr].DatenVN->Rk[2].k.Vu[1];     /* erster Periode  */
2737   }                                   /* END:   if ( gperiod == 1 )*/
2738   else                     /* Periode > 1: Verhaltensregel anwenden */
2739   {
2740     s1  = ( sw1 > normal() ) * ( a1 + b1 * normal() ); /* Schaeden  */
2741     s2  = ( sw2 > normal() ) * ( c1 + d1 * normal() ); /* erzeugen  */
2742     vr1 = ( sw1 > e1       );    /* Versicherungsstatus fuer beide  */
2743     vr2 = ( sw2 > f1       );    /* Risiken aus subj. Wahrschlktn.  */
2744     a1  = 0; m = 0.0;            /* Praeferenzwahl fuer Risiko 1 :  */
```

```
2745      for(i=1;i<=MAXVU;i++)        /* Absolute Werbeaufwendungen der  */
2746       {                           /* in t aktiven VUs in pf1[]       */
2747        if ( aktiVU[i] == 1 )      /* sammeln und aufaddieren ...     */
2748         { pf1[i] = VU[i].DatenVU->Sp[1].l.Wa[gperiod];
2749           m = m + pf1[i];
2750         }
2751        else
2752         { pf1[i] = 0.0;  }
2753       }
2754      for(i=0;i<=MAXVU;i++ )       /* Die Praeferenzen aus relativen  */
2755       {                           /* Werbeaufwand in Sparte 1        */
2756        if (m != 0.0) { pf1[i] = pf1[i] / m ; }  /* berechnen         */
2757       }
2758      m = 0.0;
2759      for(i=0;i<=MAXVU;i++ )       /* Maximalen relativen Werbeaufwand*/
2760       {                           /* finden und den dazugehoerenden  */
2761        if ( pf1[i] > m )          /* Versicherer speichern           */
2762         { m  = pf1[i];
2763           r1 = i;
2764         }
2765       }
2766      if ( r1 == 0 )               /* Wenn kein Versicherer mit       */
2767       {                           /* pf1[] > 0.0 gefunden wurde,weil */
2768        do {                       /* kein VU Werbung betreibt, dann  */
2769         do {                      /* muss ein aktiver VU zufaellig   */
2770             r1 = ( myrnd() % akvu ) + 1; /* gewaehlt werden          */
2771           } while ( ( r1 < 1 ) || ( r1 > MAXVU ) );
2772          } while ( aktiVU[r1] == 0  );
2773       }
2774      r2  = 0; m = 0.0;            /* Praeferenzwahl fuer Risiko 2 :  */
2775      for(i=1;i<=MAXVU;i++)        /* Absolute Werbeaufwendungen der  */
2776       {                           /* in t aktiven VUs in pf2[]       */
2777        if ( aktiVU[i] == 1 )      /* sammeln und aufaddieren ...     */
2778         { pf2[i] = VU[i].DatenVU->Sp[2].k.Wa[gperiod];
2779           m = m + pf1[i];
2780         }
2781        else
2782         { pf2[i] = 0.0;  }
2783       }
2784      for(i=0;i<=MAXVU;i++ )       /* Die Praeferenzen aus relativen  */
2785       {                           /* Werbeaufwand in Sparte 1        */
2786        if (m != 0.0) { pf2[i] = pf2[i] / m ; }  /* berechnen         */
2787       }
2788      m = 0.0;
2789      for(i=0;i<=MAXVU;i++ )       /* Maximalen relativen Werbeaufwand*/
```

```
2790      {                           /* finden und den dazugehoerenden  */
2791       if ( pf2[i] > m )          /* Versicherer speichern           */
2792        { m  = pf2[i];
2793          r2 = i;
2794        }
2795      }
2796     if ( r2 == 0 )               /* Wenn kein Versicherer mit       */
2797      {                           /* pf2[] > 0.0 gefunden wurde,weil */
2798       do {                       /* kein VU Werbung betreibt, dann  */
2799        do {                      /* muss ein aktiver VU zufaellig   */
2800            r2 = ( myrnd() % akvu ) + 1; /* gewaehlt werden          */
2801           } while ( ( r2 < 1 ) || ( r2 > MAXVU ) );
2802          } while ( aktiVU[r2] == 0  );
2803      }
2804   }                                  /* END: else if (gperiod == 1)*/
2805  }                                   /* END: else Aenderungsschock */
2806  if ( vr1 == 1 )
2807   {                                        /* ABRECHNUNG: Risiko 1 */
2808    if ( s1 > 0.0   )
2809     {                          /* FALL:  Fremdversicherter Schaden: */
2810                                                /* VU-Vektoren:      */
2811       VU[r1].DatenVU->Sp[1].l.Rs[gperiod] =    /* Reserven um       */
2812       VU[r1].DatenVU->Sp[1].l.Rs[gperiod] +    /* Praemie erhoehen  */
2813       VU[r1].DatenVU->Sp[1].l.Pr[gperiod] -    /* und Schaden       */
2814       s1;                                      /* abziehen...       */
2815       VU[r1].DatenVU->Sp[1].l.Sh[gperiod] =    /* Periodenschadens- */
2816       VU[r1].DatenVU->Sp[1].l.Sh[gperiod] +    /* summe um den Scha-*/
2817       s1;                                      /* den erhoehen      */
2818       VU[r1].DatenVU->Sp[1].l.Sa[gperiod]++;   /* Schadenanzahl um  */
2819                                                /* eins erhoehen     */
2820       VU[r1].DatenVU->Sp[1].l.Vn[gperiod]++;   /* Versichertenanzahl*/
2821                                                /* um eins erhoehen  */
2822                                                /* VN-Vektoren :     */
2823       VN[gOwnnr].DatenVN->Rk[1].l.Vu[gperiod]= /* Gehabten Versiche-*/
2824       r1;                                      /* rer merken        */
2825       VN[gOwnnr].DatenVN->Rk[1].l.Vs[gperiod]= /* Versicherungssta- */
2826       vr1;                                     /* tus merken        */
2827       VN[gOwnnr].DatenVN->Rk[1].l.Vp[gperiod]= /* Gezahlte Praemie  */
2828       VU[r1].DatenVU->Sp[1].l.Pr[gperiod];     /* merken            */
2829       VN[gOwnnr].DatenVN->Rk[1].l.Sh[gperiod]= /* Gehabten Schaden  */
2830       s1;                                      /* merken            */
2831     }  /* END: if ( s1 > 0.0   ) */
2832    else
2833     {                      /* FALL: Fremdversicherter Nicht-Schaden */
2834                                                /* VU-Vektoren:      */
```

```
2835        VU[r1].DatenVU->Sp[1].l.Rs[gperiod] =    /* Reserven um       */
2836        VU[r1].DatenVU->Sp[1].l.Rs[gperiod] +    /* Praemie erhoehen  */
2837        VU[r1].DatenVU->Sp[1].l.Pr[gperiod];
2838        VU[r1].DatenVU->Sp[1].l.Vn[gperiod]++;   /* Versichertenanzahl*/
2839                                                 /* um eins erhoehen  */
2840                                                 /* VN-Vektoren :     */
2841        VN[gOwnnr].DatenVN->Rk[1].l.Vu[gperiod]= /* Gehabten Versiche-*/
2842        r1;                                      /* rer merken        */
2843        VN[gOwnnr].DatenVN->Rk[1].l.Vs[gperiod]= /* Versicherungssta- */
2844        vr1;                                     /* tus merken        */
2845        VN[gOwnnr].DatenVN->Rk[1].l.Vp[gperiod]= /* Gezahlte Praemie  */
2846        VU[r1].DatenVU->Sp[1].l.Pr[gperiod];     /* merken            */
2847        VN[gOwnnr].DatenVN->Rk[1].l.Sh[gperiod]= /* Kein Schaden ge-  */
2848        0.0;                                     /* habt              */
2849      }                                       /* END: else (s1 > 0.0  */
2850    }                                         /* END: if ( vr1==1 )   */
2851    else
2852    {
2853     if ( s1 > 0.0   )
2854      {                            /* FALL: Eigenversicherter Schaden: */
2855        VN[gOwnnr].DatenVN->Rk[1].l.Vu[gperiod]= /* Keinen Versicherer*/
2856        0;                                       /* gehabt            */
2857        VN[gOwnnr].DatenVN->Rk[1].l.Vs[gperiod]= /* Versicherungssta- */
2858        vr1;                                     /* tus merken        */
2859        VN[gOwnnr].DatenVN->Rk[1].l.Ev[gperiod]= /* Eigenversicherten */
2860        s1;                                      /* Schaden merken    */
2861        VN[gOwnnr].DatenVN->Rk[1].l.Sh[gperiod]= /* Gehabten Schaden  */
2862        s1;                                      /* merken            */
2863      }                                     /* END: if ( s1 > 0.0   ) */
2864     else
2865      {                       /* FALL: Eigenversicherter Nicht-Schaden */
2866                                               /* NICHTS abzurechnen*/
2867      }                                       /* END: else (s1 > 0.0  */
2868    }                                         /* END: else ( vr1==1 ) */
2869   if ( vr2 == 1 )
2870    {                                        /* ABRECHNUNG: Risiko 2 */
2871     if ( s2 > 0.0   )
2872      {                            /* FALL: Fremdversicherter Schaden: */
2873                                               /* VU-Vektoren:      */
2874        VU[r2].DatenVU->Sp[2].k.Rs[gperiod] =    /* Reserven um       */
2875        VU[r2].DatenVU->Sp[2].k.Rs[gperiod] +    /* Praemie erhoehen  */
2876        VU[r2].DatenVU->Sp[2].k.Pr[gperiod] -    /* und Schaden       */
2877        s2;                                      /* abziehen...       */
2878        VU[r2].DatenVU->Sp[2].k.Sh[gperiod] =    /* Periodenschadens- */
2879        VU[r2].DatenVU->Sp[2].k.Sh[gperiod] +    /* summe um den Scha-*/
```

```
2880        s2;                                        /* den erhoehen      */
2881        VU[r2].DatenVU->Sp[2].k.Sa[gperiod]++;     /* Schadenanzahl um  */
2882                                                   /* eins erhoehen     */
2883        VU[r2].DatenVU->Sp[2].k.Vn[gperiod]++;     /* Versichertenanzahl*/
2884                                                   /* um eins erhoehen  */
2885                                                   /* VN-Vektoren :     */
2886        VN[gOwnnr].DatenVN->Rk[2].k.Vu[gperiod]= /* Gehabten Versiche-*/
2887        r2;                                        /* rer merken        */
2888        VN[gOwnnr].DatenVN->Rk[2].k.Vs[gperiod]= /* Versicherungssta- */
2889        vr2;                                       /* tus merken        */
2890        VN[gOwnnr].DatenVN->Rk[2].k.Vp[gperiod]= /* Gezahlte Praemie  */
2891        VU[r2].DatenVU->Sp[2].k.Pr[gperiod];       /* merken            */
2892        VN[gOwnnr].DatenVN->Rk[2].k.Sh[gperiod]= /* Gehabten Schaden  */
2893        s2;                                        /* merken            */
2894      }                                      /* END: if ( s2 > 0.0   ) */
2895     else
2896      {                        /* FALL: Fremdversicherter Nicht-Schaden */
2897                                                   /* VU-Vektoren:      */
2898        VU[r2].DatenVU->Sp[2].k.Rs[gperiod] =      /* Reserven um       */
2899        VU[r2].DatenVU->Sp[2].k.Rs[gperiod] +      /* Praemie erhoehen  */
2900        VU[r2].DatenVU->Sp[2].k.Pr[gperiod];
2901        VU[r2].DatenVU->Sp[2].k.Vn[gperiod]++;     /* Versichertenanzahl*/
2902                                                   /* um eins erhoehen  */
2903                                                   /* VN-Vektoren :     */
2904        VN[gOwnnr].DatenVN->Rk[2].k.Vu[gperiod]= /* Gehabten Versiche-*/
2905        r2;                                        /* rer merken        */
2906        VN[gOwnnr].DatenVN->Rk[2].k.Vs[gperiod]= /* Versicherungssta- */
2907        vr2;                                       /* tus merken        */
2908        VN[gOwnnr].DatenVN->Rk[2].k.Vp[gperiod]= /* Gezahlte Praemie  */
2909        VU[r2].DatenVU->Sp[2].k.Pr[gperiod];       /* merken            */
2910        VN[gOwnnr].DatenVN->Rk[2].k.Sh[gperiod]= /* Kein Schaden ge-  */
2911        0.0;                                       /* habt              */
2912      }                                        /* END: else (s2 > 0.0 ) */
2913    }                                          /* END: if ( vr2 == 1 )  */
2914    else
2915    {
2916     if ( s2 > 0.0   )
2917      {                            /* FALL: Eigenversicherter Schaden: */
2918        VN[gOwnnr].DatenVN->Rk[2].k.Vu[gperiod]= /* Keinen Versicherer*/
2919        0;                                         /* gehabt            */
2920        VN[gOwnnr].DatenVN->Rk[2].k.Vs[gperiod]= /* Versicherungssta- */
2921        vr2;                                       /* tus merken        */
2922        VN[gOwnnr].DatenVN->Rk[2].k.Ev[gperiod]= /* Eigenversicherten */
2923        s2;                                        /* Schaden merken    */
2924        VN[gOwnnr].DatenVN->Rk[2].k.Sh[gperiod]= /* Gehabten Schaden  */
```

```
2925        s2;                                        /* merken            */
2926      }                                       /* END: if ( s2 > 0.0   ) */
2927     else
2928      {                        /* FALL: Eigenversicherter Nicht-Schaden */
2929                                                 /* NICHTS abzurechnen*/
2930      }                                       /* END: else (s2 > 0.0    */
2931    }                                         /* END: else ( vr2 == 1 ) */
2932   VN[gOwnnr].DatenVN->Vm[gperiod]            =    /* Kummuliertes Ver-*/
2933   VN[gOwnnr].DatenVN->Vm[gperiod-1]          -    /* moegen berechnen */
2934   s1                                         -
2935   s2                                         -
2936   VN[gOwnnr].DatenVN->Rk[1].l.Vp[gperiod]    -    /* Falls EigemVS,ist*/
2937   VN[gOwnnr].DatenVN->Rk[2].k.Vp[gperiod]    ;    /* Vp[] = 0.0 ini-  */
2938                                                    /* tialisiert       */
2939    /* Bei VR: SUCHE hier auch die Informationskosten addieren        */
2940  }                                          /* END: IF (gperiod>=... */
2941 }                                                  /* END: Vrvn03      */
2942
2943 /* ************************************************************** */
2944 /* act: Vrvn04:  Versicherungsnehmer   Verhaltensregel: Totale       */
2945 /*                                                      Erinnerung   */
2946 /*                            vgl. Kapitel 3.3.2.2                   */
2947 /* ************************************************************** */
2948 act Vrvn04 {                        /* Vereinbarung der Hilfsvariable */
2949 float s1,s2,sw1,sw2,a0,a1,b0,b1,c0,c1,d0,d1,e0,e1,f0,f1,g0,g1,
2950       h0,h1,m;
2951 int vr1,vr2,r1,r2,i;
2952 if(gperiod>=((VN[gOwnnr].DatenVN->Ap)*(VN[gOwnnr].DatenVN->Al[rl])))
2953  {/* LANGE IMS-Variable in KURZE Variable fuer VR uebersetzen        */
2954   a0 = VN[gOwnnr].DatenVN->Pv[0 ]; a1 = VN[gOwnnr].DatenVN->Pv[1 ];
2955   b0 = VN[gOwnnr].DatenVN->Pv[2 ]; b1 = VN[gOwnnr].DatenVN->Pv[3 ];
2956   c0 = VN[gOwnnr].DatenVN->Pv[4 ]; c1 = VN[gOwnnr].DatenVN->Pv[5 ];
2957   d0 = VN[gOwnnr].DatenVN->Pv[6 ]; d1 = VN[gOwnnr].DatenVN->Pv[7 ];
2958   e0 = VN[gOwnnr].DatenVN->Pv[8 ]; e1 = VN[gOwnnr].DatenVN->Pv[9 ];
2959   f0 = VN[gOwnnr].DatenVN->Pv[10]; f1 = VN[gOwnnr].DatenVN->Pv[11];
2960   g0 = VN[gOwnnr].DatenVN->Pv[12]; g1 = VN[gOwnnr].DatenVN->Pv[13];
2961   h0 = VN[gOwnnr].DatenVN->Pv[14]; h1 = VN[gOwnnr].DatenVN->Pv[15];
2962   sw1 = VN[gOwnnr].DatenVN->Rk[1].l.Sw[gperiod];
2963   sw2 = VN[gOwnnr].DatenVN->Rk[2].k.Sw[gperiod];
2964   vr1 = 0; vr2 = 0; r1 = 0; r2 = 0;
2965   /* Aktivitaetsmeldung an die Aggregattabellen der class BAV: ...   */
2966   BAV[1].DatenBAV.Vnag1[ gOwnnr                 ]++;
2967   BAV[1].DatenBAV.Vnag2[ VN[gOwnnr].DatenVN->Vr ]++;
2968   BAV[1].DatenBAV.Vnag3[ VN[gOwnnr].DatenVN->Vk ]++;
2969  if ( !( BAV[1].DatenBAV.As[gperiod] * BAV[1].DatenBAV.Ar[rl]))
```

```
2970  {                                          /* KEIN Aenderungsschock */
2971   if ( gperiod == 1 )         /* Erste Periode: Startwerte benutzen */
2972   {
2973     s1  = ( sw1 > normal() ) * ( a0 + b0 * normal() ); /* Schaeden  */
2974     s2  = ( sw2 > normal() ) * ( c0 + d0 * normal() ); /* erzeugen  */
2975     vr1 = VN[gOwnnr].DatenVN->Rk[1].l.Vs[1];     /* Status in erster*/
2976     vr2 = VN[gOwnnr].DatenVN->Rk[2].k.Vs[1];     /* Periode: ...    */
2977     r1  = VN[gOwnnr].DatenVN->Rk[1].l.Vu[1];     /* Versicherer in  */
2978     r2  = VN[gOwnnr].DatenVN->Rk[2].k.Vu[1];     /* erster Periode  */
2979   }
2980   else                      /* Periode > 1: Verhaltensregel anwenden */
2981   {
2982     s1  = ( sw1 > normal() ) * ( a0 + b0 * normal() ); /* Schaeden  */
2983     s2  = ( sw2 > normal() ) * ( c0 + d0 * normal() ); /* erzeugen  */
2984     vr1 = ( sw1 > e0       );    /* Versicherungsstatus aus subjek- */
2985     vr2 = ( sw2 > f0       );    /* tiven Wahrscheinlichkeiten....  */
2986     r1 = 0; m = 1000.0;          /* Erinnerungswahl fuer Risiko 1:..*/
2987     for(i=1; i<=(gperiod-1) ;i++ )
2988      {
2989       if (VN[gOwnnr].DatenVN->Rk[1].l.Vs[i] == 1 ) /* Wenn VN in    */
2990        {         /* der Periode i versichert war und die Praemie    */
2991         if ( VN[gOwnnr].DatenVN->Rk[1].l.Vp[i] < m )/* kleiner war..*/
2992          {  /* ...als die nie erreichbare Praemie von 1000.0 ...    */
2993           m  = VN[gOwnnr].DatenVN->Rk[1].l.Vp[i];/* Praemie und ... */
2994           r1 = VN[gOwnnr].DatenVN->Rk[1].l.Vu[i];/* Anbieter merken */
2995          }                                      /* ... und         */
2996        }                                        /* ..weitersuchen. */
2997      }
2998     if ( r1 == 0 )                /* Falls trotzdem keine VU gefun- */
2999      {                            /* den wurde: Zufallswahl !       */

3000       do {
3001        do {
3002            r1 = ( myrnd() % akvu ) + 1;
3003           } while ( ( r1 < 1 ) || ( r1 > MAXVU ) );
3004          } while ( aktiVU[r1] == 0  );
3005      }
3006     r2 = 0; m = 1000.0;          /* Erinnerungswahl fuer Risiko 2:..*/
3007     for(i=1; i<=(gperiod-1) ;i++ )
3008      {
3009       if (VN[gOwnnr].DatenVN->Rk[2].k.Vs[i] == 1 ) /* Wenn VN in    */
3010        {         /* der Periode i versichert war und die Praemie    */
3011         if ( VN[gOwnnr].DatenVN->Rk[2].k.Vp[i] < m )/* kleiner war..*/
3012          {  /* ...als die nie erreichbare Praemie von 1000.0 ...    */
3013           m  = VN[gOwnnr].DatenVN->Rk[2].k.Vp[i];/* Praemie und ... */
3014           r2 = VN[gOwnnr].DatenVN->Rk[2].k.Vu[i];/* Anbieter merken */
```

```
3015          }                                        /* ... und         */
3016        }                                          /* ..weitersuchen. */
3017      }
3018      if ( r2 == 0 )                /* Falls trotzdem keine VU gefun- */
3019      {                             /* den wurde: Zufallswahl !       */
3020       do {
3021        do {
3022            r2 = ( myrnd() % akvu ) + 1;
3023           } while ( ( r2 < 1 ) || ( r2 > MAXVU ) );
3024          } while ( aktiVU[r2] == 0  );
3025      }
3026   }                            /* END: else ( gperiod == 1 )         */
3027  }                        /* END: if ( !( BAV[1].Daten.As[gperiod]...*/
3028  else                          /* Aenderungsschock hat stattgefunden */
3029  {
3030   if ( gperiod == 1 )          /* Erste Periode: Startwerte benutzen */
3031   {
3032     s1  = ( sw1 > normal() ) * ( a1 + b1 * normal() ); /* Schaeden  */
3033     s2  = ( sw2 > normal() ) * ( c1 + d1 * normal() ); /* erzeugen  */
3034     vr1 = VN[gOwnnr].DatenVN->Rk[1].l.Vs[1];     /* Status in erster*/
3035     vr2 = VN[gOwnnr].DatenVN->Rk[2].k.Vs[1];     /* Periode: ...    */
3036     r1  = VN[gOwnnr].DatenVN->Rk[1].l.Vu[1];     /* Versicherer in  */
3037     r2  = VN[gOwnnr].DatenVN->Rk[2].k.Vu[1];     /* erster Periode  */
3038   }                                    /* END:   if ( gperiod == 1 )*/
3039   else                       /* Periode > 1: Verhaltensregel anwenden */
3040   {
3041     s1  = ( sw1 > normal() ) * ( a1 + b1 * normal() ); /* Schaeden  */
3042     s2  = ( sw2 > normal() ) * ( c1 + d1 * normal() ); /* erzeugen  */
3043     vr1 = ( sw1 > e1       );    /* Versicherungsstatus aus subjek- */
3044     vr2 = ( sw2 > f1       );    /* tiven Wahrscheinlichkeiten....  */
3045     r1 = 0; m = 1000.0;          /* Erinnerungswahl fuer Risiko 1:..*/
3046     for(i=1; i<=(gperiod-1) ;i++ )
3047      {
3048       if (VN[gOwnnr].DatenVN->Rk[1].l.Vs[i] == 1 ) /* Wenn VN in    */
3049        {         /* der Periode i versichert war und die Praemie    */
3050         if ( VN[gOwnnr].DatenVN->Rk[1].l.Vp[i] < m )/* kleiner war..*/
3051          {  /* ...als die nie erreichbare Praemie von 1000.0 ...    */
3052           m  = VN[gOwnnr].DatenVN->Rk[1].l.Vp[i];/* Praemie und ... */
3053           r1 = VN[gOwnnr].DatenVN->Rk[1].l.Vu[i];/* Anbieter merken */
3054          }                                      /* ... und         */
3055        }                                        /* ..weitersuchen. */
3056      }
3057      if ( r1 == 0 )                /* Falls trotzdem keine VU gefun- */
3058      {                             /* den wurde: Zufallswahl !       */
3059       do {
```

```
3060        do {
3061            r1 = ( myrnd() % akvu ) + 1;
3062           } while ( ( r1 < 1 ) || ( r1 > MAXVU ) );
3063          } while ( aktiVU[r1] == 0  );
3064      }
3065     r2 = 0; m = 1000.0;          /* Erinnerungswahl fuer Risiko 2:..*/
3066     for(i=1; i<=(gperiod-1) ;i++ )
3067      {
3068       if (VN[gOwnnr].DatenVN->Rk[2].k.Vs[i] == 1 ) /* Wenn VN in    */
3069        {         /* der Periode i versichert war und die Praemie    */
3070         if ( VN[gOwnnr].DatenVN->Rk[2].k.Vp[i] < m )/* kleiner war..*/
3071          {  /* ...als die nie erreichbare Praemie von 1000.0 ...    */
3072           m  = VN[gOwnnr].DatenVN->Rk[2].k.Vp[i];/* Praemie und ... */
3073           r2 = VN[gOwnnr].DatenVN->Rk[2].k.Vu[i];/* Anbieter merken */
3074          }                                      /* ... und         */
3075        }                                        /* ..weitersuchen. */
3076      }
3077     if ( r2 == 0 )                /* Falls trotzdem keine VU gefun- */
3078      {                            /* den wurde: Zufallswahl !       */
3079       do {
3080        do {
3081            r2 = ( myrnd() % akvu ) + 1;
3082           } while ( ( r2 < 1 ) || ( r2 > MAXVU ) );
3083          } while ( aktiVU[r2] == 0  );
3084      }
3085   }                                  /* END: else if (gperiod == 1)*/
3086  }                                   /* END: else Aenderungsschock */
3087  if ( vr1 == 1 )
3088   {                                        /* ABRECHNUNG: Risiko 1 */
3089    if ( s1 > 0.0   )
3090     {                          /* FALL:  Fremdversicherter Schaden: */
3091                                             /* VU-Vektoren:      */
3092       VU[r1].DatenVU->Sp[1].l.Rs[gperiod] =    /* Reserven um       */
3093       VU[r1].DatenVU->Sp[1].l.Rs[gperiod] +    /* Praemie erhoehen  */
3094       VU[r1].DatenVU->Sp[1].l.Pr[gperiod] -    /* und Schaden       */
3095       s1;                                      /* abziehen...       */
3096       VU[r1].DatenVU->Sp[1].l.Sh[gperiod] =    /* Periodenschadens- */
3097       VU[r1].DatenVU->Sp[1].l.Sh[gperiod] +    /* summe um den Scha-*/
3098       s1;                                      /* den erhoehen      */
3099       VU[r1].DatenVU->Sp[1].l.Sa[gperiod]++;   /* Schadenanzahl um  */
3100                                                /* eins erhoehen     */
3101       VU[r1].DatenVU->Sp[1].l.Vn[gperiod]++;   /* Versichertenanzahl*/
3102                                                /* um eins erhoehen  */
3103                                                /* VN-Vektoren :     */
3104       VN[gOwnnr].DatenVN->Rk[1].l.Vu[gperiod]= /* Gehabten Versiche-*/
```

```
3105        r1;                                        /* rer merken        */
3106        VN[gOwnnr].DatenVN->Rk[1].l.Vs[gperiod]= /* Versicherungssta- */
3107        vr1;                                       /* tus merken        */
3108        VN[gOwnnr].DatenVN->Rk[1].l.Vp[gperiod]= /* Gezahlte Praemie  */
3109        VU[r1].DatenVU->Sp[1].l.Pr[gperiod];     /* merken            */
3110        VN[gOwnnr].DatenVN->Rk[1].l.Sh[gperiod]= /* Gehabten Schaden  */
3111        s1;                                        /* merken            */
3112      }  /* END: if ( s1 > 0.0   ) */
3113     else
3114      {                       /* FALL: Fremdversicherter Nicht-Schaden */
3115                                                   /* VU-Vektoren:      */
3116        VU[r1].DatenVU->Sp[1].l.Rs[gperiod] =    /* Reserven um       */
3117        VU[r1].DatenVU->Sp[1].l.Rs[gperiod] +    /* Praemie erhoehen  */
3118        VU[r1].DatenVU->Sp[1].l.Pr[gperiod];
3119        VU[r1].DatenVU->Sp[1].l.Vn[gperiod]++;   /* Versichertenanzahl*/
3120                                                   /* um eins erhoehen  */
3121                                                   /* VN-Vektoren :     */
3122        VN[gOwnnr].DatenVN->Rk[1].l.Vu[gperiod]= /* Gehabten Versiche-*/
3123        r1;                                        /* rer merken        */
3124        VN[gOwnnr].DatenVN->Rk[1].l.Vs[gperiod]= /* Versicherungssta- */
3125        vr1;                                       /* tus merken        */
3126        VN[gOwnnr].DatenVN->Rk[1].l.Vp[gperiod]= /* Gezahlte Praemie  */
3127        VU[r1].DatenVU->Sp[1].l.Pr[gperiod];     /* merken            */
3128        VN[gOwnnr].DatenVN->Rk[1].l.Sh[gperiod]= /* Kein Schaden ge-  */
3129        0.0;                                       /* habt              */
3130      }                                        /* END: else (s1 > 0.0  */
3131    }                                          /* END: if ( vr1==1 )   */
3132    else
3133    {
3134     if ( s1 > 0.0   )
3135      {                           /* FALL: Eigenversicherter Schaden: */
3136        VN[gOwnnr].DatenVN->Rk[1].l.Vu[gperiod]= /* Keinen Versicherer*/
3137        0;                                         /* gehabt            */
3138        VN[gOwnnr].DatenVN->Rk[1].l.Vs[gperiod]= /* Versicherungssta- */
3139        vr1;                                       /* tus merken        */
3140        VN[gOwnnr].DatenVN->Rk[1].l.Ev[gperiod]= /* Eigenversicherten */
3141        s1;                                        /* Schaden merken    */
3142        VN[gOwnnr].DatenVN->Rk[1].l.Sh[gperiod]= /* Gehabten Schaden  */
3143        s1;                                        /* merken            */
3144      }                                    /* END: if ( s1 > 0.0   ) */
3145     else
3146      {                       /* FALL: Eigenversicherter Nicht-Schaden */
3147                                                   /* NICHTS abzurechnen*/
3148      }                                        /* END: else (s1 > 0.0  */
3149    }                                          /* END: else ( vr1==1 ) */
```

```
3150   if ( vr2 == 1 )
3151    {                                          /* ABRECHNUNG: Risiko 2 */
3152     if ( s2 > 0.0   )
3153      {                            /* FALL: Fremdversicherter Schaden: */
3154                                                /* VU-Vektoren:      */
3155        VU[r2].DatenVU->Sp[2].k.Rs[gperiod] =   /* Reserven um       */
3156        VU[r2].DatenVU->Sp[2].k.Rs[gperiod] +   /* Praemie erhoehen  */
3157        VU[r2].DatenVU->Sp[2].k.Pr[gperiod] -   /* und Schaden       */
3158        s2;                                     /* abziehen...       */
3159        VU[r2].DatenVU->Sp[2].k.Sh[gperiod] =   /* Periodenschadens- */
3160        VU[r2].DatenVU->Sp[2].k.Sh[gperiod] +   /* summe um den Scha-*/
3161        s2;                                     /* den erhoehen      */
3162        VU[r2].DatenVU->Sp[2].k.Sa[gperiod]++;  /* Schadenanzahl um  */
3163                                                /* eins erhoehen     */
3164        VU[r2].DatenVU->Sp[2].k.Vn[gperiod]++;  /* Versichertenanzahl*/
3165                                                /* um eins erhoehen  */
3166                                                /* VN-Vektoren :     */
3167        VN[gOwnnr].DatenVN->Rk[2].k.Vu[gperiod]= /* Gehabten Versiche-*/
3168        r2;                                     /* rer merken        */
3169        VN[gOwnnr].DatenVN->Rk[2].k.Vs[gperiod]= /* Versicherungssta- */
3170        vr2;                                    /* tus merken        */
3171        VN[gOwnnr].DatenVN->Rk[2].k.Vp[gperiod]= /* Gezahlte Praemie  */
3172        VU[r2].DatenVU->Sp[2].k.Pr[gperiod];    /* merken            */
3173        VN[gOwnnr].DatenVN->Rk[2].k.Sh[gperiod]= /* Gehabten Schaden  */
3174        s2;                                     /* merken            */
3175      }                                    /* END: if ( s2 > 0.0   ) */
3176     else
3177      {                        /* FALL: Fremdversicherter Nicht-Schaden */
3178                                                /* VU-Vektoren:      */
3179        VU[r2].DatenVU->Sp[2].k.Rs[gperiod] =   /* Reserven um       */
3180        VU[r2].DatenVU->Sp[2].k.Rs[gperiod] +   /* Praemie erhoehen  */
3181        VU[r2].DatenVU->Sp[2].k.Pr[gperiod];
3182        VU[r2].DatenVU->Sp[2].k.Vn[gperiod]++;  /* Versichertenanzahl*/
3183                                                /* um eins erhoehen  */
3184                                                /* VN-Vektoren :     */
3185        VN[gOwnnr].DatenVN->Rk[2].k.Vu[gperiod]= /* Gehabten Versiche-*/
3186        r2;                                     /* rer merken        */
3187        VN[gOwnnr].DatenVN->Rk[2].k.Vs[gperiod]= /* Versicherungssta- */
3188        vr2;                                    /* tus merken        */
3189        VN[gOwnnr].DatenVN->Rk[2].k.Vp[gperiod]= /* Gezahlte Praemie  */
3190        VU[r2].DatenVU->Sp[2].k.Pr[gperiod];    /* merken            */
3191        VN[gOwnnr].DatenVN->Rk[2].k.Sh[gperiod]= /* Kein Schaden ge-  */
3192        0.0;                                    /* habt              */
3193      }                                    /* END: else (s2 > 0.0 ) */
3194    }                                      /* END: if ( vr2 == 1 )  */
```

```
3195    else
3196    {
3197     if ( s2 > 0.0   )
3198      {                           /* FALL: Eigenversicherter Schaden: */
3199        VN[gOwnnr].DatenVN->Rk[2].k.Vu[gperiod]= /* Keinen Versicherer*/
3200        0;                                       /* gehabt            */
3201        VN[gOwnnr].DatenVN->Rk[2].k.Vs[gperiod]= /* Versicherungssta- */
3202        vr2;                                     /* tus merken        */
3203        VN[gOwnnr].DatenVN->Rk[2].k.Ev[gperiod]= /* Eigenversicherten */
3204        s2;                                      /* Schaden merken    */
3205        VN[gOwnnr].DatenVN->Rk[2].k.Sh[gperiod]= /* Gehabten Schaden  */
3206        s2;                                      /* merken            */
3207      }                                    /* END: if ( s2 > 0.0   ) */
3208     else
3209      {                       /* FALL: Eigenversicherter Nicht-Schaden */
3210                                               /* NICHTS abzurechnen*/
3211      }                                    /* END: else (s2 > 0.0    */
3212    }                                      /* END: else ( vr2 == 1 ) */
3213   VN[gOwnnr].DatenVN->Vm[gperiod]           =    /* Kummuliertes Ver-*/
3214   VN[gOwnnr].DatenVN->Vm[gperiod-1]         -    /* moegen berechnen */
3215   s1                                        -
3216   s2                                        -
3217   VN[gOwnnr].DatenVN->Rk[1].l.Vp[gperiod]   -    /* Falls EigemVS,ist*/
3218   VN[gOwnnr].DatenVN->Rk[2].k.Vp[gperiod]   ;    /* Vp[] = 0.0 ini-  */
3219                                                  /* tialisiert       */
3220    /* Bei VR: SUCHE hier auch die Informationskosten addieren        */
3221  }                                        /* END: IF (gperiod>=... */
3222 }                                          /* END: Vrvn04      */
3223
3224 /* ************************************************************** */
3225 /* act: Vrvn05:  Versicherungsnehmer   Verhaltensregel:  Suche       */
3226 /*                                                                   */
3227 /*                           vgl. Kapitel 3.3.2.3                    */
3228 /* ************************************************************** */
3229 act Vrvn05 {                        /* Vereinbarung der Hilfsvariable */
3230 float s1,s2,p1[MAXVU+1],p2[MAXVU+1],sw1,sw2,a0,a1,b0,b1,c0,c1,
3231       d0,d1,e0,e1,f0,f1,g0,g1,h0,h1,ik,m;
3232 int vr1,vr2,r1,r2,i,j,k;
3233 if(gperiod>=((VN[gOwnnr].DatenVN->Ap)*(VN[gOwnnr].DatenVN->Al[rl])))
3234  {/* LANGE IMS-Variable in KURZE Variable fuer VR uebersetzen        */
3235   a0 = VN[gOwnnr].DatenVN->Pv[0 ]; a1 = VN[gOwnnr].DatenVN->Pv[1 ];
3236   b0 = VN[gOwnnr].DatenVN->Pv[2 ]; b1 = VN[gOwnnr].DatenVN->Pv[3 ];
3237   c0 = VN[gOwnnr].DatenVN->Pv[4 ]; c1 = VN[gOwnnr].DatenVN->Pv[5 ];
3238   d0 = VN[gOwnnr].DatenVN->Pv[6 ]; d1 = VN[gOwnnr].DatenVN->Pv[7 ];
3239   e0 = VN[gOwnnr].DatenVN->Pv[8 ]; e1 = VN[gOwnnr].DatenVN->Pv[9 ];
```

```
3240   f0 = VN[gOwnnr].DatenVN->Pv[10]; f1 = VN[gOwnnr].DatenVN->Pv[11];
3241   g0 = VN[gOwnnr].DatenVN->Pv[12]; g1 = VN[gOwnnr].DatenVN->Pv[13];
3242   h0 = VN[gOwnnr].DatenVN->Pv[14]; h1 = VN[gOwnnr].DatenVN->Pv[15];
3243   sw1 = VN[gOwnnr].DatenVN->Rk[1].l.Sw[gperiod];
3244   sw2 = VN[gOwnnr].DatenVN->Rk[2].k.Sw[gperiod];
3245   vr1 = 0; vr2 = 0; r1 = 0; r2 = 0; ik = 0.0;
3246   /* Aktivitaetsmeldung an die Aggregattabellen der class BAV: ...   */
3247   BAV[1].DatenBAV.Vnag1[ gOwnnr                 ]++;
3248   BAV[1].DatenBAV.Vnag2[ VN[gOwnnr].DatenVN->Vr ]++;
3249   BAV[1].DatenBAV.Vnag3[ VN[gOwnnr].DatenVN->Vk ]++;
3250  if ( !( BAV[1].DatenBAV.As[gperiod] * BAV[1].DatenBAV.Ar[rl]))
3251   {                                        /* KEIN Aenderungsschock */
3252    if ( gperiod == 1 )         /* Erste Periode: Startwerte benutzen */
3253    {
3254      s1  = ( sw1 > normal() ) * ( a0 + b0 * normal() ); /* Schaeden  */
3255      s2  = ( sw2 > normal() ) * ( c0 + d0 * normal() ); /* erzeugen  */
3256      vr1 = VN[gOwnnr].DatenVN->Rk[1].l.Vs[1];     /* Status in erster*/
3257      vr2 = VN[gOwnnr].DatenVN->Rk[2].k.Vs[1];     /* Periode: ...    */
3258      r1  = VN[gOwnnr].DatenVN->Rk[1].l.Vu[1];     /* Versicherer in  */
3259      r2  = VN[gOwnnr].DatenVN->Rk[2].k.Vu[1];     /* erster Periode  */
3260    }
3261    else                     /* Periode > 1: Verhaltensregel anwenden */
3262    {
3263      s1  = ( sw1 > normal() ) * ( a0 + b0 * normal() ); /* Schaeden  */
3264      s2  = ( sw2 > normal() ) * ( c0 + d0 * normal() ); /* erzeugen  */
3265      vr1 = ( BAV[1].DatenBAV.Dg[0] <= e0);
3266      vr2 = ( BAV[1].DatenBAV.Dg[0] <= f0);
3267      for(i=1;i<=MAXVU;i++)                     /* Suchwahl: Risiko 1 */
3268       {                       /* Weil ein Minimum gesucht wird: zu-  */
3269        p1[i] = 1000.0;        /* naechst Suchvektor auf 1000.0 init. */
3270       }
3271      for(j=1;j<=g0;j++)       /* Jetzt Stichprobe mit dem Umfang g0  */
3272       {                       /* aus allen aktiven VUs ziehen:       */
3273        do {                   /* Doppelziehungen sind moeglich !     */
3274         do {
3275             r1 = ( myrnd() % akvu ) + 1;
3276            } while ( ( r1 < 1 ) || ( r1 > MAXVU ) );
3277           } while ( aktiVU[r1] == 0  );       /* ... und die Ergeb-  */
3278        p1[r1] = VU[r1].DatenVU->Sp[1].l.Pr[gperiod]; /* nisse merken.*/
3279        ik = ik + BAV[1].DatenBAV.Sk[gperiod];
3280       }
3281      r1 = 0; m = 1000.0;      /* ... aus der gefundenen Stichprobe   */
3282      for(i=1;i<=MAXVU;i++)    /* das Minimum ermitteln und den       */
3283       {                       /* entsprechenden Versicherer aus-     */
3284        if ( p1[i] < m )       /* waehlen.                            */
```

```
3285        {
3286          m  = p1[i];
3287          r1 = i;
3288        }                       /* Mindestens EIN VU muss aktiv sein,  */
3289      }                         /* sonst ist a1 hier immer noch Null ! */
3290     for(i=1;i<=MAXVU;i++)                       /* Suchwahl: Risiko 2 */
3291      {                         /* Weil ein Minimum gesucht wird: zu-  */
3292       p2[i] = 1000.0;          /* naechst Suchvektor auf 1000.0 init. */
3293      }
3294     for(j=1;j<=h0;j++)        /* Jetzt Stichprobe mit dem Umfang h0  */
3295      {                         /* aus allen aktiven VUs ziehen:       */
3296       do {                     /* Doppelziehungen sind moeglich !     */
3297        do {
3298            r2 = ( myrnd() % akvu ) + 1;
3299           } while ( ( r2 < 1 ) || ( r2 > MAXVU ) );

3300          } while ( aktiVU[r2] == 0  );       /* ... und die Ergeb-  */
3301       p2[r2] = VU[r2].DatenVU->Sp[2].k.Pr[gperiod]; /* nisse merken.*/
3302       ik = ik + BAV[1].DatenBAV.Sk[gperiod];
3303      }
3304     r2 = 0; m = 1000.0;       /* ... aus der gefundenen Stichprobe   */
3305     for(i=1;i<=MAXVU;i++)     /* das Minimum ermitteln und den       */
3306      {                         /* entsprechenden Versicherer aus-     */
3307       if ( p2[i] < m )         /* waehlen.                            */
3308        {
3309          m  = p2[i];
3310          r2 = i;
3311        }                       /* Mindestens EIN VU muss aktiv sein,  */
3312      }                         /* sonst ist a1 hier immer noch Null ! */
3313   }                             /* END: else ( gperiod == 1 )          */
3314  }                         /* END: if ( !( BAV[1].Daten.As[gperiod]...*/
3315  else                           /* Aenderungsschock hat stattgefunden */
3316  {
3317   if ( gperiod == 1 )           /* Erste Periode: Startwerte benutzen */
3318   {
3319     s1  = ( sw1 > normal() ) * ( a1 + b1 * normal() ); /* Schaeden  */
3320     s2  = ( sw2 > normal() ) * ( c1 + d1 * normal() ); /* erzeugen  */
3321     vr1 = VN[gOwnnr].DatenVN->Rk[1].l.Vs[1];     /* Status in erster*/
3322     vr2 = VN[gOwnnr].DatenVN->Rk[2].k.Vs[1];     /* Periode: ...    */
3323     r1  = VN[gOwnnr].DatenVN->Rk[1].l.Vu[1];     /* Versicherer in  */
3324     r2  = VN[gOwnnr].DatenVN->Rk[2].k.Vu[1];     /* erster Periode  */
3325   }                                   /* END:   if ( gperiod == 1 )*/
3326   else                      /* Periode > 1: Verhaltensregel anwenden */
3327   {
3328     s1  = ( sw1 > normal() ) * ( a1 + b1 * normal() ); /* Schaeden  */
3329     s2  = ( sw2 > normal() ) * ( c1 + d1 * normal() ); /* erzeugen  */
```

```
3330      vr1 = ( BAV[1].DatenBAV.Dg[0] <= e1);
3331      vr2 = ( BAV[1].DatenBAV.Dg[0] <= f1);
3332      for(i=1;i<=MAXVU;i++)                      /* Suchwahl: Risiko 1 */
3333       {                        /* Weil ein Minimum gesucht wird: zu-  */
3334        p1[i] = 1000.0;         /* naechst Suchvektor auf 1000.0 init. */
3335       }
3336      for(j=1;j<=g1;j++)        /* Jetzt Stichprobe mit dem Umfang g0  */
3337       {                        /* aus allen aktiven VUs ziehen:       */
3338        do {                    /* Doppelziehungen sind moeglich !     */
3339         do {
3340             r1 = ( myrnd() % akvu ) + 1;
3341            } while ( ( r1 < 1 ) || ( r1 > MAXVU ) );
3342           } while ( aktiVU[r1] == 0  );       /* ... und die Ergeb-  */
3343        p1[r1] = VU[r1].DatenVU->Sp[1].l.Pr[gperiod]; /* nisse merken.*/
3344        ik = ik + BAV[1].DatenBAV.Sk[gperiod];
3345       }
3346      r1 = 0; m = 1000.0;       /* ... aus der gefundenen Stichprobe   */
3347      for(i=1;i<=MAXVU;i++)     /* das Minimum ermitteln und den       */
3348       {                        /* entsprechenden Versicherer aus-     */
3349        if ( p1[i] < m )        /* waehlen.                            */
3350         {
3351           m  = p1[i];
3352           r1 = i;
3353         }                      /* Mindestens EIN VU muss aktiv sein,  */
3354       }                        /* sonst ist a1 hier immer noch Null ! */
3355      for(i=1;i<=MAXVU;i++)                      /* Suchwahl: Risiko 2 */
3356       {                        /* Weil ein Minimum gesucht wird: zu-  */
3357        p2[i] = 1000.0;         /* naechst Suchvektor auf 1000.0 init. */
3358       }
3359      for(j=1;j<=h1;j++)        /* Jetzt Stichprobe mit dem Umfang h0  */
3360       {                        /* aus allen aktiven VUs ziehen:       */
3361        do {                    /* Doppelziehungen sind moeglich !     */
3362         do {
3363             r2 = ( myrnd() % akvu ) + 1;
3364            } while ( ( r2 < 1 ) || ( r2 > MAXVU ) );
3365           } while ( aktiVU[r2] == 0  );       /* ... und die Ergeb-  */
3366        p2[r2] = VU[r2].DatenVU->Sp[2].k.Pr[gperiod]; /* nisse merken.*/
3367        ik = ik + BAV[1].DatenBAV.Sk[gperiod];
3368       }
3369      r2 = 0; m = 1000.0;       /* ... aus der gefundenen Stichprobe   */
3370      for(i=1;i<=MAXVU;i++)     /* das Minimum ermitteln und den       */
3371       {                        /* entsprechenden Versicherer aus-     */
3372        if ( p2[i] < m )        /* waehlen.                            */
3373         {
3374           m  = p2[i];
```

```
3375          r2 = i;
3376        }                       /* Mindestens EIN VU muss aktiv sein,  */
3377      }                         /* sonst ist a1 hier immer noch Null ! */
3378   }                                    /* END: else if (gperiod == 1)*/
3379  }                                     /* END: else Aenderungsschock */
3380  if ( vr1 == 1 )
3381   {                                          /* ABRECHNUNG: Risiko 1 */
3382    if ( s1 > 0.0   )
3383     {                          /* FALL:  Fremdversicherter Schaden: */
3384                                                /* VU-Vektoren:      */
3385       VU[r1].DatenVU->Sp[1].l.Rs[gperiod] =    /* Reserven um       */
3386       VU[r1].DatenVU->Sp[1].l.Rs[gperiod] +    /* Praemie erhoehen  */
3387       VU[r1].DatenVU->Sp[1].l.Pr[gperiod] -    /* und Schaden       */
3388       s1;                                      /* abziehen...       */
3389       VU[r1].DatenVU->Sp[1].l.Sh[gperiod] =    /* Periodenschadens- */
3390       VU[r1].DatenVU->Sp[1].l.Sh[gperiod] +    /* summe um den Scha-*/
3391       s1;                                      /* den erhoehen      */
3392       VU[r1].DatenVU->Sp[1].l.Sa[gperiod]++;   /* Schadenanzahl um  */
3393                                                /* eins erhoehen     */
3394       VU[r1].DatenVU->Sp[1].l.Vn[gperiod]++;   /* Versichertenanzahl*/
3395                                                /* um eins erhoehen  */
3396                                                /* VN-Vektoren :     */
3397       VN[gOwnnr].DatenVN->Rk[1].l.Vu[gperiod]= /* Gehabten Versiche-*/
3398       r1;                                      /* rer merken        */
3399       VN[gOwnnr].DatenVN->Rk[1].l.Vs[gperiod]= /* Versicherungssta- */
3400       vr1;                                     /* tus merken        */
3401       VN[gOwnnr].DatenVN->Rk[1].l.Vp[gperiod]= /* Gezahlte Praemie  */
3402       VU[r1].DatenVU->Sp[1].l.Pr[gperiod];     /* merken            */
3403       VN[gOwnnr].DatenVN->Rk[1].l.Sh[gperiod]= /* Gehabten Schaden  */
3404       s1;                                      /* merken            */
3405     }  /* END: if ( s1 > 0.0   ) */
3406    else
3407     {                       /* FALL: Fremdversicherter Nicht-Schaden */
3408                                                /* VU-Vektoren:      */
3409       VU[r1].DatenVU->Sp[1].l.Rs[gperiod] =    /* Reserven um       */
3410       VU[r1].DatenVU->Sp[1].l.Rs[gperiod] +    /* Praemie erhoehen  */
3411       VU[r1].DatenVU->Sp[1].l.Pr[gperiod];
3412       VU[r1].DatenVU->Sp[1].l.Vn[gperiod]++;   /* Versichertenanzahl*/
3413                                                /* um eins erhoehen  */
3414                                                /* VN-Vektoren :     */
3415       VN[gOwnnr].DatenVN->Rk[1].l.Vu[gperiod]= /* Gehabten Versiche-*/
3416       r1;                                      /* rer merken        */
3417       VN[gOwnnr].DatenVN->Rk[1].l.Vs[gperiod]= /* Versicherungssta- */
3418       vr1;                                     /* tus merken        */
3419       VN[gOwnnr].DatenVN->Rk[1].l.Vp[gperiod]= /* Gezahlte Praemie  */
```

```
3420        VU[r1].DatenVU->Sp[1].l.Pr[gperiod];     /* merken            */
3421        VN[gOwnnr].DatenVN->Rk[1].l.Sh[gperiod]= /* Kein Schaden ge-  */
3422        0.0;                                     /* habt              */
3423      }                                       /* END: else (s1 > 0.0  */
3424    }                                         /* END: if ( vr1==1 )   */
3425    else
3426    {
3427     if ( s1 > 0.0   )
3428      {                            /* FALL: Eigenversicherter Schaden: */
3429        VN[gOwnnr].DatenVN->Rk[1].l.Vu[gperiod]= /* Keinen Versicherer*/
3430        0;                                       /* gehabt            */
3431        VN[gOwnnr].DatenVN->Rk[1].l.Vs[gperiod]= /* Versicherungssta- */
3432        vr1;                                     /* tus merken        */
3433        VN[gOwnnr].DatenVN->Rk[1].l.Ev[gperiod]= /* Eigenversicherten */
3434        s1;                                      /* Schaden merken    */
3435        VN[gOwnnr].DatenVN->Rk[1].l.Sh[gperiod]= /* Gehabten Schaden  */
3436        s1;                                      /* merken            */
3437      }                                     /* END: if ( s1 > 0.0   ) */
3438     else
3439      {                       /* FALL: Eigenversicherter Nicht-Schaden */
3440                                               /* NICHTS abzurechnen*/
3441      }                                       /* END: else (s1 > 0.0  */
3442    }                                         /* END: else ( vr1==1 ) */
3443   if ( vr2 == 1 )
3444    {                                        /* ABRECHNUNG: Risiko 2 */
3445     if ( s2 > 0.0   )
3446      {                            /* FALL: Fremdversicherter Schaden: */
3447                                               /* VU-Vektoren:      */
3448        VU[r2].DatenVU->Sp[2].k.Rs[gperiod] =    /* Reserven um       */
3449        VU[r2].DatenVU->Sp[2].k.Rs[gperiod] +    /* Praemie erhoehen  */
3450        VU[r2].DatenVU->Sp[2].k.Pr[gperiod] -    /* und Schaden       */
3451        s2;                                      /* abziehen...       */
3452        VU[r2].DatenVU->Sp[2].k.Sh[gperiod] =    /* Periodenschadens- */
3453        VU[r2].DatenVU->Sp[2].k.Sh[gperiod] +    /* summe um den Scha-*/
3454        s2;                                      /* den erhoehen      */
3455        VU[r2].DatenVU->Sp[2].k.Sa[gperiod]++;   /* Schadenanzahl um  */
3456                                                 /* eins erhoehen     */
3457        VU[r2].DatenVU->Sp[2].k.Vn[gperiod]++;   /* Versichertenanzahl*/
3458                                                 /* um eins erhoehen  */
3459                                                 /* VN-Vektoren :     */
3460        VN[gOwnnr].DatenVN->Rk[2].k.Vu[gperiod]= /* Gehabten Versiche-*/
3461        r2;                                      /* rer merken        */
3462        VN[gOwnnr].DatenVN->Rk[2].k.Vs[gperiod]= /* Versicherungssta- */
3463        vr2;                                     /* tus merken        */
3464        VN[gOwnnr].DatenVN->Rk[2].k.Vp[gperiod]= /* Gezahlte Praemie  */
```

```
3465        VU[r2].DatenVU->Sp[2].k.Pr[gperiod];     /* merken            */
3466        VN[gOwnnr].DatenVN->Rk[2].k.Sh[gperiod]= /* Gehabten Schaden  */
3467        s2;                                      /* merken            */
3468      }                                     /* END: if ( s2 > 0.0   ) */
3469     else
3470      {                      /* FALL: Fremdversicherter Nicht-Schaden */
3471                                               /* VU-Vektoren:      */
3472        VU[r2].DatenVU->Sp[2].k.Rs[gperiod] =    /* Reserven um       */
3473        VU[r2].DatenVU->Sp[2].k.Rs[gperiod] +    /* Praemie erhoehen  */
3474        VU[r2].DatenVU->Sp[2].k.Pr[gperiod];
3475        VU[r2].DatenVU->Sp[2].k.Vn[gperiod]++;   /* Versichertenanzahl*/
3476                                               /* um eins erhoehen  */
3477                                               /* VN-Vektoren :     */
3478        VN[gOwnnr].DatenVN->Rk[2].k.Vu[gperiod]= /* Gehabten Versiche-*/
3479        r2;                                    /* rer merken        */
3480        VN[gOwnnr].DatenVN->Rk[2].k.Vs[gperiod]= /* Versicherungssta- */
3481        vr2;                                   /* tus merken        */
3482        VN[gOwnnr].DatenVN->Rk[2].k.Vp[gperiod]= /* Gezahlte Praemie  */
3483        VU[r2].DatenVU->Sp[2].k.Pr[gperiod];     /* merken            */
3484        VN[gOwnnr].DatenVN->Rk[2].k.Sh[gperiod]= /* Kein Schaden ge-  */
3485        0.0;                                   /* habt              */
3486      }                                     /* END: else (s2 > 0.0 ) */
3487    }                                       /* END: if ( vr2 == 1 )  */
3488    else
3489    {
3490     if ( s2 > 0.0   )
3491      {                          /* FALL: Eigenversicherter Schaden: */
3492        VN[gOwnnr].DatenVN->Rk[2].k.Vu[gperiod]= /* Keinen Versicherer*/
3493        0;                                     /* gehabt            */
3494        VN[gOwnnr].DatenVN->Rk[2].k.Vs[gperiod]= /* Versicherungssta- */
3495        vr2;                                   /* tus merken        */
3496        VN[gOwnnr].DatenVN->Rk[2].k.Ev[gperiod]= /* Eigenversicherten */
3497        s2;                                    /* Schaden merken    */
3498        VN[gOwnnr].DatenVN->Rk[2].k.Sh[gperiod]= /* Gehabten Schaden  */
3499        s2;                                    /* merken            */
3500      }                                     /* END: if ( s2 > 0.0   ) */
3501     else
3502      {                      /* FALL: Eigenversicherter Nicht-Schaden */
3503                                               /* NICHTS abzurechnen*/
3504      }                                     /* END: else (s2 > 0.0    */
3505    }                                       /* END: else ( vr2 == 1 ) */
3506   VN[gOwnnr].DatenVN->Vm[gperiod]           =    /* Kummuliertes Ver-*/
3507   VN[gOwnnr].DatenVN->Vm[gperiod-1]         -    /* moegen berechnen */
3508   s1                                        -
3509   s2                                        -
```

```
3510   VN[gOwnnr].DatenVN->Rk[1].l.Vp[gperiod]   -    /* Praemien in den  */
3511   VN[gOwnnr].DatenVN->Rk[2].k.Vp[gperiod]   -    /* Sparten,         */
3512   ik                                        ;    /* Suchkosten       */
3513  }                                         /* END: IF (gperiod>=... */
3514 }                                               /* END: Vrvn05      */
3515
3516 /* ************************************************************** */
3517 /* act: Vrvn06:  Versicherungsnehmer   Verhaltensregel:  Beste      */
3518 /*                                                      Information */
3519 /*                           vgl. Kapitel 3.3.2.3                   */
3520 /* ************************************************************** */
3521 act Vrvn06 {                        /* Vereinbarung der Hilfsvariable */
3522 float s1,s2,p1[MAXVU+1],p2[MAXVU+1],sw1,sw2,a0,a1,b0,b1,c0,c1,
3523       d0,d1,e0,e1,f0,f1,g0,g1,h0,h1,ik,m;
3524 int vr1,vr2,i,j,k,r1,r2;
3525 if(gperiod>=((VN[gOwnnr].DatenVN->Ap)*(VN[gOwnnr].DatenVN->Al[rl])))
3526  {/* LANGE IMS-Variable in KURZE Variable fuer VR uebersetzen        */
3527   a0 = VN[gOwnnr].DatenVN->Pv[0 ]; a1 = VN[gOwnnr].DatenVN->Pv[1 ];
3528   b0 = VN[gOwnnr].DatenVN->Pv[2 ]; b1 = VN[gOwnnr].DatenVN->Pv[3 ];
3529   c0 = VN[gOwnnr].DatenVN->Pv[4 ]; c1 = VN[gOwnnr].DatenVN->Pv[5 ];
3530   d0 = VN[gOwnnr].DatenVN->Pv[6 ]; d1 = VN[gOwnnr].DatenVN->Pv[7 ];
3531   e0 = VN[gOwnnr].DatenVN->Pv[8 ]; e1 = VN[gOwnnr].DatenVN->Pv[9 ];
3532   f0 = VN[gOwnnr].DatenVN->Pv[10]; f1 = VN[gOwnnr].DatenVN->Pv[11];
3533   g0 = VN[gOwnnr].DatenVN->Pv[12]; g1 = VN[gOwnnr].DatenVN->Pv[13];
3534   h0 = VN[gOwnnr].DatenVN->Pv[14]; h1 = VN[gOwnnr].DatenVN->Pv[15];
3535   sw1 = VN[gOwnnr].DatenVN->Rk[1].l.Sw[gperiod];
3536   sw2 = VN[gOwnnr].DatenVN->Rk[2].k.Sw[gperiod];
3537   vr1 = 0; vr2 = 0; ik = 0.0; r1 = 0; r2 = 0;
3538   /* Aktivitaetsmeldung an die Aggregattabellen der class BAV: ...   */
3539   BAV[1].DatenBAV.Vnag1[ gOwnnr                 ]++;
3540   BAV[1].DatenBAV.Vnag2[ VN[gOwnnr].DatenVN->Vr ]++;
3541   BAV[1].DatenBAV.Vnag3[ VN[gOwnnr].DatenVN->Vk ]++;
3542  if ( !( BAV[1].DatenBAV.As[gperiod] * BAV[1].DatenBAV.Ar[rl]))
3543   {                                         /* KEIN Aenderungsschock */
3544    if ( gperiod == 1 )         /* Erste Periode: Startwerte benutzen */
3545    {
3546      s1  = ( sw1 > normal() ) * ( a0 + b0 * normal() ); /* Schaeden  */
3547      s2  = ( sw2 > normal() ) * ( c0 + d0 * normal() ); /* erzeugen  */
3548      vr1 = VN[gOwnnr].DatenVN->Rk[1].l.Vs[1];     /* Status in erster*/
3549      vr2 = VN[gOwnnr].DatenVN->Rk[2].k.Vs[1];     /* Periode: ...    */
3550      r1  = VN[gOwnnr].DatenVN->Rk[1].l.Vu[1];     /* Versicherer in  */
3551      r2  = VN[gOwnnr].DatenVN->Rk[2].k.Vu[1];     /* erster Periode  */
3552    }
3553    else                      /* Periode > 1: Verhaltensregel anwenden */
3554    {
```

```
3555      s1  = ( sw1 > normal() ) * ( a0 + b0 * normal() ); /* Schaeden  */
3556      s2  = ( sw2 > normal() ) * ( c0 + d0 * normal() ); /* erzeugen  */
3557      vr1 = ( BAV[1].DatenBAV.Dg[0] <= e0);
3558      vr2 = ( BAV[1].DatenBAV.Dg[0] <= f0);
3559      for(i=1;i<=MAXVU;i++)    /* Minimum-Praemie :          Risiko 1 */
3560       {                       /* Weil ein Minimum gesucht wird: zu-  */
3561        p1[i] = 1000.0;        /* naechst Suchvektor auf 1000.0 init. */
3562       }
3563      for(j=1;j<=MAXVU;j++)    /* Praemieninformationen aller Ver-    */
3564       {                       /* sicherer der Periode holen und      */
3565        p1[j] = VU[j].DatenVU->Sp[1].l.Pr[gperiod]; /* die Kosten da  */
3566        ik = ik + BAV[1].DatenBAV.Sk[gperiod]; /* fuer merken ...     */
3567       }
3568      r1 = 0; m = 1000.0;      /* ... aus der gefundenen Stichprobe   */
3569      for(i=1;i<=MAXVU;i++)    /* das Minimum ermitteln und den       */
3570       {                       /* entsprechenden Versicherer waehlen..*/
3571        if ( (p1[i] < m) && (aktiVU[i] != 0) ) /* ..wenn dieser       */
3572         {                     /* Versicherer auch aktiv ist...       */
3573           m  = p1[i];
3574           r1 = i;
3575         }                     /* Mindestens EIN VU muss aktiv sein,  */
3576       }                       /* sonst ist a1 hier immer noch Null ! */
3577      for(i=1;i<=MAXVU;i++)    /* Minimum-Praemie :          Risiko 2 */
3578       {                       /* Weil ein Minimum gesucht wird: zu-  */
3579        p2[i] = 1000.0;        /* naechst Suchvektor auf 1000.0 init. */
3580       }
3581      for(j=1;j<=MAXVU;j++)    /* Praemieninformationen aller Ver-    */
3582       {                       /* sicherer der Periode holen und      */
3583        p2[j] = VU[j].DatenVU->Sp[2].k.Pr[gperiod]; /* die Kosten da  */
3584        ik = ik + BAV[1].DatenBAV.Sk[gperiod]; /* fuer merken ...     */
3585       }
3586      r2 = 0; m = 1000.0;      /* ... aus der gefundenen Stichprobe   */
3587      for(i=1;i<=MAXVU;i++)    /* das Minimum ermitteln und den       */
3588       {                       /* entsprechenden Versicherer waehlen..*/
3589        if ( (p2[i] < m) && (aktiVU[i] != 0) ) /* ..wenn dieser       */
3590         {                     /* Versicherer auch aktiv ist...       */
3591           m  = p2[i];
3592           r2 = i;
3593         }                     /* Mindestens EIN VU muss aktiv sein,  */
3594       }                       /* sonst ist a1 hier immer noch Null ! */
3595    }                           /* END: else ( gperiod == 1 )          */
3596   }                        /* END: if ( !( BAV[1].Daten.As[gperiod]...*/
3597   else                        /* Aenderungsschock hat stattgefunden */
3598   {
3599    if ( gperiod == 1 )        /* Erste Periode: Startwerte benutzen */
```

```
3600    {
3601      s1  = ( sw1 > normal() ) * ( a1 + b1 * normal() ); /* Schaeden  */
3602      s2  = ( sw2 > normal() ) * ( c1 + d1 * normal() ); /* erzeugen  */
3603      vr1 = VN[gOwnnr].DatenVN->Rk[1].l.Vs[1];     /* Status in erster*/
3604      vr2 = VN[gOwnnr].DatenVN->Rk[2].k.Vs[1];     /* Periode: ...    */
3605      r1  = VN[gOwnnr].DatenVN->Rk[1].l.Vu[1];     /* Versicherer in  */
3606      r2  = VN[gOwnnr].DatenVN->Rk[2].k.Vu[1];     /* erster Periode  */
3607    }                                     /* END:   if ( gperiod == 1 )*/
3608    else                      /* Periode > 1: Verhaltensregel anwenden */
3609    {
3610      s1  = ( sw1 > normal() ) * ( a1 + b1 * normal() ); /* Schaeden  */
3611      s2  = ( sw2 > normal() ) * ( c1 + d1 * normal() ); /* erzeugen  */
3612      vr1 = ( BAV[1].DatenBAV.Dg[0] <= e1);
3613      vr2 = ( BAV[1].DatenBAV.Dg[0] <= f1);
3614      for(i=1;i<=MAXVU;i++)    /* Minimum-Praemie :          Risiko 1 */
3615       {                       /* Weil ein Minimum gesucht wird: zu-  */
3616        p1[i] = 1000.0;        /* naechst Suchvektor auf 1000.0 init. */
3617       }
3618      for(j=1;j<=MAXVU;j++)    /* Praemieninformationen aller Ver-    */
3619       {                       /* sicherer der Periode holen und      */
3620        p1[j] = VU[j].DatenVU->Sp[1].l.Pr[gperiod]; /* die Kosten da  */
3621        ik = ik + BAV[1].DatenBAV.Sk[gperiod]; /* fuer merken ...     */
3622       }
3623      r1 = 0; m = 1000.0;      /* ... aus der gefundenen Stichprobe   */
3624      for(i=1;i<=MAXVU;i++)    /* das Minimum ermitteln und den       */
3625       {                       /* entsprechenden Versicherer waehlen..*/
3626        if ( (p1[i] < m) && (aktiVU[i] != 0) ) /* ..wenn dieser       */
3627         {                     /* Versicherer auch aktiv ist...       */
3628           m  = p1[i];
3629           r1 = i;
3630         }                     /* Mindestens EIN VU muss aktiv sein,  */
3631       }                       /* sonst ist a1 hier immer noch Null ! */
3632      for(i=1;i<=MAXVU;i++)    /* Minimum-Praemie :          Risiko 2 */
3633       {                       /* Weil ein Minimum gesucht wird: zu-  */
3634        p2[i] = 1000.0;        /* naechst Suchvektor auf 1000.0 init. */
3635       }
3636      for(j=1;j<=MAXVU;j++)    /* Praemieninformationen aller Ver-    */
3637       {                       /* sicherer der Periode holen und      */
3638        p2[j] = VU[j].DatenVU->Sp[2].k.Pr[gperiod]; /* die Kosten da  */
3639        ik = ik + BAV[1].DatenBAV.Sk[gperiod]; /* fuer merken ...     */
3640       }
3641      r2 = 0; m = 1000.0;      /* ... aus der gefundenen Stichprobe   */
3642      for(i=1;i<=MAXVU;i++)    /* das Minimum ermitteln und den       */
3643       {                       /* entsprechenden Versicherer waehlen..*/
3644        if ( (p2[i] < m) && (aktiVU[i] != 0) ) /* ..wenn dieser       */
```

```
3645        {                       /* Versicherer auch aktiv ist...       */
3646          m  = p2[i];
3647          r2 = i;
3648        }                       /* Mindestens EIN VU muss aktiv sein,  */
3649      }                         /* sonst ist a1 hier immer noch Null ! */
3650   }                                    /* END: else if (gperiod == 1)*/
3651  }                                     /* END: else Aenderungsschock */
3652  if ( vr1 == 1 )
3653   {                                          /* ABRECHNUNG: Risiko 1 */
3654    if ( s1 > 0.0   )
3655     {                           /* FALL:  Fremdversicherter Schaden: */
3656                                                /* VU-Vektoren:      */
3657       VU[r1].DatenVU->Sp[1].l.Rs[gperiod] =    /* Reserven um       */
3658       VU[r1].DatenVU->Sp[1].l.Rs[gperiod] +    /* Praemie erhoehen  */
3659       VU[r1].DatenVU->Sp[1].l.Pr[gperiod] -    /* und Schaden       */
3660       s1;                                      /* abziehen...       */
3661       VU[r1].DatenVU->Sp[1].l.Sh[gperiod] =    /* Periodenschadens- */
3662       VU[r1].DatenVU->Sp[1].l.Sh[gperiod] +    /* summe um den Scha-*/
3663       s1;                                      /* den erhoehen      */
3664       VU[r1].DatenVU->Sp[1].l.Sa[gperiod]++;   /* Schadenanzahl um  */
3665                                                /* eins erhoehen     */
3666       VU[r1].DatenVU->Sp[1].l.Vn[gperiod]++;   /* Versichertenanzahl*/
3667                                                /* um eins erhoehen  */
3668                                                /* VN-Vektoren :     */
3669       VN[gOwnnr].DatenVN->Rk[1].l.Vu[gperiod]= /* Gehabten Versiche-*/
3670       r1;                                      /* rer merken        */
3671       VN[gOwnnr].DatenVN->Rk[1].l.Vs[gperiod]= /* Versicherungssta- */
3672       vr1;                                     /* tus merken        */
3673       VN[gOwnnr].DatenVN->Rk[1].l.Vp[gperiod]= /* Gezahlte Praemie  */
3674       VU[r1].DatenVU->Sp[1].l.Pr[gperiod];     /* merken            */
3675       VN[gOwnnr].DatenVN->Rk[1].l.Sh[gperiod]= /* Gehabten Schaden  */
3676       s1;                                      /* merken            */
3677     }  /* END: if ( s1 > 0.0   ) */
3678    else
3679     {                       /* FALL: Fremdversicherter Nicht-Schaden */
3680                                                /* VU-Vektoren:      */
3681       VU[r1].DatenVU->Sp[1].l.Rs[gperiod] =    /* Reserven um       */
3682       VU[r1].DatenVU->Sp[1].l.Rs[gperiod] +    /* Praemie erhoehen  */
3683       VU[r1].DatenVU->Sp[1].l.Pr[gperiod];
3684       VU[r1].DatenVU->Sp[1].l.Vn[gperiod]++;   /* Versichertenanzahl*/
3685                                                /* um eins erhoehen  */
3686                                                /* VN-Vektoren :     */
3687       VN[gOwnnr].DatenVN->Rk[1].l.Vu[gperiod]= /* Gehabten Versiche-*/
3688       r1;                                      /* rer merken        */
3689       VN[gOwnnr].DatenVN->Rk[1].l.Vs[gperiod]= /* Versicherungssta- */
```

```
3690        vr1;                                        /* tus merken        */
3691        VN[gOwnnr].DatenVN->Rk[1].l.Vp[gperiod]= /* Gezahlte Praemie  */
3692        VU[r1].DatenVU->Sp[1].l.Pr[gperiod];     /* merken            */
3693        VN[gOwnnr].DatenVN->Rk[1].l.Sh[gperiod]= /* Kein Schaden ge-  */
3694        0.0;                                        /* habt              */
3695      }                                         /* END: else (s1 > 0.0  */
3696    }                                           /* END: if ( vr1==1 )   */
3697    else
3698    {
3699     if ( s1 > 0.0   )
3700      {                             /* FALL: Eigenversicherter Schaden: */
3701        VN[gOwnnr].DatenVN->Rk[1].l.Vu[gperiod]= /* Keinen Versicherer*/
3702        0;                                        /* gehabt            */
3703        VN[gOwnnr].DatenVN->Rk[1].l.Vs[gperiod]= /* Versicherungssta- */
3704        vr1;                                      /* tus merken        */
3705        VN[gOwnnr].DatenVN->Rk[1].l.Ev[gperiod]= /* Eigenversicherten */
3706        s1;                                       /* Schaden merken    */
3707        VN[gOwnnr].DatenVN->Rk[1].l.Sh[gperiod]= /* Gehabten Schaden  */
3708        s1;                                       /* merken            */
3709      }                                       /* END: if ( s1 > 0.0   ) */
3710     else
3711      {                         /* FALL: Eigenversicherter Nicht-Schaden */
3712                                                  /* NICHTS abzurechnen*/
3713      }                                       /* END: else (s1 > 0.0  */
3714    }                                         /* END: else ( vr1==1 ) */
3715   if ( vr2 == 1 )
3716    {                                         /* ABRECHNUNG: Risiko 2 */
3717     if ( s2 > 0.0   )
3718      {                             /* FALL: Fremdversicherter Schaden: */
3719                                                  /* VU-Vektoren:      */
3720        VU[r2].DatenVU->Sp[2].k.Rs[gperiod] =     /* Reserven um       */
3721        VU[r2].DatenVU->Sp[2].k.Rs[gperiod] +     /* Praemie erhoehen  */
3722        VU[r2].DatenVU->Sp[2].k.Pr[gperiod] -     /* und Schaden       */
3723        s2;                                       /* abziehen...       */
3724        VU[r2].DatenVU->Sp[2].k.Sh[gperiod] =     /* Periodenschadens- */
3725        VU[r2].DatenVU->Sp[2].k.Sh[gperiod] +     /* summe um den Scha-*/
3726        s2;                                       /* den erhoehen      */
3727        VU[r2].DatenVU->Sp[2].k.Sa[gperiod]++;    /* Schadenanzahl um  */
3728                                                  /* eins erhoehen     */
3729        VU[r2].DatenVU->Sp[2].k.Vn[gperiod]++;    /* Versichertenanzahl*/
3730                                                  /* um eins erhoehen  */
3731                                                  /* VN-Vektoren :     */
3732        VN[gOwnnr].DatenVN->Rk[2].k.Vu[gperiod]= /* Gehabten Versiche-*/
3733        r2;                                       /* rer merken        */
3734        VN[gOwnnr].DatenVN->Rk[2].k.Vs[gperiod]= /* Versicherungssta- */
```

```
3735        vr2;                                        /* tus merken        */
3736        VN[gOwnnr].DatenVN->Rk[2].k.Vp[gperiod]= /* Gezahlte Praemie  */
3737        VU[r2].DatenVU->Sp[2].k.Pr[gperiod];     /* merken            */
3738        VN[gOwnnr].DatenVN->Rk[2].k.Sh[gperiod]= /* Gehabten Schaden  */
3739        s2;                                         /* merken            */
3740      }                                       /* END: if ( s2 > 0.0   ) */
3741     else
3742      {                       /* FALL: Fremdversicherter Nicht-Schaden */
3743                                                    /* VU-Vektoren:      */
3744        VU[r2].DatenVU->Sp[2].k.Rs[gperiod] =    /* Reserven um       */
3745        VU[r2].DatenVU->Sp[2].k.Rs[gperiod] +    /* Praemie erhoehen  */
3746        VU[r2].DatenVU->Sp[2].k.Pr[gperiod];
3747        VU[r2].DatenVU->Sp[2].k.Vn[gperiod]++;   /* Versichertenanzahl*/
3748                                                    /* um eins erhoehen  */
3749                                                    /* VN-Vektoren :     */
3750        VN[gOwnnr].DatenVN->Rk[2].k.Vu[gperiod]= /* Gehabten Versiche-*/
3751        r2;                                         /* rer merken        */
3752        VN[gOwnnr].DatenVN->Rk[2].k.Vs[gperiod]= /* Versicherungssta- */
3753        vr2;                                        /* tus merken        */
3754        VN[gOwnnr].DatenVN->Rk[2].k.Vp[gperiod]= /* Gezahlte Praemie  */
3755        VU[r2].DatenVU->Sp[2].k.Pr[gperiod];     /* merken            */
3756        VN[gOwnnr].DatenVN->Rk[2].k.Sh[gperiod]= /* Kein Schaden ge-  */
3757        0.0;                                        /* habt              */
3758      }                                     /* END: else (s2 > 0.0 ) */
3759    }                                       /* END: if ( vr2 == 1 )  */
3760    else
3761    {
3762     if ( s2 > 0.0   )
3763      {                           /* FALL: Eigenversicherter Schaden: */
3764        VN[gOwnnr].DatenVN->Rk[2].k.Vu[gperiod]= /* Keinen Versicherer*/
3765        0;                                          /* gehabt            */
3766        VN[gOwnnr].DatenVN->Rk[2].k.Vs[gperiod]= /* Versicherungssta- */
3767        vr2;                                        /* tus merken        */
3768        VN[gOwnnr].DatenVN->Rk[2].k.Ev[gperiod]= /* Eigenversicherten */
3769        s2;                                         /* Schaden merken    */
3770        VN[gOwnnr].DatenVN->Rk[2].k.Sh[gperiod]= /* Gehabten Schaden  */
3771        s2;                                         /* merken            */
3772      }                                       /* END: if ( s2 > 0.0   ) */
3773     else
3774      {                       /* FALL: Eigenversicherter Nicht-Schaden */
3775                                                    /* NICHTS abzurechnen*/
3776      }                                       /* END: else (s2 > 0.0    */
3777    }                                         /* END: else ( vr2 == 1 ) */
3778   VN[gOwnnr].DatenVN->Vm[gperiod]            =    /* Kummuliertes Ver-*/
3779   VN[gOwnnr].DatenVN->Vm[gperiod-1]          -    /* moegen berechnen */
```

```
3780   s1                                          -
3781   s2                                          -
3782   VN[gOwnnr].DatenVN->Rk[1].l.Vp[gperiod]     -    /* Praemien in den  */
3783   VN[gOwnnr].DatenVN->Rk[2].k.Vp[gperiod]     -    /* Sparten,         */
3784   ik                                          ;    /* Suchkosten       */
3785  }                                            /* END: IF (gperiod>=... */
3786 }                                                  /* END: Vrvn06      */
3787
3788 /* ************************************************************** */
3789 /* act: Vrvn07:  Versicherungsnehmer   Verhaltensregel:  undefiniert */
3790 /*                                                                  */
3791 /*                            vgl. Kapitel 3.3.2.x                  */
3792 /* ************************************************************** */
3793 act Vrvn07 {                        /* Vereinbarung der Hilfsvariable */
3794 float s1,s2,pf1[MAXVU+1],pf2[MAXVU+1],sw1,sw2,a0,a1,b0,b1,c0,c1,
3795       d0,d1,e0,e1,f0,f1,g0,g1,h0,h1;
3796 int vr1,vr2,r1,r2;
3797 if(gperiod>=((VN[gOwnnr].DatenVN->Ap)*(VN[gOwnnr].DatenVN->Al[rl])))
3798  {/* LANGE IMS-Variable in KURZE Variable fuer VR uebersetzen        */
3799   a0 = VN[gOwnnr].DatenVN->Pv[0 ]; a1 = VN[gOwnnr].DatenVN->Pv[1 ];
3800   b0 = VN[gOwnnr].DatenVN->Pv[2 ]; b1 = VN[gOwnnr].DatenVN->Pv[3 ];
3801   c0 = VN[gOwnnr].DatenVN->Pv[4 ]; c1 = VN[gOwnnr].DatenVN->Pv[5 ];
3802   d0 = VN[gOwnnr].DatenVN->Pv[6 ]; d1 = VN[gOwnnr].DatenVN->Pv[7 ];
3803   e0 = VN[gOwnnr].DatenVN->Pv[8 ]; e1 = VN[gOwnnr].DatenVN->Pv[9 ];
3804   f0 = VN[gOwnnr].DatenVN->Pv[10]; f1 = VN[gOwnnr].DatenVN->Pv[11];
3805   g0 = VN[gOwnnr].DatenVN->Pv[12]; g1 = VN[gOwnnr].DatenVN->Pv[13];
3806   h0 = VN[gOwnnr].DatenVN->Pv[14]; h1 = VN[gOwnnr].DatenVN->Pv[15];
3807   sw1 = VN[gOwnnr].DatenVN->Rk[1].l.Sw[gperiod];
3808   sw2 = VN[gOwnnr].DatenVN->Rk[2].k.Sw[gperiod];
3809   vr1 = 0; vr2 = 0; r1 = 0; r2 = 0;
3810   /* Aktivitaetsmeldung an die Aggregattabellen der class BAV: ...   */
3811   BAV[1].DatenBAV.Vnag1[ gOwnnr                 ]++;
3812   BAV[1].DatenBAV.Vnag2[ VN[gOwnnr].DatenVN->Vr ]++;
3813   BAV[1].DatenBAV.Vnag3[ VN[gOwnnr].DatenVN->Vk ]++;
3814  if ( !( BAV[1].DatenBAV.As[gperiod] * BAV[1].DatenBAV.Ar[rl]))
3815   {                                          /* KEIN Aenderungsschock */
3816    if ( gperiod == 1 )         /* Erste Periode: Startwerte benutzen */
3817    {
3818      s1  = ( sw1 > normal() ) * ( a0 + b0 * normal() ); /* Schaeden  */
3819      s2  = ( sw2 > normal() ) * ( c0 + d0 * normal() ); /* erzeugen  */
3820      vr1 = VN[gOwnnr].DatenVN->Rk[1].l.Vs[1];     /* Status in erster*/
3821      vr2 = VN[gOwnnr].DatenVN->Rk[2].k.Vs[1];     /* Periode: ...    */
3822      r1  = VN[gOwnnr].DatenVN->Rk[1].l.Vu[1];     /* Versicherer in  */
3823      r2  = VN[gOwnnr].DatenVN->Rk[2].k.Vu[1];     /* erster Periode  */
3824    }
```

```
3825    else                      /* Periode > 1: Verhaltensregel anwenden */
3826    {
3827      s1  = ( sw1 > normal() ) * ( a0 + b0 * normal() ); /* Schaeden  */
3828      s2  = ( sw2 > normal() ) * ( c0 + d0 * normal() ); /* erzeugen  */
3829      /*
3830      vr1 = 0; HIER KOENNEN DIE EIGENENTWICKLUNGEN REALISIERT WERDEN
3831      vr2 = 0;
3832      r1  = 0;
3833      r2  = 0;
3834      */
3835    }                           /* END: else ( gperiod == 1 )         */
3836   }                       /* END: if ( !( BAV[1].Daten.As[gperiod]...*/
3837   else                         /* Aenderungsschock hat stattgefunden */
3838   {
3839    if ( gperiod == 1 )         /* Erste Periode: Startwerte benutzen */
3840    {
3841      s1  = ( sw1 > normal() ) * ( a1 + b1 * normal() ); /* Schaeden  */
3842      s2  = ( sw2 > normal() ) * ( c1 + d1 * normal() ); /* erzeugen  */
3843      vr1 = VN[gOwnnr].DatenVN->Rk[1].l.Vs[1];     /* Status in erster*/
3844      vr2 = VN[gOwnnr].DatenVN->Rk[2].k.Vs[1];     /* Periode: ...    */
3845      r1  = VN[gOwnnr].DatenVN->Rk[1].l.Vu[1];     /* Versicherer in  */
3846      r2  = VN[gOwnnr].DatenVN->Rk[2].k.Vu[1];     /* erster Periode  */
3847    }                                   /* END:   if ( gperiod == 1 )*/
3848    else                      /* Periode > 1: Verhaltensregel anwenden */
3849    {
3850      s1  = ( sw1 > normal() ) * ( a1 + b1 * normal() ); /* Schaeden  */
3851      s2  = ( sw2 > normal() ) * ( c1 + d1 * normal() ); /* erzeugen  */
3852      /*
3853      vr1 = 0;  HIER KOENNEN DIE EIGENENTWICKLUNGEN REALISIERT WERDEN
3854      vr2 = 0;
3855      r1  = 0;
3856      r2  = 0;
3857      */
3858    }                                 /* END: else if (gperiod == 1)*/
3859   }                                  /* END: else Aenderungsschock */
3860   if ( vr1 == 1 )
3861    {                                       /* ABRECHNUNG: Risiko 1 */
3862     if ( s1 > 0.0   )
3863      {                          /* FALL:  Fremdversicherter Schaden: */
3864                                                /* VU-Vektoren:      */
3865        VU[r1].DatenVU->Sp[1].l.Rs[gperiod] =    /* Reserven um       */
3866        VU[r1].DatenVU->Sp[1].l.Rs[gperiod] +    /* Praemie erhoehen  */
3867        VU[r1].DatenVU->Sp[1].l.Pr[gperiod] -    /* und Schaden       */
3868        s1;                                      /* abziehen...       */
3869        VU[r1].DatenVU->Sp[1].l.Sh[gperiod] =    /* Periodenschadens- */
```

```
3870        VU[r1].DatenVU->Sp[1].l.Sh[gperiod] +    /* summe um den Scha-*/
3871        s1;                                      /* den erhoehen      */
3872        VU[r1].DatenVU->Sp[1].l.Sa[gperiod]++;   /* Schadenanzahl um  */
3873                                                 /* eins erhoehen     */
3874        VU[r1].DatenVU->Sp[1].l.Vn[gperiod]++;   /* Versichertenanzahl*/
3875                                                 /* um eins erhoehen  */
3876                                                 /* VN-Vektoren :     */
3877        VN[gOwnnr].DatenVN->Rk[1].l.Vu[gperiod]= /* Gehabten Versiche-*/
3878        r1;                                      /* rer merken        */
3879        VN[gOwnnr].DatenVN->Rk[1].l.Vs[gperiod]= /* Versicherungssta- */
3880        vr1;                                     /* tus merken        */
3881        VN[gOwnnr].DatenVN->Rk[1].l.Vp[gperiod]= /* Gezahlte Praemie  */
3882        VU[r1].DatenVU->Sp[1].l.Pr[gperiod];     /* merken            */
3883        VN[gOwnnr].DatenVN->Rk[1].l.Sh[gperiod]= /* Gehabten Schaden  */
3884        s1;                                      /* merken            */
3885      }  /* END: if ( s1 > 0.0   ) */
3886     else
3887      {                       /* FALL: Fremdversicherter Nicht-Schaden */
3888                                                 /* VU-Vektoren:      */
3889        VU[r1].DatenVU->Sp[1].l.Rs[gperiod] =    /* Reserven um       */
3890        VU[r1].DatenVU->Sp[1].l.Rs[gperiod] +    /* Praemie erhoehen  */
3891        VU[r1].DatenVU->Sp[1].l.Pr[gperiod];
3892        VU[r1].DatenVU->Sp[1].l.Vn[gperiod]++;   /* Versichertenanzahl*/
3893                                                 /* um eins erhoehen  */
3894                                                 /* VN-Vektoren :     */
3895        VN[gOwnnr].DatenVN->Rk[1].l.Vu[gperiod]= /* Gehabten Versiche-*/
3896        r1;                                      /* rer merken        */
3897        VN[gOwnnr].DatenVN->Rk[1].l.Vs[gperiod]= /* Versicherungssta- */
3898        vr1;                                     /* tus merken        */
3899        VN[gOwnnr].DatenVN->Rk[1].l.Vp[gperiod]= /* Gezahlte Praemie  */

3900        VU[r1].DatenVU->Sp[1].l.Pr[gperiod];     /* merken            */
3901        VN[gOwnnr].DatenVN->Rk[1].l.Sh[gperiod]= /* Kein Schaden ge-  */
3902        0.0;                                     /* habt              */
3903      }                                       /* END: else (s1 > 0.0  */
3904    }                                         /* END: if ( vr1==1 )   */
3905    else
3906    {
3907     if ( s1 > 0.0   )
3908      {                           /* FALL: Eigenversicherter Schaden: */
3909        VN[gOwnnr].DatenVN->Rk[1].l.Vu[gperiod]= /* Keinen Versicherer*/
3910        0;                                       /* gehabt            */
3911        VN[gOwnnr].DatenVN->Rk[1].l.Vs[gperiod]= /* Versicherungssta- */
3912        vr1;                                     /* tus merken        */
3913        VN[gOwnnr].DatenVN->Rk[1].l.Ev[gperiod]= /* Eigenversicherten */
3914        s1;                                      /* Schaden merken    */
```

```
3915        VN[gOwnnr].DatenVN->Rk[1].l.Sh[gperiod]= /* Gehabten Schaden  */
3916        s1;                                      /* merken            */
3917      }                                    /* END: if ( s1 > 0.0   ) */
3918     else
3919      {                       /* FALL: Eigenversicherter Nicht-Schaden */
3920                                               /* NICHTS abzurechnen*/
3921      }                                      /* END: else (s1 > 0.0  */
3922    }                                        /* END: else ( vr1==1 ) */
3923   if ( vr2 == 1 )
3924    {                                       /* ABRECHNUNG: Risiko 2 */
3925     if ( s2 > 0.0   )
3926      {                            /* FALL: Fremdversicherter Schaden: */
3927                                               /* VU-Vektoren:      */
3928        VU[r2].DatenVU->Sp[2].k.Rs[gperiod] =    /* Reserven um       */
3929        VU[r2].DatenVU->Sp[2].k.Rs[gperiod] +    /* Praemie erhoehen  */
3930        VU[r2].DatenVU->Sp[2].k.Pr[gperiod] -    /* und Schaden       */
3931        s2;                                      /* abziehen...       */
3932        VU[r2].DatenVU->Sp[2].k.Sh[gperiod] =    /* Periodenschadens- */
3933        VU[r2].DatenVU->Sp[2].k.Sh[gperiod] +    /* summe um den Scha-*/
3934        s2;                                      /* den erhoehen      */
3935        VU[r2].DatenVU->Sp[2].k.Sa[gperiod]++;   /* Schadenanzahl um  */
3936                                                 /* eins erhoehen     */
3937        VU[r2].DatenVU->Sp[2].k.Vn[gperiod]++;   /* Versichertenanzahl*/
3938                                                 /* um eins erhoehen  */
3939                                                 /* VN-Vektoren :     */
3940        VN[gOwnnr].DatenVN->Rk[2].k.Vu[gperiod]= /* Gehabten Versiche-*/
3941        r2;                                      /* rer merken        */
3942        VN[gOwnnr].DatenVN->Rk[2].k.Vs[gperiod]= /* Versicherungssta- */
3943        vr2;                                     /* tus merken        */
3944        VN[gOwnnr].DatenVN->Rk[2].k.Vp[gperiod]= /* Gezahlte Praemie  */
3945        VU[r2].DatenVU->Sp[2].k.Pr[gperiod];     /* merken            */
3946        VN[gOwnnr].DatenVN->Rk[2].k.Sh[gperiod]= /* Gehabten Schaden  */
3947        s2;                                      /* merken            */
3948      }                                    /* END: if ( s2 > 0.0   ) */
3949     else
3950      {                       /* FALL: Fremdversicherter Nicht-Schaden */
3951                                               /* VU-Vektoren:      */
3952        VU[r2].DatenVU->Sp[2].k.Rs[gperiod] =    /* Reserven um       */
3953        VU[r2].DatenVU->Sp[2].k.Rs[gperiod] +    /* Praemie erhoehen  */
3954        VU[r2].DatenVU->Sp[2].k.Pr[gperiod];
3955        VU[r2].DatenVU->Sp[2].k.Vn[gperiod]++;   /* Versichertenanzahl*/
3956                                                 /* um eins erhoehen  */
3957                                                 /* VN-Vektoren :     */
3958        VN[gOwnnr].DatenVN->Rk[2].k.Vu[gperiod]= /* Gehabten Versiche-*/
3959        r2;                                      /* rer merken        */
```

```
3960        VN[gOwnnr].DatenVN->Rk[2].k.Vs[gperiod]= /* Versicherungssta- */
3961        vr2;                                     /* tus merken        */
3962        VN[gOwnnr].DatenVN->Rk[2].k.Vp[gperiod]= /* Gezahlte Praemie  */
3963        VU[r2].DatenVU->Sp[2].k.Pr[gperiod];     /* merken            */
3964        VN[gOwnnr].DatenVN->Rk[2].k.Sh[gperiod]= /* Kein Schaden ge-  */
3965        0.0;                                     /* habt              */
3966      }                                       /* END: else (s2 > 0.0 ) */
3967    }                                         /* END: if ( vr2 == 1 )  */
3968    else
3969    {
3970     if ( s2 > 0.0   )
3971      {                            /* FALL: Eigenversicherter Schaden: */
3972        VN[gOwnnr].DatenVN->Rk[2].k.Vu[gperiod]= /* Keinen Versicherer*/
3973        0;                                       /* gehabt            */
3974        VN[gOwnnr].DatenVN->Rk[2].k.Vs[gperiod]= /* Versicherungssta- */
3975        vr2;                                     /* tus merken        */
3976        VN[gOwnnr].DatenVN->Rk[2].k.Ev[gperiod]= /* Eigenversicherten */
3977        s2;                                      /* Schaden merken    */
3978        VN[gOwnnr].DatenVN->Rk[2].k.Sh[gperiod]= /* Gehabten Schaden  */
3979        s2;                                      /* merken            */
3980      }                                       /* END: if ( s2 > 0.0   ) */
3981     else
3982      {                       /* FALL: Eigenversicherter Nicht-Schaden */
3983                                               /* NICHTS abzurechnen*/
3984      }                                       /* END: else (s2 > 0.0    */
3985    }                                         /* END: else ( vr2 == 1 ) */
3986   VN[gOwnnr].DatenVN->Vm[gperiod]            =    /* Kummuliertes Ver-*/
3987   VN[gOwnnr].DatenVN->Vm[gperiod-1]          -    /* moegen berechnen */
3988   s1                                         -
3989   s2                                         -
3990   VN[gOwnnr].DatenVN->Rk[1].l.Vp[gperiod]    -    /* Falls EigemVS,ist*/
3991   VN[gOwnnr].DatenVN->Rk[2].k.Vp[gperiod]    ;    /* Vp[] = 0.0 ini-  */
3992                                                   /* tialisiert       */
3993    /* Bei VR: SUCHE hier auch die Informationskosten addieren        */
3994    /* fprintf(runreport,
3995               "[%d,%d] R1[%2d] R2[%2d] V1[%1d] V2[%1d] %2.1f %2.1f\n",
3996               gOwnnr, gperiod, r1,r2,vr1,vr2,s1,s2);
3997     */
3998  }                                          /* END: IF (gperiod>=... */
3999 }                                                /* END: Vrvn07      */
4000
4001 /* ************************************************************** */
4002 /*            Beginn der Service- und Menuroutinen                */
4003 /* ************************************************************** */
4004 /* ************************************************************** */
```

```
4005 /* func Mkload() : Laden einer Modellkonfiguration. Aus einer Datei  */
4006 /*                 werden alle dynamischen ESL-Subjetke(VU,VN) an die*/
4007 /*                 im Hauptspeicher reservierten Stellen geladen.    */
4008 /* ************************************************************** */
4009 func Mkload() {
4010  int i;                                        /* Schleifenzaehler */
4011  clearScreen();
4012  printfAt( 1,24, "[ Marktkonfiguration laden ]");
4013  konfig = fopen("konfig.ims","rb");
4014  if ( konfig == (file *) 0)
4015    {
4016     fprintf(runreport,"func Mkload(): konfig.ims nicht geoeffnet\n");
4017    }
4018  if ( konfig != (file *) 0)
4019   {
4020    for(i=1;i<=MAXVU;i++)
4021     {
4022      if ( VU[i].DatenVU != (struct s1 *) 0 )
4023       {
4024        fread(VU[i].DatenVU, sizeof(struct s1), 1, konfig);
4025       }
4026     }
4027    for(i=1;i<=MAXVN;i++)
4028     {
4029      if ( VN[i].DatenVN != (struct s2 *) 0 )
4030       {
4031        fread(VN[i].DatenVN, sizeof(struct s2), 1, konfig);
4032       }
4033      }
4034    fread(&BAV[1].DatenBAV, sizeof(struct s7), 1, konfig);
4035   fclose(konfig);
4036   for(i=1;i<=MAXVU;i++)       /* Automatische agvux[.]-Zuweisung an  */
4037    {                          /* die Aggregatdateien, die gebraucht  */
4038     BAV[1].DatenBAV.Vuag2[ VU[i].DatenVU->Vr ]++; /* werden          */
4039     BAV[1].DatenBAV.Vuag3[ VU[i].DatenVU->Vk ]++;
4040    }
4041   for( i = 1;  i <=    10 ; i++ )
4042    { if ( BAV[1].DatenBAV.Vuag2[i] > 0 ) { agvu2[i] = 1;} }
4043   for( i = 1;  i <=     3 ; i++ )
4044    { if ( BAV[1].DatenBAV.Vuag3[i] > 0 ) { agvu3[i] = 1;} }
4045   agvu4[1] = 1; /* Die Subjektklasse der Versicherer: immer erzeugen */
4046   for(i=1;i<=MAXVN;i++)       /* Automatische agvnx[.]-Zuweisung an  */
4047    {                          /* die Aggregatdateien, die gebraucht  */
4048     BAV[1].DatenBAV.Vnag2[ VN[i].DatenVN->Vr ]++; /* werden          */
4049     BAV[1].DatenBAV.Vnag3[ VN[i].DatenVN->Vk ]++;
```

```
4050    }
4051   for( i = 1;  i <=     7 ; i++ )
4052    { if ( BAV[1].DatenBAV.Vnag2[i] > 0 ) { agvn2[i] = 1;} }
4053   for( i = 1;  i <=     3 ; i++ )
4054    { if ( BAV[1].DatenBAV.Vnag3[i] > 0 ) { agvn3[i] = 1;} }
4055   agvn4[1] = 1; /* Subjektklasse Versicherungsnehmer: immer erzeugen */
4056   load = 1; /* TAG-Variable fuer die Aktionen: MyinitVU,MyinitVN     */
4057   } /* end if ( konfig != (file *) 0) */
4058  waitKey();
4059 } /* END: func Mkload() */
4060
4061 /* ************************************************************** */
4062 /* func Mksave() : Speichern einer Modellkonfiguration              */
4063 /*                                                                 */
4064 /*                                                                 */
4065 /* ************************************************************** */
4066 func Mksave() {
4067  int i;                                        /* Schleifenzaehler */
4068  clearScreen();
4069  printfAt( 1,22, "[ Marktkonfiguration speichern ]");
4070  konfig = fopen("konfig.ims","wb");
4071  if ( konfig == (file *) 0)
4072   {
4073    fprintf(runreport,"func Mksave(): konfig.ims nicht geoeffnet\n");
4074   }
4075   else /* ( konfig != (file *) 0) */
4076   {
4077    for(i=1;i<=MAXVU;i++)
4078     {
4079      if ( VU[i].DatenVU != (struct s1 *) 0 )
4080       {
4081        fwrite(VU[i].DatenVU, sizeof(struct s1), 1, konfig);
4082       }
4083     }
4084    for(i=1;i<=MAXVN;i++)
4085     {
4086      if ( VN[i].DatenVN != (struct s2 *) 0 )
4087       {
4088        fwrite(VN[i].DatenVN, sizeof(struct s2), 1, konfig);
4089       }
4090     }
4091     fwrite(&BAV[1].DatenBAV, sizeof(struct s7), 1, konfig);
4092    fclose(konfig);
4093   } /* END: else, if ( konfig == (file *) 0) */
4094  waitKey();
```

```
4095 } /* END: func Mksave() */
4096
4097 /* ************************************************************** */
4098 /* func Bavauin() : Automatische Initialisierung der class BAV.      */
4099 /*                  Wird von  allen funcs fuer die vordefinierten    */
4100 /*                  Modelle verwendet.                               */
4101 /* ************************************************************** */
4102 func Bavauin(p,l,k,z,r)
4103 int    p;           /* Periode mit Aenderungssschock                */
4104 int    l;           /* Simulationslauf mit Aenderungsschock         */
4105 float  k;           /* Kosten pro Suchschritt des VNs               */
4106 float  z;           /* Zinssatz der Reservenverzinsung              */
4107 int    r;           /* Sim.Wiederholungen zugelassen = 1, sonst 0   */
4108 {
4109 int j;              /* Schleifenzaehler                             */
4110  for( j = 1; j <= SIMLAENGE ; j++)  /* Beispiele :                  */
4111   {                                 /* Bavauin(100,100) keine Aender-*/
4112    if ( j < p )                     /* ungsschocks in keinem Lauf   */
4113     { BAV[1].DatenBAV.As[j] = 0; }  /* Bavauin(0,0) in jedem Lauf in */
4114     else                            /* jeder Periode Aenderungsschock*/
4115      { BAV[1].DatenBAV.As[j] = 1; } /* Bavauin(50,50) in den ersten  */
4116   }                                 /* 50 Laeufen KEIN Schock, in den*/
4117  for( j = 1; j <= SIMLAENGE ; j++)  /* letzten 50 Laeufen ab Periode */
4118   {                                 /* 50 in jeder Periode Aender-   */
4119    if ( j < l )                     /* ungsschock                    */
4120     { BAV[1].DatenBAV.Ar[j] = 0; }
4121     else
4122      { BAV[1].DatenBAV.Ar[j] = 1; }
4123   }
4124  for( j = 1; j <= SIMLAENGE; j++)
4125   {
4126    BAV[1].DatenBAV.Sk[j] = k;
4127    BAV[1].DatenBAV.Zs[j] = z;
4128   }
4129  BAV[1].DatenBAV.Rp = r;
4130 } /* END: func Bavauin() */
4131
4132 /* ************************************************************** */
4133 /* func Vuauini() : Automatische Initialisierung einer Versicherung  */
4134 /*                  Wird von  allen funcs fuer die vordefinierten    */
4135 /*                  Modelle verwendet.                               */
4136 /* ************************************************************** */
4137 func Vuauini(i,st,ap,al,a11,a12,a13,a21,a22,a23,p1,w1,p2,w2,
4138              v0,v1,v2,v3,v4,v5,v6,v7,v8,v9,v10,v11,v12,v13,v14,v15 )
4139 int i;             /* Nummer der Versicherung, die initialisiert wird*/
```

```
4140 int st;            /* Nummer der Verhaltensregel, die VU ausfuehrt   */
4141 int ap;            /* Aktivierungsperiode der Versicherung           */
4142 int al;            /* Aktivierungslaeufe:alle < al: Al[j] = 1;sonst 0*/
4143 float a11,a12,a13,a21,a22,a23;/* Anspruchsnivaus der Mark-Up Regeln  */
4144 float p1,w1,p2,w2; /* Startwerte fuer Praemien, Werbung in 2 Sparten */
4145 float v0,v1,v2,v3,v4,v5,v6,v7,v8,v9,v10,v11,v12,v13,v14,v15; /* Die  */
4146                    /* Elemente des Parametervektors                  */
4147 {
4148 int j;                                /* Schleifenzaehler           */
4149  VU[i].DatenVU->Ac[1].st = st;        /* Verhaltensregel und ihr...  */
4150  VU[i].DatenVU->Ac[1].lt = 1;         /* Ausfuehrungszeitpunkt...    */
4151  for( j = 2; j <= MAXNLOG; j++ )      /* und den Rest des Aktions-   */
4152   {                                   /* vektors auf 0 initialisieren*/
4153    VU[i].DatenVU->Ac[j].lt  = 0;
4154    VU[i].DatenVU->Ac[j].st  = 0;
4155   }
4156  VU[i].DatenVU->Ap       = ap;        /* Aktivierungszeitpunkt       */
4157  for( j = 1; j <= SIMLAENGE; j++)     /* Aktivierungslaeufe          */
4158   {
4159    if ( j <= al )
4160    { VU[i].DatenVU->Al[j] = 1; }
4161    else
4162     { VU[i].DatenVU->Al[j] = 0; }
4163   }
4164  VU[i].DatenVU->Vr       = st;        /* Zuordnung zu den Aggregaten */
4165  VU[i].DatenVU->Vk       = vkrvu[st]; /* der Stufe II und III        */
4166  VU[i].DatenVU->A1[1] = a11;          /* Anspruchsniveaus der Sparten*/
4167  VU[i].DatenVU->A1[2] = a12;          /* NUR A[1..3] NICHT A[0]      */
4168  VU[i].DatenVU->A1[3] = a13;
4169  VU[i].DatenVU->A2[1] = a21;
4170  VU[i].DatenVU->A2[2] = a22;
4171  VU[i].DatenVU->A2[3] = a23;
4172  VU[i].DatenVU->Sp[1].l.Pr[1] = p1 ; /* Startwerte:Sparte 1 Praemien */
4173  VU[i].DatenVU->Sp[1].l.Wa[1] = w1 ; /*                     Werbung  */
4174  VU[i].DatenVU->Sp[2].k.Pr[1] = p2 ; /*            Sparte 2 Praemien */
4175  VU[i].DatenVU->Sp[2].k.Wa[1] = w2 ; /*                     Werbung  */
4176  VU[i].DatenVU->Pv[ 0] = v0;  VU[i].DatenVU->Pv[ 8] = v8 ; /* Initia-*/
4177  VU[i].DatenVU->Pv[ 1] = v1;  VU[i].DatenVU->Pv[ 9] = v9 ; /* lisie- */
4178  VU[i].DatenVU->Pv[ 2] = v2;  VU[i].DatenVU->Pv[10] = v10; /* rung   */
4179  VU[i].DatenVU->Pv[ 3] = v3;  VU[i].DatenVU->Pv[11] = v11; /* des    */
4180  VU[i].DatenVU->Pv[ 4] = v4;  VU[i].DatenVU->Pv[12] = v12; /* Para-  */
4181  VU[i].DatenVU->Pv[ 5] = v5;  VU[i].DatenVU->Pv[13] = v13; /* meter- */
4182  VU[i].DatenVU->Pv[ 6] = v6;  VU[i].DatenVU->Pv[14] = v14; /* vektors*/
4183  VU[i].DatenVU->Pv[ 7] = v7;  VU[i].DatenVU->Pv[15] = v15;
4184 } /* END: func Vuauini() */
```

```
4185
4186 /* ************************************************************** */
4187 /* func Vnauini() : Automatische Initialisierung eines Versicherten  */
4188 /*                  Wird von  allen funcs fuer die vordefinierten    */
4189 /*                  Modelle verwendet.                               */
4190 /* ************************************************************** */
4191 func Vnauini(i,st,ap,al,vs1,vu1,vs2,vu2,vm,
4192             v0,v1,v2,v3,v4,v5,v6,v7,v8,v9,v10,v11,v12,v13,v14,v15 )
4193 int i;             /* Nummer des Versicherten, der initialisiert wird*/
4194 int st;            /* Nummer der Verhaltensregel, die VN ausfuehrt   */
4195 int ap;            /* Aktivierungsperiode des Versicherten           */
4196 int al;            /* Aktivierungslaeufe:alle < al: Al[j] = 1;sonst 0*/
4197 int vs1,vu1,vs2,vu2; /* Startwerte: Versicherer,Status in 2 Sparten  */
4198 float vm;          /* Startwert des Vermoegens des Versicherten      */
4199 float v0,v1,v2,v3,v4,v5,v6,v7,v8,v9,v10,v11,v12,v13,v14,v15; /* Die  */

4200                    /* Elemente des Parametervektors                  */
4201 {
4202 int j;                                /* Schleifenzaehler           */
4203  VN[i].DatenVN->Ac[1].st = st;        /* Verhaltensregel und ihr...  */
4204  VN[i].DatenVN->Ac[1].lt = 1;         /* Ausfuehrungszeitpunkt...    */
4205  for( j = 2; j <= MAXNLOG; j++ )      /* Rest des Aktionsvektors auf */
4206   {                                   /* {0} initialisieren...       */
4207    VN[i].DatenVN->Ac[j].lt = 0;
4208    VN[i].DatenVN->Ac[j].st = 0;
4209   }
4210  VN[i].DatenVN->Ap = ap;              /* Aktivierungsperiode = 1..   */
4211  for( j = 1; j <= SIMLAENGE; j++ )    /* Aktivierungslaeufe          */
4212   {                                   /* Beispiel: Vnauini(5,..,1,.):*/
4213    if ( j <= al)                      /* Versicherter 5 ist NUR im   */
4214     { VN[i].DatenVN->Al[j] = 1; }     /* ersten Lauf aktiv ...       */
4215     else
4216      { VN[i].DatenVN->Al[j] = 0; }
4217   }
4218  VN[i].DatenVN->Vr = st;              /* Zuordnung zu den Aggregaten */
4219  VN[i].DatenVN->Vk = vkrvn[st];       /* der Stufe II und III...     */
4220  VN[i].DatenVN->Rk[1].l.Vu[1]= vu1;   /* Startewerte Risiko 1: Status*/
4221  VN[i].DatenVN->Rk[1].l.Vs[1]= vs1;   /*                  Versicherer*/
4222  VN[i].DatenVN->Rk[2].k.Vu[1]= vu2;   /*             Risiko 2: Status*/
4223  VN[i].DatenVN->Rk[2].k.Vs[1]= vs2;   /*                  Versicherer*/
4224  VN[i].DatenVN->Vm[0] = vm;           /* Startvermoegen              */
4225  VN[i].DatenVN->Pv[ 0] = v0 ;  VN[i].DatenVN->Pv[ 8] = v8 ; /* Initi-*/
4226  VN[i].DatenVN->Pv[ 1] = v1 ;  VN[i].DatenVN->Pv[ 9] = v9 ; /* lisi- */
4227  VN[i].DatenVN->Pv[ 2] = v2 ;  VN[i].DatenVN->Pv[10] = v10; /* rung  */
4228  VN[i].DatenVN->Pv[ 3] = v3 ;  VN[i].DatenVN->Pv[11] = v11; /* des   */
4229  VN[i].DatenVN->Pv[ 4] = v4 ;  VN[i].DatenVN->Pv[12] = v12; /* Para- */
```

```
4230  VN[i].DatenVN->Pv[ 5] = v5 ;  VN[i].DatenVN->Pv[13] = v13; /* meter-*/
4231  VN[i].DatenVN->Pv[ 6] = v6 ;  VN[i].DatenVN->Pv[14] = v14; /*vektors*/
4232  VN[i].DatenVN->Pv[ 7] = v7 ;  VN[i].DatenVN->Pv[15] = v15;
4233 } /* END: func Vnauini() */
4234
4235 /* ************************************************************** */
4236 /* func Vdefmd1() : Vordefinitiertes Modell 1                     */
4237 /*                  Homogenes Angebot bei homogener Nachfrage      */
4238 /*                  Aufgerufen von func Mkbear()                   */
4239 /* ************************************************************** */
4240 func Vdefmd1() {
4241  int j;                                /* Schleifenzaehler         */
4242  Bavauin(100,100,0.0,0.0,0);/* In KEINER Periode, KEINEM Lauf Aen-   */
4243                             /*derungsschock, Suchkosten,Zins = 0.0   */
4244                             /* KEINE Wiederholungen                  */
4245  for( j = 1; j <= MAXVU; j++ )
4246   {
4247    Vuauini(j, 3, 1, 1, 0.0,0.0,0.0,0.0,0.0,0.0,  30.0,0.0,30.0,0.0,
4248            1.03,0.0 ,0.97,0.0 ,1.08,0.0 ,0.92,0.0 , /* Pramienreakt. */
4249            0.0 ,0.0 ,0.0 ,0.0 ,0.0 ,0.0 ,0.0 ,0.0 );/* Werbereaktion */
4250   }
4251  for( j = 1; j <= MAXVN; j++ )
4252   {
4253    Vnauini(j, 1, 1, 1, 0,0,0,0, 0.0,
4254            30.0, 0.0, 5.0, 0.0, 30.0, 0.0, 10.0, 0.0,  /* Schadenfkt.*/
4255             0.0, 0.0, 0.0, 0.0, 0.0 , 0.0, 0.0 , 0.0); /* VR-Paramet.*/
4256   }
4257  for( j = 0; j <= MAXVU; j++ ) { agvu1[j] = 0; } /* Alle Aggregat-   */
4258  for( j = 0; j <= 10;    j++ ) { agvu2[j] = 0; } /* vektoren auf Null*/
4259  for( j = 0; j <= 3;     j++ ) { agvu3[j] = 0; } /* setzen, damit bei*/
4260  for( j = 0; j <= 1;     j++ ) { agvu4[j] = 0; } /* wiederholter Mo- */
4261  for( j = 0; j <= MAXVN; j++ ) { agvn1[j] = 0; } /* dellwahl nicht   */
4262  for( j = 0; j <= 7;     j++ ) { agvn2[j] = 0; } /* die Aggregate des*/
4263  for( j = 0; j <= 3;     j++ ) { agvn3[j] = 0; } /* letzten Modells  */
4264  for( j = 0; j <= 1;     j++ ) { agvn4[j] = 0; } /* erzeugt werden   */
4265  agvu1[ 1] = 1; agvu1[17] = 1; /* Aggregaterzeugung:Versicherer: 1,17*/
4266  agvu2[ 3] = 1;                /* Alle VUs mit VR: Mark-Up I         */
4267  agvu3[ 2] = 1;                /* Alle VUs der VK Eigeninfo-regeln   */
4268  agvu4[ 1] = 1;                /* Alle VUs                           */
4269  agvn1[43] = 1;                /* Aggregaterzeugung:Versicherte: 43  */
4270  agvn2[ 1] = 1;                /* Alle VNs mit VR: Zufall I          */
4271  agvn3[ 1] = 1;                /* Alle VNs mit VK: Infofreie Regeln  */
4272  agvn4[ 1] = 1;                /* Alle VNs                           */
4273  clearScreen();
4274  printfAt( 1,14, "[ Homogenes Angebot bei homogener Nachfrage ]");
```

```
4275  printfAt( 5,15, "Modellkonfiguration:                          ");
4276  printfAt( 6,15, "Versicherer:         1 ... 25 : Mark Up I      ");
4277  printfAt( 7,15, "Versicherungsnehmer: 1 ... 200: Zufall I       ");
4278  printfAt( 8,15, "Besonderheiten:                                ");
4279  printfAt( 9,15, " - Versicherer starten mit P1[0]=P2[0] = 30.0   ");
4280  printfAt(10,15, " - Versicherer legen keine Werbeausgaben fest   ");
4281  printfAt(11,15, " - Der Parametervektor fuer die Aendungsschock- ");
4282  printfAt(12,15, "   reaktion ist: (0..0)                         ");
4283  printfAt(13,15, " - Die Anspruchsniveaureaktion ist in den bei-  ");
4284  printfAt(14,15, "   den Sparten unterschiedlich                  ");
4285  printfAt(15,15, " - Die Versicherungsnehmer haben fuer beide     ");
4286  printfAt(16,15, "   Risiken unterschiedliche Schadenmodelle      ");
4287  printfAt(17,15, "Erzeugte Aggregatdateien von:                   ");
4288  printfAt(18,15, "VU[1], VU[17], VU-Verhaltensregel 3, allen VUs  ");
4289  printfAt(19,15, "die Verhaltensregeln mit Eigeninformation haben,");
4290  printfAt(20,15, "allen VUs, VN[1], VN-Verhaltensregel 1, allen   ");
4291  printfAt(21,15, "VNs mit informationsfreien Verhaltensregeln und ");
4292  printfAt(22,15, "allen VNs                                       ");
4293  waitKey(); waitKey();
4294 } /* END: func Vdefmd1() */
4295
4296 /* ************************************************************** */
4297 /* func Vdefmd2() : Vordefinitiertes Modell 2                     */
4298 /*                  Homogenes Angebot bei heterogener Nachfrage   */
4299 /*                  Aufgerufen von func Mkbear()                  */
4300 /* ************************************************************** */
4301 func Vdefmd2() {
4302  int j;                                /* Schleifenzaehler         */
4303  Bavauin(100,100,0.0,0.0,0); /* In KEINER Periode, KEINEM Lauf Aen-  */
4304                              /* derungsschock, Zins, Suchkosten=0.0  */
4305                              /* KEINE Wiederholungen                 */
4306  for( j = 1; j <= MAXVU; j++ )
4307   {                                                /* Mark Up II    */
4308    Vuauini(j, 4, 1, 1, 0.0,0.0,0.0,0.0,0.0,0.0, 20.0,10.0,22.0,12.0,
4309            0.99,0.0 ,1.03,0.0 ,0.98,0.0 ,1.02,0.0 , /* Pramienreakt. */
4310            0.99,0.0 ,1.02,0.0 ,0.99,0.0 ,1.03,0.0 );/* Werbereaktion */
4311   }
4312  for( j = 1; j <= 100; j++ )
4313   {                                                /* Zufall II  */
4314    Vnauini(j, 2, 1, 1, 0,0,0,0, 0.0,
4315            27.0, 0.0, 5.0, 0.0, 23.0, 0.0, 10.0, 0.0,  /* Schadenfkt.*/
4316             0.0, 0.0, 0.0, 0.0, 0.0 , 0.0, 0.0 , 0.0); /* VR-Paramet.*/
4317   }
4318  for( j = 101; j <= MAXVN; j++ )
4319   {                                                /* Praeferenz */
```

```
4320    Vnauini(j, 3, 1, 1, 0,0,0,0, 0.0,
4321            23.0, 0.0, 5.0, 0.0, 27.0, 0.0,  5.0, 0.0,  /* Schadenfkt.*/
4322             0.5, 0.5, 0.5, 0.5,  0.0, 0.0,  0.0, 0.0); /* VR-Paramet.*/
4323   }
4324  for( j = 0; j <= MAXVU; j++ ) { agvu1[j] = 0; } /* Alle Aggregat-   */
4325  for( j = 0; j <= 10;    j++ ) { agvu2[j] = 0; } /* vektoren auf Null*/
4326  for( j = 0; j <= 3;     j++ ) { agvu3[j] = 0; } /* setzen, damit bei*/
4327  for( j = 0; j <= 1;     j++ ) { agvu4[j] = 0; } /* wiederholter Mo- */
4328  for( j = 0; j <= MAXVN; j++ ) { agvn1[j] = 0; } /* dellwahl nicht   */
4329  for( j = 0; j <= 7;     j++ ) { agvn2[j] = 0; } /* die Aggregate des*/
4330  for( j = 0; j <= 3;     j++ ) { agvn3[j] = 0; } /* letzten Modells  */
4331  for( j = 0; j <= 1;     j++ ) { agvn4[j] = 0; } /* erzeugt werden   */
4332  agvu1[ 4] = 1;                /* Aggregaterzeugung:Versicherer:  4  */
4333  agvu2[ 4] = 1;                /* Alle VUs mit VR: Mark-Up II        */
4334  agvu4[ 1] = 1;                /* Alle VUs                           */
4335  agvn1[56] = 1;                /* Aggregaterzeugung:Versicherte: 56  */
4336  agvn2[ 2] = 1;                /* Alle VNs mit VR: Zufall II         */
4337  agvn2[ 3] = 1;                /* Alle VNs mit VR: Praeferenz        */
4338  agvn4[ 1] = 1;                /* Alle VNs                           */
4339  clearScreen();
4340  printfAt( 1,14, "[ Homogenes Angebot bei heterogener Nachfrage ]");
4341  printfAt( 5,15, "Modellkonfiguration:                            ");
4342  printfAt( 6,15, "Versicherer:          1.. 25: Mark Up II         ");
4343  printfAt( 7,15, "Versicherungsnehmer:  1..100: Zufall II          ");
4344  printfAt( 8,15, "                    101..200: Praeferenz         ");
4345  printfAt( 9,15, "Besonderheiten:                                  ");
4346  printfAt(10,15, "- Die Praemienreaktionen der VUs sind in beiden  ");
4347  printfAt(11,15, "  Sparten unterschiedlich                        ");
4348  printfAt(12,15, "- Alle VUs haben die gleichen Startwerte         ");
4349  printfAt(13,15, "- VNs mit VR Zufall II haben groessere Schadener-");
4350  printfAt(14,15, "  wartungswerte bei Risiko 1, VNs mit VR Praefe- ");
4351  printfAt(15,15, "  renz haben groessere Schadenerwartung bei      ");
4352  printfAt(16,15, "  Risiko 2                                       ");
4353  printfAt(17,15, "- Nur die VNs mit der VR Praeferenz reagieren    ");
4354  printfAt(18,15, "  auf Werbeausgaben                              ");
4355  printfAt(19,15, "Erzeugte Aggregatdateien von:                    ");
4356  printfAt(20,15, "VU[4], allen VUs mit VR Mark Up II, allen VUs,   ");
4357  printfAt(21,15, "VN[56], allen VNs mit VR Zufall II und allen VNs ");
4358  printfAt(22,15, "mit VR Praeferenz, allen VNs                     ");
4359  waitKey(); waitKey();
4360 } /* END: func Vdefmd2() */
4361
4362 /* ************************************************************** */
4363 /* func Vdefmd3() : Vordefinitiertes Modell 3                     */
4364 /*                  Heterogenes Angebot bei homogener Nachfrage    */
```

```
4365 /*                    Aufgerufen von func Mkbear()                    */
4366 /* ************************************************************** */
4367 func Vdefmd3() {
4368  int j;                                /* Schleifenzaehler          */
4369  Bavauin(100,100,0.0,0.02,0); /* In KEINER Periode, KEINEM Lauf Aen- */
4370                            /* derungsschock, Zins = 2 Prozent,      */
4371                            /* KEINE Suchkosten und Wiederholungen   */
4372  for( j = 1; j <= 8; j++ )                          /* Mark Up III   */
4373   {
4374    Vuauini(j, 5, 1, 1, 0.0,0.0,0.06,0.0,0.0,0.06, 34.0, 0.0, 8.0, 0.0,
4375            1.008,0.0 ,0.99,0.0 ,1.008,0.0 ,0.99,0.0 , /* Pramienreakt. */
4376            0.0 ,0.0 ,0.0 ,0.0 ,0.0 ,0.0 ,0.0 ,0.0 );/* Werbereaktion */
4377   }
4378  for( j = 9; j <=16; j++ )                       /* Erwartungsschaden*/
4379   {
4380    Vuauini(j, 6, 1, 1, 0.0,0.0,0.0,0.0,0.0,0.0,   34.0, 0.0, 8.0, 0.0,
4381            1.01,0.0 ,1.0 ,0.0 ,1.01,0.0 ,1.00,0.0 , /* Pramienreakt. */
4382            0.0 ,0.0 ,0.0 ,0.0 ,0.0 ,0.0 ,0.0 ,0.0 );/* Werbereaktion */
4383   }
4384  for( j =17; j <=MAXVU; j++ )                         /* DUMPING      */
4385   {
4386    Vuauini(j, 7, 1, 1, 0.0,0.0,0.0,0.0,0.0,0.0,   34.0, 0.0, 8.0, 0.0,
4387            1.00,0.0 ,1.00,0.0 ,1.00,0.0 ,1.00,0.0 , /* Pramienreakt. */
4388            0.0 ,0.0 ,0.0 ,0.0 ,0.0 ,0.0 ,0.0 ,0.0 );/* Werbereaktion */
4389   }
4390  for( j = 1; j <= MAXVN; j++ )                   /* Zufall II         */
4391   {
4392    Vnauini(j, 2, 1, 1, 0,0, 0,0, 0.0,
4393            40.0, 0.0, 5.0, 0.0, 10.0, 0.0,  1.0, 0.0,  /* Schadenfkt.*/
4394             0.0, 0.0, 0.0, 0.0,  0.0, 0.0,  0.0, 0.0); /* VR-Paramet.*/
4395   }
4396  for( j = 0; j <= MAXVU; j++ ) { agvu1[j] = 0; } /* Alle Aggregat-   */
4397  for( j = 0; j <= 10;    j++ ) { agvu2[j] = 0; } /* vektoren auf Null*/
4398  for( j = 0; j <= 3;     j++ ) { agvu3[j] = 0; } /* setzen, damit bei*/
4399  for( j = 0; j <= 1;     j++ ) { agvu4[j] = 0; } /* wiederholter Mo- */
4400  for( j = 0; j <= MAXVN; j++ ) { agvn1[j] = 0; } /* dellwahl nicht   */
4401  for( j = 0; j <= 7;     j++ ) { agvn2[j] = 0; } /* die Aggregate des*/
4402  for( j = 0; j <= 3;     j++ ) { agvn3[j] = 0; } /* letzten Modells  */
4403  for( j = 0; j <= 1;     j++ ) { agvn4[j] = 0; } /* erzeugt werden   */
4404  agvu1[12] = 1;                /* Aggregaterzeugung:Versicherer:12   */
4405  agvu2[ 5] = 1;                /* Alle VUs mit VR: Mark-Up III       */
4406  agvu2[ 6] = 1;                /* Alle VUs mit VR: Erwartungsschaden */
4407  agvu2[ 7] = 1;                /* Alle VUs mit VR: Dumping           */
4408  agvn3[ 1] = 1;                /* Alle VUs in VK:1 (Info.freie VR)   */
4409  agvu4[ 1] = 1;                /* Alle VUs                           */
```

```
4410  agvn1[99] = 1;                /* Aggregaterzeugung:Versicherte: 99  */
4411  agvn2[ 2] = 1;                /* Alle VNs mit VR: Zufall II         */
4412  clearScreen();
4413  printfAt( 1,14, "[ Heterogenes Angebot bei homogener Nachfrage ]");
4414  printfAt( 5,15, "Modellkonfiguration:                            ");
4415  printfAt( 6,15, "Versicherer:          1..  8: Mark Up III       ");
4416  printfAt( 7,15, "                      9.. 16: Erwartungsschaden ");
4417  printfAt( 8,15, "                     17.. 25: Dumping           ");
4418  printfAt( 9,15, "Versicherungsnehmer:  1..200: Zufall II         ");
4419  printfAt(10,15, "Besonderheiten:                                 ");
4420  printfAt(11,15, "- Die VUs mit VR Mark Up III streben einen      ");
4421  printfAt(12,15, "  ueberdurchschnittlichen Marktanteil an        ");
4422  printfAt(13,15, "- Keine VU betreibt Werbung aber die verschie-  ");
4423  printfAt(14,15, "  denen VU-VRs haben gleiche Startwerte und     ");
4424  printfAt(15,15, "  unterschiedliche Preamienreaktionparameter    ");
4425  printfAt(16,15, "- Alle VUs[DUMPING] bieten immer genau die      ");
4426  printfAt(17,15, "  Minimalpraemie der Vorperiode an ohne Auf-    ");
4427  printfAt(18,15, "  oder Abschlaege                               ");
4428  printfAt(19,15, "Erzeugte Aggregatdateien von:                   ");
4429  printfAt(20,15, "Versicherung[12], alle VU-VR: Mark Up III, VU-VR");
4430  printfAt(21,15, "Erwartungsschaden, VU-VR:Dumping, VU-VK:Eigen-  ");
4431  printfAt(22,15, "information, VU-VK: Fremdinformation, alle VUs  ");
4432  printfAt(23,15, "VN[99] und alle VN                              ");
4433  waitKey(); waitKey();
4434 } /* END: func Vdefmd3() */
4435
4436 /* ************************************************************** */
4437 /* func Vdefmd4() : Vordefinitiertes Modell 4                     */
4438 /*                  Einfacher Aenderungsschock, homogene Nachfrage  */
4439 /*                  Aufgerufen von func Mkbear()                   */
4440 /* ************************************************************** */
4441 func Vdefmd4() {
4442  int j;                              /* Schleifenzaehler          */
4443  Bavauin( 50,0, 0.0, 0.02,1); /* Ab Periode 50 in jedem Lauf:  Aen-  */
4444                               /* derungsschock, Zins = 2 Prozent ,   */
4445                               /* KEINE Suchkosten, Wiederholungen    */
4446  for( j = 1; j <= MAXVU; j++ )
4447   {                                        /* Mark Up I           */
4448    Vuauini(j, 3, 1,100,0.0,0.0,0.05,0.0,0.0,0.05, 30.0,0.0,30.0,0.0,
4449            1.03,1.08,0.97,0.8 ,1.01,1.10,0.98,0.90, /* Pramienreakt. */
4450            1.01,1.03,0.98,0.97,1.03,1.08,0.97,0.8 );/* Werbereaktion */
4451   }
4452  for( j = 1; j <= MAXVN; j++ )
4453   {
4454    Vnauini(j, 3, 1,100,0,0,0,0, 0.0,          /* Praeferenz         */
```

```
4455            30.0,15.0,10.0,10.0, 30.0,55.0, 15.0,15.0,  /* Schadenfkt.*/
4456             0.5, 0.7, 0.5, 0.7,  0.0, 0.0,  0.0, 0.0); /* VR-Paramet.*/
4457   }
4458  for( j = 0; j <= MAXVU; j++ ) { agvu1[j] = 0; } /* Alle Aggregat-   */
4459  for( j = 0; j <= 10;    j++ ) { agvu2[j] = 0; } /* vektoren auf Null*/
4460  for( j = 0; j <= 3;     j++ ) { agvu3[j] = 0; } /* setzen, damit bei*/
4461  for( j = 0; j <= 1;     j++ ) { agvu4[j] = 0; } /* wiederholter Mo- */
4462  for( j = 0; j <= MAXVN; j++ ) { agvn1[j] = 0; } /* dellwahl nicht   */
4463  for( j = 0; j <= 7;     j++ ) { agvn2[j] = 0; } /* die Aggregate des*/
4464  for( j = 0; j <= 3;     j++ ) { agvn3[j] = 0; } /* letzten Modells  */
4465  for( j = 0; j <= 1;     j++ ) { agvn4[j] = 0; } /* erzeugt werden   */
4466  agvu1[  7] = 1;agvu1[19] = 1; /* Aggregaterzeugung:Versicherer:7,19 */
4467  agvu2[  5] = 1;               /* Alle VUs mit VR: Mark-Up III       */
4468  agvu4[  1] = 1;               /* Alle VUs                           */
4469  agvn1[119] = 1;               /* Aggregaterzeugung:Versicherte: 119 */
4470  agvn2[  3] = 1;               /* Alle VNs mit VR: Praeferenz        */
4471  agvn4[  1] = 1;               /* Alle VNs                           */
4472  clearScreen();
4473  printfAt( 1,13, "[ Einfacher Aenderungsschock, homogene Nachfrage ]");
4474  printfAt( 5,15, "Modellkonfiguration:                            ");
4475  printfAt( 6,15, "Versicherer:          1..25 : Mark Up I         ");
4476  printfAt( 7,15, "                     mit Aenderungsschock       ");
4477  printfAt( 8,15, "Versicherungsnehmer:  1..200: Praeferenz        ");
4478  printfAt( 9,15, "                     mit Aenderungsschock       ");
4479  printfAt(10,15, "Besonderheiten:                                 ");
4480  printfAt(11,15, "- Die VUs mit VR Mark Up I  streben ausge-      ");
4481  printfAt(12,15, "  glichene Reserven in beiden Sparten an        ");
4482  printfAt(13,15, "- Auch die Werbeaufwandsparameter sind durch den");
4483  printfAt(14,15, "  Aenderungsschock beeinflusst                  ");
4484  printfAt(15,15, "- Praemien und Schaeden reagieren in beiden     ");
4485  printfAt(16,15, "  Sparten unterschiedlich auf den Schock        ");
4486  printfAt(17,15, "- Der Schock beeinflusst die Statuswahlfunktion ");
4487  printfAt(18,15, "  und die Schadenfunktion beider Risiken        ");
4488  printfAt(19,15, "Erzeugte Aggregatdateien von:                   ");
4489  printfAt(20,15, "VU[7] und VU[19], alle VUs mit der Verhaltens-  ");
4490  printfAt(21,15, "regel: Mark Up I,alle VUs,VUs,VN[119] und alle  ");
4491  printfAt(22,15, "VN mit der Verhaltensregel: Praeferenz          ");
4492  waitKey(); waitKey();
4493 } /* END: func Vdefmd4() */
4494
4495 /* ************************************************************** */
4496 /* func Vdefmd5() : Vordefinitiertes Modell 5                     */
4497 /*                  Einfacher Aktivierungsschock, homogenes Angebot  */
4498 /*                  Aufgerufen von func Mkbear()                   */
4499 /* ************************************************************** */
```

```
4500 func Vdefmd5() {
4501  int j;                                /* Schleifenzaehler         */
4502  Bavauin(100,100,1.0,0.03,1
4503  ); /* In KEINER Periode, KEINEM Lauf Aen- */
4504                              /* derungsschock, Kosten pro Praemien- */
4505                              /* information = 1.0, Zins = 3 Prozent */
4506                              /* Wiederholungen                      */
4507  for( j = 1; j <= MAXVU; j++ )
4508   {                                          /* Erwartungsschaden   */
4509    Vuauini(j, 6, 1, 100, 0.0,0.0,0.0,0.0,0.0,0.0, 40.0,0.0,35.0,0.0,
4510            1.03,0.0 ,0.97,0.0 ,1.05,0.0 ,0.95,0.0 , /* Pramienreakt. */
4511            0.0 ,0.0 ,0.0 ,0.0 ,0.0 ,0.0 ,0.0 ,0.0 );/* Werbereaktion */
4512   }
4513  for( j = 1;   j <= 100  ; j++ )
4514   {
4515    Vnauini(j, 2, 1, 100, 0,0,0,0, 0.0,        /* Zufall II           */
4516            30.0, 0.0,20.0, 0.0, 30.0, 0.0,20.0 , 0.0,  /* Schadenfkt.*/
4517             0.5, 0.0, 0.5, 0.0, 0.0 , 0.0, 0.0 , 0.0); /* VR-Paramet.*/
4518   }
4519  for( j = 101; j <= MAXVN; j++ )
4520   {
4521    Vnauini(j, 1,50, 100, 0,0,0,0, 0.0,        /* Zufall I            */
4522            50.0, 0.0, 5.0, 0.0, 50.0, 0.0,  5.0, 0.0,  /* Schadenfkt.*/
4523             0.0, 0.0, 0.0, 0.0, 0.0 , 0.0, 0.0 , 0.0); /* VR-Paramet.*/
4524   }
4525  for( j = 0; j <= MAXVU; j++ ) { agvu1[j] = 0; } /* Alle Aggregat-   */
4526  for( j = 0; j <= 10;    j++ ) { agvu2[j] = 0; } /* vektoren auf Null*/
4527  for( j = 0; j <= 3;     j++ ) { agvu3[j] = 0; } /* setzen, damit bei*/
4528  for( j = 0; j <= 1;     j++ ) { agvu4[j] = 0; } /* wiederholter Mo- */
4529  for( j = 0; j <= MAXVN; j++ ) { agvn1[j] = 0; } /* dellwahl nicht   */
4530  for( j = 0; j <= 7;     j++ ) { agvn2[j] = 0; } /* die Aggregate des*/
4531  for( j = 0; j <= 3;     j++ ) { agvn3[j] = 0; } /* letzten Modells  */
4532  for( j = 0; j <= 1;     j++ ) { agvn4[j] = 0; } /* erzeugt werden   */
4533  for( j = 1; j <= MAXVU; j++ ) { agvu1[j] = 1; } /* Alle Einzel-VUs  */
4534  agvu4[  1] = 1;               /* Alle VUs                          */
4535  agvn2[  1] = 1;               /* Alle VNs mit VR: Zufall I         */
4536  agvn2[  2] = 1;               /* Alle VNs mit VR: Zufall II        */
4537  agvn4[  1] = 1;               /* Alle VNs                          */
4538  clearScreen();
4539  printfAt( 1,13, "[ Einfacher Aktivierungsschock, homogenes Angebot ]");
4540  printfAt( 5,15, "Modellkonfiguration:                            ");
4541  printfAt( 6,15, "Versicherer:          1..25 : Erwartungsschaden ");
4542  printfAt( 7,15, "Versicherungsnehmer:  1..100: Zufall II         ");
4543  printfAt( 8,15, "                    101..200: Zufall I, t > 50  ");
4544  printfAt( 9,15, "                    mit Aktivierungsschock      ");
```

```
4545  printfAt(10,15, "Besonderheiten:                                  ");
4546  printfAt(11,15, "- Die VUs haben fuer beide Sparten unterschied- ");
4547  printfAt(12,15, "  liche Startwerte in den Praemien              ");
4548  printfAt(13,15, "- Keine VU betreibt Werbung                     ");
4549  printfAt(14,15, "- Es findet nur ein Aktivierungsschock auf der  ");
4550  printfAt(15,15, "  Nachfrageseite statt, kein Aenderungsschock   ");
4551  printfAt(16,15, "- Die VNs[101..200], die in t>50 aktiviert wer- ");
4552  printfAt(17,15, "  den, haben andere Schadenparamter als die     ");
4553  printfAt(18,15, "  VNs[1..100]                                   ");
4554  printfAt(19,15, "Erzeugte Aggregatdateien von:                   ");
4555  printfAt(20,15, "Alle VUs (Aggregat I), alle VUs (Aggregat IV),  ");
4556  printfAt(21,15, "alle VNs mit VR[Zufall I] und VR[Zufall II] und ");
4557  printfAt(22,15, "alle VNs (Aggregat IV)                          ");
4558  waitKey(); waitKey();
4559 } /* END: func Vdefmd5() */
4560
4561 /* ************************************************************** */
4562 /* func Vdefmd6() : Vordefinitiertes Modell 6                     */
4563 /*                  Komplexer Marktschock: Die Wiedervereinigung  */
4564 /*                  Aufgerufen von func Mkbear()                  */
4565 /* ************************************************************** */
4566 func Vdefmd6() {
4567  int j;                                          /*Schleifenzaehler */
4568  Bavauin( 50,0, 0.8, 0.02,1); /* Ab Periode 50 in jedem Lauf:  Aen-  */
4569                               /* derungsschock, Kosten pro Praemien- */
4570                               /* information = 0.8, Zins = 2 Prozent */
4571                               /* Wiederholungen sind erlaubt         */
4572  for ( j =  1; j <=  2   ; j++)
4573   {                                              /* Zufall I        */
4574    Vuauini(j, 1,1,100, 0.0,0.0,0.0,0.0,0.0,0.0,  40.0,0.0,40.0,0.0,
4575            60.0,70.0,50.0,40.0,20.0,20.0,20.0,20.0, /*Pramienreaktion*/
4576            0.0 ,0.0 ,0.0 ,0.0 ,0.0 ,0.0 ,0.0 ,0.0 );/*Werbereaktion  */
4577   }
4578  for ( j =  3; j <=  4   ; j++)
4579   {                                               /* Zufall II      */
4580    Vuauini(j, 2,1,100, 0.0,0.0,0.0,0.0,0.0,0.0,  40.0,10.0,40.0,10.0,
4581            30.0,40.0,5.0 ,7.0 ,30.0,40.0,5.0 ,8.0 , /*Pramienreaktion*/
4582            10.0,10.0,0.0 ,0.0 ,10.0,10.0,0.0 ,0.0 );/*Werbereaktion  */
4583   }
4584  for ( j =  5; j <=  7   ; j++)
4585   {                                              /* Mark Up I       */
4586    Vuauini(j, 3,1,100, 0.0,0.0,0.0,0.0,0.0,0.0,  40.0,10.0,40.0,10.0,
4587            1.03,1.1 ,0.97,0.9 ,1.04,1.05,0.97,0.99, /*Pramienreaktion*/
4588            1.03,1.1 ,0.97,0.9 ,1.08,1.01,0.92,0.99);/*Werbereaktion  */
4589   }
```

```
4590  for ( j =  8; j <= 10   ; j++)
4591   {                                                /* Mark Up II      */
4592    Vuauini(j, 4,1,100, 0.0,2.0,0.0,0.2,0.0,0.0,  40.0,10.0,40.0,10.0,
4593            1.008,1.01,0.99,0.98,1.008,1.01,0.99,0.98,/*Pramienreaktion*/
4594            1.008,1.01,0.97,1.0 ,1.05,1.0 ,0.95,1.0 );/*Werbereaktion  */
4595   }
4596  for ( j = 11; j <= 13   ; j++)
4597   {                                                /* Mark Up III     */
4598    Vuauini( j,5,1,100, 0.0,0.0,0.04,0.0,0.0,0.04,  40.0,10.0,40.0,10.0,
4599            1.01,1.02,0.99,0.98,1.008,1.01,0.99,0.98,/*Pramienreaktion*/
4600            1.02,1.03,0.98,1.01,1.02,1.03,0.98,1.0 );/*Werbereaktion  */
4601   }
4602                                /* VU[14] = Allianz: Erwartungsschaden */
4603  Vuauini( 14,6,1,100, 0.0,0.0,0.0,0.0,0.0,0.0,  40.0,10.0,40.0,10.0,
4604          1.02,1.03,0.98,0.99,1.02,1.01,0.98,0.99, /*Pramienreaktion*/
4605          1.0 ,1.06,1.03,1.06,1.0 ,1.06,1.03,1.06);/*Werbereaktion  */
4606  for ( j = 15; j <= 16   ; j++)
4607   {                                                /*Erwartungsschaden*/
4608    Vuauini( j,6,1,100, 0.0,0.0,0.0,0.0,0.0,0.0,  40.0,10.0,40.0,10.0,
4609            1.02,1.03,0.98,0.99,1.02,1.01,0.98,0.99, /*Pramienreaktion*/
4610            1.05,1.08,0.95,0.97,1.09,1.06,0.95,0.97);/*Werbereaktion  */
4611   }
4612  for ( j = 17; j <= 19   ; j++)
4613   {                                                /* Dumping         */
4614    Vuauini( j,7,1,100, 0.0,0.0,0.0,0.0,0.0,0.0,  40.0, 0.0,40.0, 0.0,
4615            1.07,1.04,1.07,1.04,1.07,1.04,1.07,1.04, /*Pramienreaktion*/
4616            0.0 ,0.0 ,0.0 ,0.0 ,0.0 ,0.0 ,0.0 ,0.0 );/* Werbereaktion */
4617   }
4618  for ( j = 20; j <= 22   ; j++)
4619   {                                                /* Durchschnitt    */
4620    Vuauini( j,8,1,100, 0.0,0.0,0.0,0.0,0.0,0.0,  40.0,10.0,40.0,10.0,
4621            0.0 ,0.0 ,1.0 ,1.0 ,0.0 ,0.0 ,1.0 ,1.0 ,/*Pramienreaktion */
4622            0.0 ,0.0 ,1.0 ,1.0 ,0.0 ,0.0 ,1.0 ,1.0 );/* Werbereaktion */
4623   }
4624  for ( j = 23; j <= MAXVU; j++)
4625   {                                                /* Angriff         */
4626    Vuauini( j,9,1,100, 0.0,0.0,0.0,0.0,0.0,0.0,  40.0,0.0,40.0,0.0,
4627          0.0 ,0.0 ,0.99,0.95,0.0 ,0.0 ,0.99,0.95, /* Pramienreaktion */
4628          0.0 ,0.0 ,0.0 ,0.0 ,0.0 ,0.0 ,0.0 ,0.0 );/* Werbereaktion   */
4629   }
4630  for( j = 1;   j <= 15  ; j++ )
4631   {Vnauini( j,1,1,100, 0,0,0,0, 0.0,          /* Zufall I            */
4632            30.0,30.0, 5.0, 5.0, 30.0,30.0, 5.0 , 5.0,  /* Schadenfkt.*/
4633             0.0, 0.0, 0.0, 0.0, 0.0 , 0.0, 0.0 , 0.0); /* VR-Paramet.*/
4634   }
```

```
4635  for( j = 16;  j <= 30  ; j++ )
4636   {Vnauini( j,2,1,100, 0,0,0,0, 0.0,          /* Zufall II           */
4637            30.0,30.0, 5.0, 5.0, 30.0,30.0, 5.0 , 5.0,  /* Schadenfkt.*/
4638             0.5, 0.5, 0.5, 0.5, 0.0 , 0.0, 0.0 , 0.0); /* VR-Paramet.*/
4639   }
4640  for( j = 31;  j <= 60  ; j++ )
4641   {Vnauini( j,3,1,100, 0,0,0,0, 0.0,          /* Praeferenz          */
4642            30.0,30.0, 5.0, 5.0, 30.0,30.0,  5.0, 5.0,  /* Schadenfkt.*/
4643             0.3, 0.3, 0.3, 0.3,  0.0, 0.0,  0.0, 0.0); /* VR-Paramet.*/
4644   }
4645  for( j = 61;  j <= 90  ; j++ )
4646   {Vnauini( j,4,1,100, 0,0,0,0, 0.0,          /* Totale Erinnerung   */
4647            30.0,30.0, 5.0, 5.0, 30.0,30.0,  5.0, 5.0,  /* Schadenfkt.*/
4648             0.3, 0.3, 0.3, 0.3,  0.0, 0.0,  0.0, 0.0); /* VR-Paramet.*/
4649   }
4650  for( j = 91;  j <= 120 ; j++ )
4651   {Vnauini( j,5,1,100, 0,0,0,0, 0.0,          /* Suche               */
4652            30.0,30.0, 5.0, 5.0, 30.0,30.0,  5.0, 5.0,  /* Schadenfkt.*/
4653             0.3, 0.3, 0.3, 0.3,  8.0,10.0,  8.0,10.0); /* VR-Paramet.*/
4654   }
4655  for( j =121;  j <= 150 ; j++ )
4656   {Vnauini( j,6,1,100, 0,0,0,0, 0.0,          /* Beste Information   */
4657            30.0,30.0, 5.0, 5.0, 30.0,30.0,  5.0, 5.0,  /* Schadenfkt.*/
4658             0.5, 0.5, 0.5, 0.5,  0.0, 0.0,  0.0, 0.0); /* VR-Paramet.*/
4659   }
4660  for( j =151;  j <= 190 ; j++ )
4661   {Vnauini( j,3,50,100, 0, 0,0, 0, 0.0,       /* Praeferenz          */
4662            50.0,50.0,15.0,15.0, 50.0,50.0, 15.0,15.0,  /* Schadenfkt.*/
4663             0.9, 0.9, 0.9, 0.9,  0.0, 0.0,  0.0, 0.0); /* VR-Paramet.*/
4664   }    /* Die aktivierten VNs gehen zunaechst alle zur Allianz       */
4665  for( j =191;  j<= MAXVN; j++ ) /* ..und haben andere Schadenparamtr.*/
4666   {Vnauini( j,2,50,100, 0, 0,0, 0, 0.0,       /* Zufall II           */
4667            50.0,50.0,15.0,15.0, 50.0,50.0, 15.0,15.0,  /* Schadenfkt.*/
4668             0.9, 0.9, 0.9, 0.9,  0.0, 0.0,  0.0, 0.0); /* VR-Paramet.*/
4669   }
4670  for( j = 0; j <= MAXVU; j++ ) { agvu1[j] = 0; } /* Alle Aggregat-   */
4671  for( j = 0; j <= 10;    j++ ) { agvu2[j] = 0; } /* vektoren auf Null*/
4672  for( j = 0; j <= 3;     j++ ) { agvu3[j] = 0; } /* setzen, damit bei*/
4673  for( j = 0; j <= 1;     j++ ) { agvu4[j] = 0; } /* wiederholter Mo- */
4674  for( j = 0; j <= MAXVN; j++ ) { agvn1[j] = 0; } /* dellwahl nicht   */
4675  for( j = 0; j <= 7;     j++ ) { agvn2[j] = 0; } /* die Aggregate des*/
4676  for( j = 0; j <= 3;     j++ ) { agvn3[j] = 0; } /* letzten Modells  */
4677  for( j = 0; j <= 1;     j++ ) { agvn4[j] = 0; } /* erzeugt werden   */
4678  agvu1[ 14] = 1;               /* Aggregaterzeugung:VU[14] = Allianz */
4679  for( j = 1; j <= 9; j++ ) { agvu2[j] = 1; } /* Alle VU-Aggregate der*/
```

```
4680  for( j = 1; j <= 3; j++ ) { agvu3[j] = 1; } /* Stufe II und III     */
4681  agvu4[  1] = 1;               /*                   Alle Versicherer */
4682  for( j = 1; j <= 6; j++ ) { agvn2[j] = 1; } /* Alle Aggregate der   */
4683  for( j = 1; j <= 3; j++ ) { agvn3[j] = 1; } /* Stufe II und III     */
4684  agvn4[1] = 1;                 /*          Alle Versicherungsnehmer  */
4685  clearScreen();
4686  printfAt( 1,13, "[ Komplexer Marktschock: Die Wiedervereinigung ]");
4687  printfAt( 5,15, "Modellkonfiguration:                            ");
4688  printfAt( 6,15, "Versicherer:                                    ");
4689  printfAt( 7,15, "   1..  2: Zufall I    14..16: Erwartungsschaden");
4690  printfAt( 8,15, "   3..  4: Zufall II   17..19: Dumping         ");
4691  printfAt( 9,15, "   5..  7: Mark Up I   20..22: Durchschnitt    ");
4692  printfAt(10,15, "   8.. 10: Mark Up II  23..25: Angriff         ");
4693  printfAt(11,15, "  11.. 13: Mark Up III                         ");
4694  printfAt(12,15, "Versicherungssnehmer:                          ");
4695  printfAt(13,15, "   1.. 15: Zufall I                             ");
4696  printfAt(14,15, "  16.. 30: Zufall II                            ");
4697  printfAt(15,15, "  31.. 60: Praeferenz                           ");
4698  printfAt(16,15, "  61.. 90: Totale Erinnerung                    ");
4699  printfAt(17,15, "  91..120: Suche                                ");
4700  printfAt(18,15, " 121..150: Beste Information                    ");
4701  printfAt(19,15, " 151..180: Praeferenz        , t > 50           ");
4702  printfAt(20,15, " 181..200: Zufall II,        , t > 50           ");
4703  printfAt(21,15, "Aenderungsschock: bei VUs und VNs in t = 50     ");
4704  printfAt(22,15, "Weitere Informationen: siehe Kapitel 5.4. ff.   ");
4705  waitKey(); waitKey();
4706 } /* END: func Vdefmd6() */
4707
4708 /* ************************************************************** */
4709 /* func Erstvu()  : Versicherer erstellen                         */
4710 /*                                                                */
4711 /*                  Aufgerufen von func Moderst()                 */
4712 /* ************************************************************** */
4713 func Erstvu() {
4714  int j,ende;                          /* Lokale Variable vereinbaren*/
4715  int i;                               /* Schleifenzaehler           */
4716  float a0,a1,b0,b1,c0,c1,d0,d1,e0,e1,f0,f1,g0,g1,h0,h1,
4717        pr1t1,wa1t1,pr2t1,wa2t1,an1,an2;
4718  int st,ap,al;
4719 do { /* while ( ende != 1) */
4720  clearScreen();                       /* Maske aufbauen             */
4721  box( 0, 0, 8,80); box( 8, 0, 8,80); box(16, 0, 8,80);
4722  box( 0, 0, 3,80); box( 8, 0, 2,80); box(16, 0, 2,80);
4723  box( 0, 0,24,40); box( 0,40,24,40);
4724  printfAt( 1,25, "[ Versicherer konfigurieren ]");
```

```
4725  printfAt( 2,10, "Versicherer          : ");
4726  printfAt( 2,50, "Startwerte d. Sparten: ");
4727  printfAt( 9,10, "Aktivierungsschockparamter: ");
4728  printfAt( 9,50, "Anspruchsniveaus     : ");
4729  printfAt(17,10, "Vor Aenderungsschock : ");
4730  printfAt(17,50, "Nach Aenderungsschock: ");
4731  printfAt( 4,10, "VR[                  ]");
4732  printfAt( 5,10, "Verhaltensregelklasse: ");
4733  printfAt( 6,10, "Aggregatdatei(0=nein): ");
4734  printfAt( 4,50, "Praemie, Sparte 1    : ");
4735  printfAt( 5,50, "Werbung, Sparte 1    : ");
4736  printfAt( 6,50, "Praemie, Sparte 2    : ");
4737  printfAt( 7,50, "Werbung, Sparte 2    : ");
4738  printfAt(11,10, "Aktivierungslauf     : ");
4739  printfAt(12,10, "Aktivierungsperiode  : ");
4740  printfAt(11,50, "Fuer Sparte 1        : ");
4741  printfAt(12,50, "Fuer Sparte 2        : ");
4742  printfAt(19,10, "a  :              e  : ");
4743  printfAt(20,10, "b  :              f  : ");
4744  printfAt(21,10, "c  :              g  : ");
4745  printfAt(22,10, "d  :              h  : ");
4746  printfAt(19,50, "a* :              e* : ");
4747  printfAt(20,50, "b* :              f* : ");
4748  printfAt(21,50, "c* :              g* : ");
4749  printfAt(22,50, "d* :              h* : ");
4750  do {                              /* Zu bearbeitenden VU erfragen    */
4751      printfAt( 2,33,"[    ]");scanfAt( 2,34,"%d",&j);
4752     } while ( ( j < 1 ) || ( j > MAXVU) );
4753  st    = VU[ j ].DatenVU->Ac[1].st;         /* Daten auslesen        */
4754  ap    = VU[ j ].DatenVU->Ap;
4755  for( i = SIMLAENGE ; i >= 1 ; i--)   /* Aktivierungslaeufe          */
4756   { if (VU[j].DatenVU->Al[i] == 0) { al = i - 1; } }
4757  pr1t1 = VU[ j ].DatenVU->Sp[1].l.Pr[1];
4758  pr2t1 = VU[ j ].DatenVU->Sp[2].k.Pr[1];
4759  wa1t1 = VU[ j ].DatenVU->Sp[1].l.Wa[1];
4760  wa2t1 = VU[ j ].DatenVU->Sp[2].k.Wa[1];
4761  switch(st) /* ... nach der Abhaengigkeit der Verhaltensregel...     */
4762   {
4763    case 3: /* Anspruchsniveau fuer MARK-UP I, II oder III auslesen...*/
4764       an1 = VU[j].DatenVU->A1[1]; an2 = VU[j].DatenVU->A2[1];
4765     break;
4766    case 4:
4767       an1 = VU[j].DatenVU->A1[2]; an2 = VU[j].DatenVU->A2[2];
4768     break;
4769    case 5:
```

```
4770       an1 = VU[j].DatenVU->A1[3]; an2 = VU[j].DatenVU->A2[3];
4771     break;
4772    default:
4773       an1 = 0.0                 ; an2 = 0.0                 ;
4774     break;
4775   } /* END: switch(st) */
4776  a0 = VU[ j ].DatenVU->Pv[0 ]; a1 = VU[ j ].DatenVU->Pv[1 ];
4777  b0 = VU[ j ].DatenVU->Pv[2 ]; b1 = VU[ j ].DatenVU->Pv[3 ];
4778  c0 = VU[ j ].DatenVU->Pv[4 ]; c1 = VU[ j ].DatenVU->Pv[5 ];
4779  d0 = VU[ j ].DatenVU->Pv[6 ]; d1 = VU[ j ].DatenVU->Pv[7 ];
4780  e0 = VU[ j ].DatenVU->Pv[8 ]; e1 = VU[ j ].DatenVU->Pv[9 ];
4781  f0 = VU[ j ].DatenVU->Pv[10]; f1 = VU[ j ].DatenVU->Pv[11];
4782  g0 = VU[ j ].DatenVU->Pv[12]; g1 = VU[ j ].DatenVU->Pv[13];
4783  h0 = VU[ j ].DatenVU->Pv[14]; h1 = VU[ j ].DatenVU->Pv[15];
4784                                          /* Daten ausgeben          */
4785  printfAt( 4, 13,"%18s" , vrvunm[st]); /* Voller Name der VR        */
4786  printfAt( 5, 34,"%1d"  , vkrvu[st] );
4787  printfAt( 6, 34,"%1d"  , agvu1[j]  );
4788  printfAt( 4, 73,"%3.1f", pr1t1     );
4789  printfAt( 5, 73,"%3.1f", wa1t1     );
4790  printfAt( 6, 73,"%3.1f", pr2t1     );
4791  printfAt( 7, 73,"%3.1f", wa2t1     );
4792  switch(st) /*... nach der Abhaengigkeit der Verhaltensregel...      */
4793   {
4794    case 3: /* Anspruchsniveau fuer MARK-UP I                        */
4795     printfAt( 9,50, "VR[ MARK-UP I   ] ");
4796     printfAt(11,74, "%3.2f", an1); printfAt(12,74, "%3.2f", an2);
4797     break;
4798    case 4:
4799     printfAt( 9,50, "VR[ MARK-UP II  ] ");

4800     printfAt(11,74, "%3.2f", an1); printfAt(12,74, "%3.2f", an2);
4801     break;
4802    case 5:
4803     printfAt( 9,50, "VR[ MARK-UP III ] ");
4804     printfAt(11,74, "%3.2f", an1); printfAt(12,74, "%3.2f", an2);
4805     break;
4806    default:
4807     printfAt( 9,50, "[Kein Anspruchsniveau ] ");
4808     printfAt(11,50, "                            ");
4809     printfAt(12,50, "                            ");
4810     break;
4811   } /* END: switch(st) */
4812  printfAt(11,34,"%3d",al);
4813  printfAt(12,34,"%3d",ap);
4814  printfAt(19,15,"%3.2f",a0); printfAt(19,33,"%3.2f",e0);
```

```
4815  printfAt(20,15,"%3.2f",b0); printfAt(20,33,"%3.2f",f0);
4816  printfAt(21,15,"%3.2f",c0); printfAt(21,33,"%3.2f",g0);
4817  printfAt(22,15,"%3.2f",d0); printfAt(22,33,"%3.2f",h0);
4818  printfAt(19,55,"%3.2f",a1); printfAt(19,73,"%3.2f",e1);
4819  printfAt(20,55,"%3.2f",b1); printfAt(20,73,"%3.2f",f1);
4820  printfAt(21,55,"%3.2f",c1); printfAt(21,73,"%3.2f",g1);
4821  printfAt(22,55,"%3.2f",d1); printfAt(22,73,"%3.2f",h1);
4822  do {                    /* Neue Parameter abfragen und  pruefen ... */
4823      printfAt( 4,34,"[    ]");
4824      scanfAt(  4,35, "%d", &st);
4825     } while ( ( st < 1 ) || ( st > 10 ) );
4826  printfAt( 4, 13,"%18s" , vrvunm[st]); /* Voller Name der VR         */
4827  printfAt( 5, 34,"%1d"  , vkrvu[st] );
4828  do {
4829      printfAt( 6,34,"[    ]"); scanfAt(  6,35, "%d", &agvu1[j]);
4830     } while ( ( agvu1[j] < 0 ) || ( agvu1[j] > 1 ) );
4831  do {
4832      printfAt( 4,73,"[    ]"); scanfAt(  4,74, "%f", &pr1t1 );
4833     } while ( ( pr1t1 < 0.0 ) || ( pr1t1 > 100.0 ) );
4834  do {
4835      printfAt( 5,73,"[    ]"); scanfAt(  5,74, "%f", &wa1t1 );
4836     } while ( ( wa1t1 < 0.0 ) || ( wa1t1 > 100.0 ) );
4837  do {
4838      printfAt( 6,73,"[    ]"); scanfAt(  6,74, "%f", &pr2t1 );
4839     } while ( (  pr2t1 < 0.0 ) || (  pr2t1 > 100.0 ) );
4840  do {
4841      printfAt( 7,73,"[    ]"); scanfAt(  7,74, "%f", &wa2t1 );
4842     } while ( ( wa2t1  < 0.0 ) || ( wa2t1 > 100.0 ) );
4843  do {
4844      printfAt(11,34,"[    ]"); scanfAt( 11,35, "%d", &al    );
4845     } while ( (    al  < 1   ) || (    al > 100   ) );
4846  do {
4847      printfAt(12,34,"[    ]"); scanfAt( 12,35, "%d", &ap    );
4848     } while ( (    ap  < 1   ) || (    ap > 100   ) );
4849  switch(st)
4850   {
4851    case 3: /* Anspruchsniveau fuer MARK-UP I  (Reserven)            */
4852     printfAt( 9,50, "Anspruchsniveaus     :  ");
4853     printfAt(11,50, "Fuer Sparte 1        : ");
4854     printfAt(12,50, "Fuer Sparte 2        : ");
4855     do {
4856         printfAt(11,73,"[    ]"); scanfAt( 11,74, "%f", &an1   );
4857        } while ( (    an1 < -1000.0 ) || (   an1 > 1000.0 ) );
4858     do {
4859         printfAt(12,73,"[    ]"); scanfAt( 12,74, "%f", &an2   );
```

```
4860        } while ( (    an2 < -1000.0 ) || (   an2 > 1000.0 ) );
4861     break;
4862    case 4: /* Anspruchsniveau fuer MARK-UP II (Nettowechsler)        */
4863     printfAt( 9,50, "Anspruchsniveaus     :  ");
4864     printfAt(11,50, "Fuer Sparte 1        : ");
4865     printfAt(12,50, "Fuer Sparte 2        : ");
4866     do {
4867         printfAt(11,73,"[    ]"); scanfAt( 11,74, "%f", &an1   );
4868        } while ( (    an1 <   -50.0 ) || (   an1 >   50.0 ) );
4869     do {
4870         printfAt(12,73,"[    ]"); scanfAt( 12,74, "%f", &an2   );
4871        } while ( (    an2 <   -50.0 ) || (   an2 >   50.0 ) );
4872     break;
4873    case 5:  /* Anspruchsniveau fuer MARK-UP III (Marktanteil)         */
4874     printfAt( 9,50, "Anspruchsniveaus     :  ");
4875     printfAt(11,50, "Fuer Sparte 1        : ");
4876     printfAt(12,50, "Fuer Sparte 2        : ");
4877     do {
4878         printfAt(11,73,"[    ]"); scanfAt( 11,74, "%f", &an1   );
4879        } while ( (    an1 <     0.0 ) || (   an1 >    1.0 ) );
4880     do {
4881         printfAt(12,73,"[    ]"); scanfAt( 12,74, "%f", &an2   );
4882        } while ( (    an2 <     0.0 ) || (   an2 >    1.0 ) );
4883     break;
4884    default:
4885     printfAt( 9,50, "[Kein Anspruchsniveau ] ");
4886     printfAt(11,50, "                            ");
4887     printfAt(12,50, "                            ");
4888     break;
4889   } /* END: switch(st) */
4890   do {
4891       printfAt(19,15,"[    ]"); scanfAt( 19,16, "%f", &a0    );
4892      } while ( (     a0 <     0.0 ) || (    a0 >  100.0 ) );
4893   do {
4894       printfAt(20,15,"[    ]"); scanfAt( 20,16, "%f", &b0    );
4895      } while ( (     b0 <     0.0 ) || (    b0 >  100.0 ) );
4896   do {
4897       printfAt(21,15,"[    ]"); scanfAt( 21,16, "%f", &c0    );
4898      } while ( (     c0 <     0.0 ) || (    c0 >  100.0 ) );
4899   do {
4900       printfAt(22,15,"[    ]"); scanfAt( 22,16, "%f", &d0    );
4901      } while ( (     d0 <     0.0 ) || (    d0 >  100.0 ) );
4902   do {
4903       printfAt(19,33,"[    ]"); scanfAt( 19,34, "%f", &e0    );
4904      } while ( (     e0 <     0.0 ) || (    e0 >  100.0 ) );
```

```
4905   do {
4906       printfAt(20,33,"[    ]"); scanfAt( 20,34, "%f", &f0    );
4907      } while ( (     f0 <     0.0 ) || (    f0 >  100.0 ) );
4908   do {
4909       printfAt(21,33,"[    ]"); scanfAt( 21,34, "%f", &g0    );
4910      } while ( (     g0 <     0.0 ) || (    g0 >  100.0 ) );
4911   do {
4912       printfAt(22,33,"[    ]"); scanfAt( 22,34, "%f", &h0    );
4913      } while ( (     h0 <     0.0 ) || (    h0 >  100.0 ) );
4914   do {
4915       printfAt(19,55,"[    ]"); scanfAt( 19,56, "%f", &a1    );
4916      } while ( (     a1 <     0.0 ) || (    a1 >  100.0 ) );
4917   do {
4918       printfAt(20,55,"[    ]"); scanfAt( 20,56, "%f", &b1    );
4919      } while ( (     b1 <     0.0 ) || (    b1 >  100.0 ) );
4920   do {
4921       printfAt(21,55,"[    ]"); scanfAt( 21,56, "%f", &c1    );
4922      } while ( (     c1 <     0.0 ) || (    c1 >  100.0 ) );
4923   do {
4924       printfAt(22,55,"[    ]"); scanfAt( 22,56, "%f", &d1    );
4925      } while ( (     d1 <     0.0 ) || (    d1 >  100.0 ) );
4926   do {
4927       printfAt(19,73,"[    ]"); scanfAt( 19,74, "%f", &e1    );
4928      } while ( (     e1 <     0.0 ) || (    e1 >  100.0 ) );
4929   do {
4930       printfAt(20,73,"[    ]"); scanfAt( 20,74, "%f", &f1    );
4931      } while ( (     f1 <     0.0 ) || (    f1 >  100.0 ) );
4932   do {
4933       printfAt(21,73,"[    ]"); scanfAt( 21,74, "%f", &g1    );
4934      } while ( (     g1 <     0.0 ) || (    g1 >  100.0 ) );
4935   do {
4936       printfAt(22,73,"[    ]"); scanfAt( 22,74, "%f", &h1    );
4937      } while ( (     h1 <     0.0 ) || (    h1 >  100.0 ) );
4938  switch(st)                 /* Aufruf der Initialisierungsfunktion : */
4939   {
4940    case 3: /* Anspruchsniveau fuer MARK-UP I  (Reserven)            */
4941    Vuauini( j,st,al,ap,an1,an2,0.0,0.0,0.0,0.0,pr1t1,wa1t1,pr2t1,wa2t1,
4942            a0  ,  a1,  b0,  b1,  c0,  c1,  d0,  d1, /*Pramienreaktion*/
4943            e0  ,  e1,  f0,  f1,  g0,  g1,  h0,  h1);/*Werbereaktion  */
4944     break;
4945    case 4: /* Anspruchsniveau fuer MARK-UP II (Nettowechsler)       */
4946    Vuauini( j,st,al,ap,0.0,0.0,an1,an2,0.0,0.0,pr1t1,wa1t1,pr2t1,wa2t1,
4947            a0  ,  a1,  b0,  b1,  c0,  c1,  d0,  d1, /*Pramienreaktion*/
4948            e0  ,  e1,  f0,  f1,  g0,  g1,  h0,  h1);/*Werbereaktion  */
4949     break;
```

```
4950    case 5: /* Anspruchsniveau fuer MARK-UP III (Marktanteil)        */
4951    Vuauini( j,st,al,ap,0.0,0.0,0.0,0.0,an1,an2,pr1t1,wa1t1,pr2t1,wa2t1,
4952            a0  ,  a1,  b0,  b1,  c0,  c1,  d0,  d1, /*Pramienreaktion*/
4953            e0  ,  e1,  f0,  f1,  g0,  g1,  h0,  h1);/*Werbereaktion  */
4954     break;
4955    default:
4956     break;
4957   } /* END: switch(st) */
4958  for(i=1;i<=MAXVU;i++)       /* Automatische agvux[.]-Zuweisung an   */
4959   {                          /* die Aggregatdateien, die gebraucht   */
4960    BAV[1].DatenBAV.Vuag2[ VU[j].DatenVU->Vr ]++; /* werden           */
4961    BAV[1].DatenBAV.Vuag3[ VU[j].DatenVU->Vk ]++;
4962   }
4963  for( i = 1;  i <=    10 ; i++ )
4964   { if ( BAV[1].DatenBAV.Vuag2[i] > 0 ) { agvu2[i] = 1;} }
4965  for( i = 1;  i <=     3 ; i++ )
4966   { if ( BAV[1].DatenBAV.Vuag3[i] > 0 ) { agvu3[i] = 1;} }
4967  agvu4[1] = 1; /* Die Subjektklasse der Versicherer: immer erzeugen  */
4968  printfAt(24,35,"[ [1] Ruecksprung [0] Weitere VU [    ]  ]");
4969  scanfAt( 24,70, "%d", &ende);
4970  } while ( ende != 1);
4971 } /* END: func Erstvu() */
4972
4973 /* ************************************************************** */
4974 /* func Erstvn()  : Versicherungsnehmer erstellen                 */
4975 /*                                                                */
4976 /*                  Aufgerufen von func Moderst()                 */
4977 /* ************************************************************** */
4978 func Erstvn() {
4979  int j,ende;                          /* Lokale Variable vereinbaren*/
4980  int i;                               /* Schleifenzaehler           */
4981  int vr1,vr2,r1,r2, st,ap,al;
4982  float a0,a1,b0,b1,c0,c1,d0,d1,e0,e1,f0,f1,g0,g1,h0,h1,vm;
4983  do { /* while ende != 1 */
4984  clearScreen();                       /* Maske aufbauen             */
4985  box( 0, 0, 8,80); box( 8, 0, 8,80); box(16, 0, 8,80);
4986  box( 0, 0, 3,80); box( 8, 0, 2,80); box(16, 0, 2,80);
4987  box( 0, 0,24,40); box( 0,40,24,40);
4988  printfAt( 1,22, "[ Versicherungsnehmer konfigurieren ]");
4989  printfAt( 2,10, "Versicherungsnehmer  : ");
4990  printfAt( 2,50, "Startwerte d. Risiken: ");
4991  printfAt( 9,10, "Aktivierungsschockparamter: ");
4992  printfAt( 9,50, "Anfangsausstattung   : ");
4993  printfAt(17,10, "Vor Aenderungsschock : ");
4994  printfAt(17,50, "Nach Aenderungsschock: ");
```

```
4995  printfAt( 4,10, "VR[                  ]");
4996  printfAt( 5,10, "Verhaltensregelklasse: ");
4997  printfAt( 6,10, "Aggregatdatei(0=nein): ");
4998  printfAt( 4,50, "Status      Risiko 1 : ");
4999  printfAt( 5,50, "Versicherer Risiko 1 : ");
5000  printfAt( 6,50, "Status      Risiko 2 : ");
5001  printfAt( 7,50, "Versicherer Risiko 2 : ");
5002  printfAt(11,10, "Aktivierungslauf     : ");
5003  printfAt(12,10, "Aktivierungsperiode  : ");
5004  printfAt(11,50, "Gesamtvermoegen      : ");
5005  printfAt(19,10, "a  :              e  : ");
5006  printfAt(20,10, "b  :              f  : ");
5007  printfAt(21,10, "c  :              g  : ");
5008  printfAt(22,10, "d  :              h  : ");
5009  printfAt(19,50, "a* :              e* : ");
5010  printfAt(20,50, "b* :              f* : ");
5011  printfAt(21,50, "c* :              g* : ");
5012  printfAt(22,50, "d* :              h* : ");
5013  do {                              /* Zu bearbeitenden VN erfragen    */
5014      scanfAt( 2,33,"%d",&j);
5015     } while ( ( j < 1 ) || ( j > MAXVN) );
5016  st    = VN[ j ].DatenVN->Ac[1].st;         /* Daten auslesen        */
5017  ap    = VN[ j ].DatenVN->Ap;
5018  for( i = SIMLAENGE ; i >= 1 ; i--)   /* Aktivierungslaeufe          */
5019   { if (VN[j].DatenVN->Al[i] == 0) { al = i - 1; } }
5020  vr1 = VN[ j ].DatenVN->Rk[1].l.Vs[1];
5021  vr2 = VN[ j ].DatenVN->Rk[2].k.Vs[1];
5022  r1  = VN[ j ].DatenVN->Rk[1].l.Vu[1];
5023  r2  = VN[ j ].DatenVN->Rk[2].k.Vu[1];
5024  vm = VN[ j ].DatenVN->Vm[1];
5025  a0 = VN[ j ].DatenVN->Pv[0 ]; a1 = VN[ j ].DatenVN->Pv[1 ];
5026  b0 = VN[ j ].DatenVN->Pv[2 ]; b1 = VN[ j ].DatenVN->Pv[3 ];
5027  c0 = VN[ j ].DatenVN->Pv[4 ]; c1 = VN[ j ].DatenVN->Pv[5 ];
5028  d0 = VN[ j ].DatenVN->Pv[6 ]; d1 = VN[ j ].DatenVN->Pv[7 ];
5029  e0 = VN[ j ].DatenVN->Pv[8 ]; e1 = VN[ j ].DatenVN->Pv[9 ];
5030  f0 = VN[ j ].DatenVN->Pv[10]; f1 = VN[ j ].DatenVN->Pv[11];
5031  g0 = VN[ j ].DatenVN->Pv[12]; g1 = VN[ j ].DatenVN->Pv[13];
5032  h0 = VN[ j ].DatenVN->Pv[14]; h1 = VN[ j ].DatenVN->Pv[15];
5033                                          /* Daten ausgeben          */
5034  printfAt( 4, 13,"%18s" , vrvnnm[st]); /* Voller Name der VR        */
5035  printfAt( 5, 34,"%1d"  , vkrvn[st] );
5036  printfAt( 6, 34,"%1d"  , agvn1[j]  );
5037  printfAt( 4, 73,"%2d"  , vr1       );
5038  printfAt( 5, 73,"%2d"  , r1        );
5039  printfAt( 6, 73,"%2d"  , vr2       );
```

```
5040  printfAt( 7, 73,"%2d"  , r2        );
5041  printfAt(11, 74,"%4.2f", vm        );
5042  printfAt(11, 34,"%3d"  ,al);
5043  printfAt(12, 34,"%3d"  ,ap);
5044  printfAt(19, 15,"%3.2f",a0); printfAt(19,33,"%3.2f",e0);
5045  printfAt(20, 15,"%3.2f",b0); printfAt(20,33,"%3.2f",f0);
5046  printfAt(21, 15,"%3.2f",c0); printfAt(21,33,"%3.2f",g0);
5047  printfAt(22, 15,"%3.2f",d0); printfAt(22,33,"%3.2f",h0);
5048  printfAt(19, 55,"%3.2f",a1); printfAt(19,73,"%3.2f",e1);
5049  printfAt(20, 55,"%3.2f",b1); printfAt(20,73,"%3.2f",f1);
5050  printfAt(21, 55,"%3.2f",c1); printfAt(21,73,"%3.2f",g1);
5051  printfAt(22, 55,"%3.2f",d1); printfAt(22,73,"%3.2f",h1);
5052  do {                    /* Neue Parameter abfragen und  pruefen ... */
5053      printfAt( 4,34,"[    ]");
5054      scanfAt(  4,35, "%d", &st);
5055     } while ( ( st < 1 ) || ( st > 7 ) );
5056  printfAt( 4, 13,"%18s" , vrvnnm[st]); /* Voller Name der VR         */
5057  printfAt( 5, 34,"%1d"  , vkrvn[st] );
5058  do {
5059      printfAt( 6,34,"[    ]"); scanfAt(  6,35, "%d", &agvn1[j]);
5060     } while ( ( agvn1[j] < 0 ) || ( agvn1[j] > 1 ) );
5061  do {
5062      printfAt( 4,73,"[    ]"); scanfAt(  4,74, "%d", &vr1   );
5063     } while ( ( vr1   < 0   ) || ( vr1   >    1  ) );
5064  do {
5065      printfAt( 5,73,"[    ]"); scanfAt(  5,74, "%d", &r1    );
5066     } while ( ( r1    < 0   ) || ( r1    > MAXVU ) );
5067  do {
5068      printfAt( 6,73,"[    ]"); scanfAt(  6,74, "%d", &vr2   );
5069     } while ( (  vr2   < 0   ) || (  vr2   >  1    ) );
5070  do {
5071      printfAt( 7,73,"[    ]"); scanfAt(  7,74, "%d", &r2    );
5072     } while ( ( r2     < 0   ) || ( r2    > MAXVU ) );
5073  do {
5074      printfAt(11,34,"[    ]"); scanfAt( 11,35, "%d", &al    );
5075     } while ( (    al  < 1   ) || (    al > 100   ) );
5076  do {
5077      printfAt(12,34,"[    ]"); scanfAt( 12,35, "%d", &ap    );
5078     } while ( (    ap  < 1   ) || (    ap > 100   ) );
5079  do {
5080      printfAt(11,73,"[    ]"); scanfAt( 11,74, "%f", &vm    );
5081     } while ( (    vm  <     0.0 ) || (   vm  > 50000.0) );
5082  do {
5083      printfAt(19,15,"[    ]"); scanfAt( 19,16, "%f", &a0    );
5084     } while ( (     a0 <     0.0 ) || (    a0 >  100.0 ) );
```

```
5085  do {
5086      printfAt(20,15,"[    ]"); scanfAt( 20,16, "%f", &b0    );
5087     } while ( (     b0 <     0.0 ) || (    b0 >  100.0 ) );
5088  do {
5089      printfAt(21,15,"[    ]"); scanfAt( 21,16, "%f", &c0    );
5090     } while ( (     c0 <     0.0 ) || (    c0 >  100.0 ) );
5091  do {
5092      printfAt(22,15,"[    ]"); scanfAt( 22,16, "%f", &d0    );
5093     } while ( (     d0 <     0.0 ) || (    d0 >  100.0 ) );
5094  do {
5095      printfAt(19,33,"[    ]"); scanfAt( 19,34, "%f", &e0    );
5096     } while ( (     e0 <     0.0 ) || (    e0 >  100.0 ) );
5097  do {
5098      printfAt(20,33,"[    ]"); scanfAt( 20,34, "%f", &f0    );
5099     } while ( (     f0 <     0.0 ) || (    f0 >  100.0 ) );

5100  do {
5101      printfAt(21,33,"[    ]"); scanfAt( 21,34, "%f", &g0    );
5102     } while ( (     g0 <     0.0 ) || (    g0 >  100.0 ) );
5103  do {
5104      printfAt(22,33,"[    ]"); scanfAt( 22,34, "%f", &h0    );
5105     } while ( (     h0 <     0.0 ) || (    h0 >  100.0 ) );
5106  do {
5107      printfAt(19,55,"[    ]"); scanfAt( 19,56, "%f", &a1    );
5108     } while ( (     a1 <     0.0 ) || (    a1 >  100.0 ) );
5109  do {
5110      printfAt(20,55,"[    ]"); scanfAt( 20,56, "%f", &b1    );
5111     } while ( (     b1 <     0.0 ) || (    b1 >  100.0 ) );
5112  do {
5113      printfAt(21,55,"[    ]"); scanfAt( 21,56, "%f", &c1    );
5114     } while ( (     c1 <     0.0 ) || (    c1 >  100.0 ) );
5115  do {
5116      printfAt(22,55,"[    ]"); scanfAt( 22,56, "%f", &d1    );
5117     } while ( (     d1 <     0.0 ) || (    d1 >  100.0 ) );
5118  do {
5119      printfAt(19,73,"[    ]"); scanfAt( 19,74, "%f", &e1    );
5120     } while ( (     e1 <     0.0 ) || (    e1 >  100.0 ) );
5121  do {
5122      printfAt(20,73,"[    ]"); scanfAt( 20,74, "%f", &f1    );
5123     } while ( (     f1 <     0.0 ) || (    f1 >  100.0 ) );
5124  do {
5125      printfAt(21,73,"[    ]"); scanfAt( 21,74, "%f", &g1    );
5126     } while ( (     g1 <     0.0 ) || (    g1 >  100.0 ) );
5127  do {
5128      printfAt(22,73,"[    ]"); scanfAt( 22,74, "%f", &h1    );
5129     } while ( (     h1 <     0.0 ) || (    h1 >  100.0 ) );
```

```
5130  Vnauini( j,st,al, ap, vr1, r1,vr2, r2,vm,
5131           a0 , a1 , b0 , b1 , c0  , c1 , d0  , d1 ,
5132           e0 , e1 , f0 , f1 ,  g0 , g1 ,  h0 , h1 );
5133  for(i=1;i<=MAXVN;i++)       /* Automatische agvnx[.]-Zuweisung an   */
5134   {                          /* die Aggregatdateien, die gebraucht   */
5135    BAV[1].DatenBAV.Vnag2[ VN[j].DatenVN->Vr ]++; /* werden           */
5136    BAV[1].DatenBAV.Vnag3[ VN[j].DatenVN->Vk ]++;
5137   }
5138  for( i = 1;  i <=     7 ; i++ )
5139   { if ( BAV[1].DatenBAV.Vnag2[i] > 0 ) { agvn2[i] = 1;} }
5140  for( i = 1;  i <=     3 ; i++ )
5141   { if ( BAV[1].DatenBAV.Vnag3[i] > 0 ) { agvn3[i] = 1;} }
5142  agvn4[1] = 1; /* Subjektklasse Versicherungsnehmer: immer erzeugen  */
5143  printfAt(24,35,"[ [1] Ruecksprung [0] Weiterer VN [    ] ]");
5144  scanfAt( 24,70, "%d", &ende);
5145  } while ( ende != 1);
5146 }
5147
5148 /* ************************************************************** */
5149 /* func Erstmkt() : Allgemeine Marktparameter konfigurieren         */
5150 /*                                                                 */
5151 /*                  Aufgerufen von func Moderst()                  */
5152 /* ************************************************************** */
5153 func Erstmkt() {
5154  int i;                                /* Schleifenzaehler         */
5155  int a, b, e;
5156  float c, d;
5157  clearScreen();
5158  printfAt( 1,25, "[ Allgemeine Marktparameter konfigurieren ]");
5159  printfAt( 5,10, "Periode mit Aenderungsschock   : > "); /* Maske... */
5160  printfAt( 8,10, "Lauf    mit Aenderungsschock   : > "); /* aufbauen */
5161  printfAt(11,10, "Kosten pro Praemieninformation :   ");
5162  printfAt(14,10, "Periodenzinssatz               :   ");
5163  printfAt(17,10, "Wiederholungslaeufe,1:Ja,0:Nein:   ");
5164  for( i =  SIMLAENGE ; i >= 1; i--)                     /* Daten an-*/
5165   { if ( BAV[1].DatenBAV.As[i] == 1 )  { a = i ; }  }    /* zeigen   */
5166  printfAt( 5,50, "%3d",a);
5167  for( i =  SIMLAENGE ; i >= 1; i--)
5168   { if ( BAV[1].DatenBAV.Ar[i] == 1 )  { b = i ; }  }
5169  printfAt( 8,50, "%3d",b);
5170  printfAt(11,50, "%01.2f",BAV[1].DatenBAV.Sk[1]);
5171  printfAt(14,50, "%01.2f",BAV[1].DatenBAV.Zs[1]);
5172  printfAt(17,50, "%1d"   ,BAV[1].DatenBAV.Rp   );
5173  do {                          /* Neue Werte einlesen und pruefen ...*/
5174      printfAt( 5,60,"[       ]");  scanfAt( 5,62, "%d", &a);
```

```
5175     } while ( ( a < 1 ) || ( a > 100 ) );
5176  do {
5177      printfAt( 8,60,"[       ]");  scanfAt( 8,62, "%d", &b);
5178     } while ( ( b < 1 ) || ( b > 100 ) );
5179  do {
5180      printfAt(11,60,"[       ]");  scanfAt(11,62, "%f", &c);
5181     } while ( ( c < 0.0) || ( c > 10.0) );
5182  do {
5183      printfAt(14,60,"[       ]");  scanfAt(14,62, "%f", &d);
5184     } while ( ( d < 0.0) || ( d > 1.0) );
5185  do {
5186      printfAt(17,60,"[       ]");  scanfAt(17,62, "%d", &e);
5187     } while ( ( e < 0  ) || ( e > 1  ) );
5188  Bavauin( a, b, c, d, e); /* Initialisierungsroutine aufrufen        */
5189 /* Moderst();  Ruecksprung zum vorherigen Menue                      */
5190 }
5191
5192 /* ************************************************************** */
5193 /* func Moderst() : Modell erstellen                               */
5194 /*                                                                 */
5195 /*                  Aufgerufen von func Mkbear()                   */
5196 /* ************************************************************** */
5197 func Moderst() {
5198  int j;                                /* Auswahlvariable           */
5199  clearScreen();
5200  printfAt( 1,30, "[ Modell erstellen ]");
5201  printfAt( 9,12, "[1] Versicherer konfigurieren                    ");
5202  printfAt(12,12, "[2] Versicherungsnehmer konfigurieren            ");
5203  printfAt(15,12, "[3] Allgemeine Marktparameter konfigurieren       ");
5204  printfAt(18,12, "[4] Zum Menue [ Marktkonfiguration bearbeiten ]   ");
5205  do {
5206      printfAt(20,12,"[ ] "); scanfAt(20,13,"%d",&j);
5207     } while ( ( j < 1 ) || ( j > 4 ) );
5208  switch(j)
5209   {
5210    case 1: /* Versicherer konfigurieren */
5211       Erstvu();
5212      break;
5213    case 2: /* Versicherungsnehmer konfigurieren */
5214       Erstvn();
5215      break;
5216    case 3: /* Allgemeine Marktparameter konfigurieren */
5217       Erstmkt();
5218      break;
5219    case 4: /* Zum Vormenue */
```

```
5220        Mkbear();
5221      break;
5222    default:
5223      printfAt(24,2, "UNBEKANNTE AUSWAHL!");
5224      break;
5225  } /* END: switch(j)  */
5226 } /* END: Moderst() */
5227
5228 /* ************************************************************** */
5229 /* func Mkbear(): Modellkonfiguration auswaehlen oder erstellen      */
5230 /*                                                                   */
5231 /*                Aufgerufen von func Mainmenu()                     */
5232 /* ************************************************************** */
5233 func Mkbear() {
5234  int j;                                /* Auswahlvariable           */
5235  clearScreen();
5236  printfAt( 1,21, "[ Marktkonfiguration bearbeiten ]");
5237  printfAt( 2,10, "Vordefinierte Modelle ohne Schocks   ");
5238  printfAt( 4,12, "[1] Homogenes Angebot bei homogener Nachfrage      ");
5239  printfAt( 6,12, "[2] Homogenes Angebot bei heterogener Nachfrage    ");
5240  printfAt( 8,12, "[3] Heterogenes Angebot bei homogener Nachfrage    ");
5241  printfAt(10,10, "Vordefinierte Modelle mit Schocks                  ");
5242  printfAt(12,12, "[4] Einfacher Aenderungsschock, homogene Nachfrage ");
5243  printfAt(14,12, "[5] Einfacher Aktivierungsschock, homogenes Angebot");
5244  printfAt(16,12, "[6] Komplexer Marktschock: Die Wiedervereinigung   ");
5245  printfAt(18,10, "Selbstdefinierte Modelle                           ");
5246  printfAt(20,12, "[7] Modell erstellen                               ");
5247  printfAt(22,12, "[8] Zum Hauptmenu                                  ");
5248  do {
5249      printfAt(23,12,"[ ] "); scanfAt(23,13,"%d",&j);
5250     } while ( ( j < 1 ) || ( j > 8 ) );
5251  switch(j)
5252   {
5253    case 1:                  /* Vordefiniertes Modell 1 auswaehlen ...*/
5254      Vdefmd1();
5255      break;
5256    case 2:                  /* Vordefiniertes Modell 2 auswaehlen ...*/
5257      Vdefmd2();
5258      break;
5259    case 3:                  /* Vordefiniertes Modell 3 auswaehlen ...*/
5260      Vdefmd3();
5261      break;
5262    case 4:                  /* Vordefiniertes Modell 4 auswaehlen ...*/
5263      Vdefmd4();
5264      break;
```

```
5265    case 5:                  /* Vordefiniertes Modell 5 auswaehlen ...*/
5266      Vdefmd5();
5267      break;
5268    case 6:                  /* Vordefiniertes Modell 6 auswaehlen ...*/
5269      Vdefmd6();             /* Das Modell der Wiedervereinigung      */
5270      break;
5271    case 7:                  /* Selbstdefiniertes Modell erstellen ...*/
5272      Moderst();
5273      break;
5274    case 8:                  /* Zurueck zum Hauptmenu ...             */
5275      Mainmenu();
5276      break;
5277    default:
5278      printfAt(24,2, "UNBEKANNTE AUSWAHL!");
5279      break;
5280  } /* END: switch(j)  */
5281 } /* END: func Mkbear() */
5282
5283 /* ************************************************************** */
5284 /* func Starts(): Eingabe von Periodenanzahl und Anzahl der Laeufe   */
5285 /*                                                                  */
5286 /*                Aufgerufen von func Mainmenu()                    */
5287 /* ************************************************************** */
5288 func Starts() {
5289  int j;
5290  clearScreen();
5291  printfAt( 6,15,"Sollen bisher erzeugte Aggregatdateien     ");
5292  printfAt( 7,15,"geloescht werden? [1] Ja [0] Nein : [     ]");
5293  do {
5294      scanfAt( 7,54, "%d", &j);
5295     } while ( ( j < 0  ) || ( j > 1  ) );
5296  if ( j == 1 )
5297   {
5298    #if defined(PC386)
5299     system("del imsv*.dat");
5300    #endif
5301    #if defined(UNIX)
5302     system("rm imsv*.dat");
5303    #endif
5304   }
5305  clearScreen(); box(22,0,2,80);
5306   do
5307   {
5308    printfAt(23,3,"Periodenanzahl der Einzelsimulation eingeben: [   ]");
5309    scanfAt(23,50,"%d", &sl);
```

```
5310   } while ( (sl < 1 ) || ( sl > SIMLAENGE ) );
5311  if ( BAV[1].DatenBAV.Rp == 1 ) /* Wiederholungen sind erlaubt ... */
5312   {
5313    do
5314     {
5315      printfAt(23,3,"Anzahl der Simulationslaeufe eingeben       : [   ]");
5316      scanfAt(23,50,"%d", &al);
5317     } while ( (al < 1 ) || ( al > SIMLAENGE ) );
5318    printfAt(23,3,"                                                  ");
5319   } /* END: if ( BAV[1].DatenBAV.Rp == 1 ) */
5320   rl  = 1;
5321   clearScreen();
5322   box(8,8,14,36);
5323   printfAt(10,10,"Aktuelle Periode:                ");
5324   printfAt(11,10,"Aktueller Lauf  :                ");
5325   printfAt(12,10,"Bisher berechnet:        Prozent ");
5326   printfAt(15,10,"Version         : %8s",version    );
5327   printfAt(17,10,"I M S   (c) Martin Koehler       ");
5328   printfAt(18,10,"Institut fuer Versicherungs-     ");
5329   printfAt(19,10,"betriebslehre und -informatik    ");
5330   printfAt(20,10,"Universitaet Hannover            ");
5331 } /* END: func Starts() */
5332
5333 /* ************************************************************** */
5334 /* func Analyvu() : Analyse der Simulationsergebnisse der          */
5335 /*                  Versicherer                                    */
5336 /*                  Aufgerufen von func Analy()                    */
5337 /* ************************************************************** */
5338 func Analyvu() {
5339  int ende=0;
5340  int a,b,c,i,k,j,l,m;
5341  int attrib[13]; /* 1..12 = Attribut, das geplottet wird = Daten-    */
5342                  /* spalte in der .dat-Datei                         */
5343                                 /* Aggregatauswahl der Versicherungen */
5344  int lagvu1[MAXVU+1], lagvu2[10], lagvu3[4], lagvu4[2];
5345  static char *vuattr[] = { "                      ",
5346   "Praemie,      Sparte 1", "Werbung,      Sparte 1",
5347   "Reserven,     Sparte 1", "Versicherte,  Sparte 1",
5348   "Schadenzahl,  Sparte 1", "Schadensumme, Sparte 1",
5349   "Praemie,      Sparte 2", "Werbung,      Sparte 2",
5350   "Reserven,     Sparte 2", "Versicherte,  Sparte 2",
5351   "Schadenzahl,  Sparte 2", "Schadensumme, Sparte 2" };
5352  file *t;                                /* temporaere GNUplot-Datei */
5353  char fname[13];                         /* GNUplot Dateiname        */
5354  for ( i = 1; i <= MAXVU; i++) { lagvu1[i] = 0; } /* Lokale Vektoren */
```

```
5355  for ( i = 1; i <=  9   ; i++) { lagvu2[i] = 0; } /* fuer die Wahl   */
5356  for ( i = 1; i <=  3   ; i++) { lagvu3[i] = 0; } /* der Aggregate   */
5357  for ( i = 0; i <=  1   ; i++) { lagvu4[i] = 0; } /* auf Null setzen */
5358  for ( i = 0; i <= 12   ; i++) { attrib[i] = 0; }
5359  do { /* BEGIN: while ( ende == 1 ) */
5360   clearScreen();
5361   printfAt( 1,15, "[ Modellergebnis der Versicherer analysieren ]");
5362   t = fopen("temp.gnu","w+t");    /* Entweder Datei neu anlegen oder */
5363   if ( t == (file *) 0 )          /* bestehende Datei ueberschreiben */
5364    {
5365     fprintf(runreport,"Analyvu(): Kann temp.gnu nicht oeffnen\n");
5366    }
5367    else
5368     {
5369      fclose(t);                   /* ... und wieder schliessen...    */
5370     }
5371   do {
5372       printfAt( 3, 5,"Aggregatstufe: I[1], II[2], III[3], IV[4][  ]");
5373        scanfAt( 3,47,"%d",&a);
5374      } while ( ( a < 1 ) || ( a > 4 ) );
5375   switch(a)
5376    {
5377     case 1:
5378       printfAt( 4, 5,"Erzeugte Aggregate der Einzelversicherungen:");
5379       k=0;
5380       for( i=1; i<=MAXVU;i++)       /* Aggregate Stufe I darstellen  */
5381        {
5382         if ( agvu1[i] == 1)
5383          {
5384           printfAt(5,5+7*k,"VU[%02d] ",i);
5385           k++;
5386          }
5387        }
5388       for( i=1; i<=MAXVU;i++)       /* ... und abfragen ...          */
5389        {
5390         if ( agvu1[i] == 1)
5391          {
5392           do {
5393            printfAt( 6, 5,
5394             "VU[%02d] nicht plotten[0], plotten[1], replotten[2][ ]",i);
5395             scanfAt( 6,55,"%d",&k);
5396              } while ( ( k < 0 )  || ( k > 2 ) );
5397           lagvu1[i] = k;
5398          }
5399        }
```

```
5400      break;
5401     case 2:
5402       printfAt( 4, 5,"Erzeugte Aggregate der Verhaltensregeln    :");
5403       k = 0;
5404       for(i = 1; i <= 9; i++)
5405        {
5406         if ( agvu2[i] == 1)
5407          {
5408           printfAt(5+k,5,"VR[%02d] %s",i,vrvunm[i]);
5409           k++;
5410          }
5411        }
5412       for( i = 1; i <= 9; i++)      /* ... und abfragen ...          */
5413        {
5414         if ( agvu2[i] == 1)
5415          {
5416           do {
5417            printfAt( 15,5,
5418             "VR[%02d] nicht plotten[0], plotten[1], replotten[2][ ]",i);
5419             scanfAt( 15,55,"%d",&k);
5420              } while ( ( k < 0 )  || ( k > 2 ) );
5421           lagvu2[i] = k;
5422          }
5423        }
5424      break;
5425     case 3:
5426       printfAt( 4, 5,"Erzeugte Aggregate der Regelklassen        :");
5427       k = 0;
5428       for( i = 1; i <= 3; i++)
5429        {
5430         if ( agvu3[i] == 1)
5431          {
5432           printfAt(5,5+7*k,"VK[%02d] ",i);
5433           k++;
5434          }
5435        }
5436       for( i = 1; i <= 3;i++)       /* ... und abfragen ...          */
5437        {
5438         if ( agvu3[i] == 1)
5439          {
5440           do {
5441            printfAt(  9,5,
5442             "VK[%02d] nicht plotten[0], plotten[1], replotten[2][ ]",i);
5443             scanfAt(  9,55,"%d",&k);
5444              } while ( ( k < 0 )  || ( k > 2 ) );
```

```
5445          lagvu3[i] = k;
5446         }
5447       }
5448     break;
5449    case 4:
5450      printfAt( 4, 5,"Erzeugte Aggregate der Subjektklasse       :");
5451      if ( agvu4[1] == 1)
5452       {
5453        printfAt( 5, 5,"Aggregat der Subjektklasse VU erzeugt");
5454       }
5455      do {
5456       printfAt( 6, 5, "Sk[1] nicht plotten[0], plotten[1], replotten[2][ ]",i);
5457            scanfAt( 6,54,"%d",&k);
5458         } while ( ( k < 0 )  || ( k > 2 ) );
5459      lagvu4[1] = k;
5460     break;
5461   }
5462  k=0;
5463  do {                            /* Attribute = Spalten auswaehlen */
5464      clearScreen();
5465      for( i = 1; i <= 12; i++ )
5466       {
5467        printfAt( 5+i,10,"[%02d] %s", i, vuattr[i] );
5468       }
5469      do {
5470          printfAt(18,10,"Attributnummer [  ]");
5471           scanfAt(18,26,"%d",&j);
5472         } while ( ( j < 0 )  || ( j > 12) );
5473      attrib[j] = 1;   /* ..weil in erster Spalte die Periode steht..*/
5474      m=j;
5475      l=j+1;       /* ... auf j wird weiter unter wieder zugegriffen */
5476     } while ( k != 0 );
5477   do {
5478       printfAt(21,10,"Simulationslauf: [1] .... [%03d]         [   ]",rl);
5479        scanfAt(21,52,"%d",&c);
5480      } while ( ( c < 1 ) || ( c > rl ) );
5481   t = fopen("temp.gnu","a+t");  /* GNUplot-Datei beschreiben        */
5482   if ( t == (file *) 0 )
5483    {
5484     fprintf(runreport,"Analyvu(): Kann temp.gnu nicht oeffnen\n");
5485    }
5486    else
5487     {
5488      fprintf(t,"# IMS v1.0 GNUplot-Steuerdatei\n");
5489      for( i = 1; i <= MAXVU; i++)
```

```
5490        {
5491         if (lagvu1[i] != 0 )       /* Versicherer i wird gedruckt !  */
5492          {
5493           fprintf(t,"set title  'IMS: Versicherer: VU[%02d]' \n",i);
5494           fprintf(t,"set xlabel 'Periode'\n");
5495           fprintf(t,"set ylabel '%s'\n", vuattr[m]);
5496           switch( lagvu1[i] )
5497            {
5498             case 1:
5499              fprintf(t,"plot [%d:%d] '%s' using 1:%d with lines\n",
5500                     (c-1)*sl, c*sl, agvu1nm[i], l );
5501              break;
5502             case 2:
5503              fprintf(t,"replot       '%s' using 1:%d with lines\n",
5504                         agvu1nm[i], l );
5505              break;
5506            }
5507            #if defined(UNIX)
5508             fprintf(t,"pause(-1) '<ENTER...>'\n");
5509            #endif
5510          } /* END: if (lagvu1[i] != 0 )*/
5511        } /* END: for( i = 1; i <= MAXVU; i++) */
5512       for( i = 1; i <=     9; i++)
5513        {
5514         if (lagvu2[i] != 0 )       /* Verhaltensregel i drucken   !  */
5515          {
5516           fprintf(t,"set title  'IMS: Verhaltensregel[%s]'\n",vrvunm[i]);
5517           fprintf(t,"set xlabel 'Periode'\n");
5518           fprintf(t,"set ylabel '%s'\n", vuattr[m]);
5519           switch( lagvu2[i] )
5520            {
5521             case 1:
5522              fprintf(t,"plot [%d:%d] '%s' using 1:%d with lines\n",
5523                     (c-1)*sl, c*sl, agvu2nm[i], l );
5524              break;
5525             case 2:
5526              fprintf(t,"replot       '%s' using 1:%d with lines\n",
5527                         agvu2nm[i], l );
5528              break;
5529            }
5530            #if defined(UNIX)
5531             fprintf(t,"pause(-1) '<ENTER...>'\n");
5532            #endif
5533          } /* END: if (lagvu2[i] != 0 )*/
5534        } /* END: for( i = 1; i <=     9; i++) */
```

```
5535      for( i = 1; i <=     3; i++)
5536       {
5537        if (lagvu3[i] != 0 )       /* Verhaltensregelklasse i drucken*/
5538         {
5539          switch(i)
5540           {
5541            case 1:
5542              fprintf(t,"set title  'IMS: Informationsfreie VRs'\n");
5543             break;
5544            case 2:
5545              fprintf(t,"set title  'IMS: VRs mit Eigeninformation'\n");
5546             break;
5547            case 3:
5548              fprintf(t,"set title  'IMS: VRs mit Fremdinformation'\n");
5549             break;
5550           }
5551          fprintf(t,"set xlabel 'Periode'\n");
5552          fprintf(t,"set ylabel '%s'\n", vuattr[m]);
5553          switch( lagvu3[i] )
5554           {
5555            case 1:
5556             fprintf(t,"plot [%d:%d] '%s' using 1:%d with lines\n",
5557                    (c-1)*sl, c*sl, agvu3nm[i], l );
5558             break;
5559            case 2:
5560             fprintf(t,"replot       '%s' using 1:%d with lines\n",
5561                        agvu3nm[i], l );
5562             break;
5563           }
5564           #if defined(UNIX)
5565            fprintf(t,"pause(-1) '<ENTER...>'\n");
5566           #endif
5567         } /* END: if (lagvu3[i] != 0 )*/
5568       } /* END: for( i = 1; i <=     3; i++) */
5569       if (lagvu4[1] != 0 )       /* Subjekteklassse der VUs drucken*/
5570        {
5571         fprintf(t,"set title  'IMS: Subjektklasse VUs'\n");
5572         fprintf(t,"set xlabel 'Periode'\n");
5573         fprintf(t,"set ylabel '%s'\n", vuattr[m]);
5574         switch( lagvu4[1] )
5575          {
5576           case 1:
5577            fprintf(t,"plot [%d:%d] '%s' using 1:%d with lines\n",
5578                   (c-1)*sl, c*sl, agvu4nm[1], l );
5579            break;
```

```
5580            case 2:
5581             fprintf(t,"replot       '%s' using 1:%d with lines\n",
5582                        agvu4nm[1], l );
5583             break;
5584           } /* END: switch( lagvu4[1] ) */
5585            #if defined(UNIX)
5586             fprintf(t,"pause(-1) '<ENTER...>'\n");
5587            #endif
5588         } /* END: if (lagvu4[1] != 0 ) */
5589      } /* END:  switch(a) */
5590   fclose(t);
5591   system("gnuplot temp.gnu");
5592   #if defined(PC386)
5593    system("del temp.gnu");
5594   #endif
5595   #if defined(UNIX)
5596    system("rm temp.gnu");
5597   #endif
5598   clearScreen();
5599   do {
5600       printfAt(23,10,"ENDE [1], weitere Grafiken [0] [ ]");
5601        scanfAt(23,42,"%d",&ende);
5602      } while ( ( ende < 0 )  || ( ende > 1) );
5603   } while ( ende != 1 );
5604 } /* END: func Analyvu() */
5605
5606 /* ************************************************************** */
5607 /* func Analyvn() : Analyse der Simulationsergebnisse der          */
5608 /*                  Versicherungsnehmer                            */
5609 /*                  Aufgerufen von func Analy()                    */
5610 /* ************************************************************** */
5611 func Analyvn() {
5612  int ende=0;
5613  int a,b,c,i,k,j,l,m;
5614  int attrib[12]; /* 1..11 = Attribut, das geplottet wird = Daten-    */
5615                  /* spalte in der .dat-Datei                         */
5616                           /* Aggregatauswahl der Versicherungsnehmer*/
5617  int lagvn1[MAXVN+1], lagvn2[7],  lagvn3[4], lagvn4[2];
5618  char *vnattr[] = { "                      ",
5619                     "Gewaehlte Versicherung,     Risiko 1",
5620                     "Status,                     Risiko 1",
5621                     "Gezahlte Praemie,           Risiko 1",
5622                     "Selbstversicherte Schaeden, Risiko 1",
5623                     "Gesamtschaeden,             Risiko 1",
5624                     "Gewaehlte Versicherung,     Risiko 2",
```

```
5625                    "Status,                     Risiko 2",
5626                    "Gezahlte Praemie,           Risiko 2",
5627                    "Selbstversicherte Schaeden, Risiko 2",
5628                    "Gesamtschaeden,             Risiko 2",
5629                    "Periodenendvermoegen                 " };
5630  file *t;                                /* temporaere GNUplot-Datei */
5631  char fname[13];                         /* GNUplot Dateiname        */
5632  for ( i = 1; i <= MAXVN; i++) { lagvn1[i] = 0; } /* Lokale Vektoren */
5633  for ( i = 1; i <=  6   ; i++) { lagvn2[i] = 0; } /* fuer die Wahl   */
5634  for ( i = 1; i <=  3   ; i++) { lagvn3[i] = 0; } /* der Aggregate   */
5635  for ( i = 0; i <=  1   ; i++) { lagvn4[i] = 0; } /* auf Null setzen */
5636  for ( i = 0; i <= 12   ; i++) { attrib[i] = 0; }
5637  do { /* BEGIN: while ( ende == 1 ) */
5638   clearScreen();
5639   printfAt( 1,15, "[ Modellergebnis der Versicherten analysieren ]");
5640   t = fopen("temp.gnu","w+t");    /* Entweder Datei neu anlegen oder */
5641   if ( t == (file *) 0 )          /* bestehende Datei ueberschreiben */
5642    {
5643     fprintf(runreport,"Analyvn(): Kann temp.gnu nicht oeffnen\n");
5644    }
5645    else
5646     {
5647      fclose(t);                   /* ... und wieder schliessen...    */
5648     }
5649   do {
5650       printfAt( 3, 5,"Aggregatstufe: I[1], II[2], III[3], IV[4][  ]");
5651        scanfAt( 3,47,"%d",&a);
5652      } while ( ( a < 1 ) || ( a > 4 ) );
5653   switch(a)
5654    {
5655     case 1:
5656       printfAt( 4, 5,"Erzeugte Aggregate der Einzelversicherten  :");
5657       k=0;
5658       for( i=1; i<=MAXVN;i++)       /* Aggregate Stufe I darstellen  */
5659        {
5660         if ( agvn1[i] == 1)
5661          {
5662           printfAt(5,5+7*k,"VN[%03d] ",i);
5663           k++;
5664          }
5665        }
5666       for( i=1; i<=MAXVN;i++)       /* ... und abfragen ...          */
5667        {
5668         if ( agvn1[i] == 1)
5669          {
```

```
5670          do {
5671           printfAt( 6, 5,
5672            "VN[%03d] nicht plotten[0], plotten[1], replotten[2][   ]",i);
5673            scanfAt( 6,57,"%d",&k);
5674             } while ( ( k < 0 )  || ( k > 2 ) );
5675          lagvn1[i] = k;
5676         }
5677       }
5678     break;
5679    case 2:
5680      printfAt( 4, 5,"Erzeugte Aggregate der Verhaltensregeln    :");
5681      k = 0;
5682      for(i = 1; i <= 6; i++)
5683       {
5684        if ( agvn2[i] == 1)
5685         {
5686          printfAt(5+k,5,"VR[%02d] %s",i,vrvnnm[i]);
5687          k++;
5688         }
5689       }
5690      for( i = 1; i <= 6; i++)      /* ... und abfragen ...          */
5691       {
5692        if ( agvn2[i] == 1)
5693         {
5694          do {
5695           printfAt( 15, 5,
5696            "VR[%02d] nicht plotten[0], plotten[1], replotten[2][   ]",i);
5697            scanfAt( 15,58,"%d",&k);
5698             } while ( ( k < 0 )  || ( k > 2 ) );
5699          lagvn2[i] = k;

5700         }
5701       }
5702     break;
5703    case 3:
5704      printfAt( 4, 5,"Erzeugte Aggregate der Regelklassen        :");
5705      k = 0;
5706      for( i = 1; i <= 3; i++)
5707       {
5708        if ( agvn3[i] == 1)
5709         {
5710          printfAt(5,5+7*k,"VK[%02d] ",i);
5711          k++;
5712         }
5713       }
5714      for( i = 1; i <= 3;i++)       /* ... und abfragen ...          */
```

```
5715        {
5716         if ( agvn3[i] == 1)
5717          {
5718           do {
5719            printfAt(  9,5,
5720             "VK[%02d] nicht plotten[0], plotten[1], replotten[2][   ]",i);
5721             scanfAt(  9,55,"%d",&k);
5722              } while ( ( k < 0 )  || ( k > 2 ) );
5723           lagvn3[i] = k;
5724          }
5725        }
5726      break;
5727     case 4:
5728       printfAt( 4, 5,"Erzeugte Aggregate der Subjektklasse       :");
5729       if ( agvn4[1] == 1)
5730        {
5731         printfAt( 5, 5,"Aggregat der Subjektklasse VN erzeugt");
5732        }
5733       do {
5734        printfAt( 6, 5,
5735             "Sk[1] nicht plotten[0], plotten[1], replotten[2][   ]",i);
5736             scanfAt( 6,55,"%d",&k);
5737          } while ( ( k < 0 )  || ( k > 2 ) );
5738       lagvn4[1] = k;
5739      break;
5740    }
5741   k=0;
5742   do {                              /* Attribute = Spalten auswaehlen */
5743       clearScreen();
5744       for( i = 1; i <= 11; i++ )
5745        {
5746         printfAt( 5+i,10,"[%02d] %s", i, vnattr[i] );
5747        }
5748       do {
5749           printfAt(18,10,"Attributnummer [   ]");
5750            scanfAt(18,26,"%d",&j);
5751          } while ( ( j < 0 )  || ( j > 11) );
5752       attrib[j] = 1;   /* ..weil in erster Spalte die Periode steht..*/
5753       m=j;
5754       l=j+1;       /* ... auf j wird weiter unter wieder zugegriffen */
5755      } while ( k != 0 );
5756    do {
5757        printfAt(21,10,"Simulationslauf: [1] .... [%03d]         [   ]",rl);
5758         scanfAt(21,52,"%d",&c);
5759       } while ( ( c < 1 ) || ( c > rl ) );
```

```
5760    t = fopen("temp.gnu","a+t");  /* GNUplot-Datei beschreiben        */
5761    if ( t == (file *) 0 )
5762     {
5763      fprintf(runreport,"Analyvn(): Kann temp.gnu nicht oeffnen\n");
5764     }
5765     else
5766      {
5767       fprintf(t,"# IMS v1.0 GNUplot-Steuerdatei\n");
5768       for( i = 1; i <= MAXVN; i++)
5769        {
5770         if (lagvn1[i] != 0 )      /* Versicherter i wird gedruckt !  */
5771          {
5772           fprintf(t,"set title  'IMS: Versicherter: VN[%03d]' \n",i);
5773           fprintf(t,"set xlabel 'Periode'\n");
5774           fprintf(t,"set ylabel '%s'\n", vnattr[m]);
5775           switch( lagvn1[i] )
5776            {
5777             case 1:
5778              fprintf(t,"plot [%d:%d] '%s' using 1:%d with lines\n",
5779                     (c-1)*sl, c*sl, agvn1nm[i], l );
5780              break;
5781             case 2:
5782              fprintf(t,"replot       '%s' using 1:%d with lines\n",
5783                         agvn1nm[i], l );
5784              break;
5785            }
5786            #if defined(UNIX)
5787             fprintf(t,"pause(-1) '<ENTER...>'\n");
5788            #endif
5789          } /* END: if (lagvn1[i] != 0 )*/
5790        } /* END: for( i = 1; i <= MAXVN; i++) */
5791       for( i = 1; i <=     6; i++)
5792        {
5793         if (lagvn2[i] != 0 )       /* Verhaltensregel i drucken   !  */
5794          {
5795           fprintf(t,"set title  'IMS: Verhaltensregel[%s]'\n",vrvnnm[i]);
5796           fprintf(t,"set xlabel 'Periode'\n");
5797           fprintf(t,"set ylabel '%s'\n", vnattr[m]);
5798           switch( lagvn2[i] )
5799            {
5800             case 1:
5801              fprintf(t,"plot [%d:%d] '%s' using 1:%d with lines\n",
5802                     (c-1)*sl, c*sl, agvn2nm[i], l );
5803              break;
5804             case 2:
```

```
5805            fprintf(t,"replot       '%s' using 1:%d with lines\n",
5806                       agvn2nm[i], l );
5807            break;
5808          }
5809          #if defined(UNIX)
5810           fprintf(t,"pause(-1) '<ENTER...>'\n");
5811          #endif
5812        } /* END: if (lagvn2[i] != 0 )*/
5813      } /* END: for( i = 1; i <=     6; i++) */
5814     for( i = 1; i <=     3; i++)
5815      {
5816       if (lagvn3[i] != 0 )       /* Verhaltensregelklasse i drucken*/
5817        {
5818         switch(i)
5819          {
5820           case 1:
5821             fprintf(t,"set title  'IMS: Informationsfreie VRs'\n");
5822            break;
5823           case 2:
5824             fprintf(t,"set title  'IMS: VRs mit Eigeninformation'\n");
5825            break;
5826           case 3:
5827             fprintf(t,"set title  'IMS: VRs mit Fremdinformation'\n");
5828            break;
5829          }
5830         fprintf(t,"set xlabel 'Periode'\n");
5831         fprintf(t,"set ylabel '%s'\n", vnattr[m]);
5832         switch( lagvn3[i] )
5833          {
5834           case 1:
5835            fprintf(t,"plot [%d:%d] '%s' using 1:%d with lines\n",
5836                   (c-1)*sl, c*sl, agvn3nm[i], l );
5837            break;
5838           case 2:
5839            fprintf(t,"replot       '%s' using 1:%d with lines\n",
5840                       agvn3nm[i], l );
5841            break;
5842          }
5843          #if defined(UNIX)
5844           fprintf(t,"pause(-1) '<ENTER...>'\n");
5845          #endif
5846        } /* END: if (lagvn3[i] != 0 )*/
5847      } /* END: for( i = 1; i <=     3; i++) */
5848      if (lagvn4[1] != 0 )       /* Subjekteklassse der VUs drucken*/
5849       {
```

```
5850          fprintf(t,"set title  'IMS: Subjektklasse VNs'\n");
5851          fprintf(t,"set xlabel 'Periode'\n");
5852          fprintf(t,"set ylabel '%s'\n", vnattr[m]);
5853          switch( lagvn4[1] )
5854           {
5855            case 1:
5856             fprintf(t,"plot [%d:%d] '%s' using 1:%d with lines\n",
5857                    (c-1)*sl, c*sl, agvn4nm[1], l );
5858             break;
5859            case 2:
5860             fprintf(t,"replot       '%s' using 1:%d with lines\n",
5861                        agvn4nm[1], l );
5862             break;
5863           } /* END: switch( lagvn4[1] ) */
5864            #if defined(UNIX)
5865             fprintf(t,"pause(-1) '<ENTER...>'\n");
5866            #endif
5867         } /* END: if (lagvn4[1] != 0 ) */
5868      } /* END:  switch(a) */
5869   fclose(t);
5870   system("gnuplot temp.gnu");
5871   #if defined(PC386)
5872    system("del temp.gnu");
5873   #endif
5874   #if defined(UNIX)
5875    system("rm temp.gnu");
5876   #endif
5877   clearScreen();
5878   do {
5879       printfAt(23,10,"ENDE [1], weitere Grafiken [0] [ ]");
5880        scanfAt(23,42,"%d",&ende);
5881      } while ( ( ende < 0 )  || ( ende > 1) );
5882   } while ( ende != 1 );
5883 } /* END: func Analyvn() */
5884
5885 /* ************************************************************** */
5886 /* func Analy() : Analyse der Simulationsergebnisse               */
5887 /*                                                                */
5888 /*                Aufgerufen von func Mainmenu()                  */
5889 /* ************************************************************** */
5890 func Analy() {
5891  int j;                                        /* Schleifenzaehler */
5892  /* for(;;)             ENDLOSschleife, die ueber [4] verlassen wird
5893   { */
5894    clearScreen();
```

```
5895    printfAt( 1,25, "[ Modellergebnis analysieren ]");
5896    printfAt( 7,10, "[1] Grafische Analyse der Versicherer         ");
5897    printfAt( 9,10, "[2] Grafische Analyse der Versicherungsnehmer ");
5898    printfAt(11,10, "[3] Grafiken interaktiv erstellen             ");
5899    printfAt(13,10, "[4] Zurueck zum Hauptmenu                     ");
5900    do {
5901        printfAt(15,10,"[ ] "); scanfAt(15,11,"%d",&j);
5902       } while ( ( j < 1 )  || ( j > 4));
5903    switch(j)
5904     {
5905      case 1:                    /* Grafische Analyse der Versicherer */
5906         Analyvu();
5907       break;
5908      case 2:            /* Grafische Analyse der Versicherungsnehmer */
5909         Analyvn();
5910       break;
5911      case 3:                       /* Grafiken interaktiv erstellen */
5912         system("gnuplot");         /* .. d.h. GNUplot starten       */
5913         break;
5914      case 4:                                /* Zurueck zum Hauptmenu */
5915         Mainmenu();
5916       break;
5917      default:
5918         printfAt(23,10, "ERROR: unbekannte Auswahl!");
5919       break;
5920     } /* END: switch(j) */
5921  /*  }  END: for(;;) */
5922 }
5923
5924 /* ************************************************************** */
5925 /* func Pginfo() : Programminformation                            */
5926 /*                                                                */
5927 /*                 Aufgerufen von func Mainmenu()                 */
5928 /* ************************************************************** */
5929 func Pginfo() {
5930  clearScreen();
5931  printfAt( 1,27,"[ Programminformation ]");
5932  printfAt( 3,10,"      I M S     The Insurance Market Simulation      ");
5933  printfAt( 5,10," Simulationsprogramm zur Analyse von exogenen Schocks ");
5934  printfAt( 6,10," auf Versicherungsmaerkten im Rahmen der Dissertation:");
5935  printfAt( 8,10,"      Eine Computersimulation zur Analyse der        ");
5936  printfAt( 9,10,"      Auswirkungen von exogenen Schocks auf          ");
5937  printfAt(10,10,"      Versicherungsmaerkte - am Beispiel der         ");
5938  printfAt(11,10,"      deutschen Wiedervereinigung                    ");
5939  printfAt(13,10,"      Dipl-Oek. Martin Koehler                       ");
```

```
5940  printfAt(14,10,"      Institut fuer Versicherungsbetriebslehre       ");
5941  printfAt(15,10,"      der Universitaet Hannover                      ");
5942  printfAt(17,10,"  Referent  : Prof. Dr. J.-M. Graf von der Schulenburg");
5943  printfAt(18,10,"              Universitaet Hannover, FB Wirtschaft   ");
5944  printfAt(19,10,"  Koreferent: Prof. Dr. P. Stahlecker                ");
5945  printfAt(20,10,"              Universitaet Hamburg,  FB Wirtschaft   ");
5946  waitKey(); waitKey();
5947 }
5948
5949 /* ************************************************************** */
5950 /* func Pgende() : Programmende                                   */
5951 /*                                                                */
5952 /*                 Aufgerufen von func Mainmenu()                 */
5953 /* ************************************************************** */
5954 func Pgende() {
5955  clearScreen();
5956  Freemem();
5957  fprintf(runreport,"********** IMS-Reportdatei ENDE *********\n");
5958  fclose(runreport);                      /* REPORTDATEI schliessen ! */
5959  printfAt(24,2,"[ Programm-ENDE ]");
5960  waitKey(); waitKey(); closeScreen(); cls();
5961  exit(0);                                /* C-Funktion aus clib      */
5962 } /* END: func Pgende() */
5963
5964 /* ************************************************************** */
5965 /* func Mainmenu() : Hauptmenu                                    */
5966 /*                                                                */
5967 /*                   Aufgerufen von func Init()                   */
5968 /* ************************************************************** */
5969 func Mainmenu() {
5970  int j, startsim = 0 ;
5971  openScreen();
5972  while ( startsim == 0 )                  /* HAUPTKONTROLL-SCHLEIFE */
5973   {
5974    clearScreen();
5975    printfAt( 1,23, "[ IMS Insurance Market Simulation ]");
5976    printfAt( 5,20, "[1] Modell laden                       ");
5977    printfAt( 7,20, "[2] Modell speichern                   ");
5978    printfAt( 9,20, "[3] Modell auswaehlen und bearbeiten   ");
5979    printfAt(11,20, "[4] Modell simulieren                  ");
5980    printfAt(13,20, "[5] Modellergebnis analysieren         ");
5981    printfAt(15,20, "[6] Programminformation                ");
5982    printfAt(17,20, "[7] Programm beenden                   ");
5983    do {
5984        printfAt(19,20,"[ ] "); scanfAt(19,21,"%d",&j);
```

```
5985       } while ( ( j<1) || (j>7));
5986    switch(j)
5987    {
5988    case 1: /* Modell laden    */
5989      Mkload();
5990      break;
5991    case 2: /* Modell speichern*/
5992      Mksave();
5993      break;
5994    case 3: /* Modell auswaehlen und bearbeiten */
5995      Mkbear();
5996      break;
5997    case 4: /* Modell simulieren */
5998      Starts();
5999      startsim = 1;

6000      break;
6001    case 5: /* Modellergebnis analysieren  */
6002      Analy();
6003      break;
6004    case 6: /* Programminformation         */
6005      Pginfo();
6006      break;
6007    case 7: /* Programmende                */
6008      Pgende();
6009      break;
6010    default:
6011      printfAt(24,2,"UNBEKANNTE AUSWAHL!");
6012      break;
6013   } /* end CASE ...*/
6014  } /* end while startsim ... */
6015 } /* END: func Mainmenu ... */
6016
6017 /* ************************************************************** */
6018 /* func Opendata() : Initialisierung der des Zufallszahlengenerators */
6019 /*                   auf dem die anderen Generatoren aussetzen.     */
6020 /*                   Oeffnen der Reportdatei der Programmablaufkontr.*/
6021 /* ************************************************************** */
6022 func Opendata() {
6023  int        seed;       /* Startwert des Zufallszahlengenerators     */
6024  time_t     x;          /* in time.h definierte Strukturvariable     */
6025  struct tm *t;
6026  time(&x);              /* Bestimmung des Startwertes der Zufalls-   */
6027  t = gmtime(&x);        /* zahlengeneratoren aus der Zeit            */
6028  seed = (t->tm_sec +1) * (t->tm_min +1) * (t->tm_mday+1);
6029  srand(seed);           /* Startwert der Generatoren setzen ...      */
```

```
6030  runreport = fopen("imsreport.dat","w");  /* Oeffnen der Reportdatei */
6031  setvbuf(stdout,(char)NULL, _IONBF,0);    /* Datenpufferung ...      */
6032  setvbuf(runreport,(char)NULL, _IONBF,0); /* ..ausschalten...        */
6033  if ( runreport == (file *) 0)
6034   {
6035     printf("func Opendata(): Kann imsreport.dat nicht oeffnen.\n");
6036   }
6037   else                  /* Dateikopf der Programmablaufkontrolldatei */
6038    {                    /* schreiben ...                             */
6039     fprintf(runreport,"IMS Version %s (c) mk, compiled on  %s %s\n\n",
6040                          version, __DATE__,__TIME__);
6041     fprintf(runreport,
6042        "********************* IMS-Reportdatei ******************\n");
6043     fprintf(runreport,"Seed : %d    \n",seed);
6044    }  /* END: if ( runreport == (file *) 0) */
6045 } /* END: func Opendata() */
6046
6047 /* ************************************************************** */
6048 /* func Freemem(): Gibt den in func Getmem() angeforderten Speicher  */
6049 /*                 wieder frei und schreibt eventuelle Fehler in die */
6050 /*                 Reportdatei                                       */
6051 /* ************************************************************** */
6052 func Freemem() {
6053 int i;
6054   for(i=1;i<=MAXVU;i++)
6055    {
6056     if ( VU[i].DatenVU != (struct s1 *) 0 )
6057      {
6058       free( VU[i].DatenVU  );
6059       fprintf(runreport,"%ld Bytes von VU[%d] freigegeben\n",
6060                                      (long) sizeof(struct s1),i);
6061      gmem = gmem - (long) sizeof(struct s1);
6062      fprintf(runreport,"Bisher allocierter Speicher: %ld Bytes\n",gmem);
6063      }
6064      else
6065      {
6066       fprintf(runreport,"Kein Speicher von VU[%d] freigegeben\n", i);
6067      }
6068     }
6069    for(i=1;i<=MAXVN;i++)
6070    {
6071     if ( VN[i].DatenVN != (struct s2 *) 0 )
6072      {
6073       free( VN[i].DatenVN  );
6074       fprintf(runreport,"%ld Bytes von VN[%d] freigegeben\n",
```

```
6075                            (long) sizeof(struct s2),i);
6076      gmem = gmem - (long) sizeof(struct s2);
6077      fprintf(runreport,"Bisher allocierter Speicher: %ld Bytes\n",gmem);
6078      }
6079      else
6080      {
6081       fprintf(runreport, "Kein Speicher von VN[%d] freigegeben\n",i);
6082      }
6083     }
6084 } /* END: func Freemem() */
6085
6086 /* ************************************************************** */
6087 /* func Getmem() : Funktion zum anfordern von Heap-Space fuer die dy-*/
6088 /*                 namischen ESL-Subjekte der class VU und class VN. */
6089 /*                                                                   */
6090 /* ************************************************************** */
6091 func Getmem() {
6092 int i;                                /* Schleifenzaehler          */
6093 if ( load == 0 )
6094 {
6095  for(i=1;i<=MAXVU;i++)
6096   {
6097    VU[i].DatenVU = (struct s1 *) 0;
6098    VU[i].DatenVU = (struct s1 *) malloc(sizeof(struct s1));
6099    if ( VU[i].DatenVU != (struct s1 *) 0 )
6100     {
6101       gmem = gmem + (long) sizeof(struct s1);
6102     }
6103     else /* if ( VU[i].DatenVU == (struct s1 *) 0 )*/
6104     {
6105      fprintf(runreport,"Kein Speicher allocierte fuer VU[%d]\n", i);
6106     }
6107   } /* for(i=1;i<=MAXVU;i++) */
6108  fprintf(runreport,"Speicher fuer %d VUs: %ld Bytes\n",MAXVU,gmem);
6109  for(i=1;i<=MAXVN;i++)
6110   {
6111    VN[i].DatenVN = (struct s2 *) 0;
6112    VN[i].DatenVN = (struct s2 *) malloc(sizeof(struct s2));
6113     if ( VN[i].DatenVN != (struct s2 *) 0 )
6114      {
6115       gmem = gmem + (long) sizeof(struct s2);
6116      }
6117     else
6118     {
6119      fprintf(runreport,"Kein Speicher allocierte fuer VN[%d]\n", i);
```

```
6120     }
6121   } /* END: for(i=1;i<=MAXVN;i++)*/
6122   fprintf(runreport,"Speicher fuer %d VNs: %ld Bytes\n",MAXVN,gmem);
6123  } /* end if ( load == 1) */
6124 } /* END: func Getmem() */
6125
6126 /* ************************************************************** */
6127 /* func Init(): Wird von ess.c(main()) aus aufgerufen. Startet vor   */
6128 /*              dem 1. Lauf das Hauptmenue und kehrt ueber ess.c(main*/
6129 /*              ()) und Fini() wieder hier hin zurueck               */
6130 /* ************************************************************** */
6131 func Init()              /* jumpback wird in imsdata.c auf 0 initia- */
6132 {                        /* lisiert. Wird beim ERSTEN Lauf auf 1 ge- */
6133  if ( jumpback == 0 )    /* setzt, damit der Speicher fuer die dyn-  */
6134  {                       /* amischen Subjekte NICHT doppelt angefor- */
6135   Opendata();            /* dert wird und die Reportdatei nur im er- */
6136   jumpback=1;            /* sten Lauf geoeffnet wird.                */
6137   Getmem();
6138  }
6139  Mainmenu();
6140 } /* END: func Init() */
6141
6142 /* ************************************************************** */
6143 /* func Fini(): Pruef-Funktion fuer Simulationsende. Wenn Fini() be- */
6144 /*              endet ist, wird in ess.c(main()) der naechste Durch- */
6145 /*              lauf gestartet und die func Init() wieder gestartet  */
6146 /* ************************************************************** */
6147 func Fini()
6148 {
6149  float prozent;
6150  if ( period <= sl )
6151   {
6152    printfAt(10,28,"%3d",period);
6153    printfAt(11,28,"%3d",rl    );
6154    if ( sl * al != 0)
6155    {
6156     prozent=100.0*((float)((rl-1)*sl+period))/((float)sl*(float)al);
6157     printfAt(12,28,"%3.2f",prozent);
6158    }
6159    else
6160    {
6161     if ( sl != 0 )
6162      {
6163       prozent=100.0*((float)((rl-1)*sl+period))/((float)sl);
6164       printfAt(12,28,"%3.2f",prozent);
```

```
6165      }
6166    }
6167    return(GO);
6168   }
6169   else
6170   {
6171    if ( rl < al )
6172     {
6173      period  = 1; /* VORHER: period  = 0 ! */
6174      logtime = 0;
6175      rl++;
6176      return(GO);
6177     }
6178    else
6179    {
6180     printfAt(22,12,"[ Simulation abgeschlossen ]");
6181     waitKey();waitKey();
6182     /*  rl  = 1; */
6183     period  = 1;
6184     logtime = 0;
6185     jumpback=1;
6186     return(STOP);
6187    }
6188   }
6189 } /* END: func Fini()  */
```

Literaturverzeichnis

[1] Ackermann, P. (1983)
Ein Vorschlag zur Messung der Betriebsgröße von Schaden- und Unfallversicherungsunternehmen, Zeitschrift für die gesamte Versicherungswissenschaft, S. 87–97

[2] Arrow, K. J., Debreu, G. (1954)
Existence of an equilibrium for a competitive economy, Econometrica, Vol. 22, No. 3, S. 265–290

[3] Bach, P. (Hrsg.) (1990)
Entwicklung und Perspektiven des Versicherungswesens in der DDR, Karlsruhe

[4] BAV (Hrsg.) (1992)
Geschäftsbericht 1991, Berlin

[5] Bonhoeffer, F.O., Goldrian, G. (1975)
Einführung in die praktische Anwendung der Korrelations- und Regressionsanalyse, München

[6] Böbel, I. (1984)
Wettbewerb und Industriestruktur, Berlin, Heidelberg, New York, Tokyo

[7] Bossel, H., (1992)
Modellbildung und Simulation, Braunschweig, Wiesbaden

[8] Chamberlin, E.H. (1962)
The theory of monopolistic competition, a reorientation of the theory of value, 8. Aufl., Cambridge/Massachusetts

[9] Crössmann, H. J. (1982)
Entscheidungsverhalten auf unvollkommenen Märkten, Frankfurt/Main (Diss.)

[10] Cyert, R.M., March, J.G. (1995)
Eine verhaltenswissenschaftliche Theorie der Unternehmung, 2. Auflage, Stuttgart

[11] Debreu, G. (1959)
Theory of Value, New York, London

[12] Dirrheimer, M., Junghanns, H. (1989)
Wettbewerbsanalyse versicherungswirtschaftlicher Einzelmärkte, in: FWU Forschungsgesellschaft für Wettbewerb und Unternehmensorganisation (Hrsg.), Versicherungsmärkte im Wandel, Baden-Baden, S. 61–206

[13] Farny, D. (1979)
Die Versicherungswirtschaft im Wettbewerbskonzept der Marktwirtschaft, Zeitschrift für die gesamte Versicherungswissenschaft, S. 31–74

[14] Ferstl, O. K. (1979)
Konstruktion und Analyse von Simulationsmodellen, Königsstein/Taunus

[15] Finsinger, J. (1983)
Versicherungsmärkte, Frankfurt/Main, New York

[16] Finsinger, J.; Schulenburg, J.-M. Graf von der (1987)
Nachfragerverhalten bei unvollständiger Preisinformation, Jahrbücher für Nationalökonomie und Statistik, Band 202/3, S. 244–256

[17] Fischer, K. (1988)
Oligopolistische Marktprozesse: Einsatz verschiedener Preis-Mengen-Strategien unter Berücksichtigung von Nachfrageträgheit, Heidelberg

[18] Flassbeck, H., Horn, G.A., Zwiener R. (1992)
Rigide Preise und flexible Mengen – Ansätze zu einer dynamischen Analyse von Angebots- und Nachfrageschocks, Deutsches Institut für Wirtschaftsforschung, Sonderheft 149, Berlin

[19] Flöthmann, E.-J. (1989)
Preisbildung auf dem Kraftversicherungsmarkt, Berlin, (Diss.)

[20] FWU Forschungsgesellschaft für Wettbewerb und Unternehmensorganisation (Hrsg.) (1989)
Versicherungsmärkte im Wandel, Baden-Baden

[21] Gaidamavicius, P. (1985)
Unternehmensmodelle für Versicherer, Versicherungswirtschaft, Heft 19, S. 1248–1257

[22] Galitz, L., Brown, M. (1981)
ASIR - The Advanced Simulation Model of Insurance and Re-insurance Operations: An Introduction, The Geneva Papers on Risk and Insurance, No. 19, S. 85–96

[23] Galitz, L. (1982)
The ASIR Model - An Introduction, The Geneva Papers on Risk and Insurance, No. 25, S. 279–289

[24] GDV (Hrsg.) (1991)
Die deutsche Versicherungswirtschaft 1991, Bonn

[25] GDV (Hrsg.) (1992)
Jahresabschlußstatistik 1991, Auswertung der Geschäftsberichte der Schaden- und Unfall-Versicherungsunternehmen, Bonn

[26] Gerlach, R. (1991)
Das Versicherungswesen in den neuen Bundesländern aus der Sicht der Aufsichtsbehörde, Versicherungswirtschaft, Heft 5, S. 892–902

[27] Goovaerts, M., de Vylder, F., Haezendonck, J. (Hrsg.) (1986)
Insurance and Risk Theory, Dordrecht, Boston, Lancaster, Toykyo

[28] Grams, T. (1992)
Simulation, Mannhein, Leipzig, Wien, Zürich

[29] Gutenberg, E. (1965)
Zur Diskussion der polypolistischen Absatzkurve, Jahrbücher für Nationalökonomie und Statistik 177, S. 289–303

[30] Haasen, U. (1990)
Zum Engagement der Allianz in der DDR, Zeitschrift für Versicherungswesen, Heft 18, S. 434–435

[31] Haid, A. (1984)
Informationsökonomik und Güterpreise, Göttingen

[32] Hansen, K. (1979)
The Insurance Game - A Business Decision Game based upon a Simulation of an Insurance Market, Etudes et Dossiers Nr. 29, Association International pour l'Etude de l'Economie de l'Assurance, Genf

[33] Hansen, K. (1982)
Versicherungswirtschaftliche Unternehmensspiele, Karlsruhe

[34] Hax, K. (1972)
Unternehmensgröße und Konzentration in der Versicherungswirtschaft, in: Schmidt, R., Sieg, K. Grundprobleme des Versicherungsrechts, Festgabe für Hans Möller, Karlsruhe, S. 261–281

[35] Holleis, W. (1985)
Das Ungleichgewicht in der Gleichgewichtstheorie, Frankfurt, New York

[36] Holler, M. J., Illing, G. (1991)
Einführung in die Spieltheorie, Berlin, Heidelberg

[37] Holzheu, Th. (1992)
Größenvorteile in der Assekuranz, Versicherungswirtschaft, Heft 2, S. 111-118

[38] Houston, D. B., Simon, R. M. (1970)
Economies of Scale in Financial Institutions: A study in life insurance, Econometrica, Vol. 38, No. 6, S. 856–864

[39] Kaluza, B. (1990)
Die Betriebsgröße - ein strategischer Erfolgsfaktor von Versicherungsunternehmen? Zeitschrift für die gesamte Versicherungswissenschaft, S. 251–273

[40] Klein, W. (1991)
Der Versicherungsmarkt in den neuen Bundesländern, Versicherungswirtschaft, Heft 4, S. 202–206

[41] Klopstech, A. (1980)
Eingeschränkt rationale Marktprozesse, Bielefeld

[42] Klösgen, W.; Quinke, H. (1985)
Sozio-ökonomische Simulation und Planung, Informatik-Spektrum, Heft 8, S. 328–336

[43] Krupp, H.-J.; Wagner, G. (1982)
Grundlagen und Anwendung mikronanalytischer Modelle, DIW, Vierteljahrhefte zur Wirtschaftsforschung, S. 5–27

[44] Krupp, H.-J. (1986)
Potential and Limitations of Microsimulation Modells, in: Orcutt, G.; Merz, J.; Quinke, H. (Hrsg.), Microanalytic Simulation Models to support social and Financial Policy, Amsterdam, New York, Oxford S. 31–41

[45] Külp, B. (1989)
Wirtschaftssimulationen in C – Spracheinführung und praktische Umsetzung, Hamburg

[46] Law, A. M., Kelton, W. D. (1982)
Simulation Modelling and Analysis, New York, London, Tokyo

[47] Lehmann-Waffenschmidt, M. (1992)
Ökonomische Evolution und Gleichgewicht – konträr oder komplementär, in: Witt, U. (Hrsg.) (1992) Studien zur Evolutorischen Ökonomik II, Schriften des Vereins für Socialpolitik, 195/II, Berlin

[48] Le Vaillant de Folleville, P. (1986)
Nicht-sequenzielle Suche, Preiswerbung und Marktgleichgewicht, Frankfurt/Main, Bern, New York

[49] Lindfield, G.R. (1991)
Educational aspects of simulation and modelling, in: Moscardini A.O., Fletcher E.J., Applied Simulation and system dynamics, Welbury, Northallerton, S. 95 – 109

[50] Ludewig, J. (1975)
Simulationsmodelle ganzer Unternehmen, Wiesbaden

[51] Mahr, W. (1971)
Markt- und Wettbewerbsprobleme in der Versicherungswirtschaft, Zeitschrift für die gesamte Versicherungswissenschaft, S. 361–406

[52] McGuinness, J. S. (1960)
A Managerial Game for an Insurance Company, Operations Research, Vol. 8, S. 196–203

[53] Meissner, W. (1970)
Zur Methodologie der Simulation, Zeitschrift für die gesamte Staatswissenschaft, 126. Band, S. 385–397

[54] Meyer-Kahlen, W. (1985)
Wettbewerb auf den Versicherungsmärkten aus der Sicht der gewerblichen Nachfrager, Zeitschrift für die gesamte Versicherungswissenschaft, S. 239–255

[55] Mresse, M. (1984)
Computersimulation: Planung und Durchführung Output, Nr. 7, S. 25–32

[56] Möller, D. (1992)
Modellbildung, Simulation und Identifikation dynamischer Systeme, Berlin, Heidelberg

[57] Moscardini A.O., Fletcher E.J. (1991)
Applied simulation and system dynamics, Welbury, Northallerton

[58] Neelamkavil, F. (1987)
Computer Simulation and Modelling, Chichester, New York, Brisbane, Toronto, Singapore

[59] Neumann, J. v., Morgenstern, O. (1973)
Spieltheorie und wirtschaftliches Verhalten, 3. unveränderte Auflage, Würzburg

[60] Ohlendorf, G. (1987)
Entwicklung und Implementierung einer Sprache zur Simulation ökonomischer Systeme, Hannover, (Diss.)

[61] Ohlendorf, G., Stahlecker, P. (1990)
ESL - A New Simulation Language for Economics Models, Computer Science in Economics and Managment, Vol. 3, S. 187–197

[62] Ohlendorf, G., Stahlecker, P. (1992)
An Application of ESL to a Stochastic Dynamic Market Modell with Free Entry and Exit, Computer Science in Economics and Managment, Heft 5, S. 105 – 118

[63] Otruba, H. (1981)
Wirtschaftliches Verhalten bei Ungleichgewicht, Wien (Diss.)

[64] Orcutt, G.; Merz, J.; Quinke, H. (Hrsg.) (1986)
Microanalytic Simulation Models to support social and Financial Policy, Amsterdam, New York, Oxford

[65] Pitacco, E. (1986)
Simulation in Insurance, in: Goovaerts, M; de Vylder, F., Haezendonck, J. (Hrsg.), Insurance and Risk Theory, Dordrecht, Boston, Lancaster, Toyko, S. 37–77

[66] Pidd, M. (1988)
Computer Simulation in Management Science, 2. Aufl., Chichester, New York, Brisbane, Toronto, Singapore

[67] Pidd, M. (1989)
Developments in discrete Simulation, in: Pidd, M. (Hrsg.) Computer Modelling for discrete Simulation, Chichester, New York, Brisbane

[68] Pidd, M. (Hrsg.) (1989)
Computer Modelling for discrete Simulation, Chichester, New York, Brisbane

[69] Rothschild, K.W (1981)
Einführung in die Ungleichgewichtstheorie, Berlin, Heidelberg, New York

[70] Schareck, B. (1979)
Modell- und computergestützte Planung im Versicherungswesen, Köln

[71] Schmalensee, R. (1986)
Advertising and Market Strukture, in: Stiglitz, J. E., Mathewson, G. F. (Hrsg.), New Depelopments in the Analysis of Market Structure, Houndmills, Basingstoke Hampshire, London, S. 373–398

[72] Schmidt, R., Sieg, K. (1972)
Grundprobleme des Versicherungsrechts, Festgabe für Hans Möller, Karlsruhe

[73] Schulenburg, J.-M. Graf von der (1987)
Marktgeschehen bei unvollständiger Nachfragerinformation, Zeitschrift für Betriebswirtschaft, 57. Jg., H. 7, S.699–719

[74] Schulenburg, J.-M. Graf von der (1989)
Die Versicherungsnachfrage als Gegenstand der ökonomischen Forschung, Zeitschrift für die gesamte Versicherungswissenschaft, S. 317–333

[75] Schulenburg, J.-M. Graf von der (1993)
Marktprozß und Marktstruktur bei unvollständiger Information, Zeitschrift für Wirtschafts- und Sozialwissenschaften, Band 113, S. 509–555

[76] Schulenburg, J.-M. Graf von der, Schattscheider, U., Knemeyer, U., Karten, W., Rautmann, N., Hipp, Ch., Wehrmann, D. (1995)
Teilnehmerhandbuch INSGAME, Hannover

[77] Schumann, J. (1987)
Grundzüge der mikroökonomischen Theorie, 5. Aufl., Berlin, Heidelberg, New York, London, Paris, Tokyo

[78] Siegert, H.-J. (1991)
Simulation zeitdiskreter Systeme, München, Wien, Oldenburg

[79] Stigler, G.J. (1961)
The economics of Information, Journal of Political Economy, Vol. 69, S. 213–225

[80] Stiglitz, J. E., Mathewson, G. F. (Hrsg.)(1986)
New Depelopments in the Analysis of Market Structure, Houndmills, Basingstoke Hampshire, London

[81] Strassl, W. (1988)
Externe Effekte auf Versicherungsmärkten. Eine allokationstheoretische Begründung staatlicher Regulierung, Tübingen

[82] Szopo, P. (1987)
Skalenerträge in der österreichischen Lebensversicherungswirtschaft, Empirica, S. 77–97

[83] Vetterle, H. (1986)
Konstruktion und Simulation mikroanalytischer Modelle: die Methode der Mikrosimulation und ihre Anwendung, Augsburg, (Diss.)

[84] Wanninger, G. (1981)
Zur Preisbildung in Oligopolen: eine theoretische und empirische Analyse, Frankfurt/Main, Bern

[85] Wilde, K. D. (1983)
Modellgestützte strategische Planung in der Versicherung, Karlsruhe

[86] Witt, U. (1980)
Marktprozesse: neoklassische versus evolutorische Theorie der Preis- und Mengendynamik, Königsstein/Taunus

[87] Wissenschaftlicher Rat der Dudenredaktion (Hrsg.)(1973)
Duden, Die Rechtsschreibung, 17. neu bearbeitete und erweiterte Auflage, Mannheim, Wien, Zürich

[88] Witt, U., Perske, J. (1982)
SMS - A Program Package for Simulation and Gaming of Stochastic Market Processes and Learing Behavior, Berlin, Heidelberg, New York

[89] Witt, U. (Hrsg.) (1992)
Studien zur Evolutorischen Ökonomik II, Schriften des Vereins für Socialpolitik, 195/II, Berlin

[90] Wesselkock, K. (1991)
Die Versicherungswirtschaft vor veränderten Rahmenbedingungen, Heft 23, S. 1450–1457

[91] Wielsch, M. (1994)
Das große Buch zu UNIX, Düsseldorf

[92] Zwicker, E. (1981)
Simulation und Analyse dynamischer Systeme in den Wirtschafts- und Sozialwissenschaften Berlin, New York